THEIVANAI AMMAL COLLEGE FOR WOMEN

(An Autonomous Institution, Affiliated to Annamalai University, Chidambaram - Tamil Nadu)

(Accredited with 'A+' Grade (4th Cycle) by NAAC, Recognized under 2(f) and 12(B) by UGC, Approved by AICTE, New Delhi)

Villupuram-Tamil Nadu

DEPARTMENT OF COMPUTER SCIENCE

This is to certify that Ms **M.KEERTHIGA (20524UCSC055)** a student of III B.Sc. Computer Science, has successfully completed the practice-based practical learning component of **Advanced Database Management System – Practical (UCSR513)** during the academic year 2026–2027.

The practical examination will be held on **16.10.2026**

Course In-charge: Dr. S Manimekalai

Head of the Department: Ms. K Manohari

CONTENT

S.NO	EXPERIMENT	PAGE NO	FACULTY SIGNATURE
1	**DDL Commands** • CREATE • ALTER • DROP • TRUNCATE		
2	**DML Commands** • INSERT • UPDATE • DELETE		
3	**DQL Commands** • SELECT • WHERE • ORDER BY • GROUP BY • HAVING		
4	**Transactions** • COMMIT • ROLLBACK • SAVEPOINT		
5	**Joins** • Inner Join • Left Join • Right Join • Full Join • Self Join		
6	**SQL Functions** • Aggregate Functions • String Functions • Date Functions		
7	**Subqueries and Nested Queries**		
8	**Views, Indexes, and Sequences**		
9	**Stored Procedures and Functions**		
10	**Triggers**		
11	**Cursor Implementation**		
12	**PL/SQL Exception Handling**		
13	**Database Normalization (1NF, 2NF, 3NF, BCNF)**		
14	**Mini Project**		
	Annexures: • **Khan Academy** • **Hacker Rank** • **Code Chef**		

EX. NO: 01 Creation of a DDL Operations using Employee Table

DATE: 3.8.2026

Aim

To create an Employee table, insert employee records, display all records, and retrieve employees whose salary is 25000 or above.

Software Used

• Oracle / MySQL / SQLite3

Description

DDL (Data Definition Language) is used to create and modify the structure of database objects.

DDL Commands

•CREATE
•ALTER
•DROP
• TRUNCATE

CREATE

The CREATE command is used to create a new table with constraints.

Syntax

```
CREATE TABLE table_name (
   column1 datatype constraints,
   column2 datatype constraints,
   column3 datatype constraints
);
```

Table Structure

```
CREATE TABLE employee (
   employee_id INTEGER PRIMARY KEY,
   employee_name VARCHAR(100),
   salary INTEGER
```

);

Table Created

ALTER

The ALTER command is used to modify the structure of a table.

ADD

Syntax

ALTER TABLE employee ADD department VARCHAR(50);

Table Altered

MODIFY

Syntax

ALTER TABLE employee MODIFY employee_name VARCHAR(150);

(MySQL / Oracle)

Or (SQL Server)

ALTER TABLE employee

ALTER COLUMN employee_name VARCHAR(150);

DROP COLUMN

Syntax

ALTER TABLE employee DROP COLUMN department;

RENAME TABLE

Syntax

ALTER TABLE employee RENAME TO employees;

OR

RENAME TABLE employee TO employees;

RENAME COLUMN

ALTER TABLE employee

RENAME COLUMN employee_name TO name;

TRUNCATE

Syntax

TRUNCATE TABLE employee;

DROP TABLE

Syntax

DROP TABLE employee;

Explanation

•Create a table named employee.
• Insert employee details into the table.
• Display all employee records.
• Retrieve employees whose salary is 25000 or above and sort them by salary.

Create Table

```
CREATE TABLE employee (
    employee_id INTEGER PRIMARY KEY,
    employee_name VARCHAR(100),
    salary INTEGER
);
```

Sample Data

```
INSERT INTO employee VALUES (201, 'Arun', 22000);
INSERT INTO employee VALUES (202, 'Priya', 28000);
INSERT INTO employee VALUES (203, 'Karthik', 35000);
INSERT INTO employee VALUES (204, 'Divya', 24000);
INSERT INTO employee VALUES (205, 'Rahul', 30000);
INSERT INTO employee VALUES (206, 'Meena', 27000);
```

SQL Query

Query 1: Display all employee records

SELECT * FROM employee;

Resultant Query

employee_id	employee_name	salary
201	Arun	22000
202	Priya	28000
203	Karthik	35000
204	Divya	24000
205	Rahul	30000
206	Meena	27000

Query 2: Display employees whose salary is 25000 or above

SELECT *

FROM employee

WHERE salary >= 25000

ORDER BY salary;

Resultant Query

employee_id	employee_name	salary
206	Meena	27000
202	Priya	28000
205	Rahul	30000
203	Karthik	35000

RESULT

Thus, the Employee table was created successfully, records were inserted, all employee details were displayed, and the employees whose salary is 25000 or above were retrieved successfully.

EX. NO: 02 **Creation of DML Operations using Student Table**

DATE: 6.8.2026

Aim

To perform DML (Data Manipulation Language) operations such as INSERT, UPDATE, DELETE, and SELECT on the Student table.

Software Used

- Oracle / MySQL / SQLite3

Description

DML (Data Manipulation Language) is used to manipulate the data stored in database tables.

DML Commands

- INSERT
- UPDATE
- DELETE

INSERT

Syntax

INSERT INTO student VALUES (student_id, student_name, mark);

Create Table

```
CREATE TABLE student (
student_id INTEGER PRIMARY KEY,
student_name VARCHAR(100),
mark INTEGER
);
```

Sample Data

```
INSERT INTO student VALUES (101,'Vijay',78);
INSERT INTO student VALUES (102,'Anitha',84);
INSERT INTO student VALUES (103,'Suresh',69);
INSERT INTO student VALUES (104,'Nandhini',92);
INSERT INTO student VALUES (105,'Mohan',73);
INSERT INTO student VALUES (106,'Kavya',87);
```

6 Rows Inserted

UPDATE

Syntax

UPDATE student SET mark = value WHERE student_id = value;

Query

UPDATE student SET mark = 89 WHERE student_id = 105;

1 Row Updated

DELETE

Syntax

DELETE FROM student WHERE student_id = value;

Query

DELETE FROM student WHERE student_id = 102;

1 Row Deleted

Query 1: Update Mohan's mark to 89

UPDATE student SET mark = 89 WHERE student_id = 105;

Query 2: Delete Anitha's record

DELETE FROM student WHERE student_id = 102;

Query 3: Display all records after UPDATE and DELETE

SELECT * FROM student;

Resultant Query

student_id	student_name	mark
101	Vijay	78
103	Suresh	69
104	Nandhini	92
105	Mohan	89
106	Kavya	87

RESULT

Thus, the DML operations (INSERT, UPDATE, DELETE) were performed successfully on the Student table, and the required results were obtained.

EX. NO: 03 **Creation of DQL Operations using Employee Table**

DATE: 11.8.2026

Aim

To create an Employee table, insert employee records, and perform DQL (Data Query Language) operations using SELECT, WHERE, ORDER BY, GROUP BY, and HAVING clauses.

Software Used

- Oracle / MySQL / SQLite3

Description

DQL (Data Query Language) is used to retrieve data from database tables.

DQL Commands

- SELECT
- WHERE
- ORDER BY
- GROUP BY
- HAVING

Create Table

```
CREATE TABLE employee (
employee_id INTEGER PRIMARY KEY,
employee_name VARCHAR(100),
department VARCHAR(50),
salary INTEGER
);
```

Sample Data

```
INSERT INTO employee VALUES (101,'Vijay','IT',35000);
INSERT INTO employee VALUES (102,'Anitha','HR',28000);
INSERT INTO employee VALUES (103,'Suresh','IT',42000);
INSERT INTO employee VALUES (104,'Nandhini','Sales',30000);
INSERT INTO employee VALUES (105,'Mohan','Finance',38000);
INSERT INTO employee VALUES (106,'Kavya','HR',32000);
```

SELECT

The SELECT command is used to retrieve all records from a table.

Syntax

SELECT * FROM employee;

Query

SELECT * FROM employee;

Resultant Query

employee_id	employee_name	department	salary
101	Vijay	IT	35000
102	Anitha	HR	28000
103	Suresh	IT	42000
104	Nandhini	Sales	30000
105	Mohan	Finance	38000
106	Kavya	HR	32000

WHERE

The WHERE clause is used to filter records.

Query

SELECT * FROM employee WHERE salary >= 35000;

Resultant Query

employee_id	employee_name	department	salary
101	Vijay	IT	35000
103	Suresh	IT	42000
105	Mohan	Finance	38000

ORDER BY

The ORDER BY clause is used to sort records.

Query

SELECT * FROM employee ORDER BY salary;

Resultant Query

employee_id	employee_name	department	salary
102	Anitha	HR	28000
104	Nandhini	Sales	30000
106	Kavya	HR	32000
101	Vijay	IT	35000
105	Mohan	Finance	38000
103	Suresh	IT	42000

GROUP BY

The GROUP BY clause is used to group records having the same department.

Query

SELECT department, COUNT(*) AS total_employees FROM employee GROUP BY department;

Resultant Query

department	total_employees
Finance	1
HR	2
IT	2
Sales	1

HAVING

The HAVING clause is used to filter grouped records.

Query

SELECT department, COUNT*() AS total_employees FROM employee GROUP BY department HAVING COUNT()* > 1;

Resultant Query

department	total_employees
HR	2
IT	2

RESULT

Thus, the DQL operations (SELECT, WHERE, ORDER BY, GROUP BY, and HAVING) were performed successfully on the Employee table, and the required results were obtained.

EX. NO: 04 Creation of Transaction Control Operations using Hospital Table

DATE: 14.8.2026

Aim

To create a Hospital table and perform Transaction Control Language (TCL) operations using COMMIT, ROLLBACK, and SAVEPOINT.

Software Used

Oracle / MySQL / SQLite3

Description

Transaction Control Language (TCL) is used to manage transactions in a database.

TCL Commands

COMMIT

ROLLBACK

SAVEPOINT

Create Table

```
CREATE TABLE hospital (
patient_id INTEGER PRIMARY KEY,
patient_name VARCHAR(100),
disease VARCHAR(100)
);
```

Sample Data

```
INSERT INTO hospital VALUES (101,'Ravi','Fever');

INSERT INTO hospital VALUES (102,'Anjali','Cold');

INSERT INTO hospital VALUES (103,'Manoj','Diabetes');

INSERT INTO hospital VALUES (104,'Kavitha','Asthma');
```

COMMIT

The COMMIT command permanently saves all changes made during the current transaction.

Syntax

COMMIT;

Query

INSERT INTO hospital VALUES (105,'Sanjay','Headache');

COMMIT;

Transaction Committed Successfully

SAVEPOINT

The SAVEPOINT command creates a point within a transaction to which you can later roll back.

Syntax

SAVEPOINT savepoint_name;

Query

SAVEPOINT sp1;

INSERT INTO hospital VALUES (106,'Divya','Allergy');

Savepoint Created Successfully

ROLLBACK

The ROLLBACK command is used to undo changes made after the last SAVEPOINT.

Syntax

ROLLBACK TO savepoint_name;

Query

ROLLBACK TO sp1;

Transaction Rolled Back Successfully

Resultant Query

SELECT * FROM hospital;

patient_id	patient_name	disease
101	Ravi	Fever
102	Anjali	Cold
103	Manoj	Diabetes
104	Kavitha	Asthma
105	Sanjay	Headache



RESULT

Thus, the Transaction Control Language (TCL) operations using COMMIT, SAVEPOINT, and ROLLBACK were performed successfully on the Hospital table.

EX. NO: 05 Creation of JOIN Operations using Employee and Project Tables

DATE: 18.8.2026

Aim

To create Employee and Project tables and perform SQL JOIN operations such as INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, and SELF JOIN.

Software Used

Oracle / MySQL / SQLite3

Description

Joins are used to combine rows from two or more tables based on a related column.

JOIN Types

INNER JOIN

LEFT JOIN

RIGHT JOIN

FULL JOIN

SELF JOIN

Create Table

```
CREATE TABLE employee (
employee_id INTEGER PRIMARY KEY,
employee_name VARCHAR(100),
department VARCHAR(50)
);

CREATE TABLE project (
project_id INTEGER PRIMARY KEY,
project_name VARCHAR(100),
employee_id INTEGER
);
```

Sample Data

```
INSERT INTO employee VALUES (1,'Vijay','Development');
```

INSERT INTO employee VALUES (2,'Anitha','Testing');

INSERT INTO employee VALUES (3,'Suresh','Designing');

INSERT INTO employee VALUES (4,'Kavya','Support');

INSERT INTO project VALUES (101,'Website Development',1);

INSERT INTO project VALUES (102,'Mobile Application',2);

INSERT INTO project VALUES (103,'Database Project',2);

INSERT INTO project VALUES (104,'Cloud Migration',5);

INNER JOIN

Syntax

SELECT * FROM employee INNER JOIN project ON employee.employee_id = project.employee_id;

Result

employee_id	employee_name	department	project_id	project_name
1	Vijay	Development	101	Website Development
2	Anitha	Testing	102	Mobile Application
2	Anitha	Testing	103	Database Project

LEFT JOIN

SELECT * FROM employee LEFT JOIN project ON employee.employee_id = project.employee_id;

Result

employee_id	employee_name	department	project_id	project_name
1	Vijay	Development	101	Website Development
2	Anitha	Testing	102	Mobile Application
2	Anitha	Testing	103	Database Project
3	Suresh	Designing	NULL	NULL
4	Kavya	Support	NULL	NULL

RIGHT JOIN

SELECT * FROM employee RIGHT JOIN project ON employee.employee_id = project.employee_id;

Result

employee_id	employee_name	department	project_id	project_name
1	Vijay	Development	101	Website Development
2	Anitha	Testing	102	Mobile Application
2	Anitha	Testing	103	Database Project
NULL	NULL	NULL	104	Cloud Migration

FULL JOIN

SELECT * FROM employee FULL OUTER JOIN project ON employee.employee_id = project.employee_id;

Result

employee_id	employee_name	department	project_id	project_name
1	Vijay	Development	101	Website Development
2	Anitha	Testing	102	Mobile Application
2	Anitha	Testing	103	Database Project
3	Suresh	Designing	NULL	NULL
4	Kavya	Support	NULL	NULL
NULL	NULL	NULL	104	Cloud Migration

SELF JOIN

CREATE TABLE staff (staff_id INTEGER PRIMARY KEY, staff_name VARCHAR(100), manager_id INTEGER);

INSERT INTO staff VALUES (1,'Vijay',NULL);

INSERT INTO staff VALUES (2,'Anitha',1);

INSERT INTO staff VALUES (3,'Suresh',1);

INSERT INTO staff VALUES (4,'Kavya',2);

SELECT A.staff_name AS Employee, B.staff_name AS Manager FROM staff A LEFT JOIN staff B ON A.manager_id = B.staff_id;

Result

Employee	**Manager**
Vijay	NULL
Anitha	Vijay
Suresh	Vijay
Kavya	Anitha

RESULT

Thus, the INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, and SELF JOIN operations were performed successfully using the Employee and Project tables.

EX. NO: 06 **Creation of SQL Functions using Mobile Table**

DATE: 21.8.2026

Aim

To create a Mobile table and perform SQL Functions such as Aggregate Functions, String Functions, and Date Functions.

Software Used

Oracle / MySQL / SQLite3

Description

SQL Functions are used to perform calculations, manipulate strings, and work with date values in a database.

SQL Functions

Aggregate Functions

String Functions

Date Functions

Create Table

CREATE TABLE mobile (mobile_id INTEGER PRIMARY KEY, mobile_name VARCHAR(100), brand VARCHAR(50), price INTEGER, purchase_date DATE);

Sample Data

INSERT INTO mobile VALUES (101,'Galaxy A15','Samsung',18000,'2024-01-12');

INSERT INTO mobile VALUES (102,'Redmi Note 13','Xiaomi',16000,'2024-02-18');

INSERT INTO mobile VALUES (103,'Nord CE 3','OnePlus',24000,'2024-03-22');

INSERT INTO mobile VALUES (104,'Moto G54','Motorola',15000,'2024-04-14');

INSERT INTO mobile VALUES (105,'Realme C55','Realme',13000,'2024-05-26');

Aggregate Functions

Aggregate functions perform calculations on multiple rows.

COUNT()

SELECT COUNT(*) AS Total_Mobiles FROM mobile;

Result

Total_Mobiles
5

SUM()

SELECT SUM(price) AS Total_Price FROM mobile;

Result

Total_Price
86000

AVG()

SELECT AVG(price) AS Average_Price FROM mobile;

Result

Average_Price
17200

MAX()

SELECT MAX(price) AS Highest_Price FROM mobile;

Result

Highest_Price
24000

MIN()

SELECT MIN(price) AS Lowest_Price FROM mobile;

Result

Lowest_Price
1300

String Functions

UPPER()

SELECT UPPER(mobile_name) FROM mobile;

Result

UPPER(mobile_name)
GALAXY A15
REDMI NOTE 13
NORD CE 3
MOTO G54
REALME C55

LOWER()

SELECT LOWER(brand) FROM mobile;

Result

LOWER(brand)
Samsung
Xiaomi
Oneplus
Motorola
Realme

LENGTH()

SELECT mobile_name, LENGTH(mobile_name) FROM mobile;

Result

mobile_name	LENGTH(mobile_name)
Galaxy A15	9
Redmi Note 13	13
Nord CE 3	9
Moto G54	8
Realme C55	10

Date Functions

Current Date

Oracle

SELECT SYSDATE FROM DUAL;

MySQL

SELECT CURDATE();

Result

SYSDATE / CURDATE()

Current System Date

Extract Year

Oracle

SELECT EXTRACT(YEAR FROM purchase_date) FROM mobile;

MySQL

SELECT YEAR(purchase_date) FROM mobile;

Result

YEAR(purchase_date)
2024
2024
2024
2024
2024

RESULT

Thus, the Aggregate Functions, String Functions, and Date Functions were performed successfully using the Mobile table.

EX. NO: 07 **Subqueries and Nested Queries**

DATE: 25.8.2026

Aim

To implement subqueries and nested queries using the COURSE table.

Software Used

Oracle / MySQL / SQLite3

Description

A Subquery is a query written inside another SQL query. The inner query is executed first and its result is used by the outer query.

TYPES

- Single-row subquery
- Multiple-row subquery
- Nested subquery

Syntax

SELECT column_name FROM table_name WHERE column_name > (SELECT column_name FROM table_name WHERE condition);

Create Table

CREATE TABLE course7 (course_id INTEGER PRIMARY KEY, course_name VARCHAR2(100), fee NUMBER);

Table Created

Sample Data

INSERT INTO course7 VALUES (401, 'Python Programming', 5000);
INSERT INTO course7 VALUES (402, 'Java Programming', 7000);
INSERT INTO course7 VALUES (403, 'Web Development', 6000);
INSERT INTO course7 VALUES (404, 'Database Management', 4500);
INSERT INTO course7 VALUES (405, 'Data Science', 8000);

COMMIT;

SQL Query

Query 1: Display all course records

SELECT * FROM course7;

Resultant Query

COURSE_ID	COURSE_NAME	FEE
401	Python Programming	5000
402	Java Programming	7000
403	Web Development	6000
404	Database Management	4500
405	Data Science	8000

Query 2: Display courses whose fee is greater than average fee

SELECT * FROM course7 WHERE fee > (SELECT AVG(fee) FROM course7);

Resultant Query

Average fee = (5000 + 7000 + 6000 + 4500 + 8000) / 5
Average fee = 6100

COURSE_ID	COURSE_NAME	FEE
402	Java Programming	7000
405	Data Science	8000

Explanation

The inner query calculates the average fee of all courses. The outer query displays courses whose fee is greater than the calculated average.

RESULT

Thus, subqueries and nested queries were executed successfully using the Course table.

EX. NO: 08 **Views, Indexes and Sequences**

DATE: 28.8.2026

Aim

To create and implement Views, Indexes and Sequences using the EMPLOYEE table.

Software Used

Oracle / MySQL / SQLite3

Description

View: A virtual table created from a SQL query.

Index: Used to improve the speed of data retrieval.

Sequence: Used to generate sequential numeric values.

Create Table

CREATE TABLE employee8 (employee_id INTEGER PRIMARY KEY, employee_name VARCHAR2(50), department VARCHAR2(50));

Table Created

Sample Data

INSERT INTO employee8 VALUES (501, 'Vijay', 'IT');

INSERT INTO employee8 VALUES (502, 'Anitha', 'HR');

INSERT INTO employee8 VALUES (503, 'Suresh', 'IT');

INSERT INTO employee8 VALUES (504, 'Kavya', 'Sales');

COMMIT;

SQL Query

Query 1: Display all employee records

SELECT * FROM employee8;

Resultant Query

EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT
501	Vijay	IT
502	Anitha	HR
503	Suresh	IT
504	Kavya	Sales

VIEW

Query

CREATE VIEW it_employees AS SELECT employee_id, employee_name, department FROM employee8 WHERE department = 'IT';

View Created

Display View

SELECT * FROM it_employees;

Resultant Query

EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT
501	Vijay	IT
503	Suresh	IT

INDEX

Query

CREATE INDEX idx_employee_name ON employee8(employee_name);

Index Created

Display Record

SELECT * FROM employee8 WHERE employee_name = 'Vijay';

Resultant Query

EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT
501	Vijay	IT

SEQUENCE

Query

CREATE SEQUENCE employee_seq START WITH 505 INCREMENT BY 1;

Sequence Created

Query

SELECT employee_seq.NEXTVAL FROM dual;

Resultant Query

EMPLOYEE_SEQ.NEXTVAL
505

Next value:

SELECT employee_seq.NEXTVAL FROM dual;

506

Explanation

- A view is created for IT employees.
- An index is created on employee name.
- A sequence generates unique employee IDs.
- NEXTVAL returns the next sequence value.

RESULT

Thus, the View, Index and Sequence were created and executed successfully using the Employee table.

EX. NO: 09 **Stored Procedures and Functions**

DATE: 31.8.2026

Aim

To create and execute stored procedures and functions using the EMPLOYEE_SALARY table**.**

Software Used

Oracle / MySQL / SQLite3

Description

A Stored Procedure is a named PL/SQL block stored in the database.

A Function is a stored PL/SQL program that returns a value.

Create Table

CREATE TABLE employee_salary9 (employee_id INTEGER PRIMARY KEY, employee_name VARCHAR2(50), salary NUMBER);

Table Created

Sample Data

INSERT INTO employee_salary9 VALUES (601, 'Vijay', 28000);

INSERT INTO employee_salary9 VALUES (602, 'Anitha', 35000);

INSERT INTO employee_salary9 VALUES (603, 'Suresh', 42000);

COMMIT;

SQL Query

Query 1: Display all employee salary records

SELECT * FROM employee_salary9;

Resultant Query

EMPLOYEE_ID	EMPLOYEE_NAME	SALARY
601	Vijay	28000
602	Anitha	35000
603	Suresh	42000

STORED PROCEDURE

Syntax

```
CREATE OR REPLACE PROCEDURE procedure_name
IS
BEGIN
statements;
END;
/
```

Query

```
CREATE OR REPLACE PROCEDURE display_employees
IS
BEGIN
FOR rec IN (
SELECT employee_id, employee_name, salary
FROM employee_salary9
)
LOOP
DBMS_OUTPUT.PUT_LINE(
'Employee ID: ' || rec.employee_id ||
' Name: ' || rec.employee_name ||
' Salary: ' || rec.salary
);
END LOOP;
END;
/
```

Procedure Created

Execute Procedure

```
SET SERVEROUTPUT ON;

EXEC display_employees;
```

Resultant Query

Employee ID: 601 Name: Vijay Salary: 28000

Employee ID: 602 Name: Anitha Salary: 35000

Employee ID: 603 Name: Suresh Salary: 42000

FUNCTION

Query

```
CREATE OR REPLACE FUNCTION get_salary
(
p_employee_id NUMBER
)
RETURN NUMBER
IS
v_salary NUMBER;
BEGIN
SELECT salary
INTO v_salary
FROM employee_salary9
WHERE employee_id = p_employee_id;

RETURN v_salary;

END;
/
```

Function Created

Execute Function

```
SELECT get_salary(601) AS salar FROM dual;
```

Resultant Query

SALARY
28000

Explanation

The procedure displays all employee salary records.

The function accepts an employee ID and returns the corresponding employee salary.

RESULT

Thus, the stored procedure and function were created and executed successfully using the Employee Salary table.

EX. NO: 10 **Triggers**

DATE: 2.9.2026

Aim

To create and implement a trigger using the PRODUCT_STOCK table to automatically validate product quantity.

Software Used

Oracle / MySQL / SQLite3

Description

A Trigger is a stored PL/SQL program that automatically executes when an event such as INSERT, UPDATE, or DELETE occurs.

TYPES OF TRIGGERS

- BEFORE Trigger
- AFTER Trigger
- INSERT Trigger
- UPDATE Trigger
- DELETE Trigger

Syntax

CREATE OR REPLACE TRIGGER trigger_name

BEFORE INSERT OR UPDATE

ON table_name

FOR EACH ROW

BEGIN

statements;

END;

/

Create Table

```
CREATE TABLE product_stock10 (
product_id INTEGER PRIMARY KEY,
product_name VARCHAR2(100),
quantity NUMBER
);
```

Table Created

Sample Data

```
INSERT INTO product_stock10 VALUES (701, 'Laptop', 10);

INSERT INTO product_stock10 VALUES (702, 'Keyboard', 15);

INSERT INTO product_stock10 VALUES (703, 'Mouse', 20);
```

COMMIT;

SQL Query

Query 1: Display all product stock records

```
SELECT * FROM product_stock10;
```

Resultant Query

PRODUCT_ID	PRODUCT_NAME	QUANTITY
701	Laptop	10
702	Keyboard	15
703	Mouse	20

Query 2: Create Trigger

```
CREATE OR REPLACE TRIGGER check_product_quantity

BEFORE INSERT OR UPDATE

ON product_stock10

FOR EACH ROW

BEGIN

IF :NEW.quantity < 0 THEN
```

```
    RAISE_APPLICATION_ERROR(
        -20001,
        'Quantity cannot be negative'
    );


END IF;
END;
/
```

Trigger Created

Query 3: Valid Insert

```
INSERT INTO product_stock10 VALUES (704, 'Monitor', 12);
```

Resultant Query

1 row inserted.

Query 4: Invalid Insert

```
INSERT INTO product_stock10 VALUES (705, 'Printer', -5);
```

Resultant Query

ORA-20001: Quantity cannot be negative

Display Records

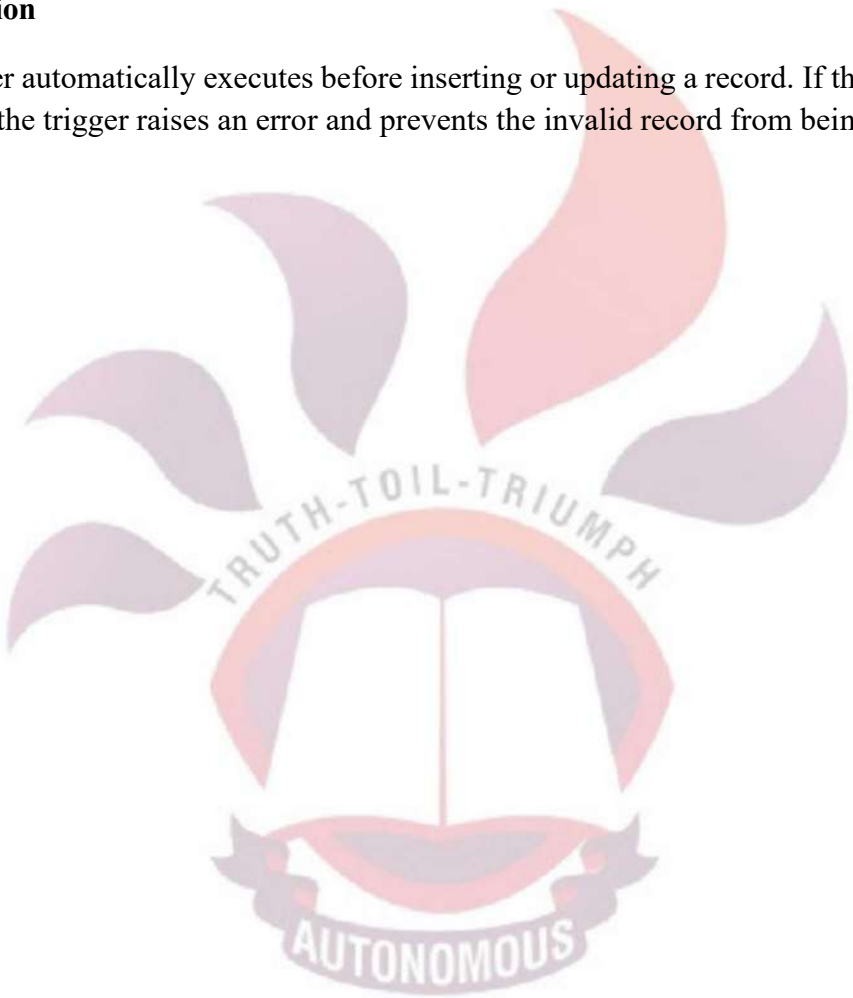
```
SELECT * FROM product_stock10;
```

Resultant Query

PRODUCT_ID	PRODUCT_NAME	QUANTITY
701	Laptop	10
702	Keyboard	15
703	Mouse	20
704	Monitor	12

Explanation

The trigger automatically executes before inserting or updating a record. If the quantity is negative, the trigger raises an error and prevents the invalid record from being inserted.



RESULT

Thus, the trigger was created successfully and automatically validated the quantity of products.

EX. NO: 11 **Cursor Implementation**

DATE: 8.9.2026

Aim

To implement a cursor in PL/SQL to retrieve and display records from a BOOK table.

Software Used

Oracle / MySQL / SQLite3

Description

A Cursor is used in PL/SQL to retrieve and process the rows returned by a SQL query one by one.

CURSOR OPERATIONS

- Declare
- Open
- Fetch
- Close

Syntax

DECLARE

```
CURSOR cursor_name IS
   SELECT column1, column2 FROM table_name;
BEGIN
OPEN cursor_name;


LOOP


   FETCH cursor_name INTO variables;
```

```
    EXIT WHEN cursor_name%NOTFOUND;


END LOOP;


CLOSE cursor_name;

END;
/
```

Create Table

```
CREATE TABLE book11 (

book_id INTEGER PRIMARY KEY,


book_name VARCHAR2(50),


price NUMBER

);
```

Table Created

Sample Data

```
INSERT INTO book11 VALUES (301, 'DBMS', 500);

INSERT INTO book11 VALUES (302, 'Java', 700);

INSERT INTO book11 VALUES (303, 'Python', 600);

INSERT INTO book11 VALUES (304, 'Oracle', 800);
```

COMMIT;

SQL Query

Query 1: Display all book records

```
SELECT * FROM book11;
```

Resultant Query

BOOK_ID	BOOK_NAME	PRICE
301	DBMS	500
302	Java	700
303	Python	600
304	Oracle	800

Query 2: Cursor Implementation

```
SET SERVEROUTPUT ON;
DECLARE
CURSOR c1 IS
   SELECT book_id, book_name, price
   FROM book11;

v_id book11.book_id%TYPE;
v_name book11.book_name%TYPE;
v_price book11.price%TYPE;
BEGIN
OPEN c1;

LOOP

   FETCH c1 INTO v_id, v_name, v_price;

   EXIT WHEN c1%NOTFOUND;

   DBMS_OUTPUT.PUT_LINE(
```

```
        'Book ID: ' || v_id ||
        ' Book: ' || v_name ||
        ' Price: ' || v_price
    );

END LOOP;

CLOSE c1;

END;
/
```

Resultant Query

Book ID: 301 Book: DBMS Price: 500

Book ID: 302 Book: Java Price: 700

Book ID: 303 Book: Python Price: 600

Book ID: 304 Book: Oracle Price: 800

Explanation

- A cursor named c1 is declared.
- The cursor retrieves book details.
- OPEN opens the cursor.
- FETCH retrieves records one by one.
- CLOSE closes the cursor.

RESULT

Thus, the cursor was implemented successfully and the book records were retrieved and displayed one by one.

EX. NO: 12 **PL/SQL Exception Handling**

DATE: 10.9.2026

Aim

To implement exception handling in PL/SQL using the STUDENT table.

Software Used

Oracle / MySQL / SQLite3

Description

Exception handling is used to handle runtime errors in a PL/SQL program.

TYPES OF EXCEPTIONS

- NO_DATA_FOUND
- TOO_MANY_ROWS
- ZERO_DIVIDE
- OTHERS

Syntax

```
BEGIN
statements;
EXCEPTION
WHEN exception_name THEN

   statements;

WHEN OTHERS THEN

   statements;
```

END;
/

Create Table

CREATE TABLE student12 (

student_id INTEGER PRIMARY KEY,

student_name VARCHAR2(50),

mark NUMBER

);

Table Created

Sample Data

INSERT INTO student12 VALUES (301, 'Arun', 85);

INSERT INTO student12 VALUES (302, 'Bala', 78);

INSERT INTO student12 VALUES (303, 'Kavitha', 92);

INSERT INTO student12 VALUES (304, 'Divya', 88);

COMMIT;

SQL Query

Query 1: Display all student records

SELECT * FROM student12;

Resultant Query

STUDENT_ID	STUDENT_NAME	MARK
301	Arun	85
302	Bala	78
303	Kavitha	92
304	Divya	88

Query 2: Exception Handling

```
SET SERVEROUTPUT ON;

DECLARE

v_mark student12.mark%TYPE;

BEGIN

SELECT mark

INTO v_mark

FROM student12

WHERE student_id = 999;


DBMS_OUTPUT.PUT_LINE('Mark = ' || v_mark);

EXCEPTION

WHEN NO_DATA_FOUND THEN

   DBMS_OUTPUT.PUT_LINE('Student not found');


WHEN TOO_MANY_ROWS THEN

   DBMS_OUTPUT.PUT_LINE('More than one student found');


WHEN OTHERS THEN

   DBMS_OUTPUT.PUT_LINE('Some other error occurred');
```

```
END;
/
```

Resultant Query

Student not found

Explanation

The program searches for student ID 999. Since the student does not exist, the NO_DATA_FOUND exception occurs and the error message is displayed.



RESULT

Thus, PL/SQL exception handling was implemented successfully using the Student table.

EX. NO: 13 **Database Normalization (1NF)**

DATE: 16.9.2026

Aim

To normalize a database table into First Normal Form (1NF) by ensuring that each attribute contains atomic values.

Software Used

Oracle / MySQL / SQLite3

Description

Normalization is the process of organizing data in a database to reduce data redundancy and avoid insertion, deletion, and update anomalies.

First Normal Form (1NF) requires that each column contains only atomic values and there are no repeating groups.

1NF RULES

- Each column must contain atomic values.
- Each row must be unique.
- There should be no repeating groups.
- Each field should contain only one value.

Syntax

CREATE TABLE table_name (

column1 datatype,

column2 datatype,

column3 datatype

);

Create Table

```
CREATE TABLE student_1nf (

student_id INTEGER,


student_name VARCHAR2(50),


subject_name VARCHAR2(50),


mark NUMBER

);
```

Table Created

Sample Data

```
INSERT INTO student_1nf VALUES (501, 'Arun', 'DBMS', 85);

INSERT INTO student_1nf VALUES (502, 'Bala', 'Java', 78);

INSERT INTO student_1nf VALUES (503, 'Kavitha', 'Python', 92);

INSERT INTO student_1nf VALUES (504, 'Divya', 'Oracle', 88);
```

COMMIT;

SQL Query

Query 1: Display all student records

```
SELECT * FROM student_1nf;
```

Resultant Query

STUDENT_ID	STUDENT_NAME	SUBJECT_NAME	MARK
501	Arun	DBMS	85
502	Bala	Java	78
503	Kavitha	Python	92
504	Divya	Oracle	88

Query 2: Display students with mark greater than 80

SELECT * FROM student_1nf WHERE mark > 80;

Resultant Query

STUDENT_ID	STUDENT_NAME	SUBJECT_NAME	MARK
501	Arun	DBMS	85
503	Kavitha	Python	92
504	Divya	Oracle	88

Explanation

- Each column contains a single value.
- There are no multiple values in a single field.
- Each student record is represented as a separate row.
- Therefore, the table satisfies the conditions of 1NF.

RESULT

Thus, the Student table was successfully normalized into First Normal Form (1NF).

Database Normalization (2NF)

Aim

To normalize a database into Second Normal Form (2NF) by removing partial dependency.

Software Used

Oracle / MySQL / SQLite3

Description

A table is in Second Normal Form (2NF) when:

- It is already in 1NF.
- There is no partial dependency.
- Every non-key attribute depends on the complete primary key.

2NF RULE

1NF + No Partial Dependency = 2NF

Create Table

CREATE TABLE purchase_2nf (

purchase_id INTEGER,

item_code INTEGER,

purchase_date DATE,

item_name VARCHAR2(50),

quantity NUMBER,

```
PRIMARY KEY (purchase_id, item_code)

);
```

Table Created

Sample Data

```
INSERT INTO purchase_2nf
VALUES (3001, 401, DATE '2026-09-05', 'Laptop', 2);

INSERT INTO purchase_2nf
VALUES (3001, 402, DATE '2026-09-05', 'Keyboard', 3);

INSERT INTO purchase_2nf
VALUES (3002, 403, DATE '2026-09-06', 'Monitor', 4);
```

COMMIT;

SQL Query

Query 1: Display all purchase records

```
SELECT * FROM purchase_2nf;
```

Resultant Query

PURCHASE_ID	ITEM_CODE	PURCHASE_DATE	ITEM_NAME	QUANTITY
3001	401	05-SEP-26	Laptop	2
3001	402	05-SEP-26	Keyboard	3
3002	403	06-SEP-26	Monitor	4

Query 2: Display items with quantity greater than 2

```
SELECT *
FROM purchase_2nf
WHERE quantity > 2;
```

Resultant Query

PURCHASE_ID	ITEM_CODE	PURCHASE_DATE	ITEM_NAME	QUANTITY
3001	402	05-SEP-26	Keyboard	3
3002	403	06-SEP-26	Monitor	4

Explanation

- The table is in 1NF.
- The combination of purchase_id and item_code forms the primary key.
- Attributes are separated based on their dependency.
- Partial dependency is removed during normalization.
- This avoids unnecessary data repetition.



RESULT

Thus, the Purchase table was successfully normalized into Second Normal Form (2NF).

Database Normalization (3NF)

Aim

To normalize a database into **Third Normal Form (3NF)** by removing transitive dependency.

Software Used

Oracle / MySQL / SQLite3

Description

A table is in **Third Normal Form (3NF)** when:

- It is already in 2NF.
- There is no transitive dependency.
- Non-key attributes depend only on the primary key.

3NF RULE

2NF + No Transitive Dependency = 3NF

Create Table

```
CREATE TABLE hospital_patient (
   patient_id INTEGER PRIMARY KEY,
   patient_name VARCHAR2(50),
   ward_id INTEGER
);
```

Table Created

```
CREATE TABLE hospital_ward (
   ward_id INTEGER PRIMARY KEY,
   ward_name VARCHAR2(50)
);
```

Table Created

Sample Data

```
INSERT INTO hospital_ward VALUES (10, 'General Ward');

INSERT INTO hospital_ward VALUES (20, 'ICU');

INSERT INTO hospital_ward VALUES (30, 'Children Ward');


INSERT INTO hospital_patient VALUES (3001, 'Arun', 10);

INSERT INTO hospital_patient VALUES (3002, 'Bala', 20);

INSERT INTO hospital_patient VALUES (3003, 'Cathy', 30);

INSERT INTO hospital_patient VALUES (3004, 'David', 10);


COMMIT;
```

SQL Query

Query 1: Display patient records

```
SELECT * FROM hospital_patient;
```

Resultant Query

PATIENT_ID	PATIENT_NAME	WARD_ID
3001	Arun	10
3002	Bala	20
3003	Cathy	30
3004	David	10

Query 2: Display ward records

```
SELECT * FROM hospital_ward;
```

Resultant Query

WARD_ID	WARD_NAME
10	General Ward
20	ICU
30	Children Ward

Query 3: Display patient and corresponding ward

```
SELECT P.patient_id,
    P.patient_name,
    W.ward_name
FROM hospital_patient P
JOIN hospital_ward W
ON P.ward_id = W.ward_id;
```

Resultant Query

PATIENT_ID	PATIENT_NAME	WARD_NAME
3001	Arun	General Ward
3002	Bala	ICU
3003	Cathy	Children Ward
3004	David	General Ward

Explanation

- Patient details and ward details are stored separately.
- patient_id is the primary key of the patient table.
- ward_id is used to connect the two tables.
- Ward name depends on ward_id, not directly on patient_id.
- Therefore, transitive dependency is removed.

RESULT

Thus, the Hospital Patient database was successfully normalized into **Third Normal Form (3NF)**.

Database Normalization (3NF)

Aim

To normalize a database into Third Normal Form (3NF) by removing transitive dependency.

Software Used

Oracle / MySQL / SQLite3

Description

A table is in Third Normal Form (3NF) when:

- It is already in 2NF.
- There is no transitive dependency.
- Non-key attributes depend only on the primary key.

3NF RULE

2NF + No Transitive Dependency = 3NF

Create Table

```
CREATE TABLE student_3nf (
student_id INTEGER PRIMARY KEY,

student_name VARCHAR2(50),

department_id INTEGER
);
```

Table Created

```
CREATE TABLE department_3nf (
department_id INTEGER PRIMARY KEY,
```

department_name VARCHAR2(50)

);

Table Created

Sample Data

INSERT INTO department_3nf VALUES (10, 'Computer Science');

INSERT INTO department_3nf VALUES (20, 'Commerce');

INSERT INTO department_3nf VALUES (30, 'Mathematics');

INSERT INTO student_3nf VALUES (4001, 'Arun', 10);

INSERT INTO student_3nf VALUES (4002, 'Bala', 20);

INSERT INTO student_3nf VALUES (4003, 'Kavitha', 30);

INSERT INTO student_3nf VALUES (4004, 'Divya', 10);

COMMIT;

SQL Query

Query 1: Display student records

SELECT * FROM student_3nf;

Resultant Query

STUDENT_ID	STUDENT_NAME	DEPARTMENT_ID
4001	Arun	10
4002	Bala	20
4003	Kavitha	30
4004	Divya	10

Query 2: Display department records

SELECT * FROM department_3nf;

Resultant Query

DEPARTMENT_ID	DEPARTMENT_NAME
10	Computer Science
20	Commerce
30	Mathematics

Query 3: Display student and corresponding department

```
SELECT S.student_id,
S.student_name,
D.department_name
FROM student_3nf S
JOIN department_3nf D
ON S.department_id = D.department_id;
```

Resultant Query

STUDENT_ID	STUDENT_NAME	DEPARTMENT_NAME
4001	Arun	Computer Science
4002	Bala	Commerce
4003	Kavitha	Mathematics
4004	Divya	Computer Science

Explanation

- Student details and department details are stored separately.
- student_id is the primary key of the student table.
- department_id is used to connect the two tables.
- Department name depends on department_id, not directly on student_id.
- Therefore, transitive dependency is removed.

RESULT

Thus, the Student database was successfully normalized into Third Normal Form (3NF).

Database Normalization (BCNF)

Aim

To normalize a database into Boyce-Codd Normal Form (BCNF) by ensuring that every determinant is a candidate key.

Software Used

Oracle / MySQL / SQLite3

Description

BCNF (Boyce-Codd Normal Form) is a stronger version of 3NF.

A relation is in BCNF if every determinant is a candidate key.

BCNF RULE

Every determinant must be a candidate key.

Create Table

CREATE TABLE hotel (

hotel_id INTEGER PRIMARY KEY,

hotel_name VARCHAR2(50),

city VARCHAR2(50),

room_type VARCHAR2(50)

);

Table Created

Sample Data

```
INSERT INTO hotel
VALUES (5001, 'Green Park', 'Chennai', 'Deluxe');

INSERT INTO hotel
VALUES (5002, 'Royal Inn', 'Madurai', 'Suite');

INSERT INTO hotel
VALUES (5003, 'Sun Palace', 'Coimbatore', 'Standard');

INSERT INTO hotel
VALUES (5004, 'Ocean View', 'Chennai', 'Suite');
```

COMMIT;

SQL Query

Query 1: Display all hotel records

```
SELECT * FROM hotel;
```

Resultant Query

HOTEL_ID	HOTEL_NAME	CITY	ROOM_TYPE
5001	Green Park	Chennai	Deluxe
5002	Royal Inn	Madurai	Suite
5003	Sun Palace	Coimbatore	Standard
5004	Ocean View	Chennai	Suite

Query 2: Display hotels located in Chennai

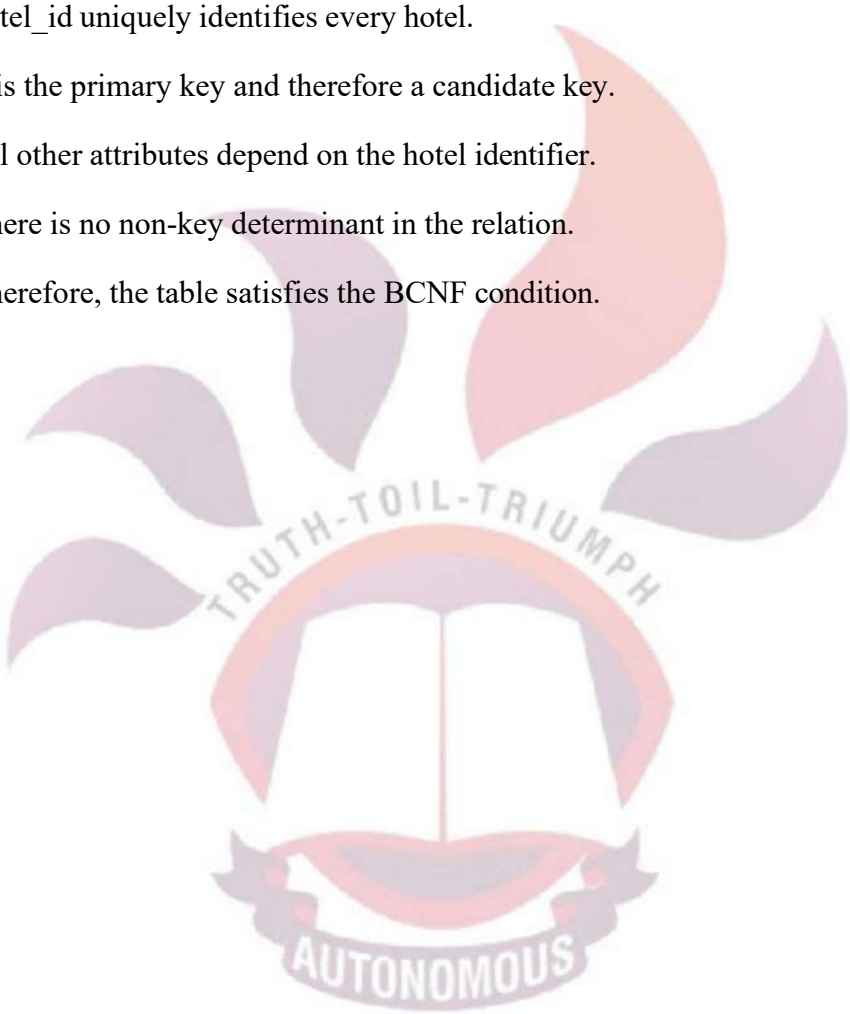
```
SELECT *
FROM hotel
WHERE city = 'Chennai';
```

Resultant Query

HOTEL_ID	HOTEL_NAME	CITY	ROOM_TYPE
5001	Green Park	Chennai	Deluxe
5004	Ocean View	Chennai	Suite

Explanation

- hotel_id uniquely identifies every hotel.
- It is the primary key and therefore a candidate key.
- All other attributes depend on the hotel identifier.
- There is no non-key determinant in the relation.
- Therefore, the table satisfies the BCNF condition.



RESULT

Thus, the Hotel table was successfully normalized into Boyce-Codd Normal

MINI PROJECT

EX. NO: 14 **RENEWABLE ENERGY PRODUCTION AND**

DATE: 4.10.2026 **CONSUMPTION**

Aim

To develop a Renewable Energy Production and Consumption system using SQL and PL/SQL to store, manage, process, and analyze renewable energy generation and consumption records.

Software Used

- GitHub Codespaces
- SQLite
- Oracle Database
- SQL
- PL/SQL

Description

The Renewable Energy Production and Consumption system is developed to monitor renewable energy generation and consumption. The system stores the energy date, energy source, generated energy, consumed energy, and location.

SQL queries are used to retrieve and analyze energy information, while PL/SQL procedures and functions are used to calculate energy surplus or deficit, process daily energy data, generate source-wise and location-wise reports, and handle exceptions.

Dataset Source

- Dataset: <write the Kaggle / UCI dataset name here> (25 records)
- Author and URL: <write the author name and URL here>
- Added fields: <write any fields you added, or write None>

Database Table

ENERGY_DATA

Column Name	Data Type	Constraint
ENERGY_ID	NUMBER(5)	PRIMARY KEY
ENERGY_DATE	DATE	NOT NULL
ENERGY_SOURCE	VARCHAR2(30)	NOT NULL
GENERATED_ENERGY	NUMBER(10,2)	NOT NULL, >= 0
CONSUMED_ENERGY	NUMBER(10,2)	NOT NULL, >= 0
LOCATION	VARCHAR2(50)	NOT NULL

TABLE CREATION

Oracle

```
CREATE TABLE energy_data (
   energy_id         NUMBER(5)      PRIMARY KEY,
   energy_date       DATE           NOT NULL,
   energy_source     VARCHAR2(30)   NOT NULL,
   generated_energy  NUMBER(10,2)   NOT NULL,
   consumed_energy   NUMBER(10,2)   NOT NULL,
   location          VARCHAR2(50)   NOT NULL,
   CONSTRAINT chk_gen   CHECK (generated_energy >= 0),
   CONSTRAINT chk_con    CHECK (consumed_energy   >= 0)
);
```

SQLite

```
CREATE TABLE energy_data (
   energy_id         INTEGER PRIMARY KEY,
   energy_date       TEXT NOT NULL,
   energy_source     TEXT NOT NULL,
   generated_energy  REAL NOT NULL,
   consumed_energy   REAL NOT NULL,
   location          TEXT NOT NULL,
   CONSTRAINT chk_gen   CHECK (generated_energy >= 0),
   CONSTRAINT chk_con    CHECK (consumed_energy   >= 0)
);
```

Table Created

The ENERGY_DATA table was created successfully with primary key, CHECK, and NOT NULL constraints.

DATASET INSERTION

```
INSERT INTO energy_data VALUES (1, DATE '2026-01-01', 'Solar', 120.50, 95.20,
'Chennai');
INSERT INTO energy_data VALUES (2, DATE '2026-01-02', 'Wind', 150.00, 110.50,
'Coimbatore');
INSERT INTO energy_data VALUES (3, DATE '2026-01-03', 'Hydro', 180.50, 140.00,
'Salem');
INSERT INTO energy_data VALUES (4, DATE '2026-01-04', 'Solar', 135.00, 100.50,
'Madurai');
```

```
INSERT INTO energy_data VALUES (5, DATE '2026-01-05', 'Wind', 160.50, 125.00,
'Trichy');
INSERT INTO energy_data VALUES (6, DATE '2026-01-06', 'Hydro', 175.00, 130.50,
'Erode');
INSERT INTO energy_data VALUES (7, DATE '2026-01-07', 'Solar', 145.50, 115.00,
'Chennai');
INSERT INTO energy_data VALUES (8, DATE '2026-01-08', 'Wind', 170.00, 135.50,
'Coimbatore');
INSERT INTO energy_data VALUES (9, DATE '2026-01-09', 'Hydro', 190.50, 150.00,
'Salem');
INSERT INTO energy_data VALUES (10, DATE '2026-01-10', 'Solar', 155.00, 120.50,
'Madurai');
INSERT INTO energy_data VALUES (11, DATE '2026-01-11', 'Wind', 165.50, 130.00,
'Trichy');
INSERT INTO energy_data VALUES (12, DATE '2026-01-12', 'Hydro', 200.00, 160.50,
'Erode');
INSERT INTO energy_data VALUES (13, DATE '2026-01-13', 'Solar', 175.50, 140.00,
'Chennai');
INSERT INTO energy_data VALUES (14, DATE '2026-01-14', 'Wind', 185.00, 145.50,
'Coimbatore');
INSERT INTO energy_data VALUES (15, DATE '2026-01-15', 'Hydro', 210.50, 170.00,
'Salem');
INSERT INTO energy_data VALUES (16, DATE '2026-01-16', 'Solar', 160.00, 125.50,
'Madurai');
INSERT INTO energy_data VALUES (17, DATE '2026-01-17', 'Wind', 195.50, 155.00,
'Trichy');
INSERT INTO energy_data VALUES (18, DATE '2026-01-18', 'Hydro', 220.00, 180.50,
'Erode');
INSERT INTO energy_data VALUES (19, DATE '2026-01-19', 'Solar', 185.50, 150.00,
'Chennai');
INSERT INTO energy_data VALUES (20, DATE '2026-01-20', 'Wind', 205.00, 165.50,
'Coimbatore');
INSERT INTO energy_data VALUES (21, DATE '2026-01-21', 'Hydro', 230.50, 190.00,
'Salem');
INSERT INTO energy_data VALUES (22, DATE '2026-01-22', 'Solar', 195.00, 155.50,
'Madurai');
INSERT INTO energy_data VALUES (23, DATE '2026-01-23', 'Wind', 215.50, 175.00,
'Trichy');
INSERT INTO energy_data VALUES (24, DATE '2026-01-24', 'Hydro', 240.00, 200.50,
'Erode');
INSERT INTO energy_data VALUES (25, DATE '2026-01-25', 'Solar', 210.50, 170.00,
'Chennai');
```

Records Inserted

A total of 25 renewable energy records were inserted into the ENERGY_DATA table.

Output

energy_id	energy_date	energy_source	generated_energy	consumed_energy	location
1	2026-01-01	Solar	120.5	95.2	Chennai
2	2026-01-02	Wind	150.0	110.5	Coimbatore
3	2026-01-03	Hydro	180.5	140.0	Salem
4	2026-01-04	Solar	135.0	100.5	Madurai
5	2026-01-05	Wind	160.5	125.0	Trichy
6	2026-01-06	Hydro	175.0	130.5	Erode
7	2026-01-07	Solar	145.5	115.0	Chennai
8	2026-01-08	Wind	170.0	135.5	Coimbatore
9	2026-01-09	Hydro	190.5	150.0	Salem
10	2026-01-10	Solar	155.0	120.5	Madurai
11	2026-01-11	Wind	165.5	130.0	Trichy
12	2026-01-12	Hydro	200.0	160.5	Erode
13	2026-01-13	Solar	175.5	140.0	Chennai
14	2026-01-14	Wind	185.0	145.5	Coimbatore
15	2026-01-15	Hydro	210.5	170.0	Salem
16	2026-01-16	Solar	160.0	125.5	Madurai
17	2026-01-17	Wind	195.5	155.0	Trichy
18	2026-01-18	Hydro	220.0	180.5	Erode
19	2026-01-19	Solar	185.5	150.0	Chennai
20	2026-01-20	Wind	205.0	165.5	Coimbatore
21	2026-01-21	Hydro	230.5	190.0	Salem
22	2026-01-22	Solar	195.0	155.5	Madurai
23	2026-01-23	Wind	215.5	175.0	Trichy
24	2026-01-24	Hydro	240.0	200.5	Erode
25	2026-01-25	Solar	210.5	170.0	Chennai

SQL Query 1: Display Energy Records

```
SELECT energy_id, energy_date, energy_source,
    generated_energy, consumed_energy, location
FROM  energy_data
ORDER  BY energy_id;
```

AUTONOMOUS

Output

energy_id	energy_date	energy_source	generated_energy	consumed_energy	location
1	2026-01-01	Solar	120.5	95.2	Chennai
2	2026-01-02	Wind	150.0	110.5	Coimbatore
3	2026-01-03	Hydro	180.5	140.0	Salem
4	2026-01-04	Solar	135.0	100.5	Madurai
5	2026-01-05	Wind	160.5	125.0	Trichy
6	2026-01-06	Hydro	175.0	130.5	Erode
7	2026-01-07	Solar	145.5	115.0	Chennai
8	2026-01-08	Wind	170.0	135.5	Coimbatore
9	2026-01-09	Hydro	190.5	150.0	Salem
10	2026-01-10	Solar	155.0	120.5	Madurai
11	2026-01-11	Wind	165.5	130.0	Trichy
12	2026-01-12	Hydro	200.0	160.5	Erode
13	2026-01-13	Solar	175.5	140.0	Chennai
14	2026-01-14	Wind	185.0	145.5	Coimbatore
15	2026-01-15	Hydro	210.5	170.0	Salem
16	2026-01-16	Solar	160.0	125.5	Madurai
17	2026-01-17	Wind	195.5	155.0	Trichy
18	2026-01-18	Hydro	220.0	180.5	Erode
19	2026-01-19	Solar	185.5	150.0	Chennai
20	2026-01-20	Wind	205.0	165.5	Coimbatore
21	2026-01-21	Hydro	230.5	190.0	Salem
22	2026-01-22	Solar	195.0	155.5	Madurai
23	2026-01-23	Wind	215.5	175.0	Trichy
24	2026-01-24	Hydro	240.0	200.5	Erode
25	2026-01-25	Solar	210.5	170.0	Chennai

SQL Query 2: Total Energy Generated by Each Source

```
SELECT energy_source,
    ROUND(SUM(generated_energy), 2) AS total_generated_energy
FROM  energy_data
GROUP  BY energy_source
ORDER  BY energy_source;
```

Output

energy_source	total_generated_e...
Hydro	1647.0
Solar	1482.5
Wind	1447.0

SQL Query 3: Daily Energy Consumption

```
SELECT energy_date,
    ROUND(SUM(consumed_energy), 2) AS daily_consumption
FROM  energy_data
GROUP  BY energy_date
ORDER  BY energy_date;
```

Output

energy_date	daily_consumption
2026-01-01	95.2
2026-01-02	110.5
2026-01-03	140.0
2026-01-04	100.5
2026-01-05	125.0
2026-01-06	130.5
2026-01-07	115.0
2026-01-08	135.5
2026-01-09	150.0
2026-01-10	120.5
2026-01-11	130.0
2026-01-12	160.5
2026-01-13	140.0
2026-01-14	145.5
2026-01-15	170.0
2026-01-16	125.5
2026-01-17	155.0
2026-01-18	180.5
2026-01-19	150.0
2026-01-20	165.5
2026-01-21	190.0
2026-01-22	155.5
2026-01-23	175.0
2026-01-24	200.5
2026-01-25	170.0

SQL Query 4: Energy Surplus or Deficit

```
SELECT energy_id, generated_energy, consumed_energy,
    ROUND(generated_energy - consumed_energy, 2) AS surplus_deficit
FROM  energy_data
ORDER  BY energy_id;
```

Output

energy_id	generated_energy	consumed_energy	surplus_deficit
1	120.5	95.2	25.299999999999997
2	150.0	110.5	39.5
3	180.5	140.0	40.5
4	135.0	100.5	34.5
5	160.5	125.0	35.5
6	175.0	130.5	44.5
7	145.5	115.0	30.5
8	170.0	135.5	34.5
9	190.5	150.0	40.5
10	155.0	120.5	34.5
11	165.5	130.0	35.5
12	200.0	160.5	39.5
13	175.5	140.0	35.5
14	185.0	145.5	39.5
15	210.5	170.0	40.5
16	160.0	125.5	34.5
17	195.5	155.0	40.5
18	220.0	180.5	39.5
19	185.5	150.0	35.5
20	205.0	165.5	39.5
21	230.5	190.0	40.5
22	195.0	155.5	39.5
23	215.5	175.0	40.5
24	240.0	200.5	39.5
25	210.5	170.0	40.5

SQL Query 5: Total Energy Generation for Each Location

```
SELECT location,
    ROUND(SUM(generated_energy), 2) AS total_generated_energy
FROM  energy_data
GROUP  BY location
ORDER  BY location;
```

Output

location	total_generated_e...
Chennai	837.5
Coimbatore	710.0
Erode	835.0
Madurai	645.0
Salem	812.0
Trichy	737.0

PL/SQL Function 1: Calculate Energy Surplus or Deficit

```
CREATE OR REPLACE FUNCTION calculate_energy_balance (p_energy_id IN
NUMBER)
RETURN NUMBER
IS
   v_generated energy_data.generated_energy%TYPE;
   v_consumed  energy_data.consumed_energy%TYPE;
BEGIN
   IF p_energy_id IS NULL THEN
      RAISE_APPLICATION_ERROR(-20002, 'Energy ID cannot be NULL.');
   END IF;
   SELECT generated_energy, consumed_energy
   INTO  v_generated, v_consumed
   FROM   energy_data
   WHERE  energy_id = p_energy_id;
   RETURN v_generated - v_consumed;
EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(-20001, 'Energy ID not found');
END calculate_energy_balance;
/
```



Execution

```
SELECT energy_id, generated_energy, consumed_energy,
      calculate_energy_balance(energy_id) AS surplus_deficit
FROM   energy_data
WHERE  energy_id IN (1, 2, 3);
```

Output

Results | Explain | Describe | Saved SQL | History

ENERGY_ID	GENERATED_ENERGY	CONSUMED_ENERGY	SURPLUS_DEFICIT
1	120.5	95.2	25.3
2	150	110.5	39.5
3	180.5	140	40.5

3 rows returned in 0.03 seconds CSV Export

PL/SQL Procedure 2: Daily Energy Generation and Consumption

```
CREATE OR REPLACE PROCEDURE daily_energy_report
IS
   v_count NUMBER := 0;
BEGIN
   DBMS_OUTPUT.PUT_LINE('DAILY  ENERGY  REPORT');
   DBMS_OUTPUT.PUT_LINE(RPAD('DATE', 14) || RPAD('GENERATED', 12) ||
'CONSUMED');
   FOR rIN (SELECT energy_date,
                   SUM(generated_energy) AS total_generated,
                   SUM(consumed_energy)  AS total_consumed
            FROM  energy_data
            GROUP  BY energy_date
            ORDER  BY energy_date)
   LOOP
      v_count := v_count + 1;
      DBMS_OUTPUT.PUT_LINE(RPAD(TO_CHAR(r.energy_date, 'DD-MON-YYYY'),
14) ||
                           RPAD(r.total_generated, 12) || r.total_consumed);
   END LOOP;
   IF v_count = 0 THEN
      RAISE_APPLICATION_ERROR(-20003, 'No energy data available.');
   END IF;
END daily_energy_report;
/
```

Results Explain Describe Saved SQL History

```
Procedure created.
```

0.10 seconds

Execution

```
SET SERVEROUTPUT ON;
BEGIN
  daily_energy_report;
END;
/
```

Output

Results Explain Describe Saved SQL History

```
DAILY ENERGY REPORT
DATE          GENERATED   CONSUMED
01-JAN-2026   120.5       95.2
02-JAN-2026   150         110.5
03-JAN-2026   180.5       140
04-JAN-2026   135         100.5
05-JAN-2026   160.5       125
06-JAN-2026   175         130.5
07-JAN-2026   145.5       115
08-JAN-2026   170         135.5
09-JAN-2026   190.5       150
10-JAN-2026   155         120.5
11-JAN-2026   165.5       130
12-JAN-2026   200         160.5
13-JAN-2026   175.5       140
14-JAN-2026   185         145.5
15-JAN-2026   210.5       170
16-JAN-2026   160         125.5
17-JAN-2026   195.5       155
18-JAN-2026   220         180.5
19-JAN-2026   185.5       150
20-JAN-2026   205         165.5
21-JAN-2026   230.5       190
22-JAN-2026   195         155.5
23-JAN-2026   215.5       175
24-JAN-2026   240         200.5
25-JAN-2026   210.5       170

Statement processed.


0.06 seconds
```

PL/SQL Procedure 3: Source-wise and Location-wise Energy Report

```
CREATE OR REPLACE PROCEDURE energy_source_location_report
IS
   v_count NUMBER := 0;
BEGIN
   DBMS_OUTPUT.PUT_LINE('SOURCE-WISE AND LOCATION-WISE ENERGY
REPORT');
   DBMS_OUTPUT.PUT_LINE(RPAD('SOURCE', 10) || RPAD('LOCATION', 14) ||
'TOTAL GENERATED');
   FOR r IN (SELECT energy_source, location,
                SUM(generated_energy) AS total_generated
           FROM   energy_data
           GROUP  BY energy_source, location
           ORDER  BY energy_source, location)
   LOOP
      v_count := v_count + 1;
      DBMS_OUTPUT.PUT_LINE(RPAD(r.energy_source, 10) || RPAD(r.location, 14) ||
                   r.total_generated);
   END LOOP;
   IF v_count = 0 THEN
      RAISE_APPLICATION_ERROR(-20004, 'No energy data available.');
   END IF;
END energy_source_location_report;
/
```




```
Procedure created.

0.10 seconds
```

Execution

```
BEGIN
  energy_source_location_report;
END;
/
```

Output

Results Explain Describe Saved SQL History

```
SOURCE-WISE AND LOCATION-WISE ENERGY REPORT
SOURCE     LOCATION       TOTAL GENERATED
Hydro      Erode          835
Hydro      Salem          812
Solar      Chennai        837.5
Solar      Madurai        645
Wind       Coimbatore     710
Wind       Trichy         737

Statement processed.
```

Exception Handling

Test Case 1 – Invalid Energy ID (missing record)

```
BEGIN
  DBMS_OUTPUT.PUT_LINE('Result: ' || calculate_energy_balance(9999));
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

Output

Results Explain Describe Saved SQL History

```
Error caught: ORA-20001: Energy ID not found

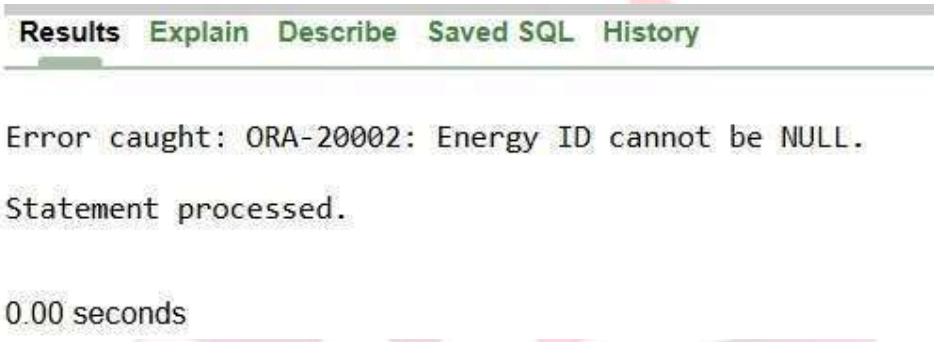
Statement processed.

0.03 seconds
```

Test Case 2 – NULL Energy ID

```
BEGIN
  DBMS_OUTPUT.PUT_LINE('Result: ' || calculate_energy_balance(NULL));
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

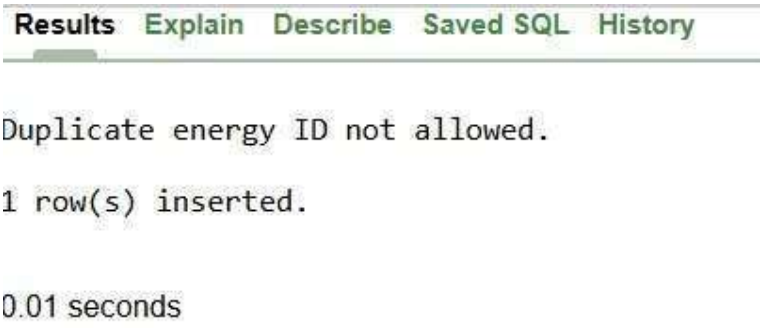
Output



Test Case 3 – Duplicate Energy ID

```
BEGIN
  INSERT INTO energy_data
  VALUES (1, DATE '2026-02-01', 'Solar', 100, 80, 'Chennai');
EXCEPTION
  WHEN DUP_VAL_ON_INDEX THEN
    DBMS_OUTPUT.PUT_LINE('Duplicate energy ID not allowed.');
END;
/
```

Output



Test Case 4 – Negative Generated Energy

```
BEGIN
  INSERT INTO energy_data
  VALUES (99, DATE '2026-02-01', 'Solar', -50, 80, 'Chennai');
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Error caught: ' || SQLERRM);
END;
/
```

Output

Results Explain Describe Saved SQL History

```
Error caught: ORA-02290: check constraint (SYSTEM.CHK_GEN) violated

1 row(s) inserted.


0.00 seconds
```

Test Case 5 – Unknown Energy Source (no records)

```
SELECT *
FROM   energy_data
WHERE  energy_source = 'Unknown';
```

Output

Results Explain Describe Saved SQL History

no data found

Dataset Insights

- The dataset contains 25 renewable energy records from three sources: Solar, Wind, and Hydro.
- Hydro produced the most energy (1647.0) and Wind the least (1447.0).
- Chennai has the highest total generation (837.5) and Madurai the lowest (645.0).
- Total generated energy is 4576.5 and total consumed energy is 3636.2, a net surplus of 940.3.

- No record shows an energy deficit; the largest surplus is 44.5 (energy ID 6) and the smallest is 25.3 (energy ID 1).
- The balance function and reports make it easy to compare generation and consumption by date, source, and location.

Result

Thus, the Renewable Energy Production and Consumption system was successfully implemented using SQL and PL/SQL. The ENERGY_DATA table was created, 25 energy records were inserted, SQL queries were executed, PL/SQL functions and procedures were implemented, and exception handling was tested successfully.

KHAN ACEDEMY

BOOK LIST DATABASE

What are your favorite books? You can make a database table to store them in! In this first step, create a table to store your list of books. It should have columns for id, name, and rating. Now, add three of your favorite books into the table.

Program:

```
CREATE TABLE books(id INTEGER PRIMARY KEY,  name TEXT, rating INTEGER);
```

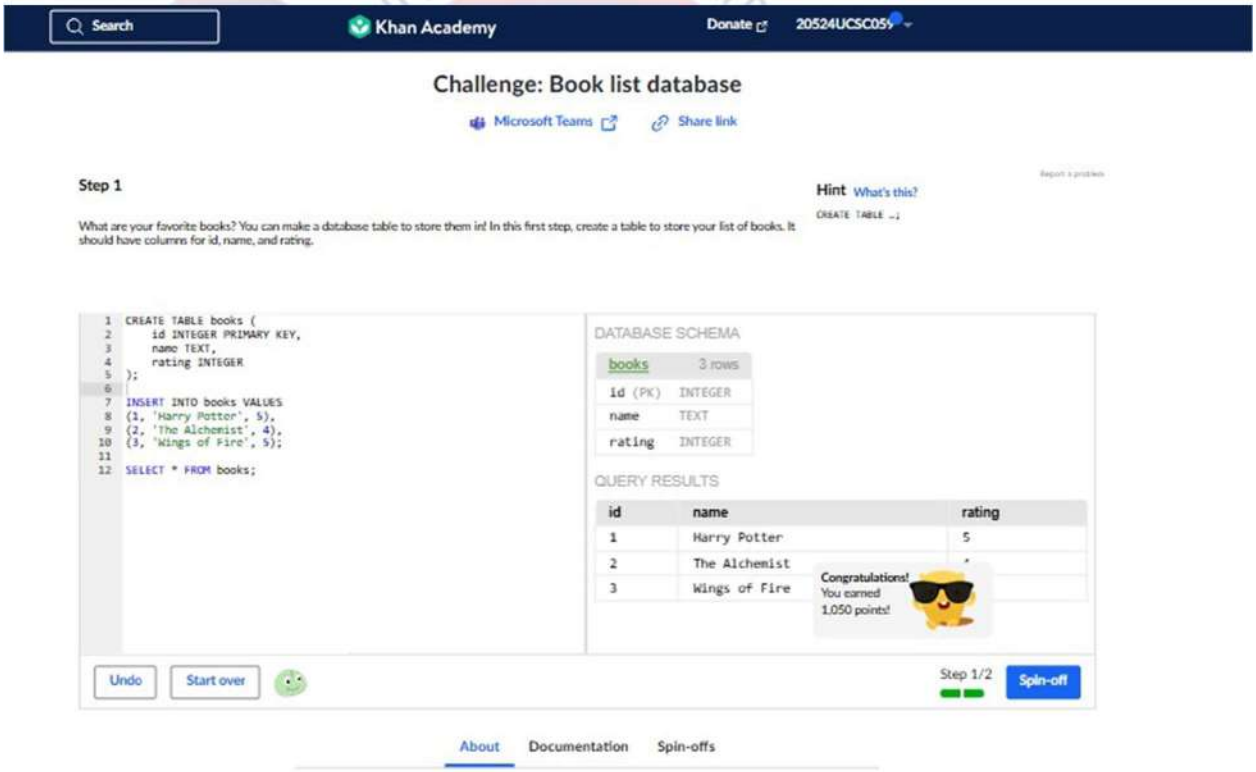

```
CREATE TABLE books(id INTEGER PRIMARY KEY,  name TEXT, rating INTEGER);
```


```
INSERT INTO books VALUES(1,'English',9/10);

INSERT INTO books VALUES(2,'Maths',10/10);

INSERT INTO books VALUES(3,'Tamil',9/10);

SELECT*FROM books;
```



BOX OFFICE HITS DATABASE

This database contains an incomplete list of box office hits and their release year. In this challenge, you're going to get the results back out of the database in different ways! In this first step, just select all the movies. Now, add a second query after the first, that retrieves only the movies that were released in the year 2000 or later, not before. Sort the results so that the earlier movies are listed first. You should have 2 SELECT statements after this step.

Program:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY, name TEXT, release_year INTEGER);

INSERT INTO movies VALUES (1, "Avatar", 2009);

INSERT INTO movies VALUES (2, "Titanic", 1997);

INSERT INTO movies VALUES (3, "Star Wars: Episode IV - A New Hope", 1977);

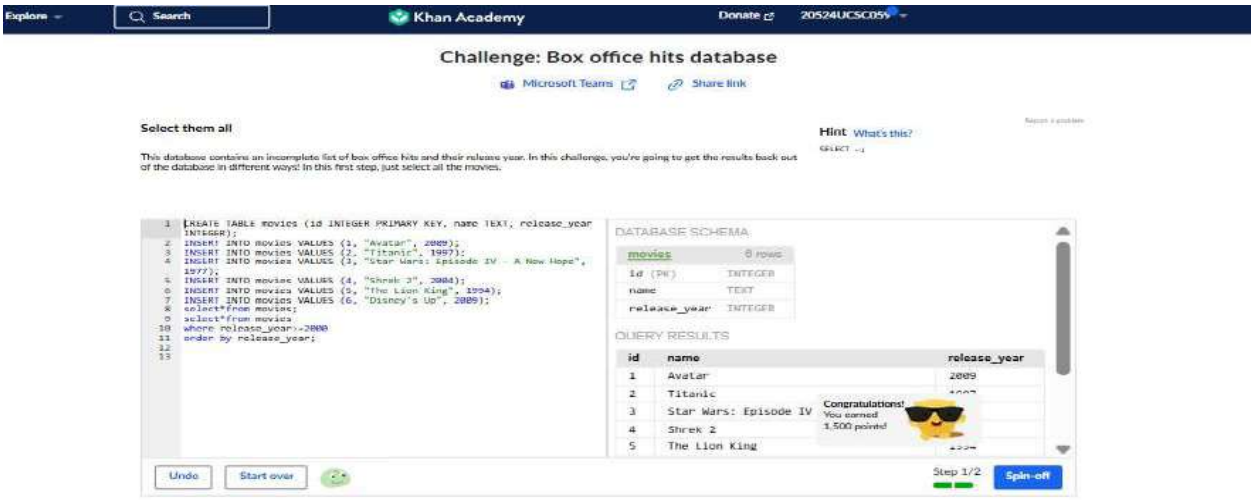
INSERT INTO movies VALUES (4, "Shrek 2", 2004);

INSERT INTO movies VALUES (5, "The Lion King", 1994);

INSERT INTO movies VALUES (6, "Disney's Up", 2009);

SELECT*FROM movies;

SELECT*FROM movies where release_year > 2000 order by release_year;
```



TRUTH-TOIL-TRIUMPH
AUTONOMOUS

TODO LISTDATABASE STATS

Here's a table containing a TODO list with the number of minutes it will take to complete each item. Insert another item to your todo list with the estimated minutes it will take. Select the SUM of minutes it will take to do all of the items on your TODO list. You should only have one SELECT statement.

Program:

```
CREATE TABLE todo_list (id INTEGER PRIMARY KEY, item TEXT, minutes INTEGER);

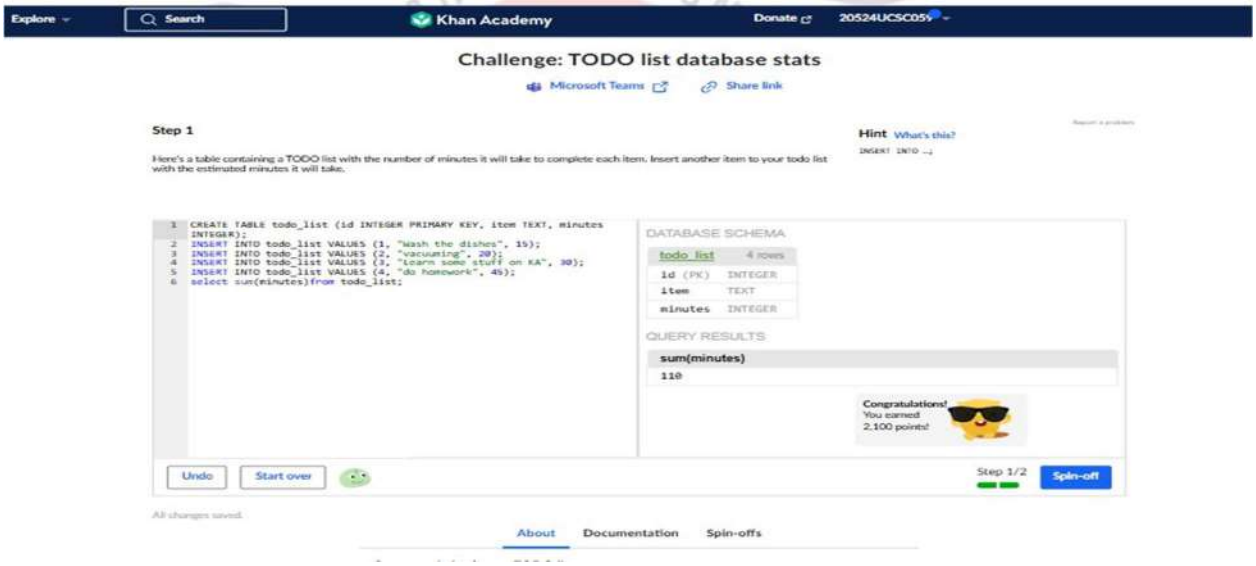
INSERT INTO todo_list VALUES (1, "Wash the dishes", 15);

INSERT INTO todo_list VALUES (2, "vacuuming", 20);

INSERT INTO todo_list VALUES (3, "Learn some stuff on KA", 30);

INSERT INTO todo_list VALUES (4, "Dry your clothes", 40);

SELECT SUM(minutes) FROM todo_list;
```



PROJECT : DESIGN A STORE DATABASE

Create your own store! Your store should sell one type of things, like clothing or bikes, whatever you want your store to specialize in. You should have a table for all the items in your store, and at least 5 columns for the kind of data you think you'd need to store. You should sell at least 15 items, and use select statements to order your items by price and show at least one statistic about the items.

Program:

```
CREATE TABLE sneakers (
    id INTEGER PRIMARY KEY,
    brand TEXT,
    model TEXT,
    size INTEGER,
    color TEXT,
    price REAL
);
INSERT INTO sneakers VALUES (1, "Nike", "Air Max", 9, "Black", 120);
INSERT INTO sneakers VALUES (2, "Adidas", "Ultraboost", 10, "White", 180);
INSERT INTO sneakers VALUES (3, "Puma", "RS-X", 8, "Blue", 110);
INSERT INTO sneakers VALUES (4, "New Balance", "574", 9, "Gray", 95);
INSERT INTO sneakers VALUES (5, "Converse", "Chuck Taylor", 10, "Red", 60);
INSERT INTO sneakers VALUES (6, "Vans", "Old Skool", 8, "Black", 70);
INSERT INTO sneakers VALUES (7, "Nike", "Jordan 1", 11, "Red", 200);
INSERT INTO sneakers VALUES (8, "Adidas", "NMD", 9, "Black", 140);
INSERT INTO sneakers VALUES (9, "Puma", "Suede Classic", 10, "Green", 75);
INSERT INTO sneakers VALUES (10, "New Balance", "990", 11, "Gray", 185);
INSERT INTO sneakers VALUES (11, "Converse", "Run Star Hike", 8, "White", 115);
INSERT INTO sneakers VALUES (12, "Vans", "Sk8-Hi", 9, "Blue", 80);
```

```
INSERT INTO sneakers VALUES (13, "Nike", "Air Force 1", 10, "White", 130);
INSERT INTO sneakers VALUES (14, "Adidas", "Superstar", 8, "Black", 100);
INSERT INTO sneakers VALUES (15, "Puma", "Future Rider", 9, "Yellow", 90);
SELECT *
FROM sneakers
ORDER BY price DESC;
SELECT
   AVG(price) AS average_price,
   MAX(price) AS highest_price,
   MIN(price) AS lowest_price
FROM sneakers;
```

Output

DATABASE SCHEMA

sneakers 15 rows

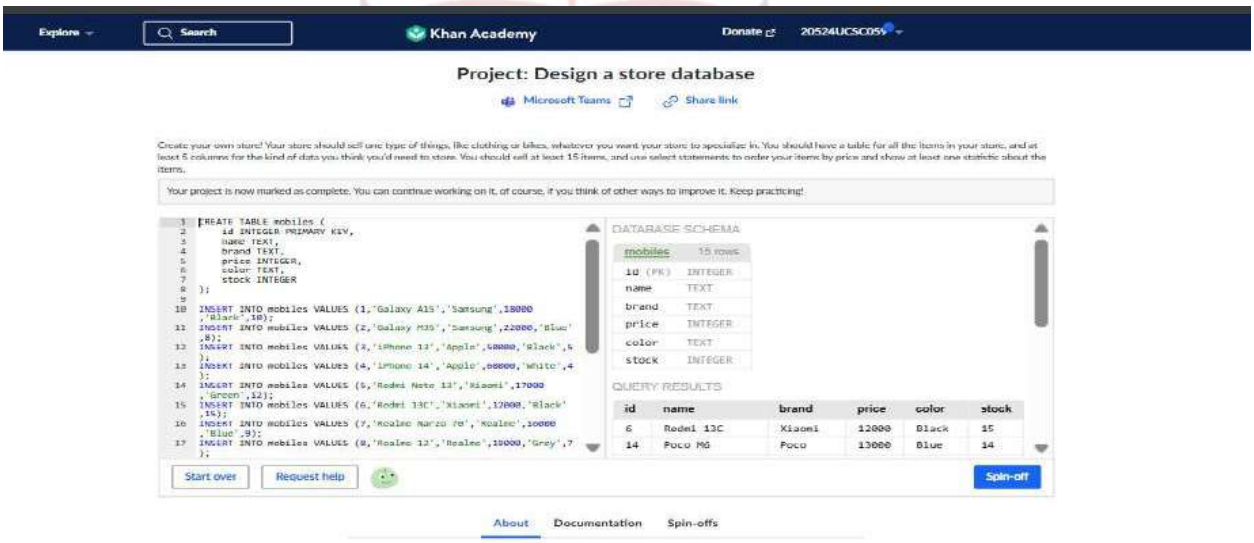
id (PK) INTEGER
brand TEXT
model TEXT
size INTEGER
color TEXT
price REAL

QUERY RESULTS

id	brand	model	size	color	price
7	Nike	Jordan 1	11	Red	200
10	New Balance	990	11	Gray	185

id	brand	model	size	color	price
2	Adidas	Ultraboost	10	White	180
8	Adidas	NMD	9	Black	140
13	Nike	Air Force 1	10	White	130
1	Nike	Air Max	9	Black	120
11	Converse	Run Star Hike	8	White	115
3	Puma	RS-X	8	Blue	110
14	Adidas	Superstar	8	Black	100
4	New Balance	574	9	Gray	95
15	Puma	Future Rider	9	Yellow	90
12	Vans	Sk8-Hi	9	Blue	80
9	Puma	Suede Classic	10	Green	75
6	Vans	Old Skool	8	Black	70
5	Converse	Chuck Taylor	10	Red	60

average_price	highest_price	lowest_price
116.66666666666667	200	60



KARAOKE SONG SELECTOR

Ever sung karaoke? It's a place where you sing songs with your friends, and it's a lot of fun. We've created a table with songs, and in this challenge, you'll use queries to decide what songs to sing. For the first step, select all the song titles. Maybe your friends only like singing either recent songs or truly epic songs. Add another SELECT that uses OR to show the titles of the songs that have an 'epic' mood or a release date after 1990. People get picky at the end of the night. Add another SELECT that uses AND to show the titles of songs that are 'epic', and released after 1990, and less than 4 minutes long.

Program:

```
CREATE TABLE songs (
    id INTEGER PRIMARY KEY,
    title TEXT,
    artist TEXT,
    mood TEXT,
    duration INTEGER,
    released INTEGER);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Bohemian Rhapsody", "Queen", "epic", 60, 1975);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Let it go", "Idina Menzel", "epic", 227, 2013);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will survive", "Gloria Gaynor", "epic", 198, 1978);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("Twist and Shout", "The Beatles", "happy", 152, 1963);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("La Bamba", "Ritchie Valens", "happy", 166, 1958);
INSERT INTO songs (title, artist, mood, duration, released)
    VALUES ("I will always love you", "Whitney Houston", "epic", 273, 1992);
INSERT INTO songs (title, artist, mood, duration, released)
```

VALUES ("Sweet Caroline", "Neil Diamond", "happy", 201, 1969);

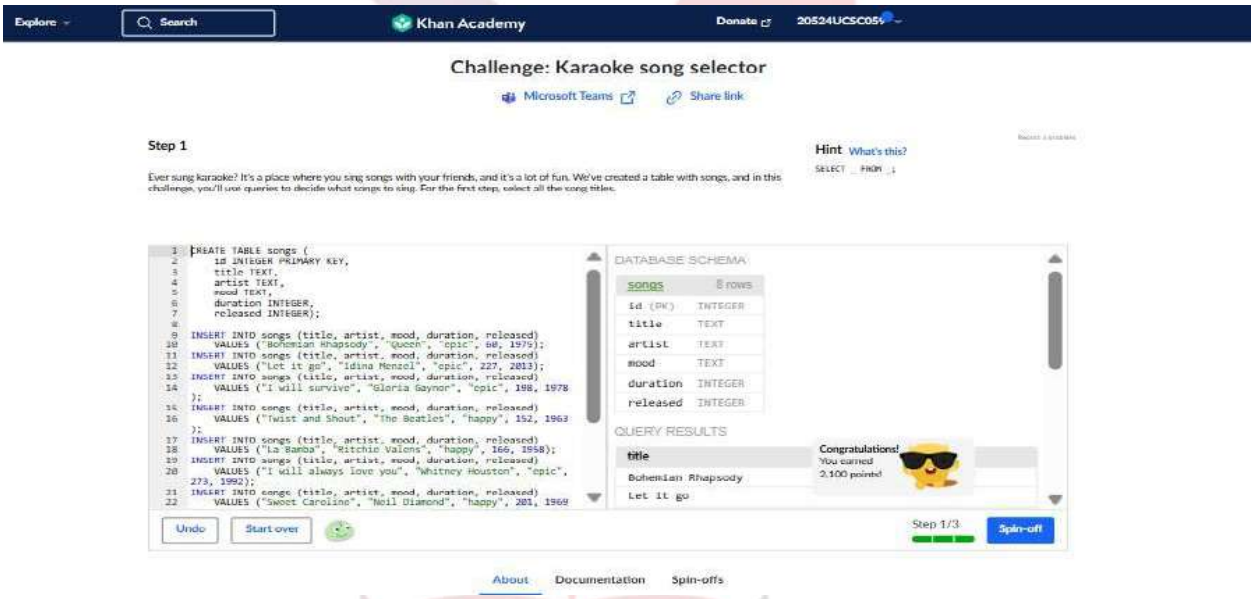
INSERT INTO songs (title, artist, mood, duration, released)

VALUES ("Call me maybe", "Carly Rae Jepsen", "happy", 193, 2011);

select title from songs;

select title from songs where mood = 'epic' or released > 1990;

select title from songs where mood = 'epic' and released > 1990 and duration < 240;



AUTONOMOUS

PLAYLIST MAKER

We've created a database of songs and artists, and you'll make playlists from them in this challenge. In this first step, select the `title` of all the songs by the `artist` named 'Queen'. Now you'll make a 'Pop' playlist. In preparation, select the `name` of all of the artists from the 'Pop' genre. To finish creating the 'Pop' playlist, add another query that will select the `title` of all the songs from the 'Pop' artists. It should use `IN` on a nested subquery that's based on your previous query.

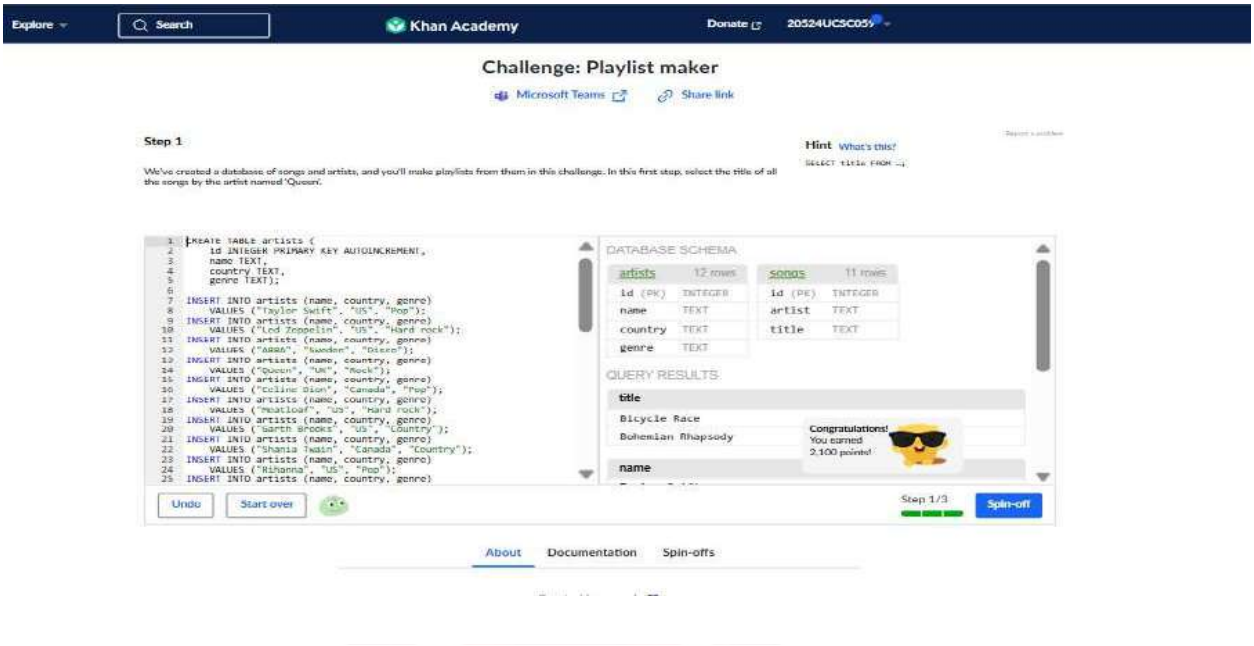
Program:

```
CREATE TABLE artists (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    country TEXT,
    genre TEXT);
INSERT INTO artists (name, country, genre)
    VALUES ("Taylor Swift", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Led Zeppelin", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("ABBA", "Sweden", "Disco");
INSERT INTO artists (name, country, genre)
    VALUES ("Queen", "UK", "Rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Celine Dion", "Canada", "Pop");
```

```
INSERT INTO artists (name, country, genre)
    VALUES ("Meatloaf", "US", "Hard rock");
INSERT INTO artists (name, country, genre)
    VALUES ("Garth Brooks", "US", "Country");
INSERT INTO artists (name, country, genre)
    VALUES ("Shania Twain", "Canada", "Country");
INSERT INTO artists (name, country, genre)
    VALUES ("Rihanna", "US", "Pop");
INSERT INTO artists (name, country, genre)
    VALUES ("Guns N' Roses", "US", "Hard rock");
 INSERT INTO artists (name, country, genre)
   VALUES ("Gloria Estefan", "US", "Pop");
 INSERT INTO artists (name, country, genre)
    VALUES ("Bob Marley", "Jamaica", "Reggae");
CREATE TABLE songs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    artist TEXT,
    title TEXT);
 INSERT INTO songs (artist, title)
    VALUES ("Taylor Swift", "Shake it off");
INSERT INTO songs (artist, title)
     VALUES ("Rihanna", "Stay");
```

```
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "My heart will go on");
INSERT INTO songs (artist, title)
    VALUES ("Celine Dion", "A new day has come");
INSERT INTO songs (artist, title)
    VALUES ("Shania Twain", "Party for two");
INSERT INTO songs (artist, title)
    VALUES ("Gloria Estefan", "Conga");
INSERT INTO songs (artist, title)
    VALUES ("Led Zeppelin", "Stairway to heaven");
INSERT INTO songs (artist, title)
    VALUES ("ABBA", "Mamma mia");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bicycle Race");
INSERT INTO songs (artist, title)
    VALUES ("Queen", "Bohemian Rhapsody");
INSERT INTO songs (artist, title)
    VALUES ("Guns N' Roses", "Don't cry");
select title from songs where artist = 'Queen';
select name from artists where genre = 'Pop';
SELECT title
FROM songs
```

```
WHERE artist IN (
    SELECT name
    FROM artists
    WHERE genre = 'Pop'
);
```



THE WORDIEST AUTHOR

We've created a database of a few popular authors and their books, with word counts for each book. In this first step, select all the authors who have written more than 1 million words, using `GROUP BY` and `HAVING`. Your results table should include the 'author' and their total word count as a 'total_words' column. Now select all the authors that write more than an average of 150,000 words per book. Your results table should include the 'author' and average words as an 'avg_words' column.

Program:

```
CREATE TABLE books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    author TEXT,
    title TEXT,
    words INTEGER);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Philosopher's Stone", 79944);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Chamber of Secrets", 85141);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Prisoner of Azkaban", 107253);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Goblet of Fire", 190637);
INSERT INTO books (author, title, words)
```

```
    VALUES ("J.K. Rowling", "Harry Potter and the Order of the Phoenix", 257045);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Half-Blood Prince", 168923);
INSERT INTO books (author, title, words)
    VALUES ("J.K. Rowling", "Harry Potter and the Deathly Hallows", 197651);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Twilight", 118501);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "New Moon", 132807);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Eclipse", 147930);
INSERT INTO books (author, title, words)
    VALUES ("Stephenie Meyer", "Breaking Dawn", 192196);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "The Hobbit", 95022);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Fellowship of the Ring", 177227);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Two Towers", 143436);
INSERT INTO books (author, title, words)
    VALUES ("J.R.R. Tolkien", "Return of the King", 134462);
SELECT author,
      SUM(words) AS total_words
```

```
FROM books

GROUP BY author

HAVING SUM(words) > 1000000;

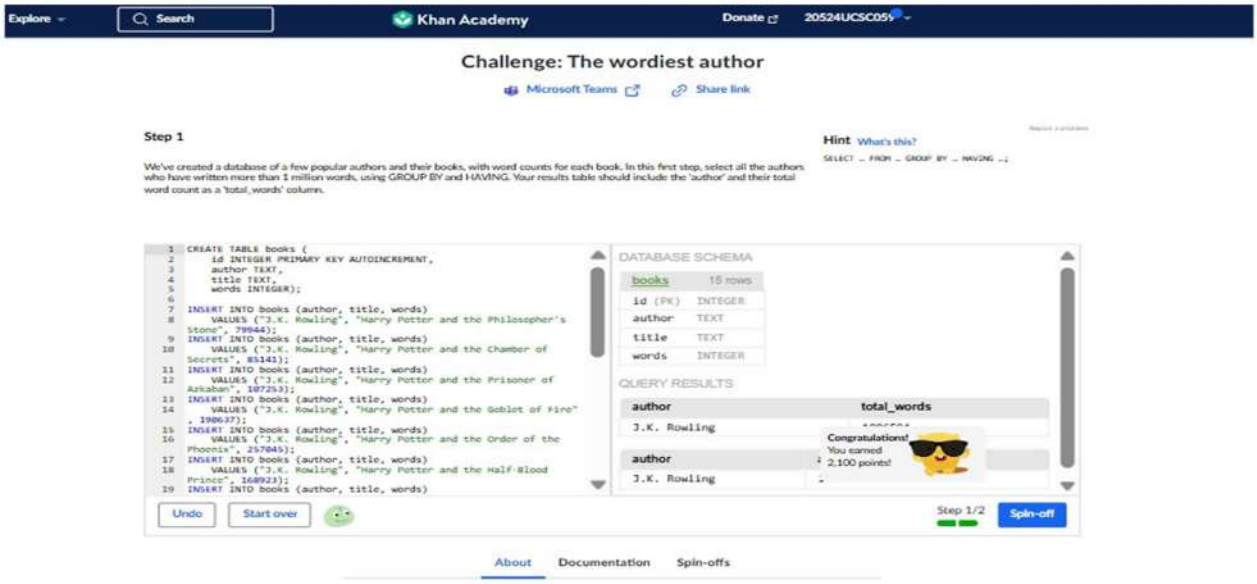
SELECT author,

    AVG(words) AS avg_words

FROM books

GROUP BY author

HAVING AVG(words) > 150000;
```



GRADEBOOK

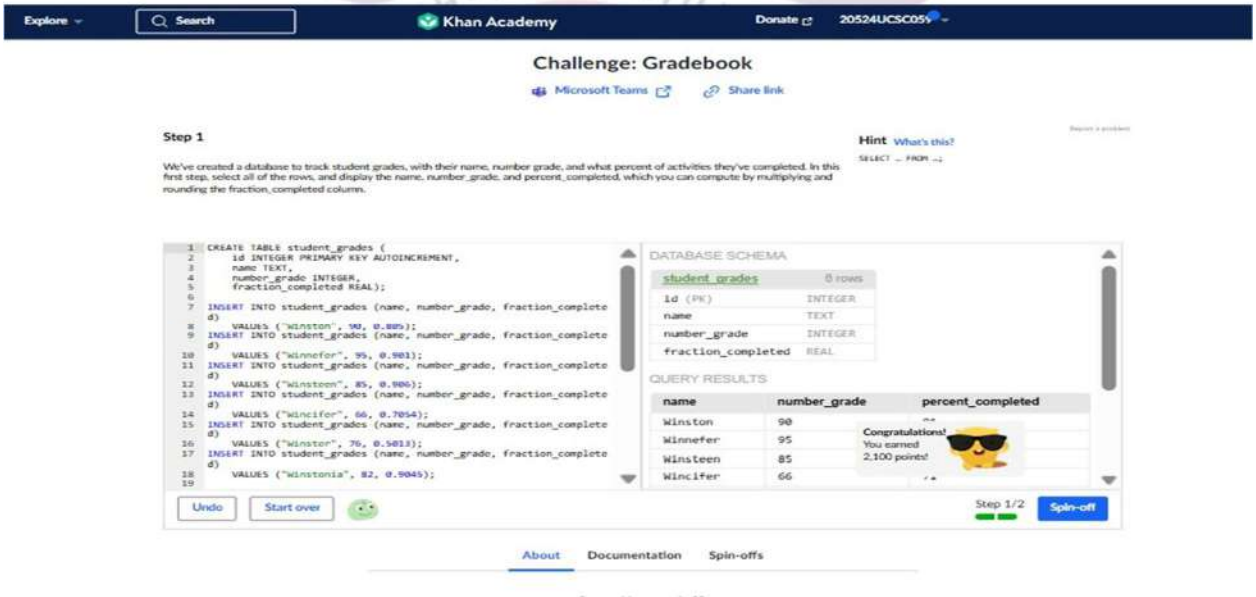
We've created a database to track student grades, with their name, number grade, and what percent of activities they've completed. In this first step, select all of the rows, and display

the name, number_grade, and percent_completed, which you can compute by multiplying and rounding the fraction_completed column. Now, this step is a little tricky. The goal is a table that shows how many students have earned which letter_grade. You can output the letter_grade by using CASE with the number_grade column, outputting 'A' for grades > 90, 'B' for grades > 80, 'C' for grades > 70, and 'F' otherwise. Then you can use COUNT with GROUP BY to show the number of students with each of those grades.

Program:

```
CREATE TABLE student_grades (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    number_grade INTEGER,
    fraction_completed REAL);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winston", 90, 0.805);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winnefer", 95, 0.901);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winsteen", 85, 0.906);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Wincifer", 66, 0.7054);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winster", 76, 0.5013);
INSERT INTO student_grades (name, number_grade, fraction_completed)
    VALUES ("Winstonia", 82, 0.9045);
SELECT name,
       number_grade,
       ROUND(fraction_completed * 100) AS percent_completed
FROM student_grades;
SELECT COUNT(*),
```

```
    CASE
        WHEN number_grade > 90 THEN 'A'
        WHEN number_grade > 80 THEN 'B'
        WHEN number_grade > 70 THEN 'C'
        ELSE 'F'
    END AS "letter_grade"
FROM student_grades
GROUP BY
    CASE
        WHEN number_grade > 90 THEN 'A'
        WHEN number_grade > 80 THEN 'B'
        WHEN number_grade > 70 THEN 'C'
        ELSE 'F'
    END;
```



DATA DIG

What are average, max, and min values in the data?
What about those numbers per category in the data (using HAVING)?
What ways are there to group the data values that don't exist yet (using CASE)?
What interesting ways are there to filter the data (using AND/OR)?

Program:

```
CREATE TABLE pokemon (
    id INTEGER PRIMARY KEY,
    name TEXT,
    type TEXT,
    hp INTEGER,
    attack INTEGER
);
INSERT INTO pokemon VALUES (1, "Pikachu", "Electric", 35, 55);
INSERT INTO pokemon VALUES (2, "Charizard", "Fire", 78, 84);
INSERT INTO pokemon VALUES (3, "Blastoise", "Water", 79, 83);
INSERT INTO pokemon VALUES (4, "Venusaur", "Grass", 80, 82);
INSERT INTO pokemon VALUES (5, "Jolteon", "Electric", 65, 65);
INSERT INTO pokemon VALUES (6, "Arcanine", "Fire", 90, 110);
INSERT INTO pokemon VALUES (7, "Gyarados", "Water", 95, 125);
```

```
INSERT INTO pokemon VALUES (8, "Sceptile", "Grass", 70, 85);
SELECT
   AVG(attack) AS avg_attack,
   MAX(attack) AS max_attack,
   MIN(attack) AS min_attack
FROM pokemon;
SELECT
   type,
   AVG(attack) AS avg_attack
FROM pokemon
GROUP BY type;
SELECT
   type,
   AVG(attack) AS avg_attack
FROM pokemon
GROUP BY type
HAVING AVG(attack) > 80;
SELECT
   name,
   attack,
   CASE
      WHEN attack >= 100 THEN "Very Strong"
      WHEN attack >= 80 THEN "Strong"
      ELSE "Average"
   END AS strength
FROM pokemon;
SELECT *
FROM pokemon
```

```
WHERE (type = "Fire" OR type = "Water")
AND attack > 80;
```

Output

DATABASE SCHEMA

pokemon 8 rows
id (PK) INTEGER
name TEXT
type TEXT
hp INTEGER
attack INTEGER

QUERY RESULTS

avg_attack	max_attack	min_attack
86.125	125	55

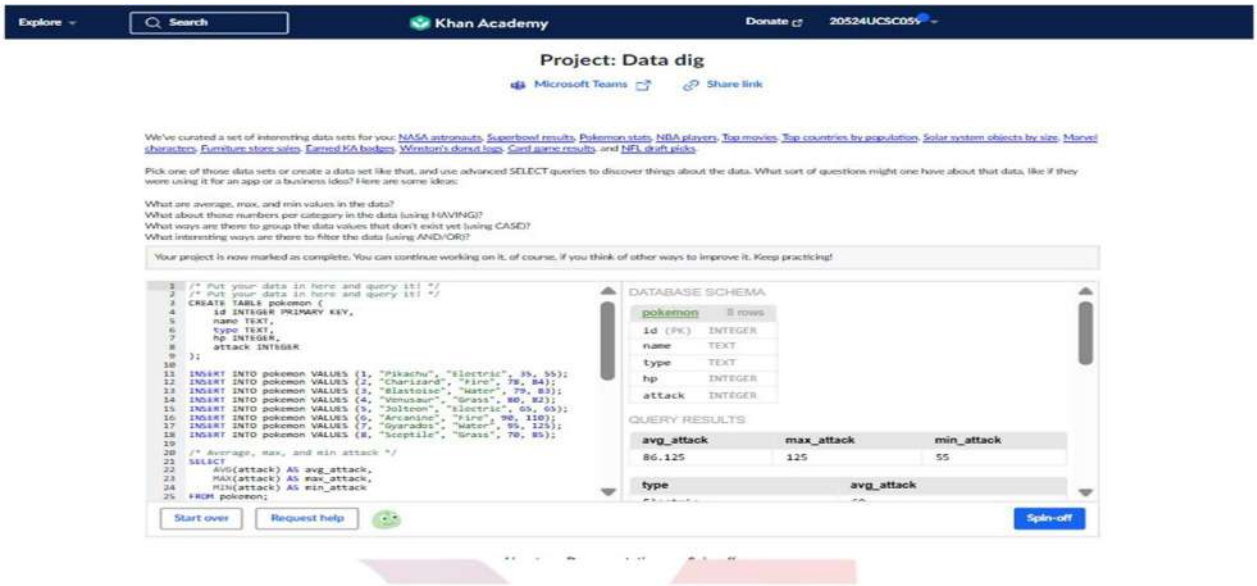
type	avg_attack
Electric	60
Fire	97
Grass	83.5

type	avg_attack
Water	104

type	avg_attack
Fire	97
Grass	83.5
Water	104

name	attack	strength
Pikachu	55	Average
Charizard	84	Strong
Blastoise	83	Strong
Venusaur	82	Strong
Jolteon	65	Average
Arcanine	110	Very Strong
Gyarados	125	Very Strong
Sceptile	85	Strong

id	name	type	hp	attack
2	Charizard	Fire	78	84
3	Blastoise	Water	79	83
6	Arcanine	Fire	90	110
7	Gyarados	Water	95	125



BOBBY'S HOBBIES

We've created a database of people and hobbies, and each row in hobbies is related to a row in persons via the person_id column. In this first step, insert one more row in persons and then one more row in hobbies that is related to the newly inserted person. Now, select the 2 tables with a join so that you can see each person's name next to their hobby. Now, add an additional query that shows only the name and hobbies of 'Bobby McBobbyFace', using JOIN combined with WHERE.

Program:

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    age INTEGER);
```

```
INSERT INTO persons (name, age) VALUES ("Bobby McBobbyFace", 12);
INSERT INTO persons (name, age) VALUES ("Lucy BoBucie", 25);
INSERT INTO persons (name, age) VALUES ("Banana FoFanna", 14);
INSERT INTO persons (name, age) VALUES ("Shish Kabob", 20);
INSERT INTO persons (name, age) VALUES ("Fluffy Sparkles", 8);
CREATE table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
INSERT INTO persons (name, age)
VALUES ("Sam Star", 18);
INSERT INTO hobbies (person_id, name)
VALUES (6, "gaming");
SELECT persons.name, hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT persons.name,
```

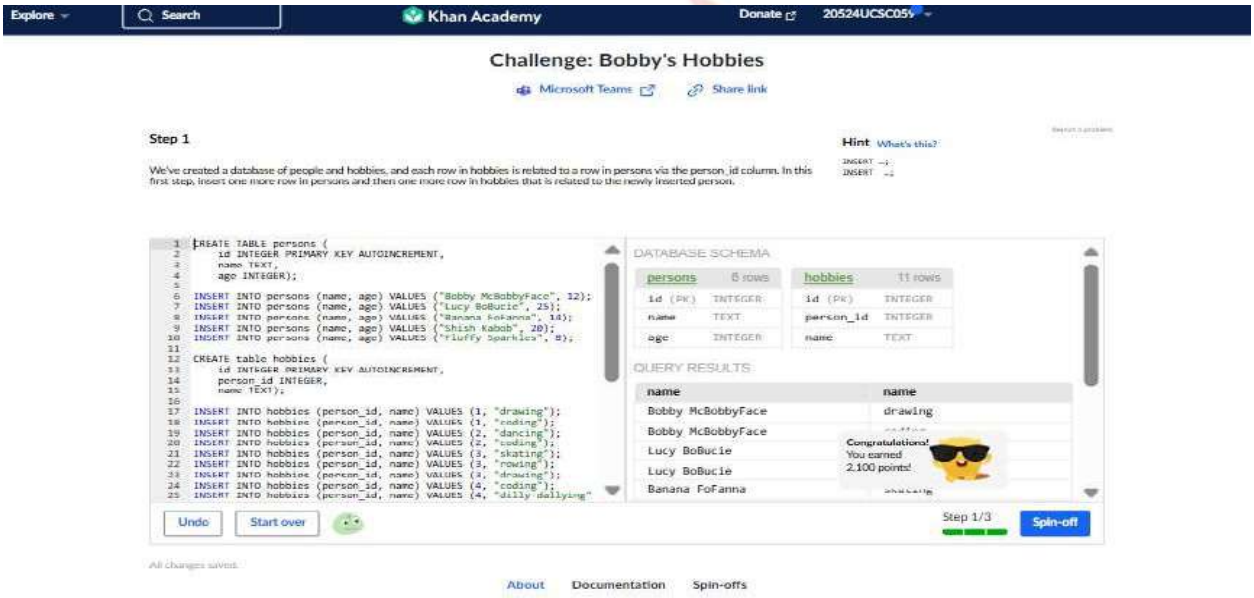
hobbies.name

FROM persons

JOIN hobbies

ON persons.id = hobbies.person_id

WHERE persons.name = "Bobby McBobbyFace";



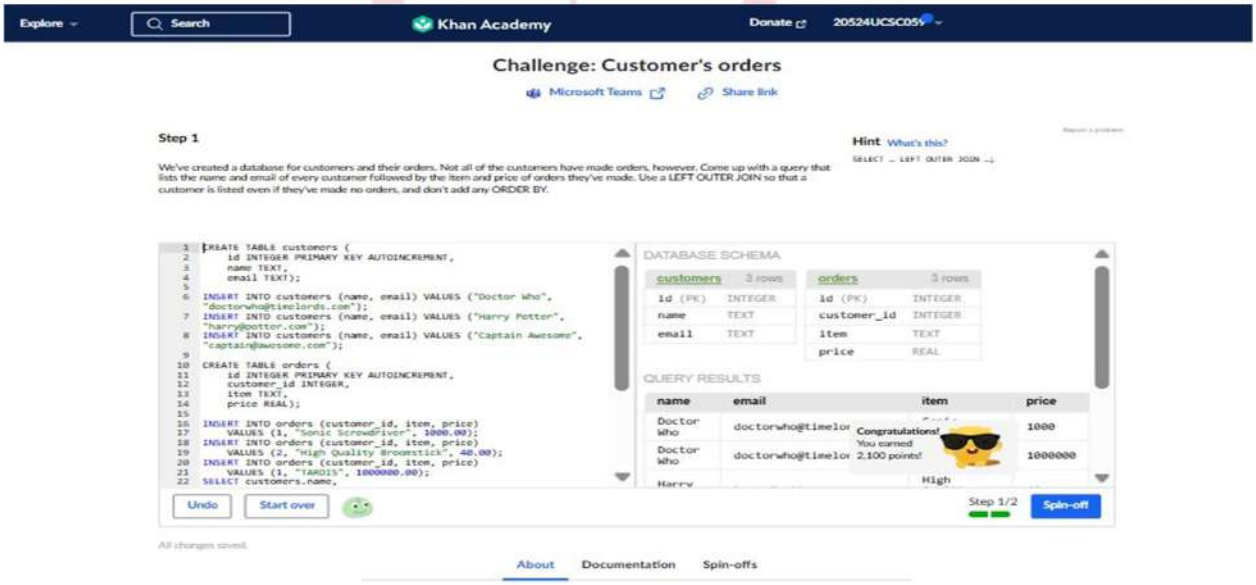
CUSTOMER'S ORDERS

We've created a database for customers and their orders. Not all of the customers have made orders, however. Come up with a query that lists the name and email of every customer followed by the item and price of orders they've made. Use a LEFT OUTER JOIN so that a customer is listed even if they've made no orders, and don't add any ORDER BY. Now, create another query that will result in one row per each customer, with their name, email, and total amount of money they've spent on orders. Sort the rows according to the total money spent, from the most spent to the least spent.

Program:

```
CREATE TABLE customers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    email TEXT);
INSERT INTO customers (name, email) VALUES ("Doctor Who",
"doctorwho@timelords.com");
INSERT INTO customers (name, email) VALUES ("Harry Potter", "harry@potter.com");
INSERT INTO customers (name, email) VALUES ("Captain Awesome",
"captain@awesome.com");
CREATE TABLE orders (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    customer_id INTEGER,
    item TEXT,
    price REAL);
INSERT INTO orders (customer_id, item, price)
    VALUES (1, "Sonic Screwdriver", 1000.00);
INSERT INTO orders (customer_id, item, price)
    VALUES (2, "High Quality Broomstick", 40.00);
INSERT INTO orders (customer_id, item, price)
```

```
    VALUES (1, "TARDIS", 1000000.00);
SELECT customers.name,
       customers.email,
       orders.item,
       orders.price
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id;
SELECT customers.name,
       customers.email,
       SUM(orders.price) AS total_spent
FROM customers
LEFT OUTER JOIN orders
ON customers.id = orders.customer_id
GROUP BY customers.id
ORDER BY total_spent DESC;
```



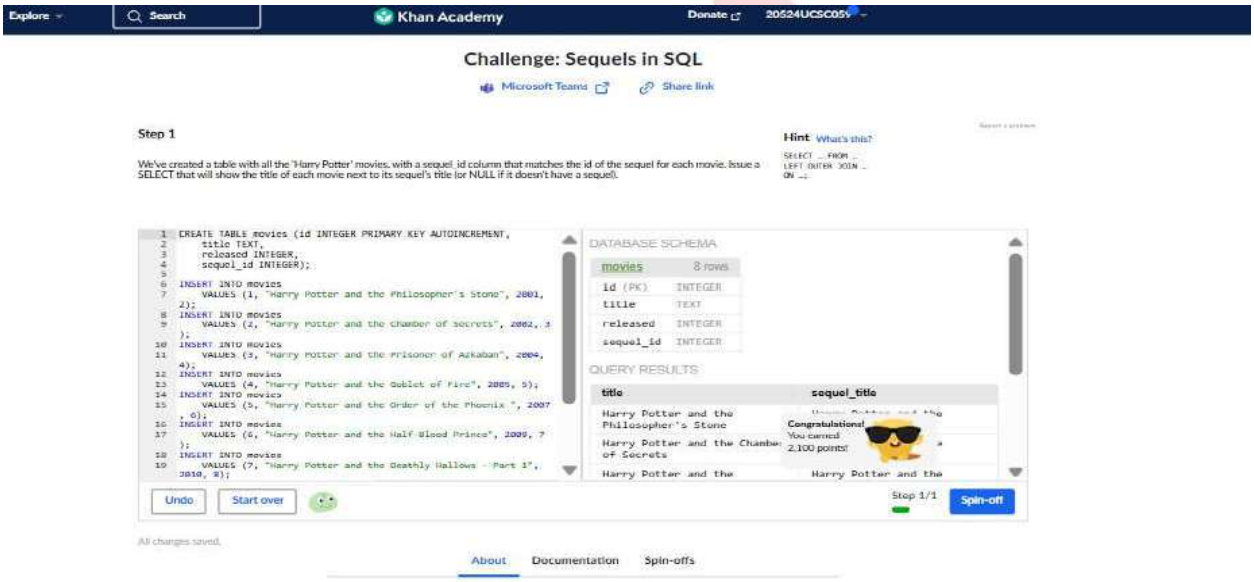
SEQUELS IN SQL

We've created a table with all the 'Harry Potter' movies, with a sequel_id column that matches the id of the sequel for each movie. Issue a SELECT that will show the title of each movie next to its sequel's title (or NULL if it doesn't have a sequel).

Program:

```
CREATE TABLE movies (id INTEGER PRIMARY KEY AUTOINCREMENT,
   title TEXT,
   released INTEGER,
   sequel_id INTEGER);
INSERT INTO movies
   VALUES (1, "Harry Potter and the Philosopher's Stone", 2001, 2);
INSERT INTO movies
   VALUES (2, "Harry Potter and the Chamber of Secrets", 2002, 3);
INSERT INTO movies
   VALUES (3, "Harry Potter and the Prisoner of Azkaban", 2004, 4);
INSERT INTO movies
   VALUES (4, "Harry Potter and the Goblet of Fire", 2005, 5);
INSERT INTO movies
   VALUES (5, "Harry Potter and the Order of the Phoenix ", 2007, 6);
INSERT INTO movies
   VALUES (6, "Harry Potter and the Half-Blood Prince", 2009, 7);
INSERT INTO movies
   VALUES (7, "Harry Potter and the Deathly Hallows – Part 1", 2010, 8);
INSERT INTO movies
   VALUES (8, "Harry Potter and the Deathly Hallows – Part 2", 2011, NULL);
SELECT m1.title,
```

```
    m2.title AS sequel_title
FROM movies AS m1
LEFT OUTER JOIN movies AS m2
ON m1.sequel_id = m2.id;
```



AUTONOMOUS

FRIENDBOOK

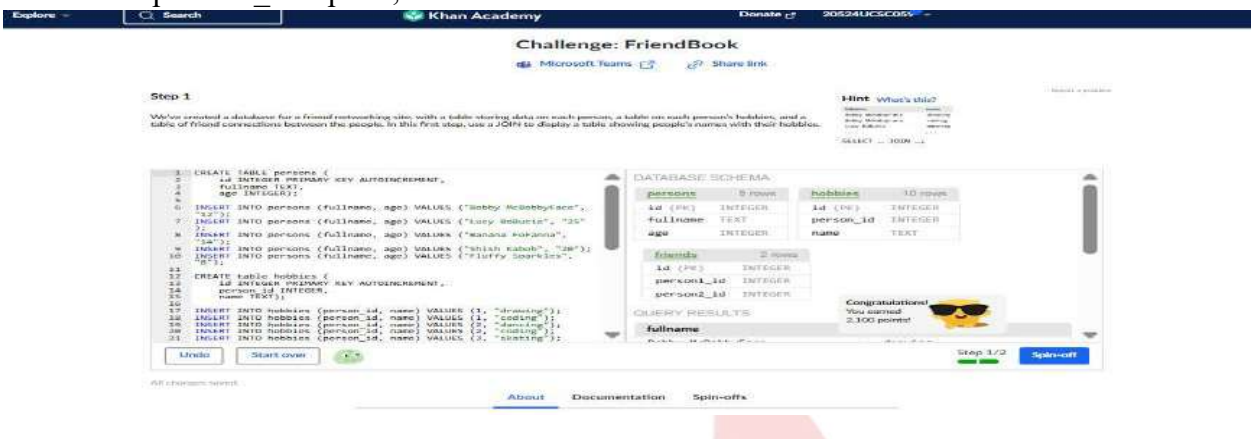
We've created a database for a friend networking site, with a table storing data on each person, a table on each person's hobbies, and a table of friend connections between the people. In this first step, use a JOIN to display a table showing people's names with their hobbies. Now, use another SELECT with a JOIN to show the names of each pair of friends, based on the data in the friends table.

Program:

```
CREATE TABLE persons (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    fullname TEXT,
    age INTEGER);
INSERT INTO persons (fullname, age) VALUES ("Bobby McBobbyFace", "12");
INSERT INTO persons (fullname, age) VALUES ("Lucy BoBucie", "25");
INSERT INTO persons (fullname, age) VALUES ("Banana FoFanna", "14");
INSERT INTO persons (fullname, age) VALUES ("Shish Kabob", "20");
INSERT INTO persons (fullname, age) VALUES ("Fluffy Sparkles", "8");
CREATE table hobbies (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    person_id INTEGER,
    name TEXT);
INSERT INTO hobbies (person_id, name) VALUES (1, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (1, "coding");
INSERT INTO hobbies (person_id, name) VALUES (2, "dancing");
INSERT INTO hobbies (person_id, name) VALUES (2, "coding");
INSERT INTO hobbies (person_id, name) VALUES (3, "skating");
INSERT INTO hobbies (person_id, name) VALUES (3, "rowing");
```

```
INSERT INTO hobbies (person_id, name) VALUES (3, "drawing");
INSERT INTO hobbies (person_id, name) VALUES (4, "coding");
INSERT INTO hobbies (person_id, name) VALUES (4, "dilly-dallying");
INSERT INTO hobbies (person_id, name) VALUES (4, "meowing");
CREATE table friends (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   person1_id INTEGER,
   person2_id INTEGER);
INSERT INTO friends (person1_id, person2_id)
   VALUES (1, 4);
INSERT INTO friends (person1_id, person2_id)
   VALUES (2, 3);
SELECT persons.fullname,
      hobbies.name
FROM persons
JOIN hobbies
ON persons.id = hobbies.person_id;
SELECT p1.fullname AS person1,
      p2.fullname AS person2
FROM friends
JOIN persons AS p1
ON friends.person1_id = p1.id
JOIN persons AS p2
```

ON friends.person2_id = p2.id;



FAMOUS PEOPLE

Movie stars: What movies are they in? Are they married to each other?

Singers: What songs did they write? Where are they from?
Authors: What books did they write?
Fictional characters: How are they related to other characters? What books do they show up in?

Program:

```
CREATE TABLE authors (
    id INTEGER PRIMARY KEY,
    name TEXT,
    country TEXT
);
INSERT INTO authors VALUES (1, "J.K. Rowling", "United Kingdom");
INSERT INTO authors VALUES (2, "George Orwell", "United Kingdom");
INSERT INTO authors VALUES (3, "Mark Twain", "United States");
INSERT INTO authors VALUES (4, "Jane Austen", "United Kingdom");
CREATE TABLE books (
    id INTEGER PRIMARY KEY,
    title TEXT,
    author_id INTEGER
);
INSERT INTO books VALUES (1, "Harry Potter and the Sorcerer's Stone", 1);
INSERT INTO books VALUES (2, "Harry Potter and the Chamber of Secrets", 1);
INSERT INTO books VALUES (3, "1984", 2);
INSERT INTO books VALUES (4, "Animal Farm", 2);
INSERT INTO books VALUES (5, "The Adventures of Tom Sawyer", 3);
INSERT INTO books VALUES (6, "Pride and Prejudice", 4);
CREATE TABLE author_friends (
    id INTEGER PRIMARY KEY,
    author1_id INTEGER,
    author2_id INTEGER
);
```

```
INSERT INTO author_friends VALUES (1, 1, 2);
INSERT INTO author_friends VALUES (2, 3, 4);
SELECT authors.name AS author,
    books.title AS book
FROM authors
JOIN books
ON authors.id = books.author_id;
SELECT name,
    country
FROM authors;
SELECT a1.name AS author1,
    a2.name AS author2
FROM author_friends
JOIN authors AS a1
ON author_friends.author1_id = a1.id
JOIN authors AS a2
ON author_friends.author2_id = a2.id;
```

Output:

DATABASE SCHEMA

authors4 rows
id (PK)INTEGER
nameTEXT
countryTEXT

books6 rows
id (PK)INTEGER
titleTEXT
author_idINTEGER

author_friends2 rows
id (PK)INTEGER
author1_idINTEGER
author2_idINTEGER

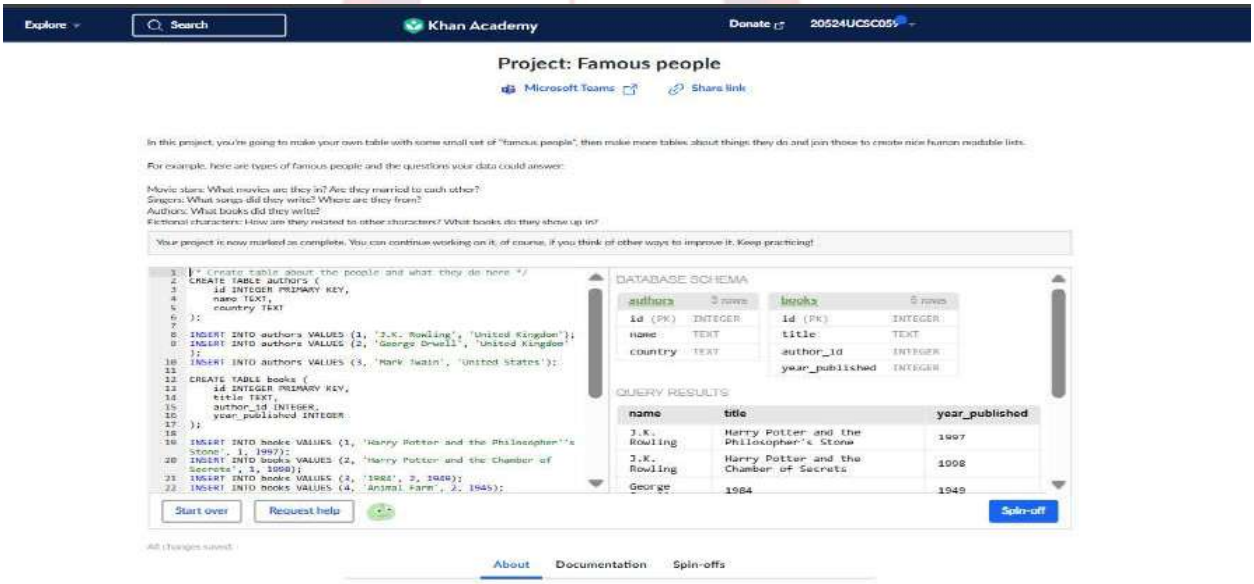
QUERY RESULTS

author	book
J.K. Rowling	Harry Potter and the Sorcerer's Stone
J.K. Rowling	Harry Potter and the Chamber of Secrets
George Orwell	1984
George Orwell	Animal Farm
Mark Twain	The Adventures of Tom Sawyer

author	book
Jane Austen	Pride and Prejudice

name	country
J.K. Rowling	United Kingdom
George Orwell	United Kingdom
Mark Twain	United States
Jane Austen	United Kingdom

author1	author2
J.K. Rowling	George Orwell
Mark Twain	Jane Austen



DYNAMIC DOCUMENTS

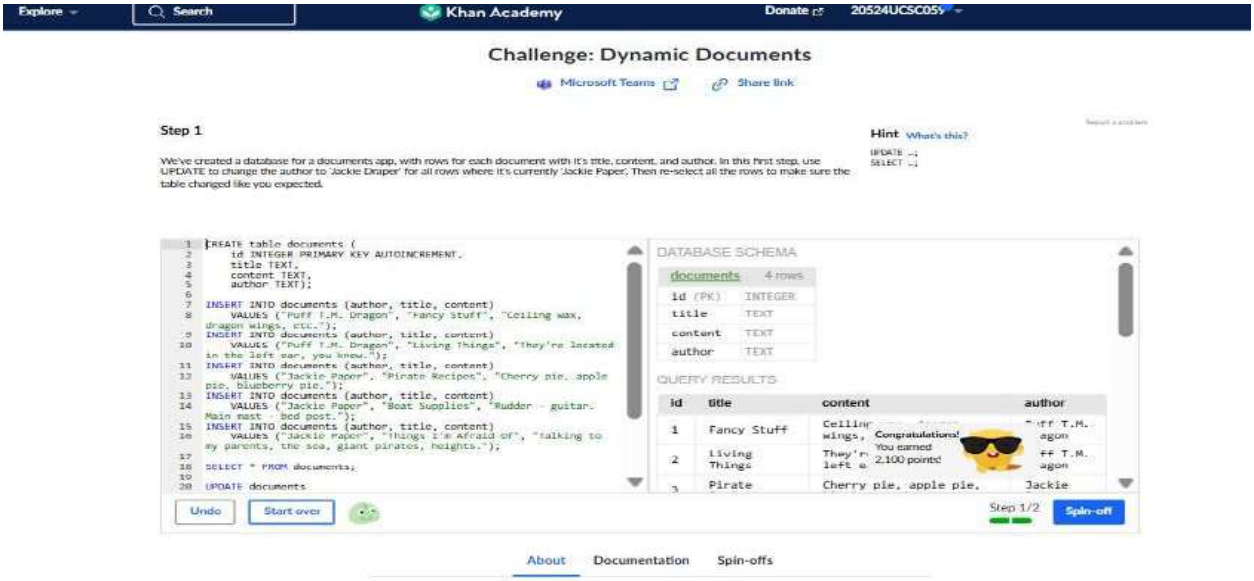
We've created a database for a documents app, with rows for each document with it's title, content, and author. In this first step, use UPDATE to change the author to 'Jackie Draper' for all rows where it's currently 'Jackie Paper'. Then re-select all the rows to make sure the table changed like you expected. Now you'll delete a row, being very careful not to delete all the rows. Only delete the row where the title is 'Things I'm Afraid Of'. Then re-select all the rows to make sure the table changed like you expected.

Program:

```
CREATE table documents (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT,
    content TEXT,
    author TEXT);
```

```
INSERT INTO documents (author, title, content)
    VALUES ("Puff T.M. Dragon", "Fancy Stuff", "Ceiling wax, dragon wings, etc.");
INSERT INTO documents (author, title, content)
    VALUES ("Puff T.M. Dragon", "Living Things", "They're located in the left ear, you know.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Pirate Recipes", "Cherry pie, apple pie, blueberry pie.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Boat Supplies", "Rudder - guitar. Main mast - bed post.");
INSERT INTO documents (author, title, content)
    VALUES ("Jackie Paper", "Things I'm Afraid Of", "Talking to my parents, the sea, giant
pirates, heights.");
SELECT * FROM documents;
UPDATE documents
SET author = "Jackie Draper"
WHERE author = "Jackie Paper";
SELECT * FROM documents;
DELETE FROM documents
WHERE title = "Things I'm Afraid Of";
SELECT * FROM documents;
```

TRUTH-TOIL-TRIUMPH
AUTONOMOUS



TRUTH-TOIL-TRIUMPH

CLOTHING ALTERATIONS

We've created a database of clothes, and decided we need a price column. Use ALTER to add a 'price' column to the table. Then select all the columns in each row to see what your table looks like now. Now assign each item a price, using UPDATE - item 1 should be 10 dollars, item 2 should be 20 dollars, item 3 should be 30 dollars. When you're done, do another SELECT of all

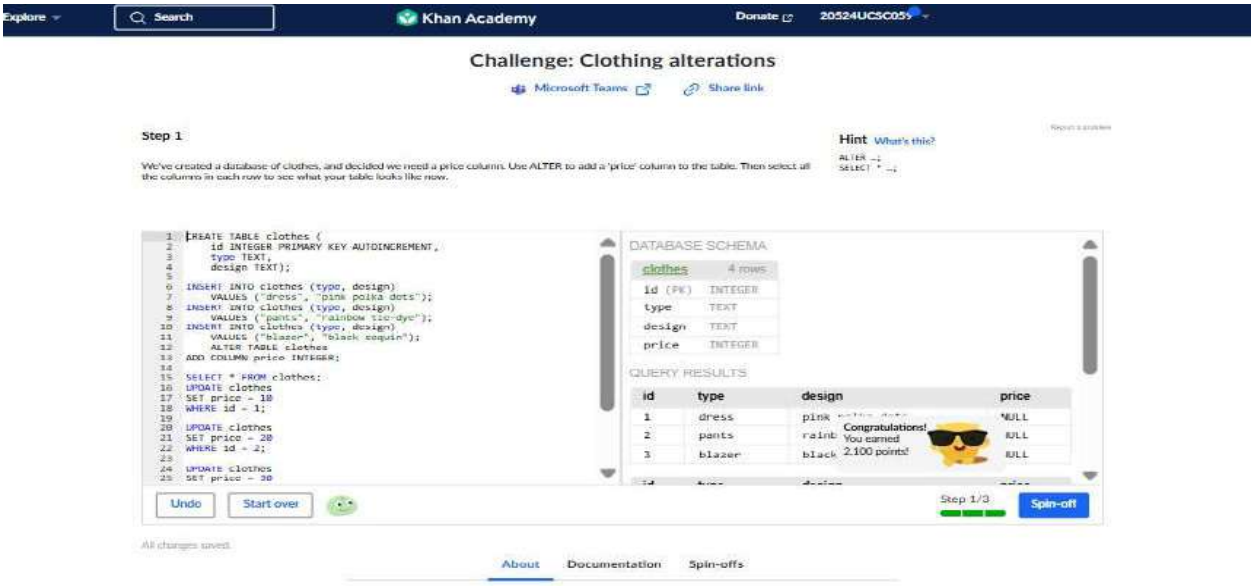
the rows to check that it worked as expected. Now insert a new item into the table that has all three attributes filled in, including 'price'. Do one final SELECT of all the rows to check it worked.

Program:

```
CREATE TABLE clothes (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   type TEXT,
   design TEXT);
INSERT INTO clothes (type, design)
   VALUES ("dress", "pink polka dots");
INSERT INTO clothes (type, design)
   VALUES ("pants", "rainbow tie-dye");
INSERT INTO clothes (type, design)
   VALUES ("blazer", "black sequin");
ALTER TABLE clothes
ADD COLUMN price REAL;
SELECT * FROM clothes;
UPDATE clothes
SET price = 10
WHERE id = 1;
UPDATE clothes
SET price = 20
WHERE id = 2;
UPDATE clothes
SET price = 30
WHERE id = 3;
SELECT * FROM clothes;
INSERT INTO clothes (type, design, price)
```

VALUES ("shirt", "blue stripes", 15);

SELECT * FROM clothes;



PROJECT: APP IMPERSONATO

CREATE a table to store the data.
INSERT a few example rows in the table.
Use an UPDATE to emulate what happens when you edit data in the app.
Use a DELETE to emulate what happens when you delete data in the app.

Program:

```
CREATE TABLE posts (
   id INTEGER PRIMARY KEY AUTOINCREMENT,
   username TEXT,
   content TEXT,
   likes INTEGER
);
INSERT INTO posts (username, content, likes)
VALUES ("alex", "Just finished my homework!", 12);
INSERT INTO posts (username, content, likes)
VALUES ("sam", "Playing my favorite game tonight.", 25);
INSERT INTO posts (username, content, likes)
VALUES ("taylor", "Had a great day at the beach.", 18);
SELECT * FROM posts;
UPDATE posts
SET content = "Just finished my homework and studied SQL!"
WHERE id = 1;
SELECT * FROM posts;
DELETE FROM posts
WHERE id = 2;
SELECT * FROM posts;
```

Output:

DATABASE SCHEMA

posts2 rows
id (PK)INTEGER
usernameTEXT
contentTEXT
likesINTEGER

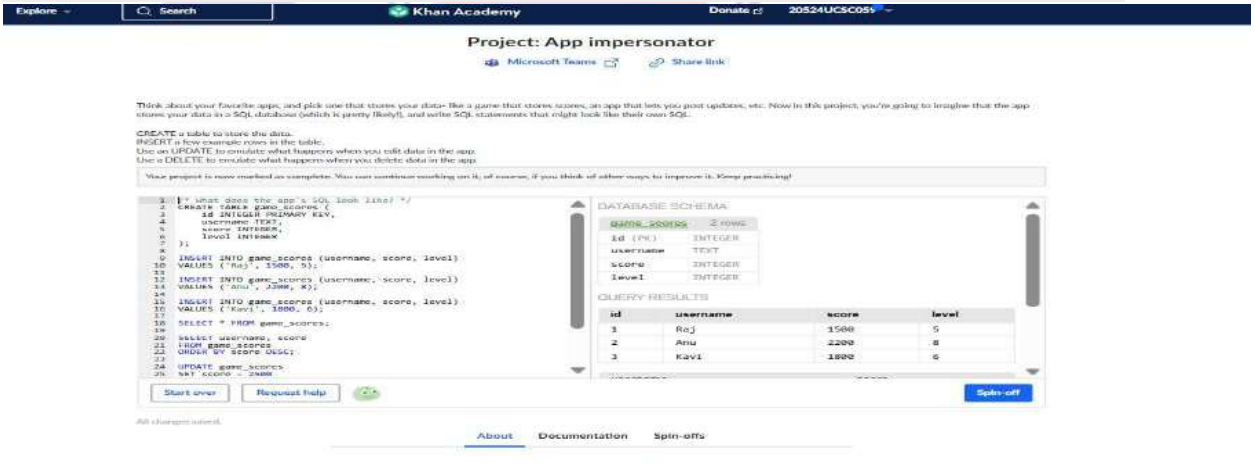
QUERY RESULTS

id	username	content	likes
1	alex	Just finished my homework!	12

id	username	content	likes
2	sam	Playing my favorite game tonight.	25
3	taylor	Had a great day at the beach.	18

id	username	content	likes
1	alex	Just finished my homework and studied SQL!	12
2	sam	Playing my favorite game tonight.	25
3	taylor	Had a great day at the beach.	18

id	username	content	likes
1	alex	Just finished my homework and studied SQL!	12
3	taylor	Had a great day at the beach.	18



HACKER RANK

REVISING THE SELECT QUERY I

Query all columns for all American cities in the CITY table with populations larger than 100000.

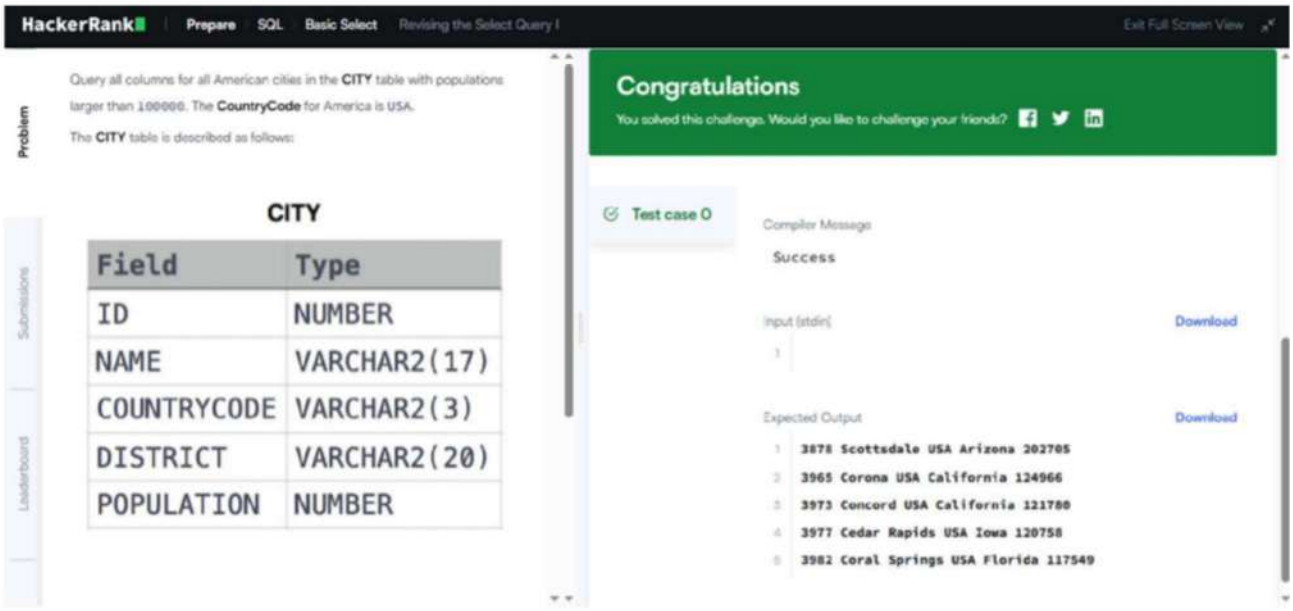
The CountryCode for America is USA.

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
select * from city where countrycode = 'usa' and population > 100000;
```



REVISING THE SELECT QUERY II

Query the NAME field for all American cities in the CITY table with populations larger than 120000.

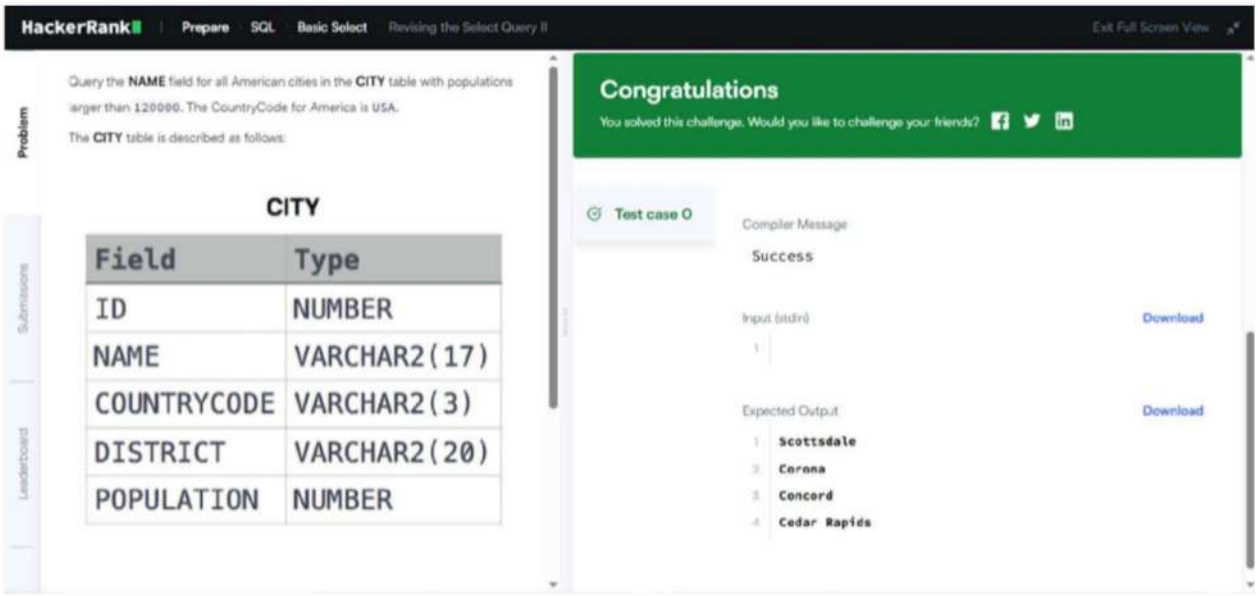
The CountryCode for America is USA.

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

select name from city where countrycode = 'usa' and population > 120000;



SELECT ALL

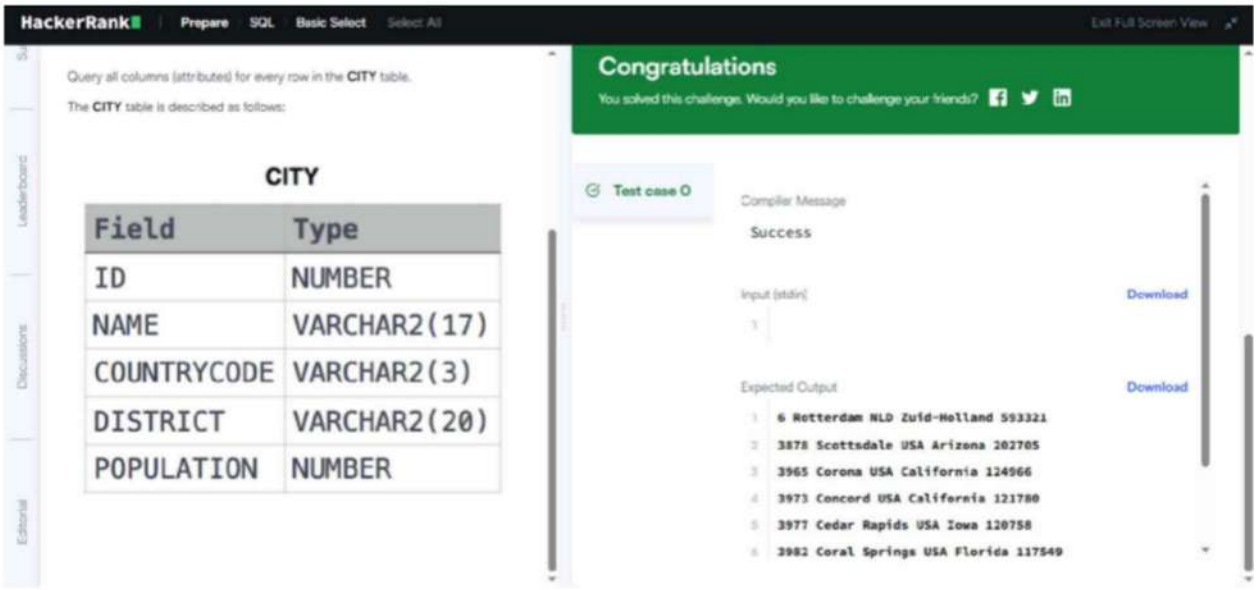
Query all columns (attributes) for every row in the CITY table.

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

select * from city;



SELECT BY ID

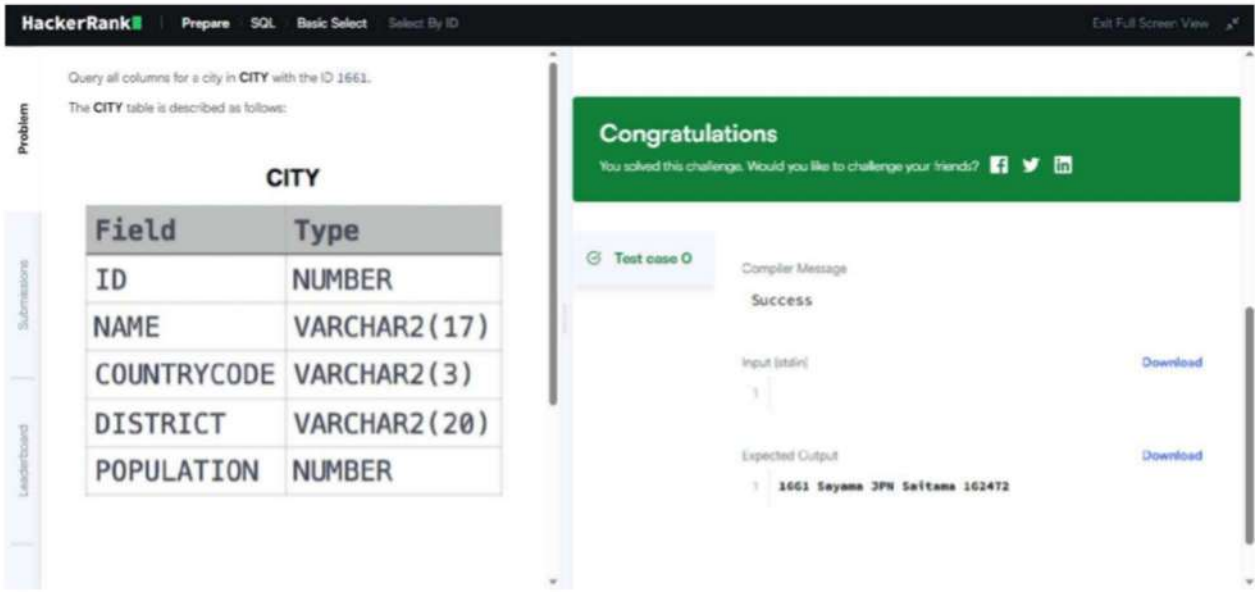
Query all columns for a city in CITY with the ID 1661.

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

select * from city where id = 1661;



JAPANESE CITIES' ATTRIBUTES

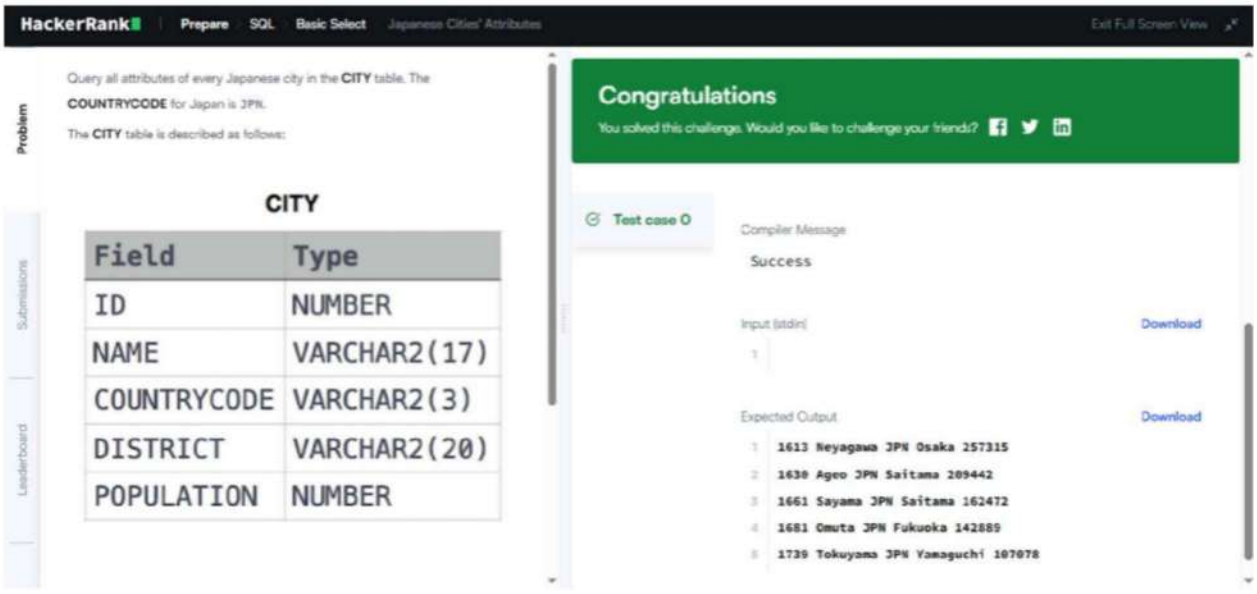
Query all attributes of every Japanese city in the CITY table. The COUNTRYCODE for Japan is JPN.

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

select * from city where countrycode = 'jpn';



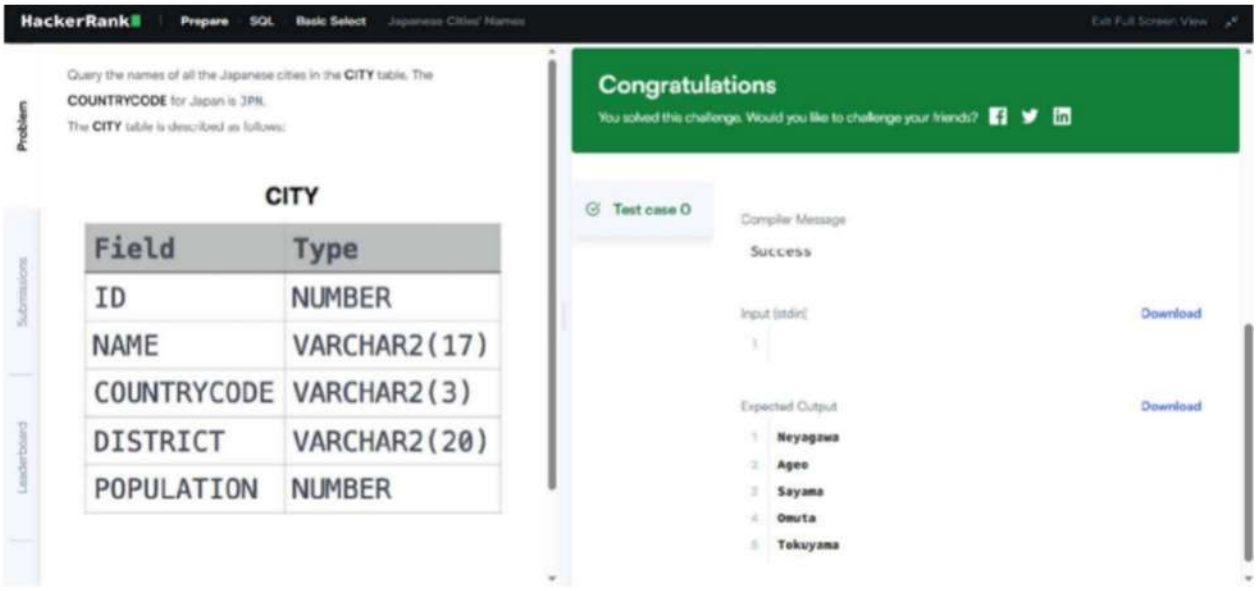
JAPANESE CITIES' NAMES

Query the names of all the Japanese cities in the CITY table. The COUNTRYCODE for Japan is JPN.
The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
select name from city where countrycode = 'jpn';
```



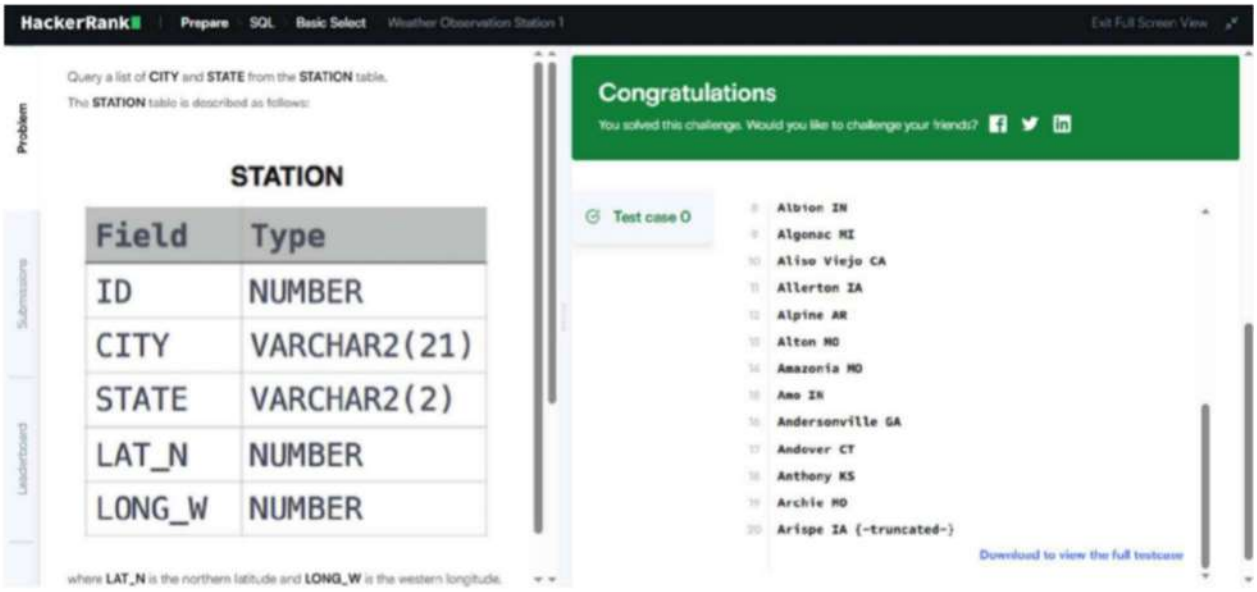
WEATHER OBSERVATION STATION 1

Query a list of CITY and STATE from the STATION table.
The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select city, state from station;



WEATHER OBSERVATION STATION 3

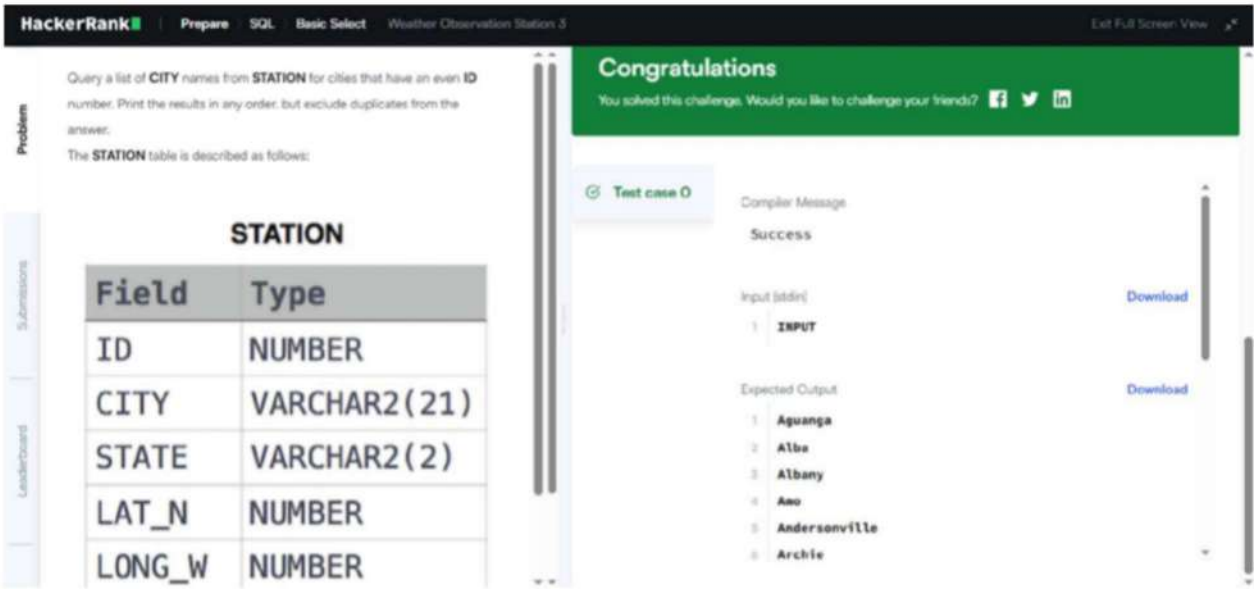
Query a list of CITY names from STATION for cities that have an even ID number. Print the results in any order, but exclude duplicates from the answer.

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where id % 2 = 0;



WEATHER OBSERVATION STATION 4

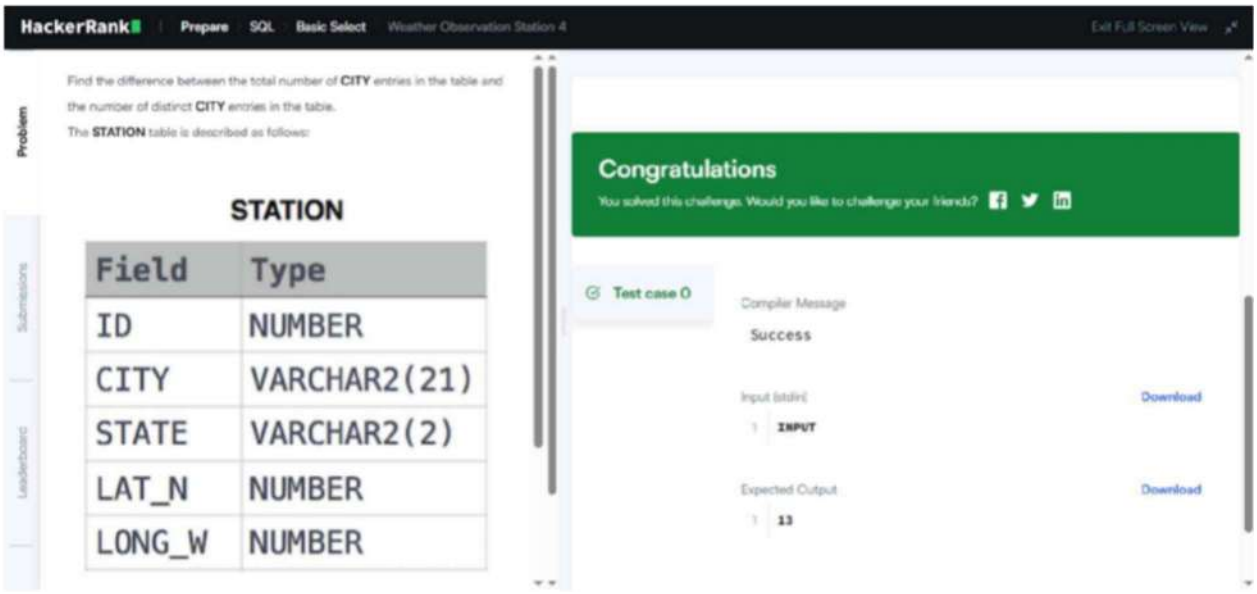
Find the difference between the total number of CITY entries in the table and the number of distinct CITY entries in the table.

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
select count(city) - count(distinct city)
from station;
```



WEATHER OBSERVATION STATION 5

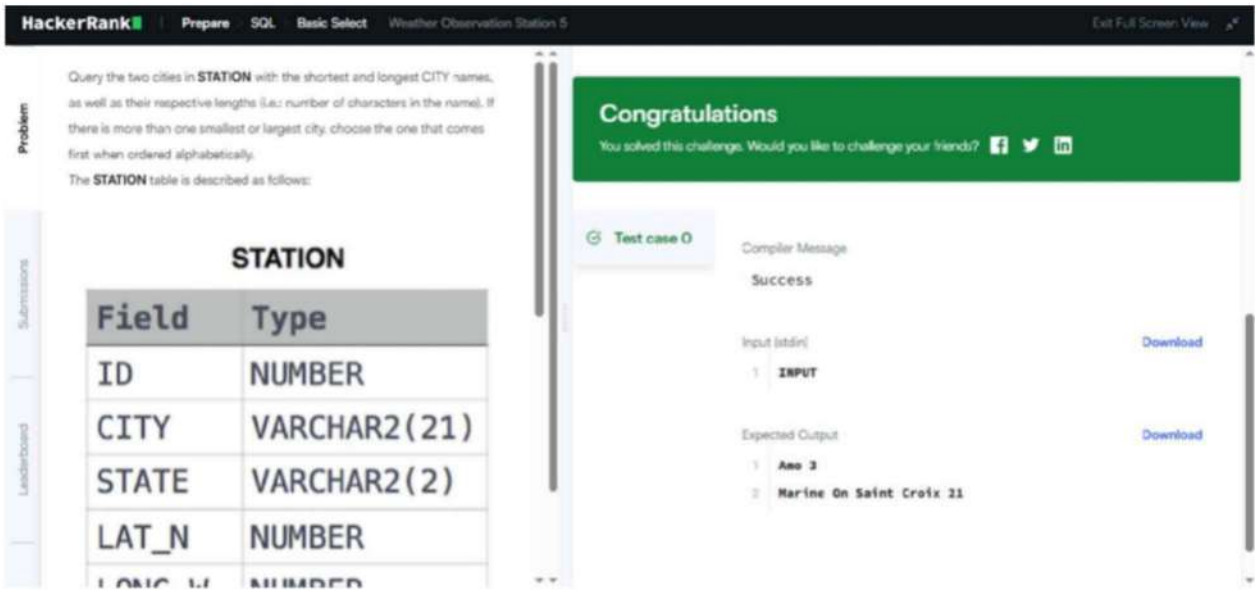
Query the two cities in STATION with the shortest and longest CITY names, as well as their respective lengths (i.e.: number of characters in the name). If there is more than one smallest or largest city, choose the one that comes first when ordered alphabetically.

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select city, length(city)from station order by length(city) asc ,city asc limit 1;
select city, length(city)from stationorder by length(city) desc, city asc limit 1;



WEATHER OBSERVATION STATION 6

Query the list of CITY

names starting with vowels (i.e., a, e, i, o, or u) from STATION. Your result cannot contain duplicates.

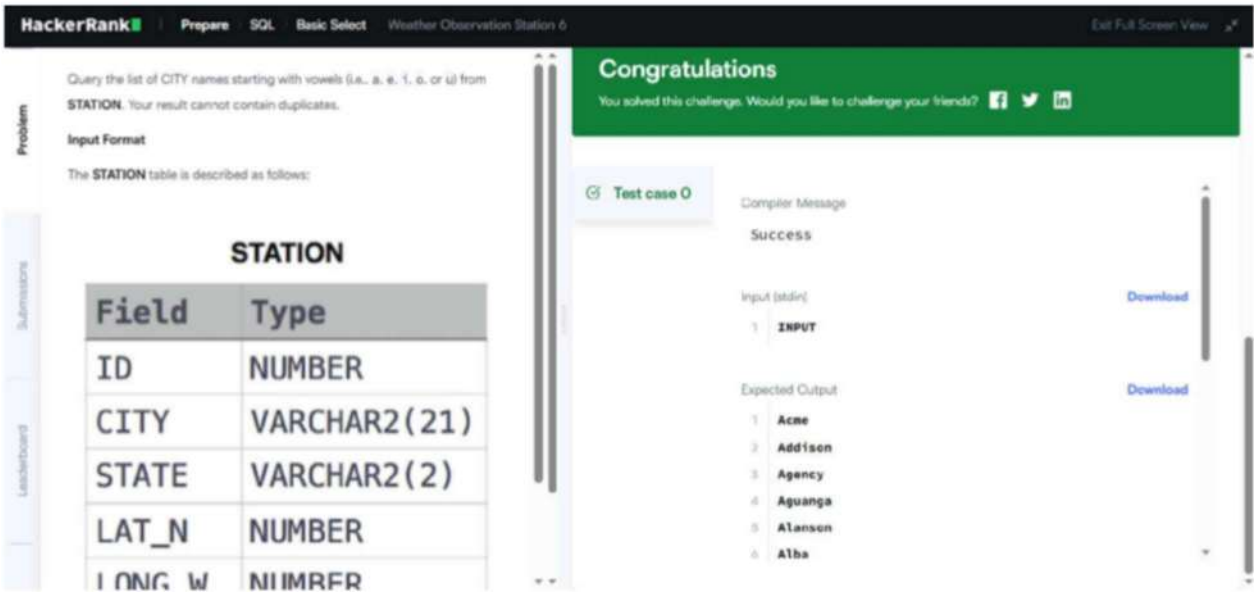
Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where city regexp '^[aeiouaeiou]';



WEATHER OBSERVATION STATION 7

Query the list of CITY

names ending with vowels (a, e, i, o, u) from STATION. Your result cannot contain duplicates.

Input Format

The STATION table is described as follows:

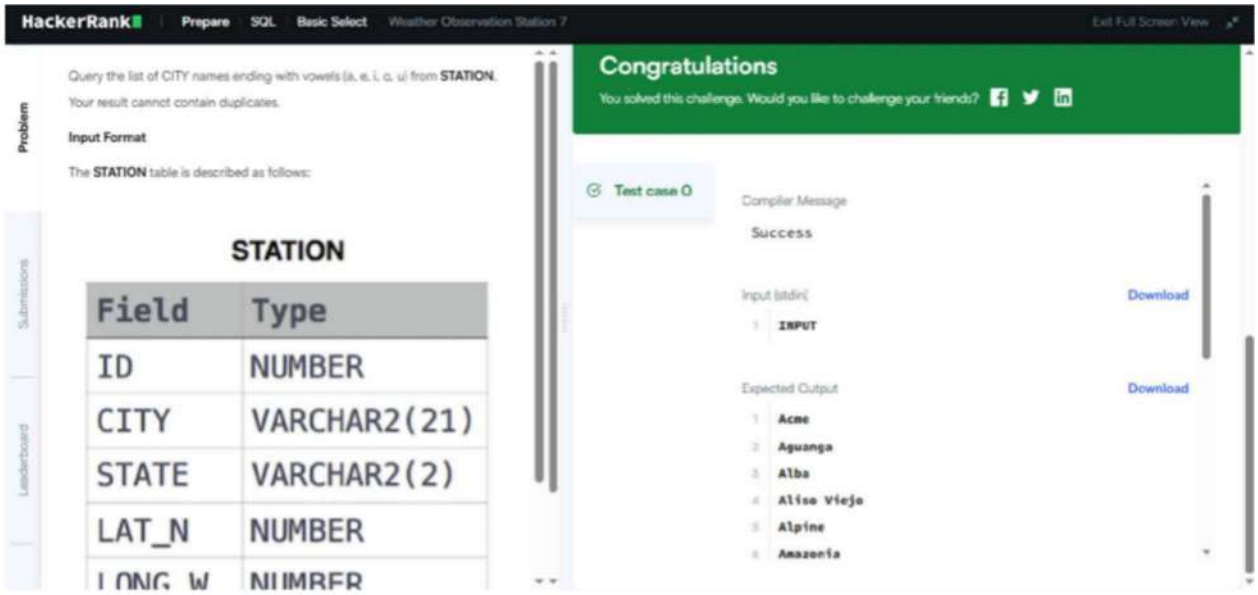
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
select distinct city from station where city regexp '[aeiouaeiou]$';
```

WEATHER OBSERVATION STATION 8

Query the list of CITY names from STATION



which have vowels (i.e., a, e, i, o, and u) as both their

first and last characters. Your result cannot contain duplicates.

Input Format

The STATION table is described as follows:

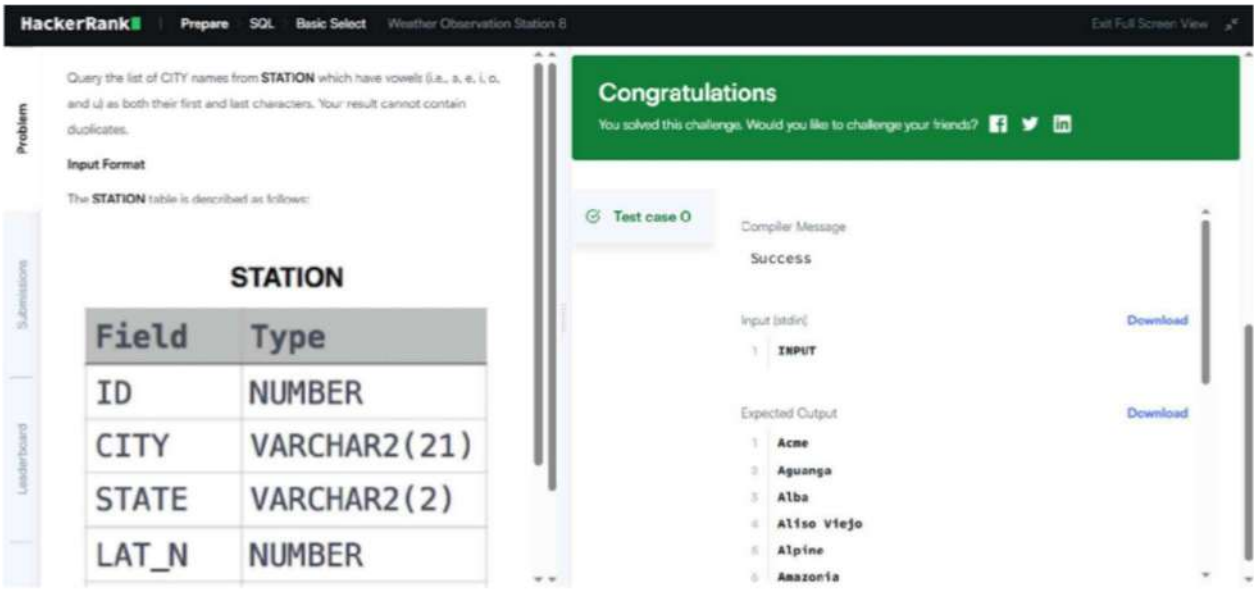
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

WEATHER OBSERVATION STATION 9

Query the list of CITY

```
select distinct city from station where city regexp '^[aeiou].*[aeiou]$';
```



WEATHER OBSERVATION STATION 10

Query the list of CITY names from STATION

that do not start with vowels. Your result cannot contain duplicates.

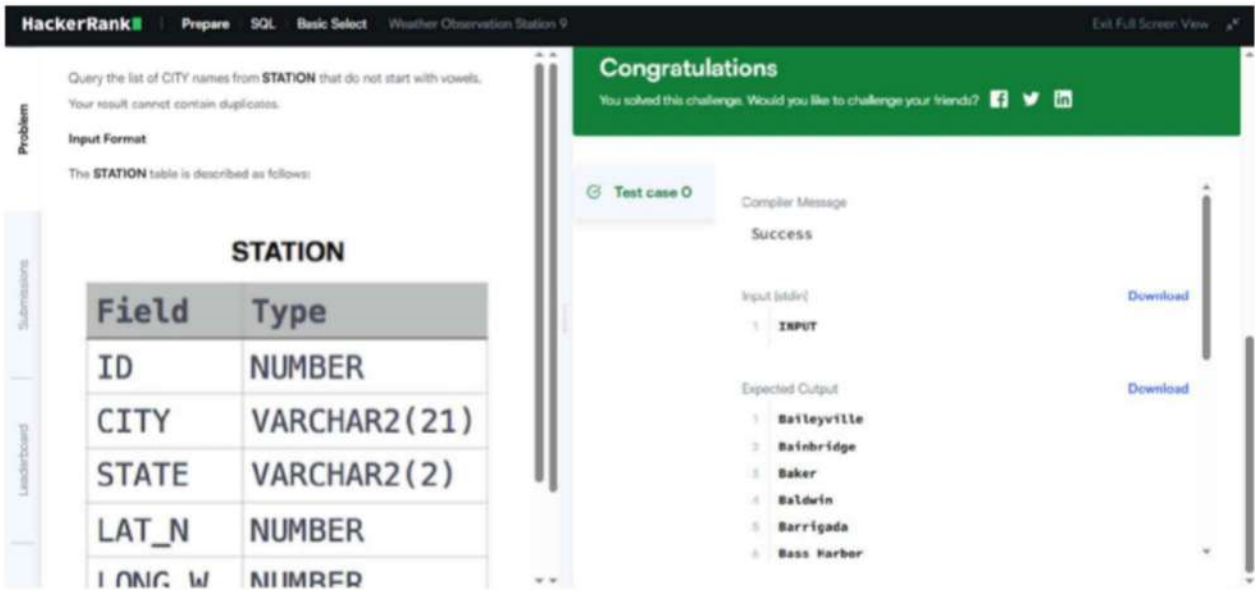
Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where city regexp '^[aeiou].*[aeiou]$';



WEATHER OBSERVATION STATION 11

Query the list of CITY names from STATION

that do not end with vowels. Your result cannot contain duplicates.

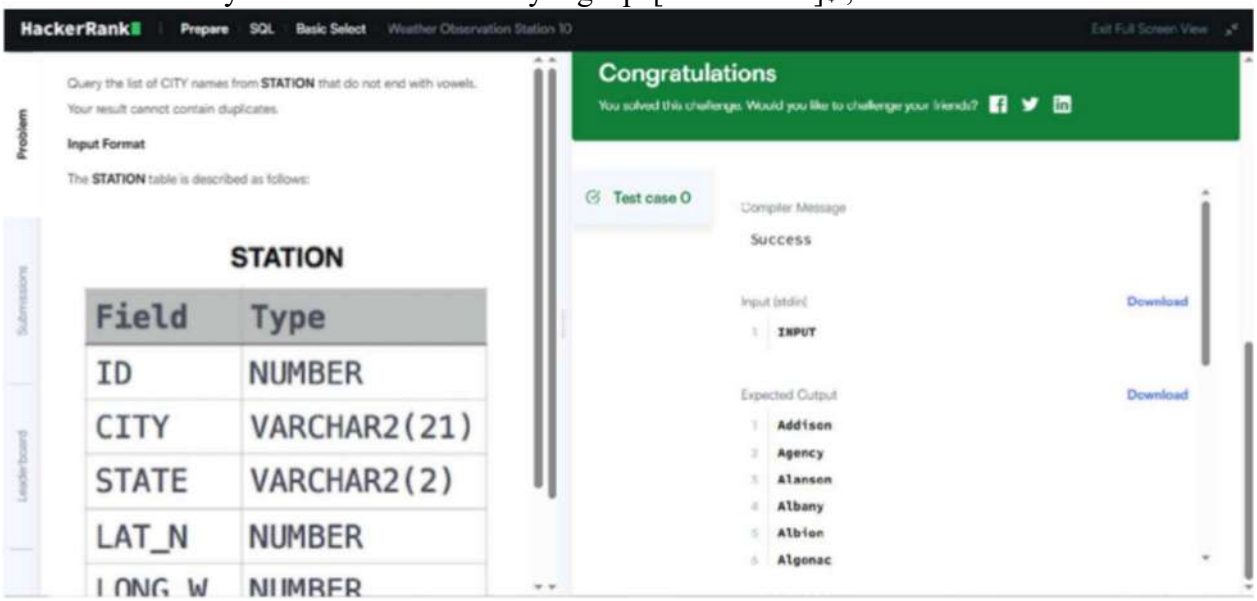
Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where city regexp '[^aeiouaeiou]$';



that either do not start with vowels or do not end with vowels.

Your result cannot contain duplicates.

WEATHER OBSERVATION STATION 12

Query the list of CITY names from STATION

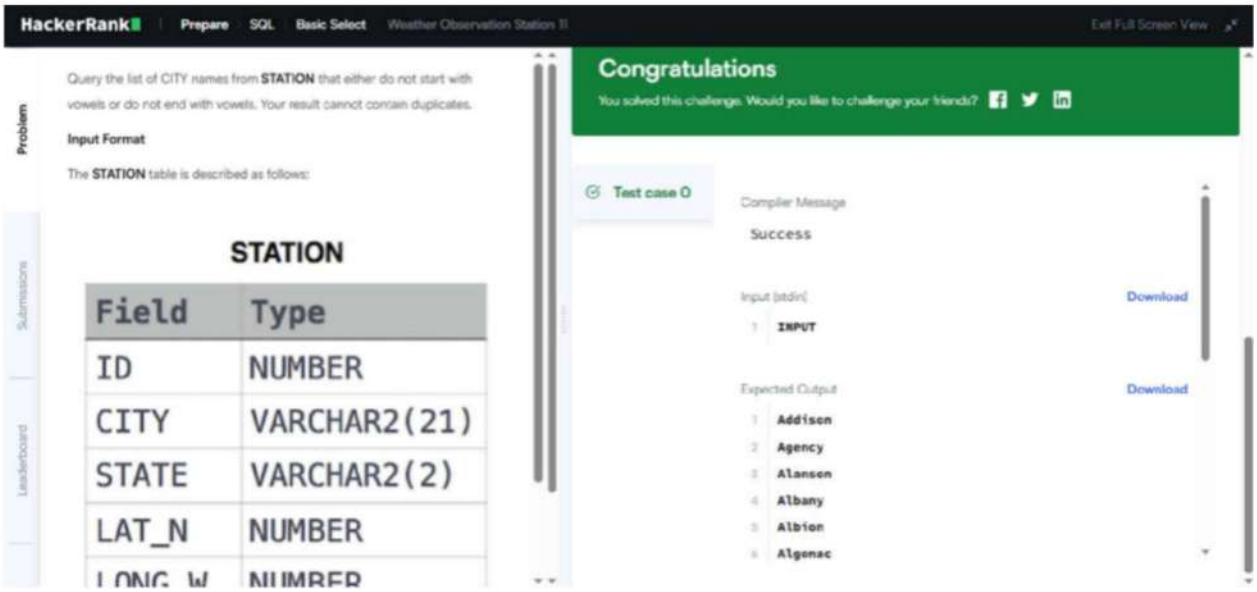
Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

select distinct city from station where left(city, 1) not in ('a','e','i','o','u','a','e','i','o','u') or right(city, 1) not in ('a','e','i','o','u','a','e','i','o','u');



WEATHER OBSERVATION STATION 13

Query the list of CITY names from STATION

Your that do not start with vowels and do not end with vowels. Your result cannot contain duplicates.

Input Format

The STATION table is described as follows:

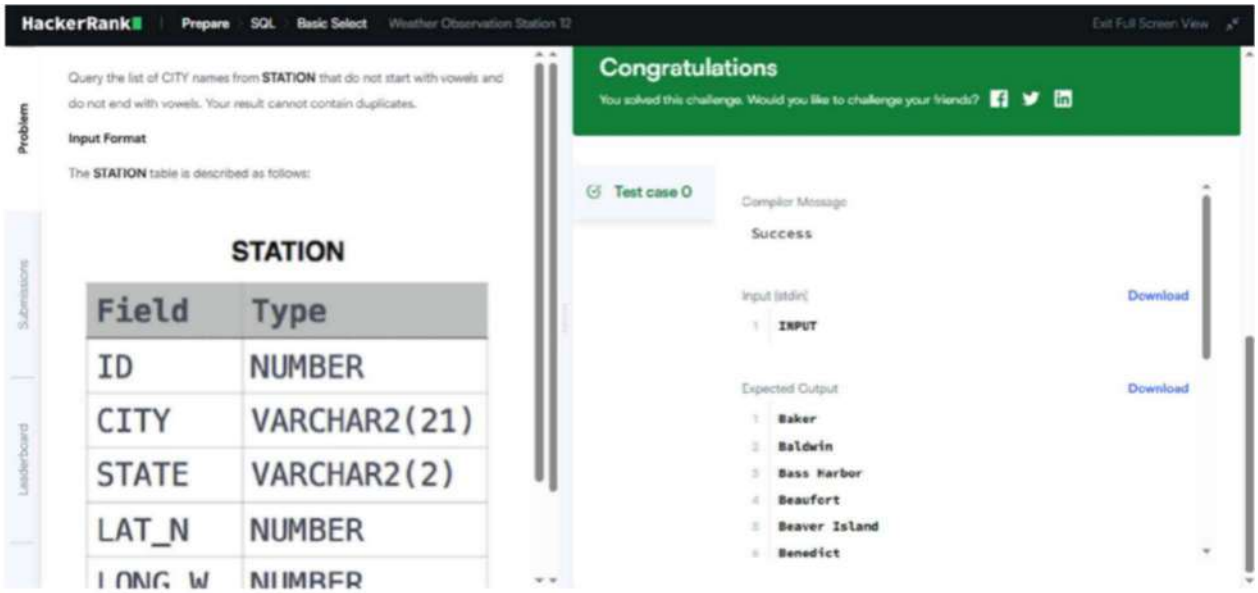
STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
select distinct city from station where city not regexp '^[aeiouaeiou]' and city not regexp
'[aeiouaeiou]$';
```

WEATHER OBSERVATION STATION 14

Query the list of CITY names from STATION



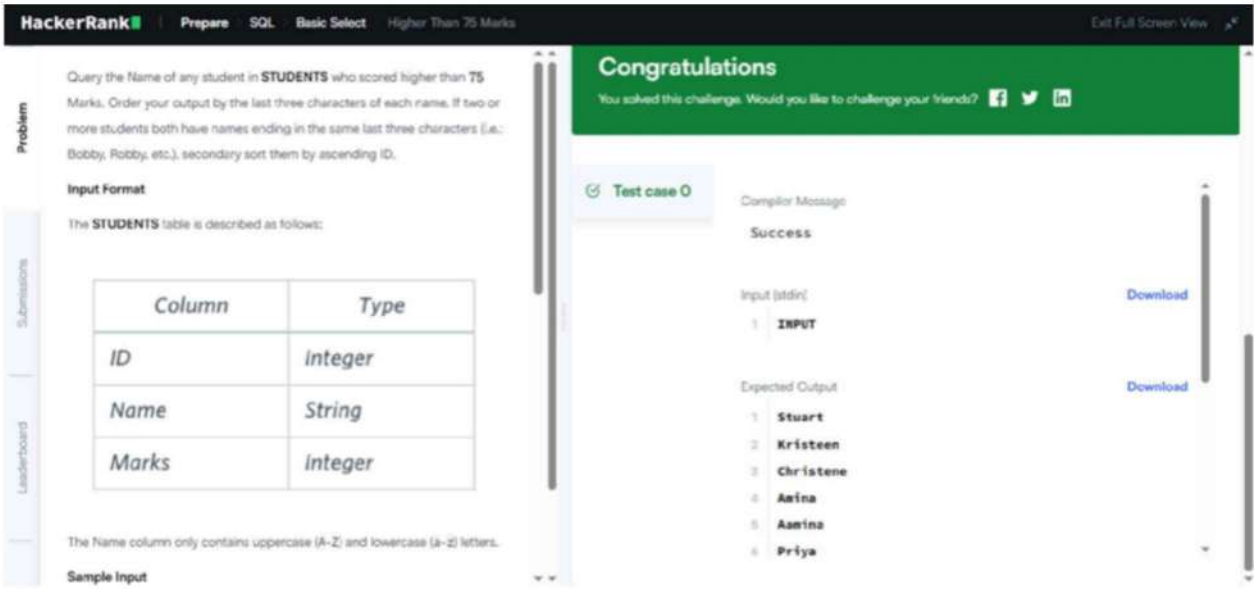
HIGHER THAN 75 MARKS

Query the Name of any student in STUDENTS who scored higher than Marks. Order your output by the last three characters of each name. If two or more students both have names ending in the same last three characters (i.e.: Bobby, Robby, etc.), secondary sort them by ascending ID.

Input Format

Column	Type
ID	Integer
Name	String
Marks	Integer

select name from students where marks > 75 order by right(name, 3), id asc;



EMPLOYEE NAMES

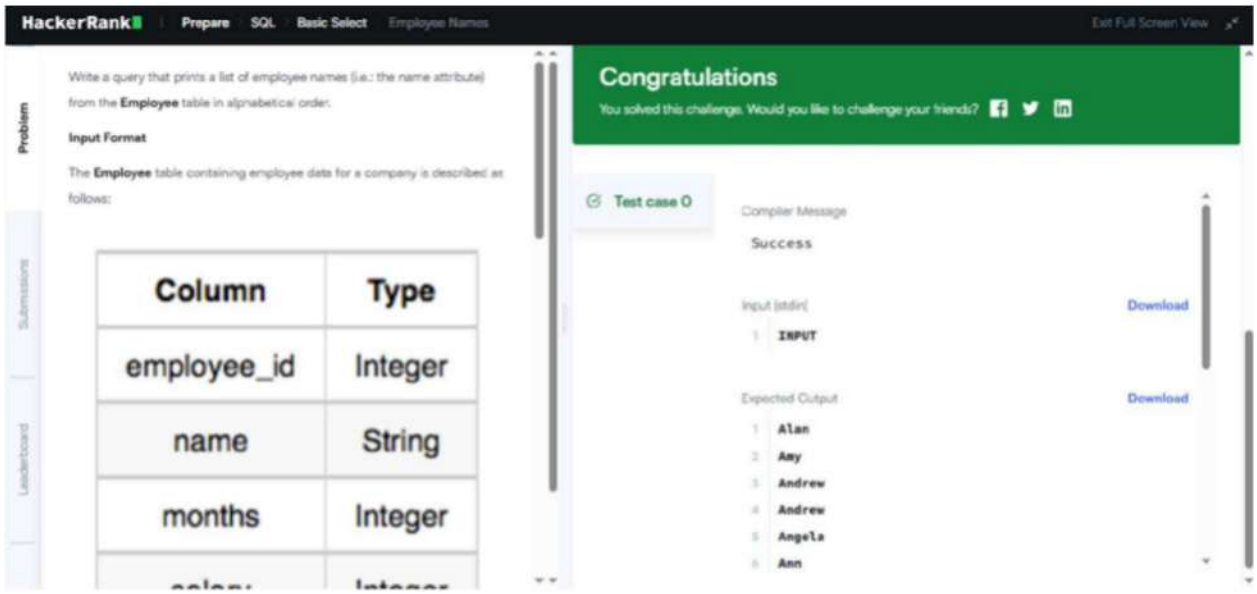
Write a query that prints a list of employee names (i.e.: the name attribute) from the Employee table in alphabetical order.

Input Format

The Employee table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

select name from employee order by name;



EMPLOYEE SALARIES

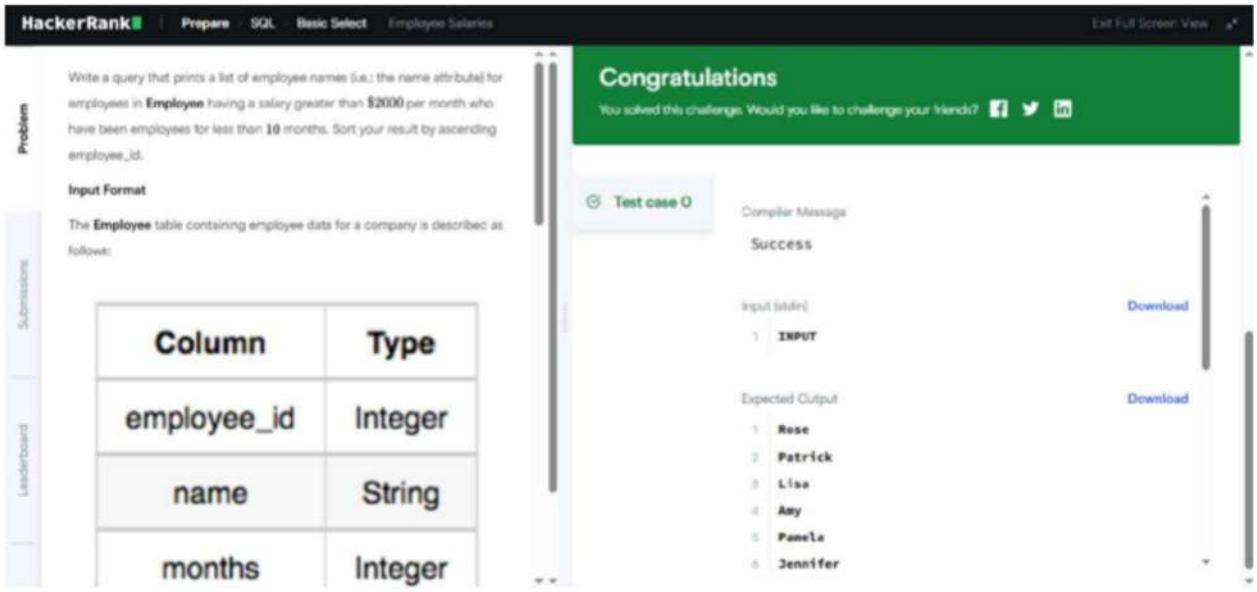
Write a query that prints a list of employee names (i.e.: the name attribute) for employees in Employee having a salary greater than per month who have been employees for less than months. Sort your result by ascending employee_id.

Input Format

The Employee table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

select name from employee where salary > 2000 and months < 10 order by employee_id asc;



TYPE OF TRIANGLE

Write a query identifying the type of each record in the TRIANGLES table using its three side lengths. Output one of the following statements for each record in the table:

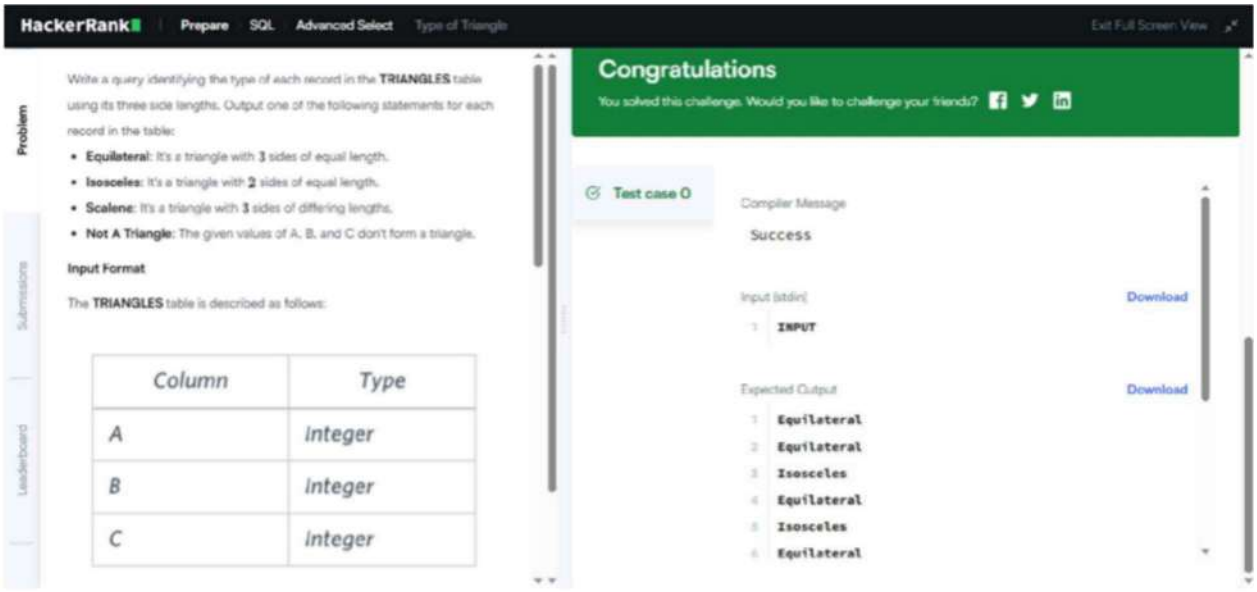
- Equilateral: It's a triangle with sides of equal length.
- Isosceles: It's a triangle with sides of equal length.
- Scalene: It's a triangle with sides of differing lengths.
- Not A Triangle: The given values of A, B, and C don't form a triangle.

Input Format

The TRIANGLES table is described as follows:

Column	Type
A	Integer
B	Integer
C	Integer

```
select
case
        when a + b <= c or a + c <= b or b + c <= a then 'not a triangle'
when a = b and b = c then 'equilateral'         when a = b or b = c or a
= c then 'isosceles'          else 'scalene'      end from triangles;
```


HackerRank
Prepare
SQL
Advanced Select
Type of Triangle
Exit Full Screen View
Problem
Submissions
Leaderboard
Write a query identifying the type of each record in the TRIANGLES table using its three side lengths. Output one of the following statements for each record in the table:
Equilateral: It's a triangle with 3 sides of equal length.
Isosceles: It's a triangle with 2 sides of equal length.
Scalene: It's a triangle with 3 sides of differing lengths.
Not A Triangle: The given values of A, B, and C don't form a triangle.
Input Format
The TRIANGLES table is described as follows:
Column
Type
A
integer
B
integer
C
integer
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
1 INPUT
Expected Output
Download
1 Equilateral
2 Equilateral
3 Isosceles
4 Equilateral
5 Isosceles
6 Equilateral

REVISING AGGREGATIONS - THE COUNT FUNCTION

Query a count of the number of cities in CITY having a Population larger than . Input

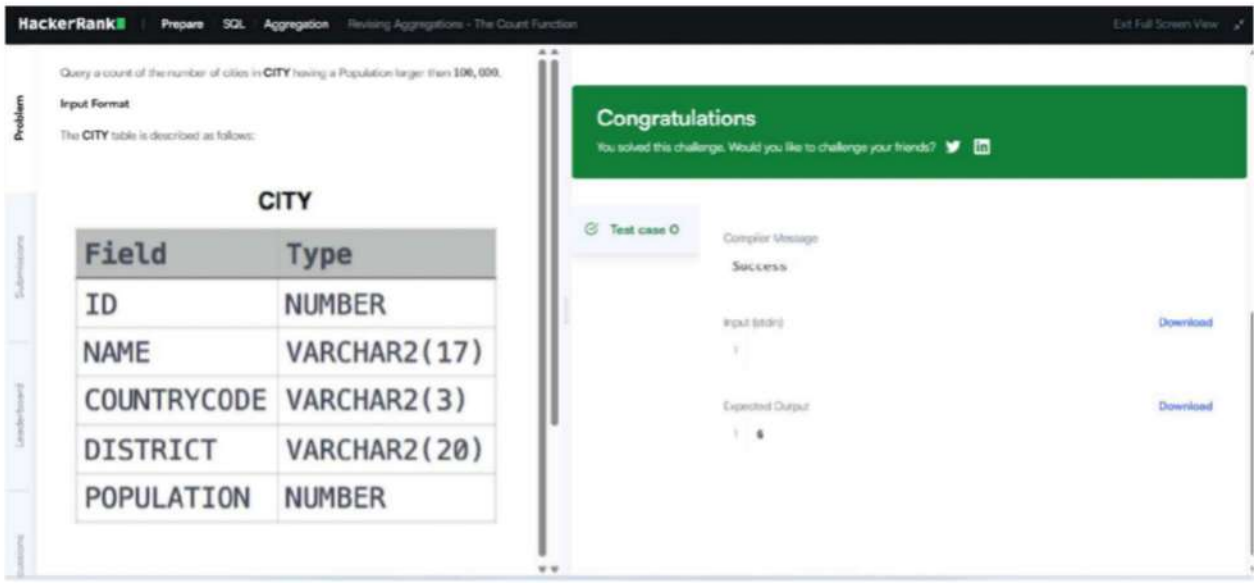
Format

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT COUNT(*) FROM CITY WHERE POPULATION > 100000;
```



REVISING AGGREGATIONS - THE SUM FUNCTION

Query the total population of all cities in CITY where District is California. Input

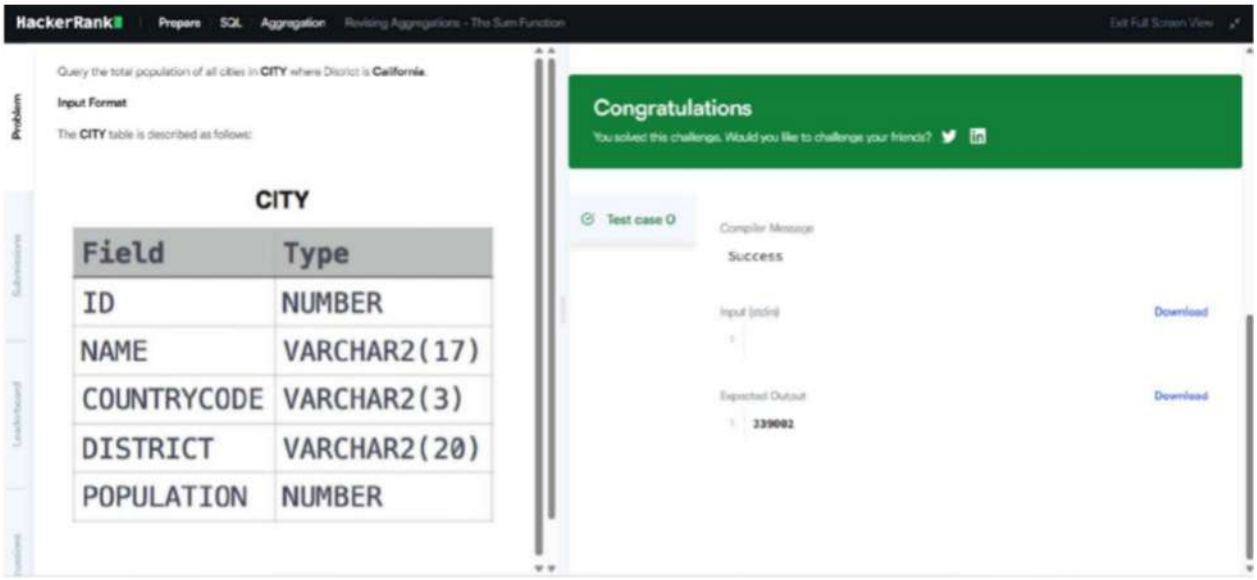
Format

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT SUM(POPULATION) FROM CITY WHERE DISTRICT = 'California';
```



REVISING AGGREGATIONS – AVERAGES

Query the average population of all cities in CITY where District is California.

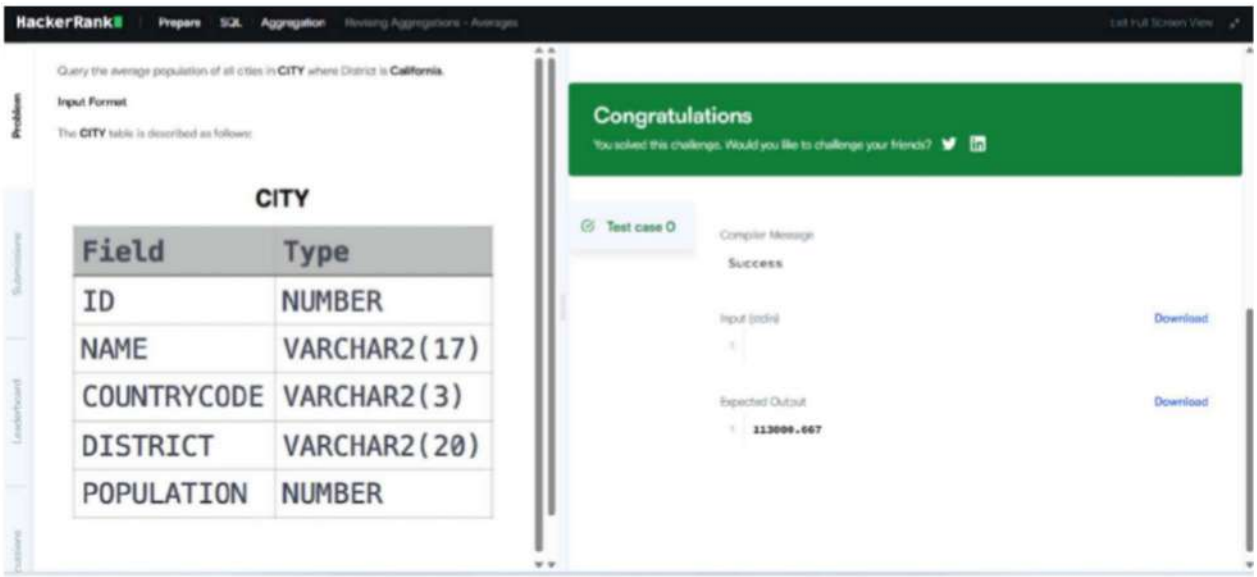
Input Format

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT AVG(POPULATION) FROM CITY WHERE DISTRICT = 'California';
```



AVERAGE POPULATION

Query the average population for all cities in CITY, rounded down to the nearest integer. Input

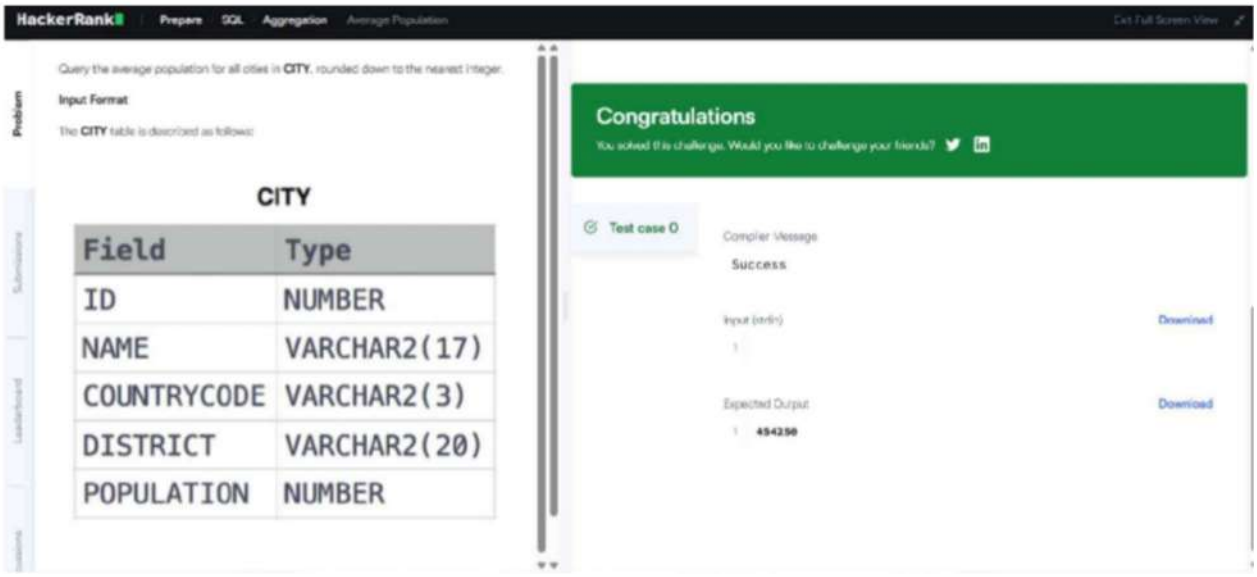
Format

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT  FLOOR (AVG(POPULATION)) FROM CITY;
```



JAPAN POPULATION

Query the sum of the populations for all Japanese cities in CITY. The COUNTRYCODE for Japan is JPN.

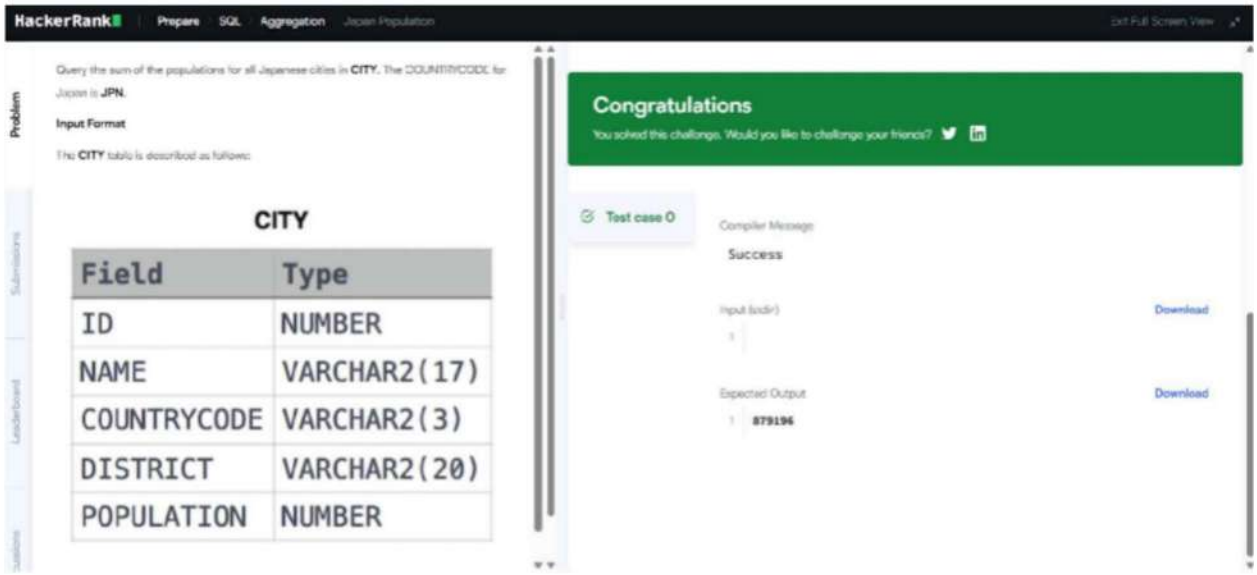
Input Format

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT SUM(POPULATION) FROM CITY WHERE COUNTRYCODE = 'JPN';
```



POPULATION DENSITY DIFFERENCE

Query the difference between the maximum and minimum populations in CITY.

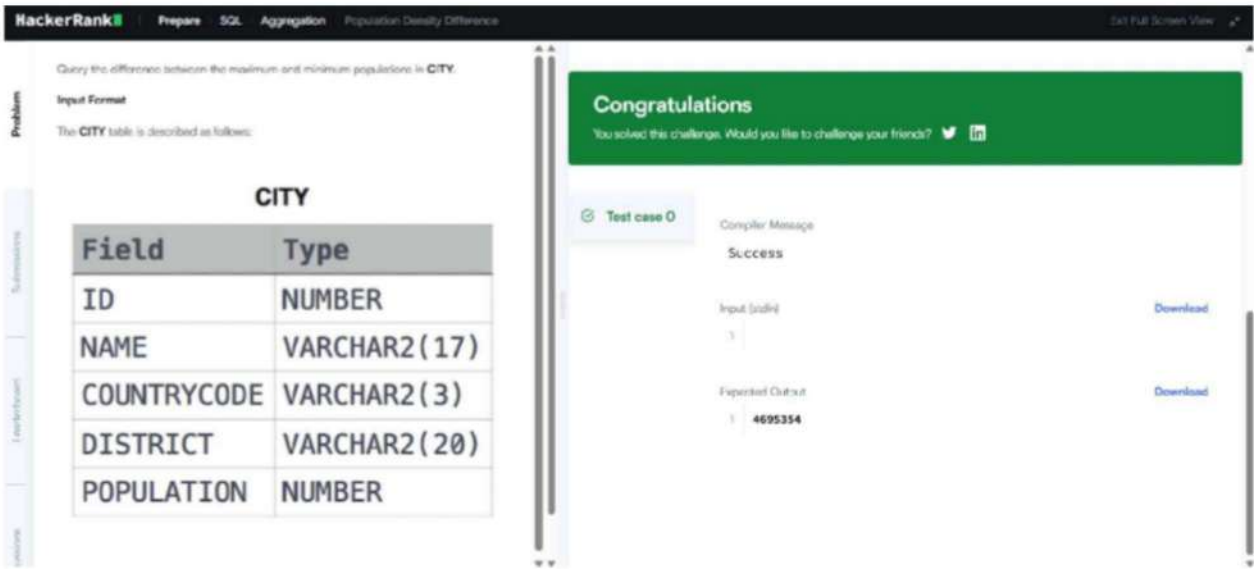
Input Format

The CITY table is described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

```
SELECT MAX(POPULATION) - MIN(POPULATION) FROM CITY;
```



THE BLUNDER

Samantha was tasked with calculating the average monthly salaries for all employees in the EMPLOYEES table, but did not realize her keyboard's key was broken until after completing the calculation. She wants your help finding the difference between her miscalculation (using salaries with any zeros removed), and the actual average salary.

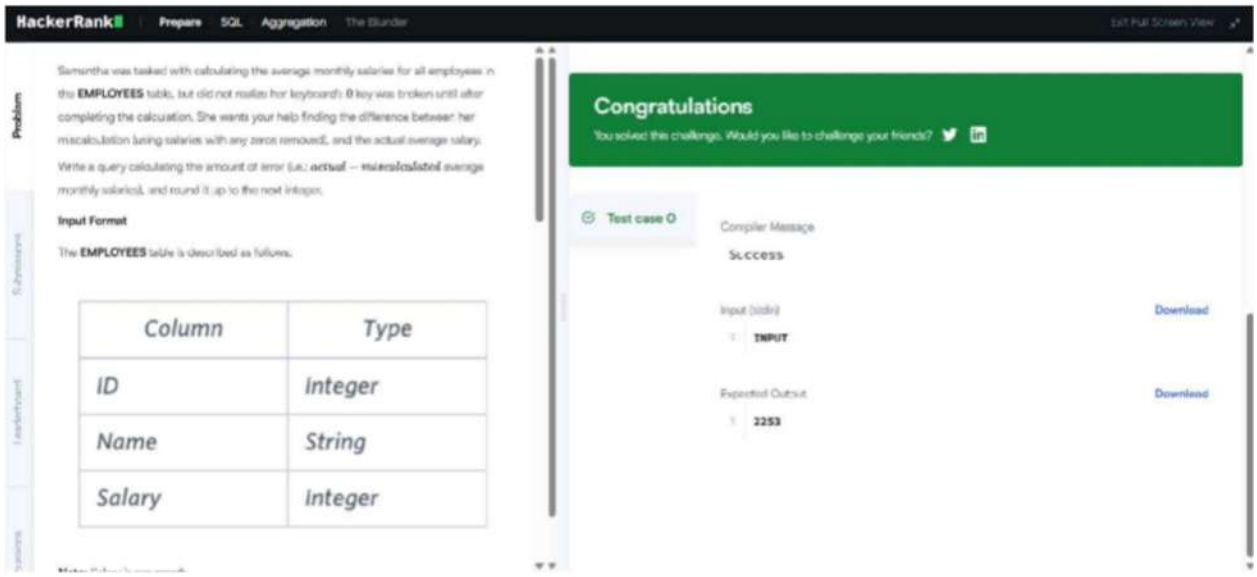
Write a query calculating the amount of error (i.e.: average monthly salaries), and round it up to the next integer.

Input Format

The EMPLOYEES table is described as follows:

Column	Type
ID	Integer
Name	String
Salary	Integer

```
SELECT CEIL( AVG(Salary) - AVG(CAST(REPLACE(Salary, '0', '') AS UNSIGNED)))FROM EMPLOYEES;
```



TOP EARNERS

We define an employee's total earnings to be their monthly worked, and the maximum total earnings to be the maximum total earnings for any employee in the Employee table. Write a query to find the maximum total earnings for all employees as well as the total number of employees who have maximum total earnings. Then print these values as space-separated integers.

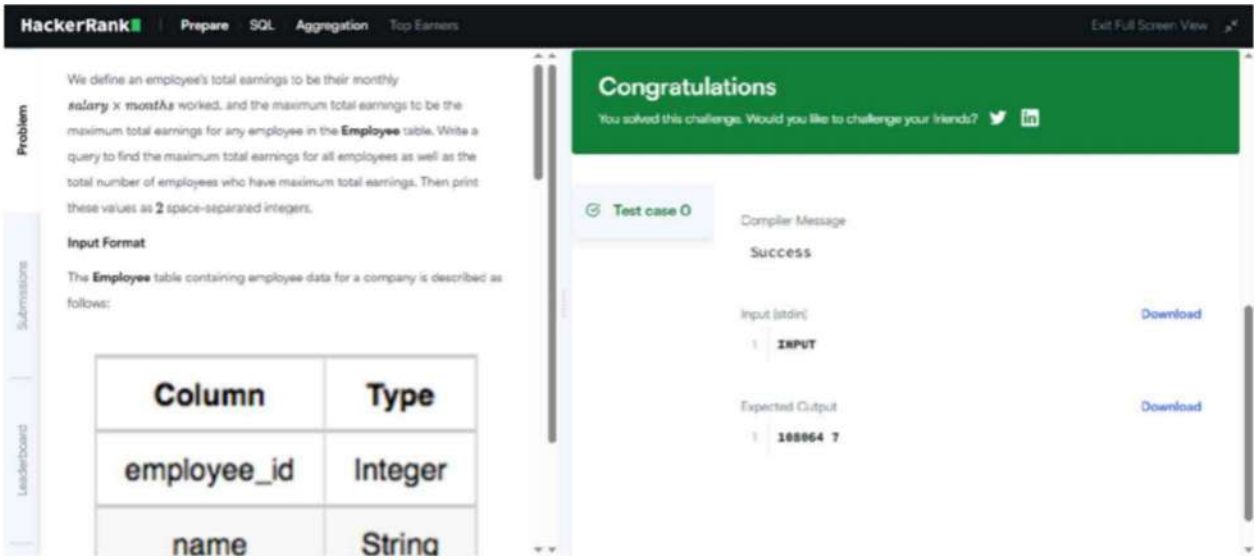
Input Format

The Employee table containing employee data for a company is described as follows:

Column	Type
employee_id	Integer
name	String
months	Integer
salary	Integer

where employee_id is an employee's ID number, name is their name, months is the total number of months they've been working for the company, and salary is the their monthly salary.

```
SELECT
    (salary * months) AS earnings,
    COUNT(*) AS total_count
FROM EMPLOYEE
GROUP BY earnings
ORDER BY earnings DESC
LIMIT 1;
```

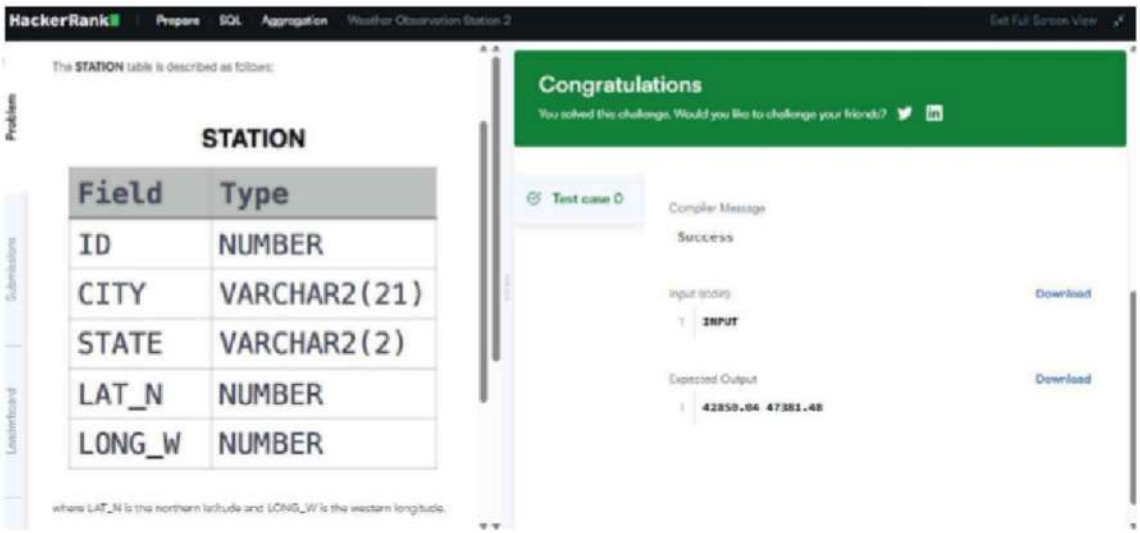

HackerRank
Prepare
SQL
Aggregation
Top Earners
Exit Full Screen View
Problem
Submissions
Leaderboard
We define an employee's total earnings to be their monthly salary x months worked, and the maximum total earnings to be the maximum total earnings for any employee in the Employee table. Write a query to find the maximum total earnings for all employees as well as the total number of employees who have maximum total earnings. Then print these values as 2 space-separated integers.
Input Format
The Employee table containing employee data for a company is described as follows:
Column
Type
employee_id
Integer
name
String
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
1 INPUT
Expected Output
Download
1 168064 7

WEATHER OBSERVATION STATION 2

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER



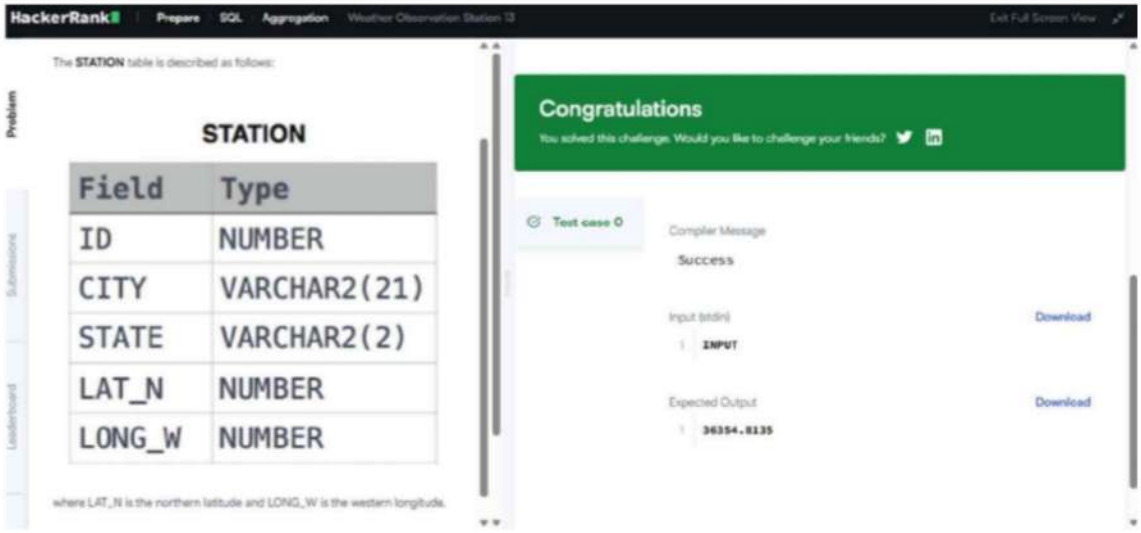
WEATHER OBSERVATION STATION 13

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



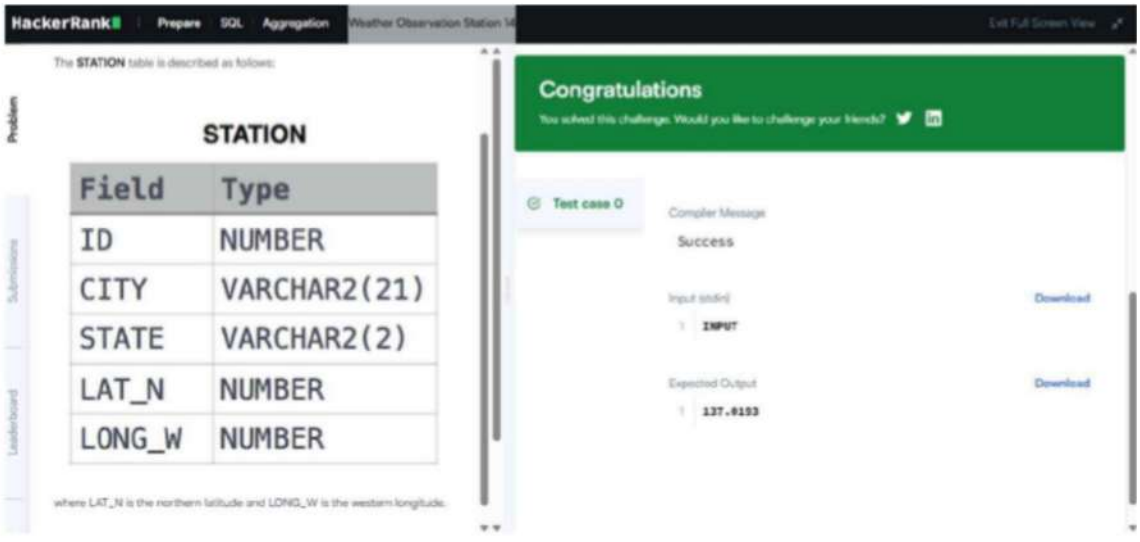
WEATHER OBSERVATION STATION 14

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



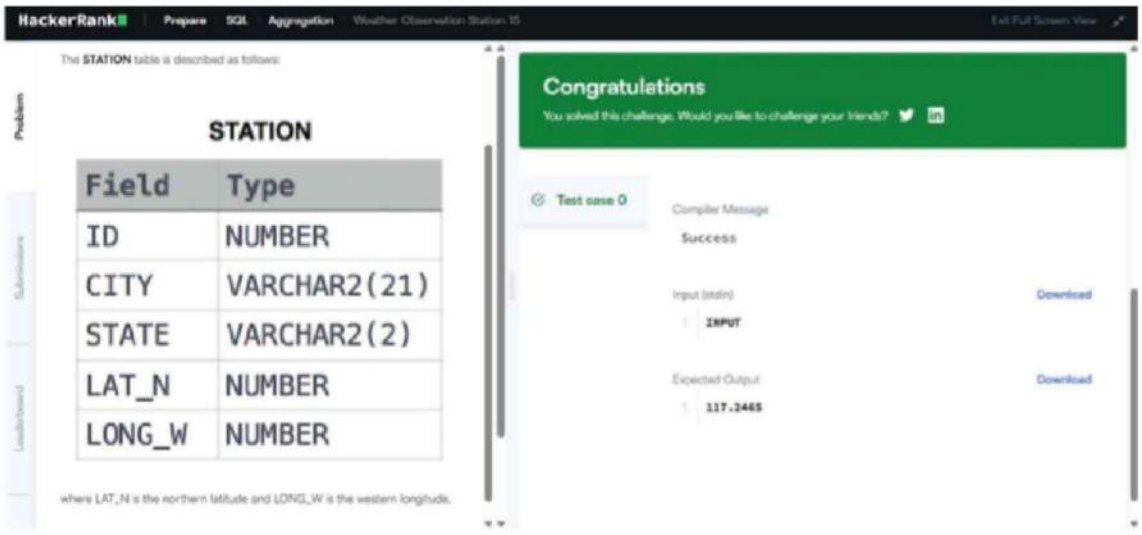
WEATHER OBSERVATION STATION 15

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



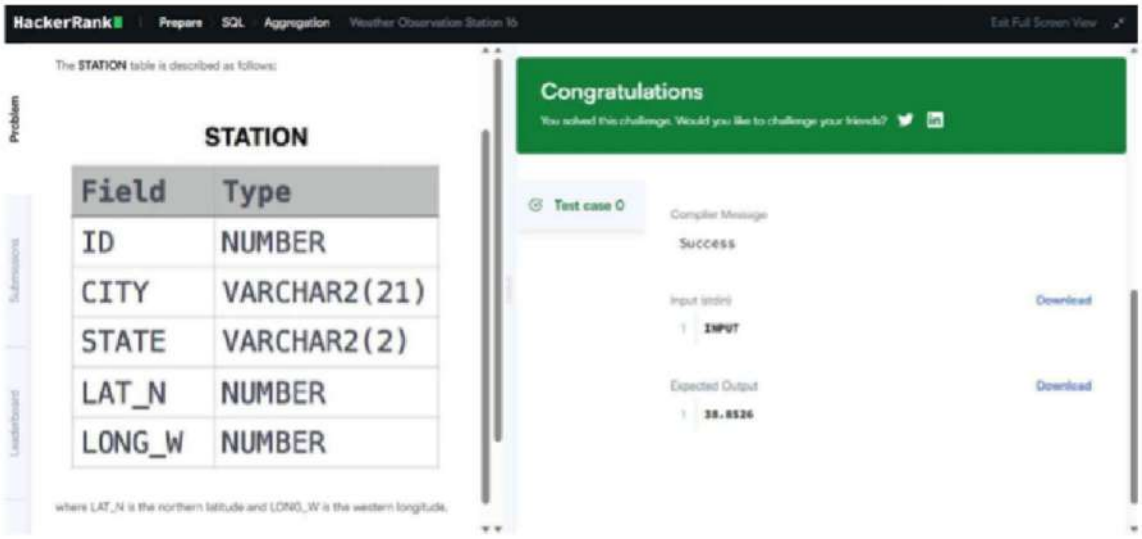
WEATHER OBSERVATION STATION 16

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



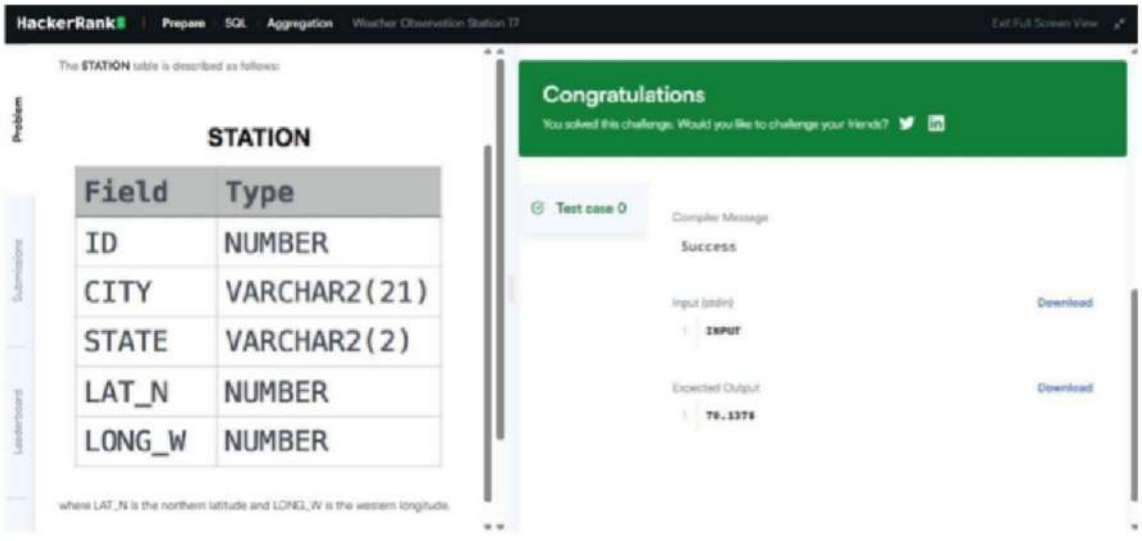
WEATHER OBSERVATION STATION 17

The **STATION** table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

where LAT_N is the northern latitude and LONG_W is the western longitude.



POPULATION CENSUS

Given the **CITY** and **COUNTRY** tables, query the sum of the populations of all cities where the CONTINENT is 'Asia'.

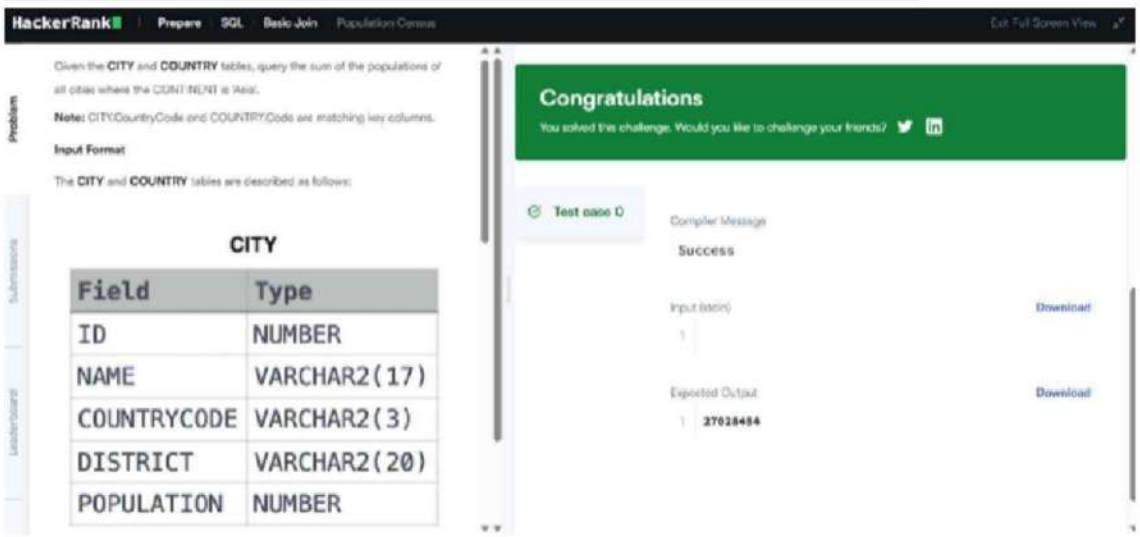
Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER



AFRICAN CITIES

Given the **CITY** and **COUNTRY** tables, query the names of all cities where the CONTINENT is 'Africa'.

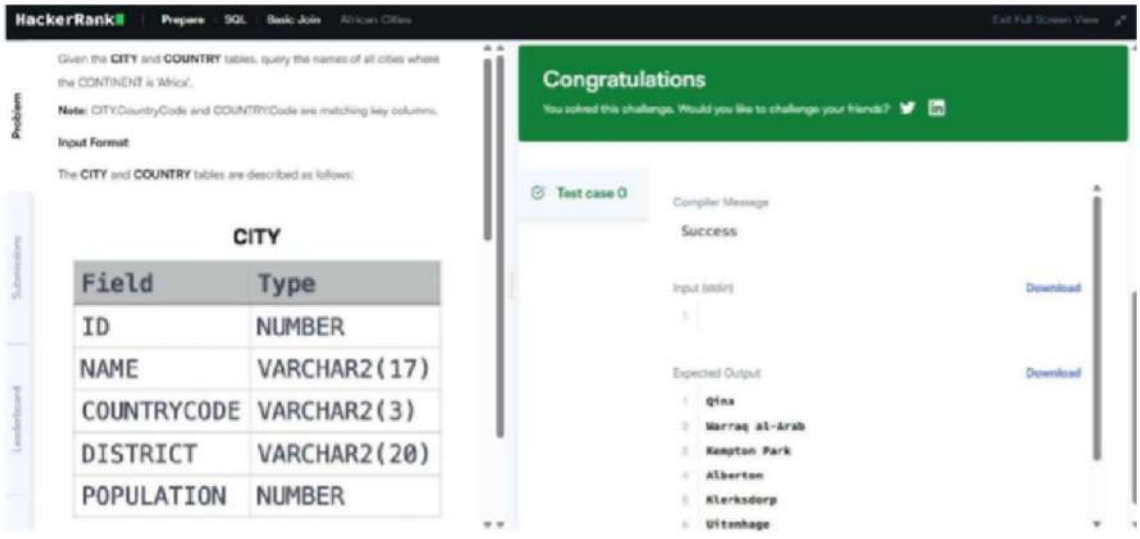
Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER



AVERAGE POPULATION OF EACH CONTINENT

Given the **CITY** and **COUNTRY** tables, query the names of all the continents (COUNTRY.Continent) and their respective average city populations (CITY.Population) rounded down to the nearest integer.

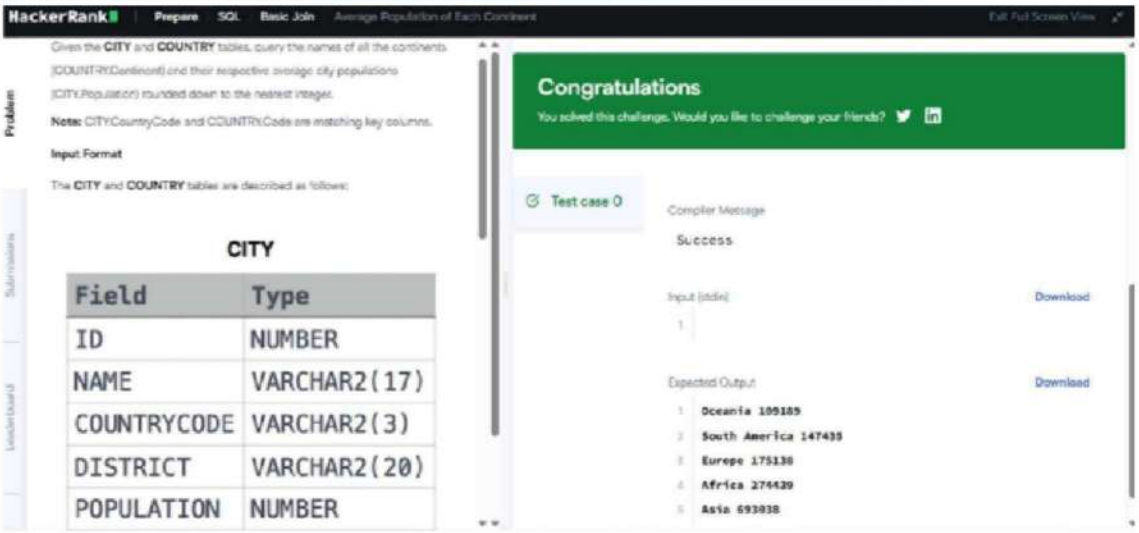
Note: CITY.CountryCode and COUNTRY.Code are matching key columns.

Input Format

The **CITY** and **COUNTRY** tables are described as follows:

CITY

Field	Type
ID	NUMBER
NAME	VARCHAR2(17)
COUNTRYCODE	VARCHAR2(3)
DISTRICT	VARCHAR2(20)
POPULATION	NUMBER

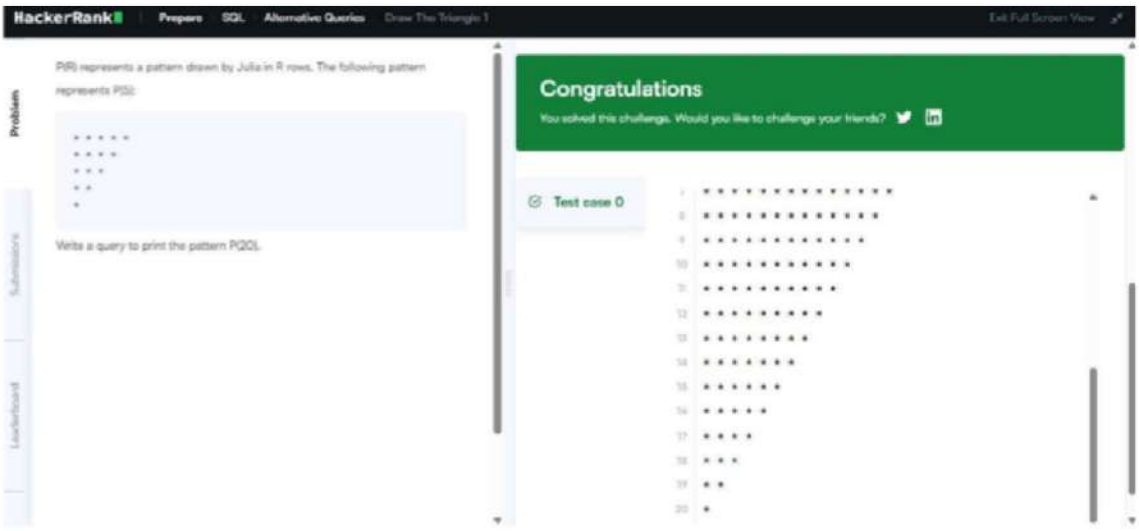


DRAW THE TRIANGLE 1

P(R) represents a pattern drawn by Julia in R rows. The following pattern represents P(5):

```
* * * * *
* * * *
* * *
* *
*
```

Write a query to print the pattern P(20).

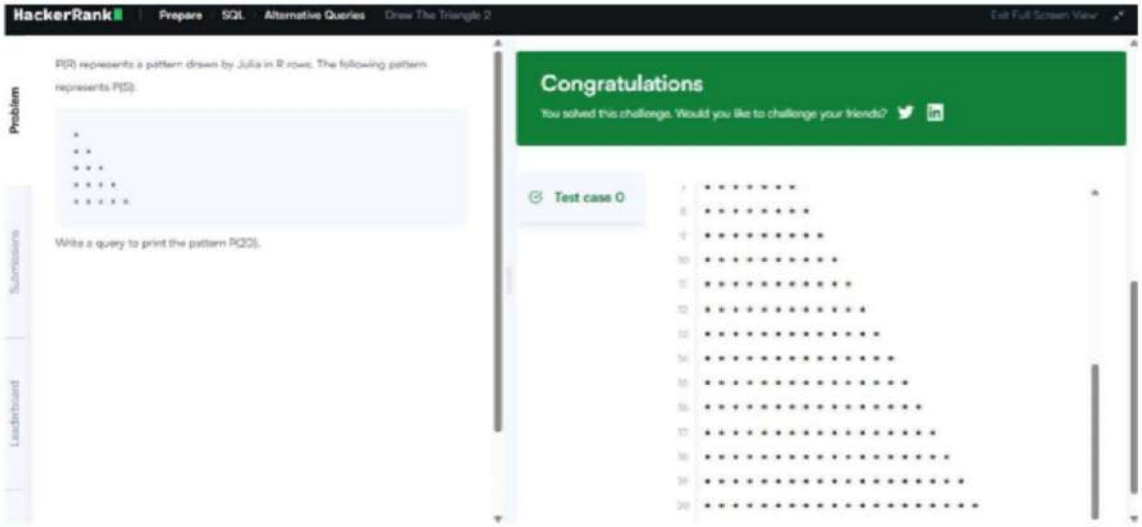


DRAW THE TRIANGLE 2

P(R) represents a pattern drawn by Julia in R rows. The following pattern represents P(5):

```
*
* *
* * *
* * * *
* * * * *
```

Write a query to print the pattern P(20).



THE PADS

Generate the following two result sets:

1. Query an alphabetically ordered list of all names in OCCUPATIONS, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: AnActorName(A), ADoctorName(D), AProfessorName(P), and ASingerName(S).

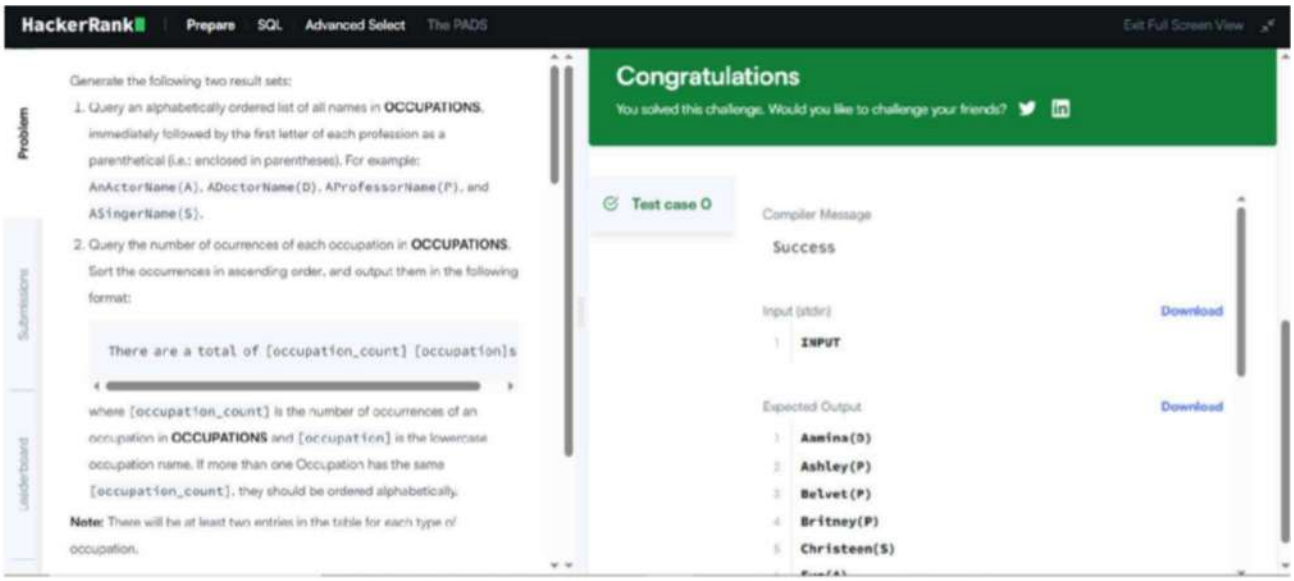
1.Query the number of ocurrences of each occupation in OCCUPATIONS. Sort the occurrences in ascending order, and output them in the following format:where [occupation_count] is the number of occurrences of an occupation in OCCUPATIONS and [occupation] is the lowercase occupation name. If more than one Occupation has the same [occupation_count], they should be ordered alphabetically.

Input Format

The OCCUPATIONS table is described as follows:

Column	Type
Name	String
Occupation	String

```
SELECT CONCAT(Name, '(', LEFT(Occupation, 1), ')')
FROM OCCUPATIONS
ORDER BY Name;
SELECT CONCAT('There are a total of ', COUNT(Occupation), ' ', LOWER(Occupation), 's.')
FROM OCCUPATIONS
GROUP BY Occupation
ORDER BY COUNT(Occupation), Occupation;
```


HackerRank
Prepare
SQL
Advanced Select
The PADS
Exit Full Screen View
Problem
Submissions
Leaderboard
Generate the following two result sets:
1. Query an alphabetically ordered list of all names in OCCUPATIONS, immediately followed by the first letter of each profession as a parenthetical (i.e.: enclosed in parentheses). For example: AnActorName(A), ADoctorName(D), AProfessorName(P), and ASingerName(S).
2. Query the number of ocurrences of each occupation in OCCUPATIONS. Sort the occurrences in ascending order, and output them in the following format:
There are a total of [occupation_count] [occupation]s
where [occupation_count] is the number of occurrences of an occupation in OCCUPATIONS and [occupation] is the lowercase occupation name. If more than one Occupation has the same [occupation_count], they should be ordered alphabetically.
Note: There will be at least two entries in the table for each type of occupation.
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
INPUT
Expected Output
Download
Aamina(D)
Ashley(P)
Belvet(P)
Britney(P)
Christeen(S)

OCCUPATIONS

The Occupation column in OCCUPATIONS so that each Name is sorted alphabetically and displayed underneath its corresponding Occupation. The output should consist of four columns (Doctor, Professor, Singer, and Actor) in that specific order, with their respective names listed alphabetically under each column.

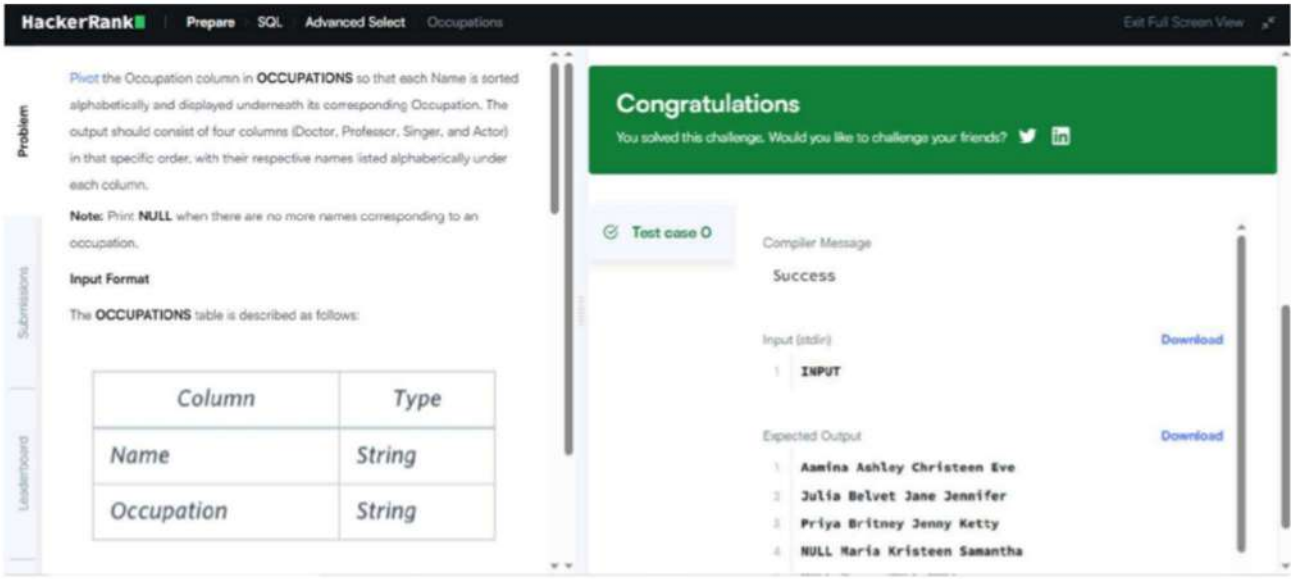
Input Format

The OCCUPATIONS table is described as follows:

Column	Type
Name	String
Occupation	String

```
SET @r1=0, @r2=0, @r3=0, @r4=0;
SELECT MIN(Doctor), MIN(Professor), MIN(Singer), MIN(Actor)
FROM (
   SELECT
      CASE
         WHEN Occupation = 'Doctor' THEN (@r1 := @r1 + 1)
         WHEN Occupation = 'Professor' THEN (@r2 := @r2 + 1)
         WHEN Occupation = 'Singer' THEN (@r3 := @r3 + 1)
         WHEN Occupation = 'Actor' THEN (@r4 := @r4 + 1)
      END AS RowNum,
      CASE WHEN Occupation = 'Doctor' THEN Name END AS Doctor,
      CASE WHEN Occupation = 'Professor' THEN Name END AS Professor,
```

```
    CASE WHEN Occupation = 'Singer' THEN Name END AS Singer,
    CASE WHEN Occupation = 'Actor' THEN Name END AS Actor
  FROM OCCUPATIONS
  ORDER BY Name
) AS Temp
GROUP BY RowNum;
```



BINARY TREE NODES

You are given a table, BST, containing two columns: N and P, where N represents the value of a node in Binary Tree, and P is the parent of N

Column	Type
N	Integer
P	Integer

.Write a query to find the node type of Binary Tree ordered by the value of the node. Output one of the following for each node:

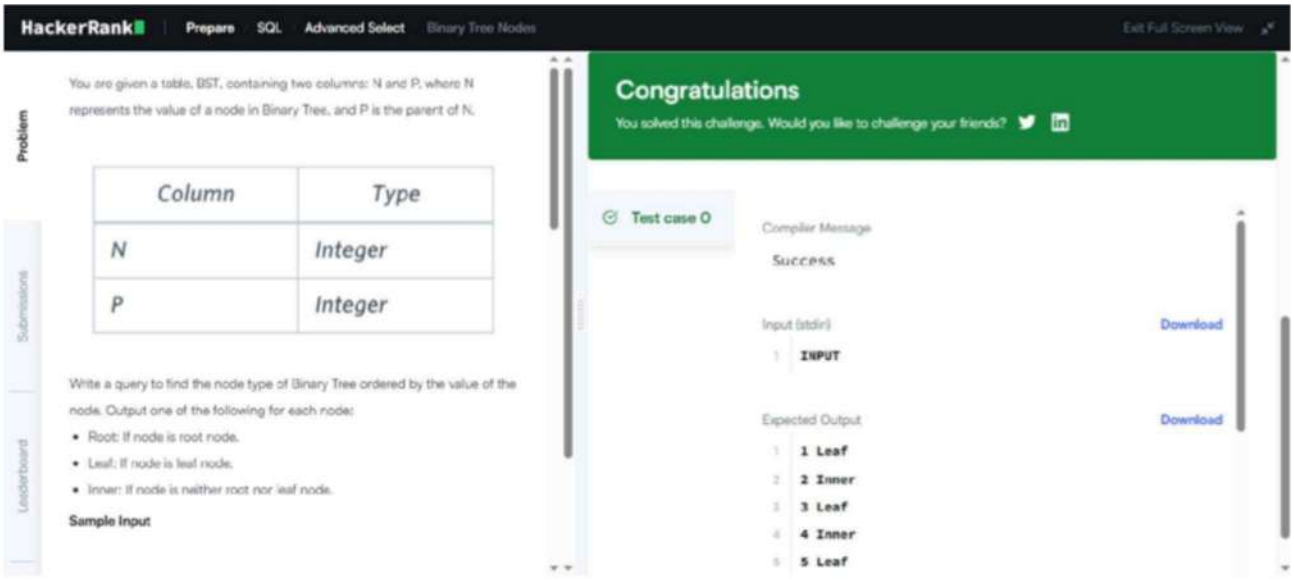
- Root: If node is root node.
- Leaf: If node is leaf node.
- Inner: If node is neither root nor leaf node.

Sample Input

N	P
1	2
3	2
6	8
9	8
2	5
8	5
5	null

SELECT N, CASE WHEN P IS NULL THEN 'Root' WHEN N IN (SELECT P FROM BST)

THEN 'Inner' ELSE 'Leaf' END FROM BST ORDER BY N;



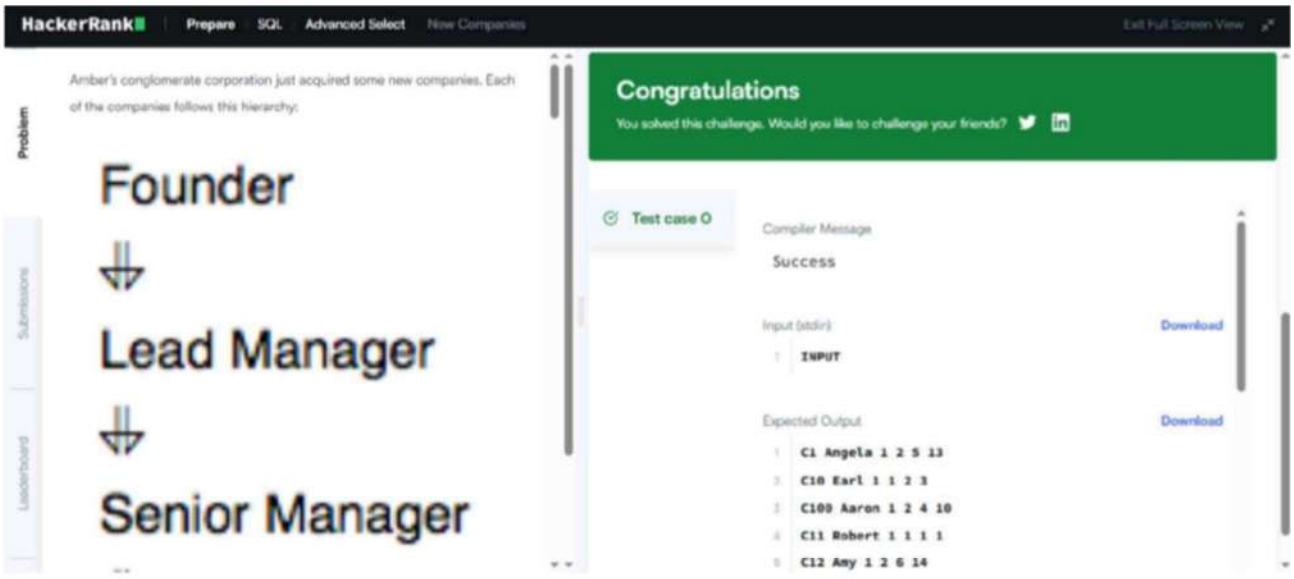
NEW COMPANIES

Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy:



Given the table schemas below, write a query to print the company_code, founder name, total number of lead managers, total number of senior managers, total number of managers, and total number of employees. Order your output by ascending company_code.

```
SELECT c.company_code, c.founder, COUNT(DISTINCT e.lead_manager_code),
COUNT(DISTINCT e.senior_manager_code), COUNT(DISTINCT e.manager_code),
COUNT(DISTINCT e.employee_code) FROM Company c LEFT JOIN Employee e ON
c.company_code = e.company_code GROUP BY c.company_code, c.founder ORDER BY
c.company_code;
```


HackerRank
Prepare
SQL
Advanced Select
New Companies
Exit Full Screen View
Problem
Submissions
Leaderboard
Amber's conglomerate corporation just acquired some new companies. Each of the companies follows this hierarchy:
Founder
Lead Manager
Senior Manager
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
1 INPUT
Expected Output
Download
1 C1 Angela 1 2 5 13
2 C10 Earl 1 1 2 3
3 C100 Aaron 1 2 4 10
4 C11 Robert 1 1 1 1
5 C12 Amy 1 2 6 14

WEATHER ONSERVATION STATION 18

Consider p1(a,b) and p2(c,d)to be two points on a 2D plane.

- A happens to equal the minimum value in Northern Latitude (LAT_N in STATION).
- B happens to equal the minimum value in Western Longitude (LONG_W in STATION).
- C happens to equal the maximum value in Northern Latitude (LAT_N in STATION).
- D happens to equal the maximum value in Western Longitude (LONG_W in STATION).

Query the Manhattan Distance between points and and round it to a scale of decimal places.

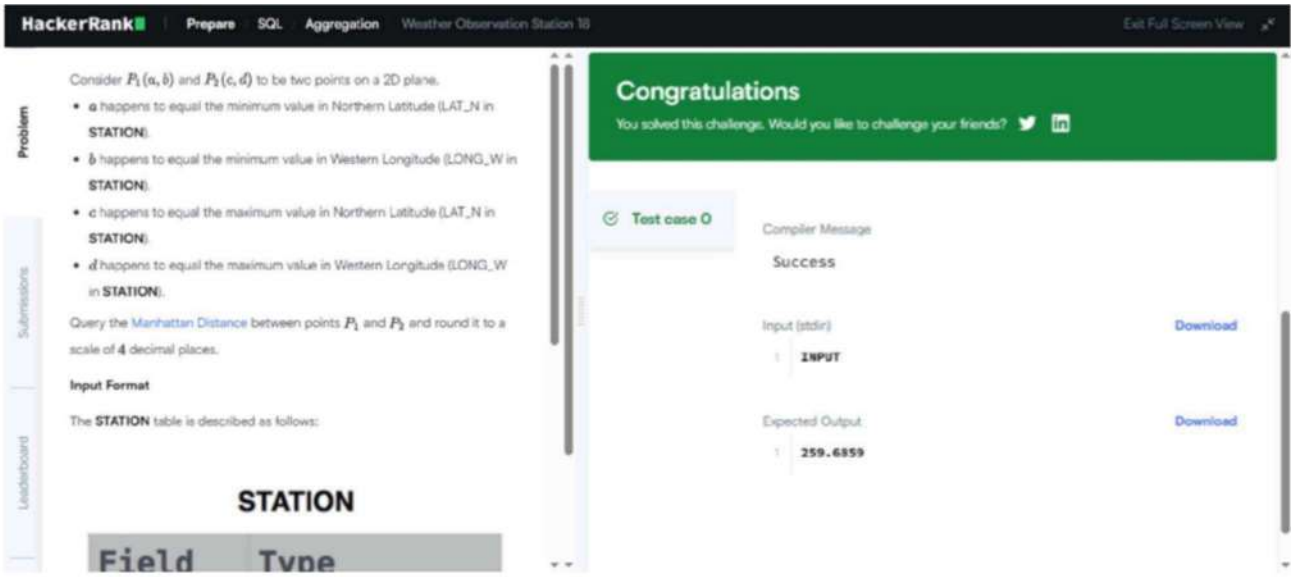
Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

SELECT ROUND(ABS(MIN(LAT_N) - MAX(LAT_N)) + ABS(MIN(LONG_W) - MAX(LONG_W)), 4) FROM STATION;


HackerRank
Prepare
SQL
Aggregation
Weather Observation Station 18
Exit Full Screen View
Problem
Submissions
Leaderboard
Consider $P_1(a, b)$ and $P_2(c, d)$ to be two points on a 2D plane.
a happens to equal the minimum value in Northern Latitude (LAT_N in STATION).
b happens to equal the minimum value in Western Longitude (LONG_W in STATION).
c happens to equal the maximum value in Northern Latitude (LAT_N in STATION).
d happens to equal the maximum value in Western Longitude (LONG_W in STATION).
Query the Manhattan Distance between points P_1 and P_2 and round it to a scale of 4 decimal places.
Input Format
The STATION table is described as follows:
STATION
Field
Type
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
1 INPUT
Expected Output
Download
1 259.6859

WEATHER ONSERVATION STATION 19

Consider ranto be two points on a 2D plane where Are the respective minimum and maximum values of Northern Latitude (LAT_N) and are the respective minimum and maximum values of Western Longitude (LONG_W) in STATION.

Query the between point and ormat your answer to display decimal digits.

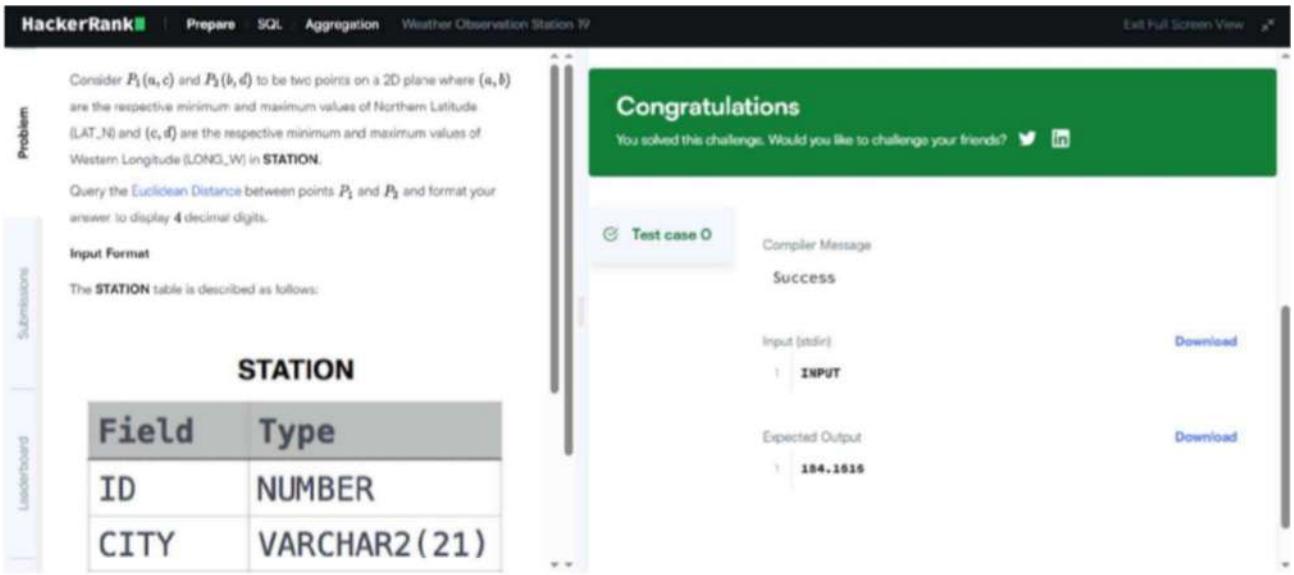
Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

SELECT ROUND(SQRT(POW(MIN(LAT_N) - MAX(LAT_N), 2) + POW(MIN(LONG_W) - MAX(LONG_W), 2)), 4) FROM STATION;


HackerRank
Prepare
SQL
Aggregation
Weather Observation Station 19
Exit Full Screen View
Problem
Submissions
Leaderboard
Consider $P_1(a, c)$ and $P_2(b, d)$ to be two points on a 2D plane where (a, b) are the respective minimum and maximum values of Northern Latitude (LAT_N) and (c, d) are the respective minimum and maximum values of Western Longitude (LONG_W) in STATION.
Query the Euclidean Distance between points P_1 and P_2 and format your answer to display 4 decimal digits.
Input Format
The STATION table is described as follows:
STATION
Field
Type
ID
NUMBER
CITY
VARCHAR2(21)
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
1 INPUT
Expected Output
Download
1 184.1615

WEATHER ONSERVATION STATION 20

A median is defined as a number separating the higher half of a data set from the lower half.

Query the median of the Northern Latitudes from STATION and round your answer to decimal places.

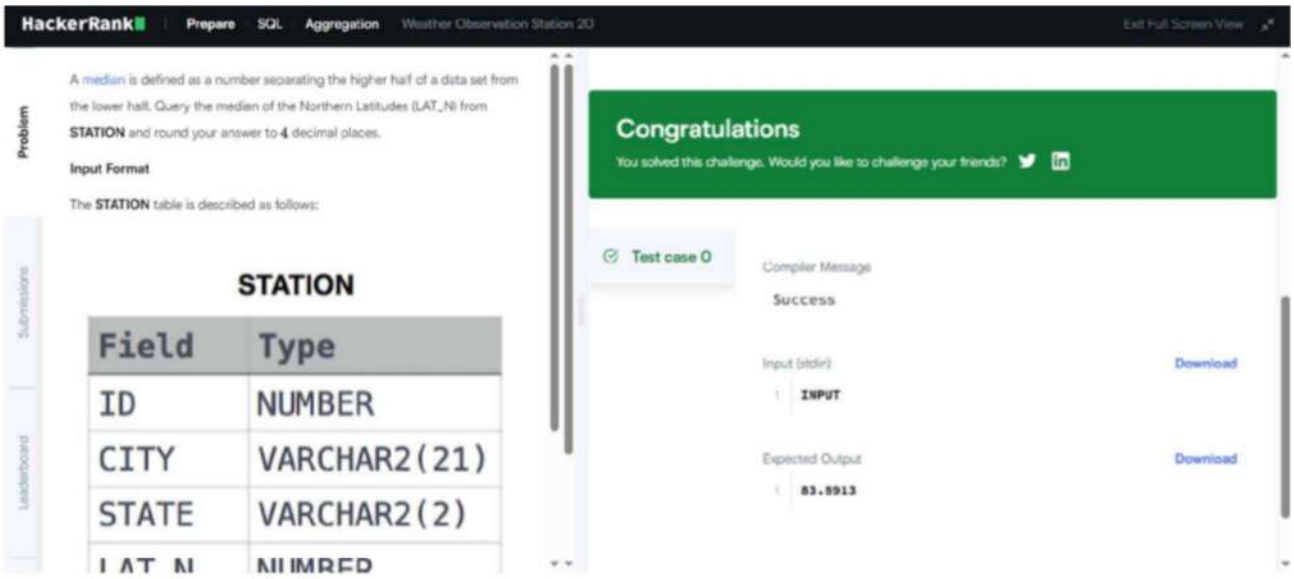
Input Format

The STATION table is described as follows:

STATION

Field	Type
ID	NUMBER
CITY	VARCHAR2(21)
STATE	VARCHAR2(2)
LAT_N	NUMBER
LONG_W	NUMBER

```
SELECT ROUND(AVG(LAT_N), 4)
FROM (
    SELECT LAT_N,
        ROW_NUMBER() OVER (ORDER BY LAT_N) AS RowNum,
        COUNT(*) OVER () AS TotalRows
    FROM STATION
) AS RankedStation
WHERE RowNum IN (FLOOR((TotalRows + 1) / 2), CEIL((TotalRows + 1) / 2));
```


HackerRank
Prepare
SQL
Aggregation
Weather Observation Station 20
Exit Full Screen View
Problem
A median is defined as a number separating the higher half of a data set from the lower half. Query the median of the Northern Latitudes (LAT_N) from STATION and round your answer to 4 decimal places.
Input Format
The STATION table is described as follows:
STATION
Field
Type
ID
NUMBER
CITY
VARCHAR2(21)
STATE
VARCHAR2(2)
Submissions
Leaderboard
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
INPUT
Expected Output
Download
83.8913

THE REPORT

You are given two tables: Students and Grades. Students contains three columns ID, Name and Marks.

Column	Type
ID	Integer
Name	String
Marks	Integer

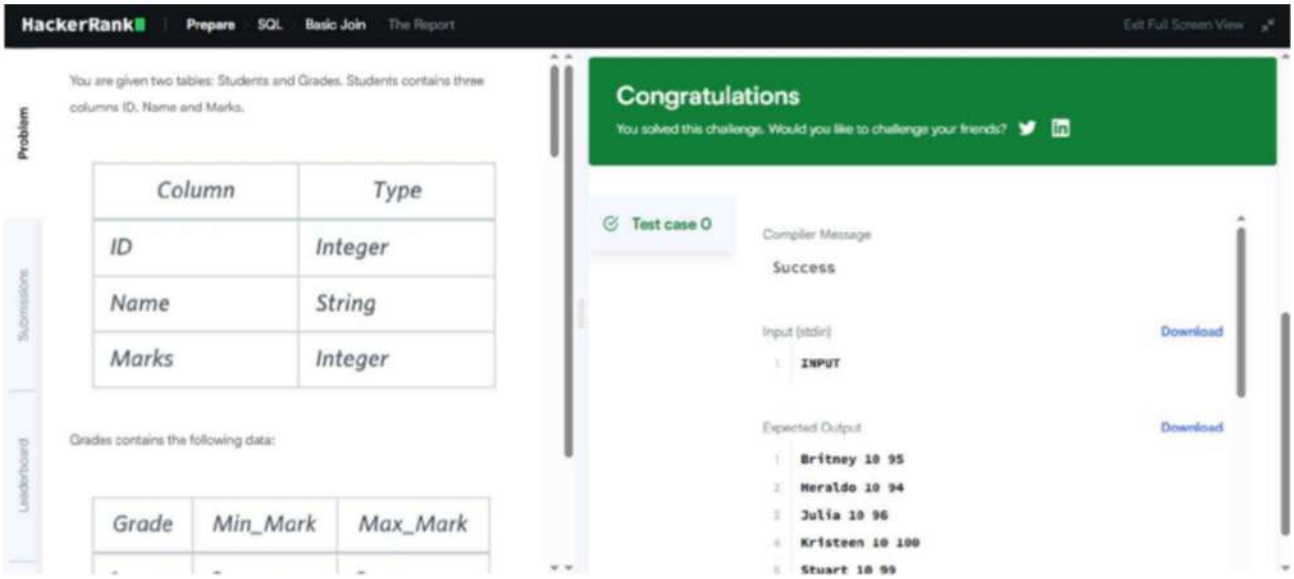
Grades contains the following data:

Grade	Min_Mark	Max_Mark
1	0	9
2	10	19
3	20	29
4	30	39
5	40	49
6	50	59
7	60	69
8	70	79
9	80	89
10	90	100

Ketty gives Eve a task to generate a report containing three columns: Name, Grade and Mark. Ketty doesn't want the NAMES of those students who received a grade lower than 8. The report must be in descenading order by grade -- i.e. higher grades are entered first. If there is more than one student with the same grade (8-10) assigned to them, order those particular students by their name alphabetically. Finally, if the grade is lower than 8, use "NULL" as their name and list them

by their grades in descending order. If there is more than one student with the same grade (1-7) assigned to them, order those particular students by their marks in ascending order.

```
SELECT
    CASE WHEN g.Grade >= 8 THEN s.Name ELSE NULL END AS Name,
g.Grade,
    s.Marks
FROM Students s
JOIN Grades g
  ON s.Marks BETWEEN g.Min_Mark AND g.Max_Mark
ORDER BY
    g.Grade DESC,
    CASE WHEN g.Grade >= 8 THEN s.Name END ASC,
    CASE WHEN g.Grade < 8 THEN s.Marks END ASC;
```



TOP COMPETITORS

Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective hacker_id and name of hackers who achieved full scores for more than one challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending hacker_id.

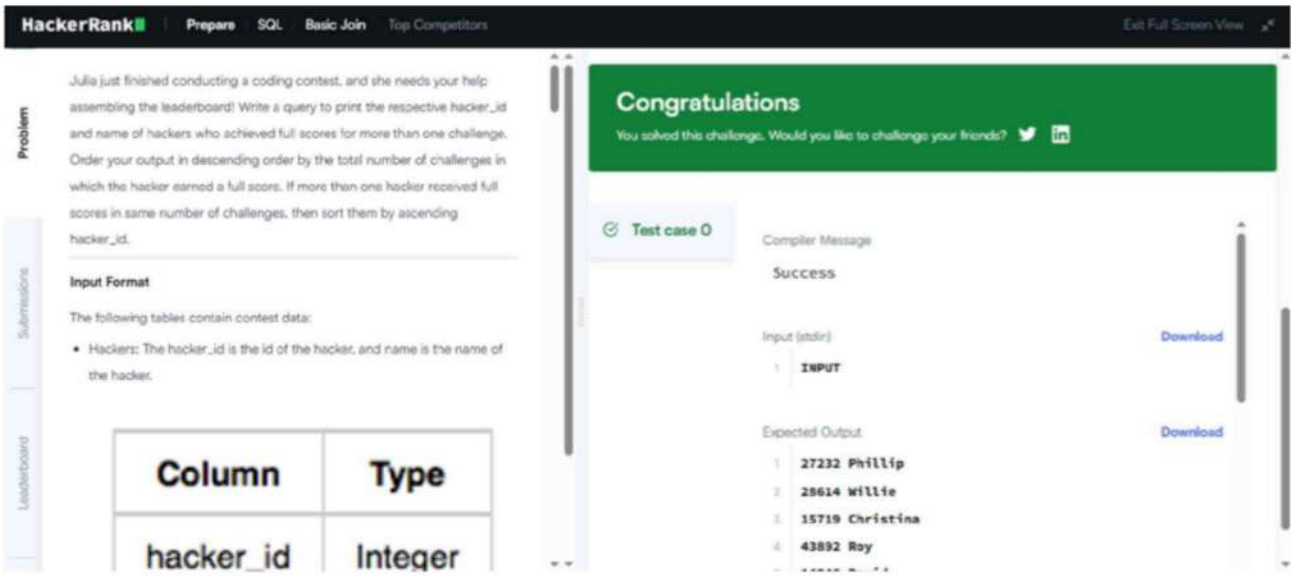
Input Format

The following tables contain contest data:

- Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

```
SELECT h.hacker_id, h.name
FROM Submissions s
JOIN Challenges c ON s.challenge_id = c.challenge_id
JOIN Difficulty d ON c.difficulty_level = d.difficulty_level
JOIN Hackers h ON s.hacker_id = h.hacker_id
WHERE s.score = d.score
GROUP BY h.hacker_id, h.name
HAVING COUNT(s.challenge_id) > 1
ORDER BY COUNT(s.challenge_id) DESC, h.hacker_id ASC;
```


HackerRank
Prepare
SQL
Basic Join
Top Competitors
Exit Full Screen View
Problem
Submissions
Leaderboard
Julia just finished conducting a coding contest, and she needs your help assembling the leaderboard! Write a query to print the respective hacker_id and name of hackers who achieved full scores for more than one challenge. Order your output in descending order by the total number of challenges in which the hacker earned a full score. If more than one hacker received full scores in same number of challenges, then sort them by ascending hacker_id.
Input Format
The following tables contain contest data:
Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.
Column
Type
hacker_id
Integer
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
1 INPUT
Expected Output
Download
1 27232 Phillip
2 28614 Willie
3 15719 Christina
4 43892 Roy

OLLIVANDER'S INVENTORY

Harry Potter and his friends are at Ollivander's with Ron, finally replacing Charlie's old broken wand.

Hermione decides the best way to choose is by determining the minimum number of gold galleons needed to buy each non-evil wand of high power and age. Write a query to print the id, age, coins_needed, and power of the wands that Ron's interested in, sorted in order of descending power. If more than one wand has same power, sort the result in order of descending age.

Input Format

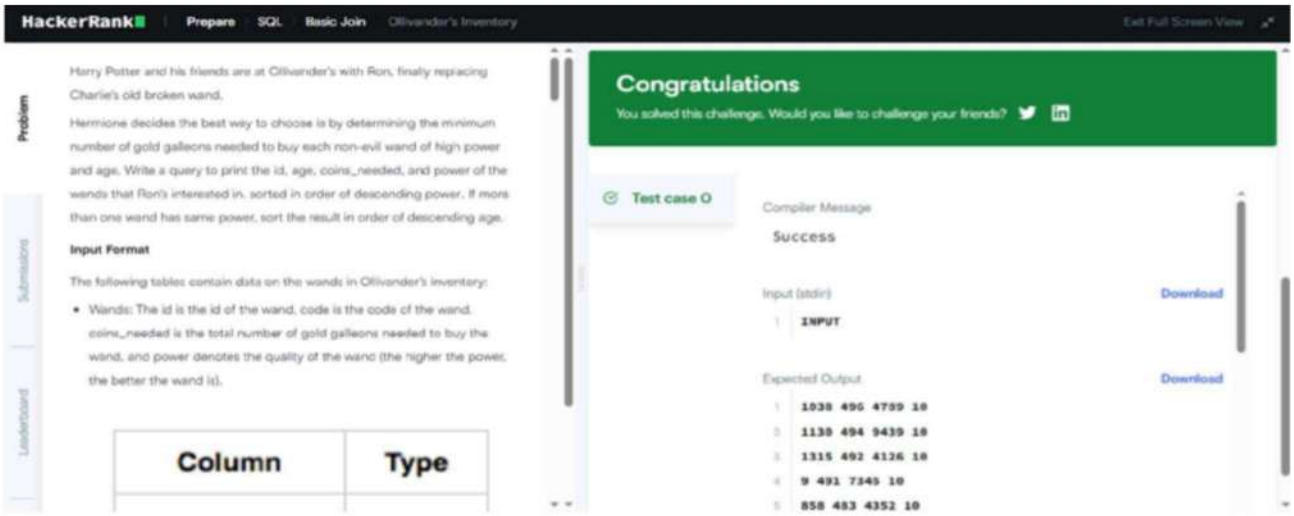
The following tables contain data on the wands in Ollivander's inventory:

- Wands: The id is the id of the wand, code is the code of the wand, coins_needed is the total number of gold galleons needed to buy the wand, and power denotes the quality of the wand (the higher the power, the better the wand is).

Column	Type
id	Integer
code	Integer
coins_needed	Integer
power	Integer

```
SELECT w.id, p.age, w.coins_needed, w.power
FROM Wands AS w
JOIN Wands_Property AS p
ON w.code = p.code
WHERE p.is_evil = 0
  AND w.coins_needed = (
```

```
    SELECT MIN(x.coins_needed)
    FROM Wands AS x
    JOIN Wands_Property AS y
      ON x.code = y.code
    WHERE x.power = w.power
      AND y.age = p.age
)
ORDER BY w.power DESC, p.age DESC;
```



CHALLENGES

Julia asked her students to create some coding challenges. Write a query to print the hacker_id, name, and the total number of challenges created by each student. Sort your results by the total number of challenges in descending order. If more than one student created the same number of challenges, then sort the result by hacker_id. If more than one student created the same number of challenges and the count is less than the maximum number of challenges created, then exclude those students from the result. Input Format

The following tables contain challenge data:

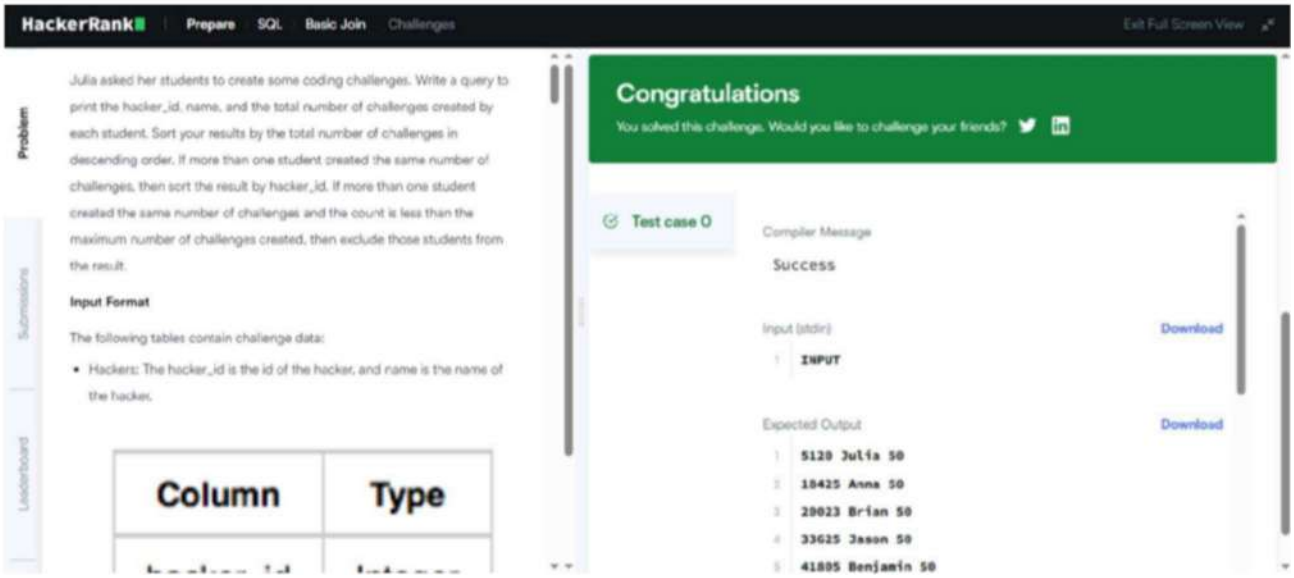
- Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

```
WITH ChallengeCounts AS (
    SELECT
        h.hacker_id,
        h.name,
        COUNT(c.challenge_id) AS total_challenges
    FROM Hackers h
    JOIN Challenges c ON h.hacker_id = c.hacker_id
    GROUP BY h.hacker_id, h.name
)

SELECT hacker_id, name, total_challenges
FROM ChallengeCounts
WHERE total_challenges = (
```

```
    /* Condition 1: Keep it if it matches the maximum challenges created by any hacker */
SELECT MAX(total_challenges) FROM ChallengeCounts
)
OR total_challenges IN (
    /* Condition 2: Keep it if the challenge count is completely unique */
    SELECT total_challenges
    FROM ChallengeCounts
    GROUP BY total_challenges
    HAVING COUNT(total_challenges) = 1
)
ORDER BY total_challenges DESC, hacker_id ASC;
```



CONTEST LEADERBOARD

You did such a great job helping Julia with her last coding contest challenge that she wants you to work on this one, too!

The total score of a hacker is the sum of their maximum scores for all of the challenges. Write a query to print the hacker_id, name, and total score of the hackers ordered by the descending score. If more than one hacker achieved the same total score, then sort the result by ascending hacker_id. Exclude all hackers with a total score of from your result.

Input Format

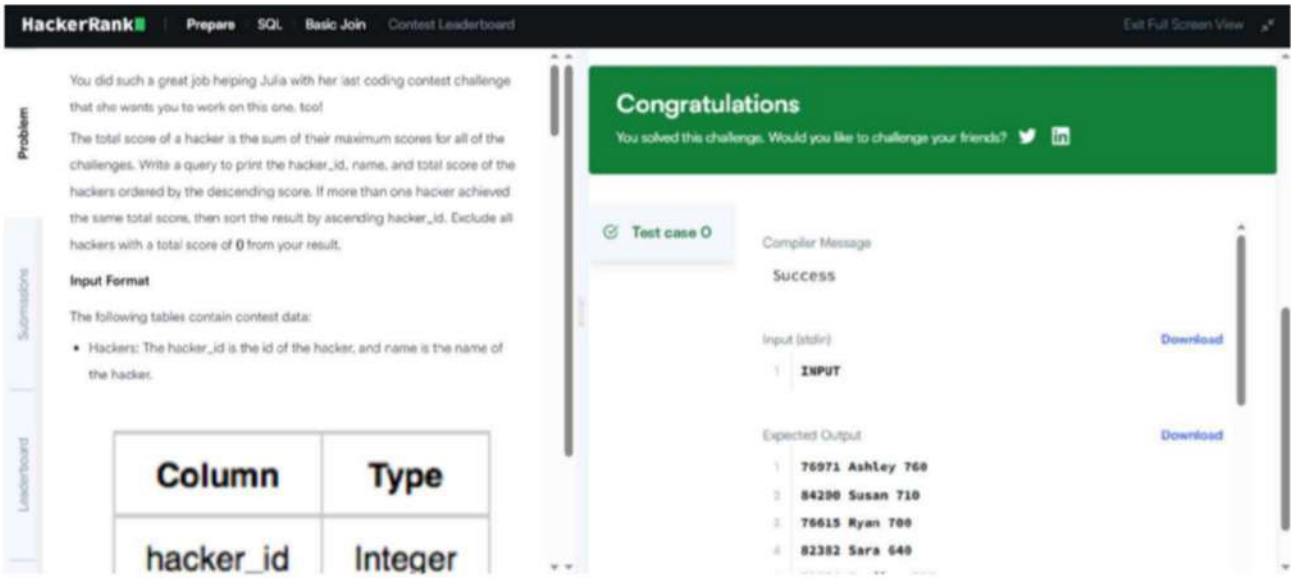
The following tables contain contest data:

- Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

```
SELECT h.hacker_id, h.name, SUM(t.max_score) AS total_score
FROM Hackers h
JOIN (
    SELECT hacker_id, challenge_id, MAX(score) AS max_score
    FROM Submissions
    GROUP BY hacker_id, challenge_id
) t ON h.hacker_id = t.hacker_id
GROUP BY h.hacker_id, h.name
HAVING total_score > 0
```

ORDER BY total_score DESC, h.hacker_id ASC;



SQL PROJECT PLANNING

You are given a table, Projects, containing three columns: Task_ID, Start_Date and End_Date. It is guaranteed that the difference between the End_Date and the Start_Date is equal to 1 day for each row in the table.

Column	Type
Task_ID	Integer
Start_Date	Date
End_Date	Date

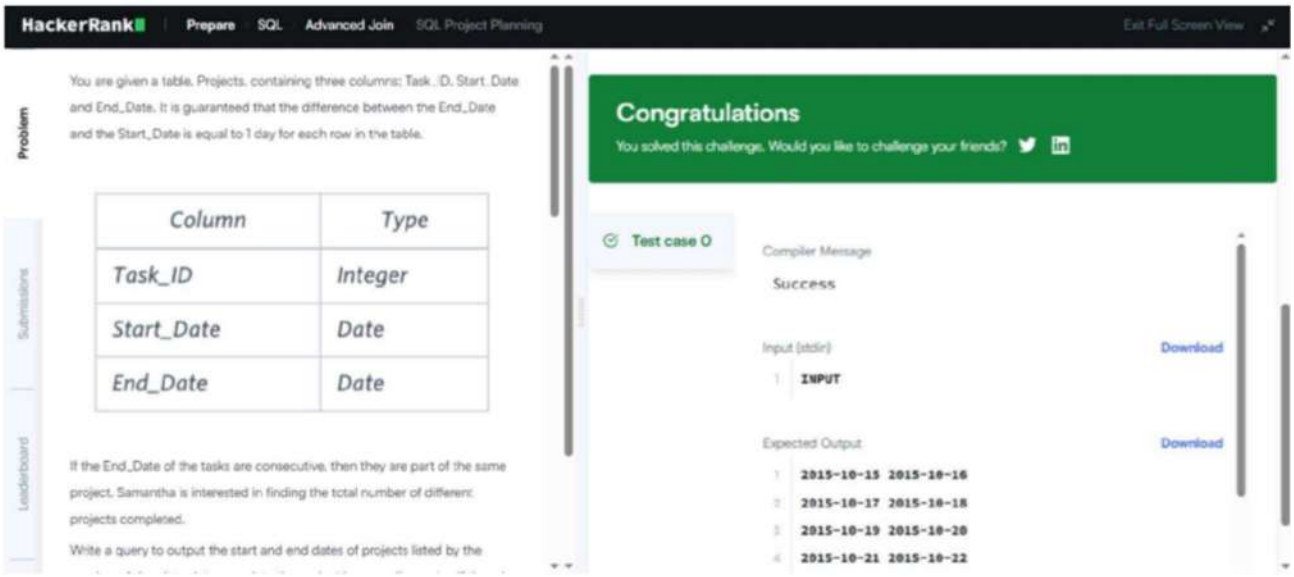
If the End_Date of the tasks are consecutive, then they are part of the same project. Samantha is interested in finding the total number of different projects completed.

Write a query to output the start and end dates of projects listed by the number of days it took to complete the project in ascending order. If there is more than one project that have the same number of completion days, then order by the start date of the project.

Sample Input

Task_ID	Start_Date	End_Date
1	2015-10-01	2015-10-02
2	2015-10-02	2015-10-03
3	2015-10-03	2015-10-04
4	2015-10-13	2015-10-14
5	2015-10-14	2015-10-15
6	2015-10-28	2015-10-29
7	2015-10-30	2015-10-31

```
SELECT A.Start_Date, MIN(B.End_Date)
FROM
    (SELECT Start_Date
    FROM Projects
    WHERE Start_Date NOT IN (SELECT End_Date FROM Projects)) AS A
    INNER JOIN
    (SELECT End_Date
    FROM Projects
    WHERE End_Date NOT IN (SELECT Start_Date FROM Projects)) AS B
WHERE A.Start_Date < B.End_Date
GROUP BY A.Start_Date
ORDER BY MIN(B.End_Date) - A.Start_Date ASC, A.Start_Date;
```



PLACEMENTS

You are given three tables: Students, Friends and Packages. Students contains two columns: ID and Name. Friends contains two columns: ID and Friend_ID (ID of the ONLY best friend). Packages contains two columns: ID and Salary (offered salary in $ thousands per month).

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

Friends

Column	Type
ID	Integer
Salary	Float

Packages

Column	Type
ID	Integer
Name	String

Students

Column	Type
ID	Integer
Friend_ID	Integer

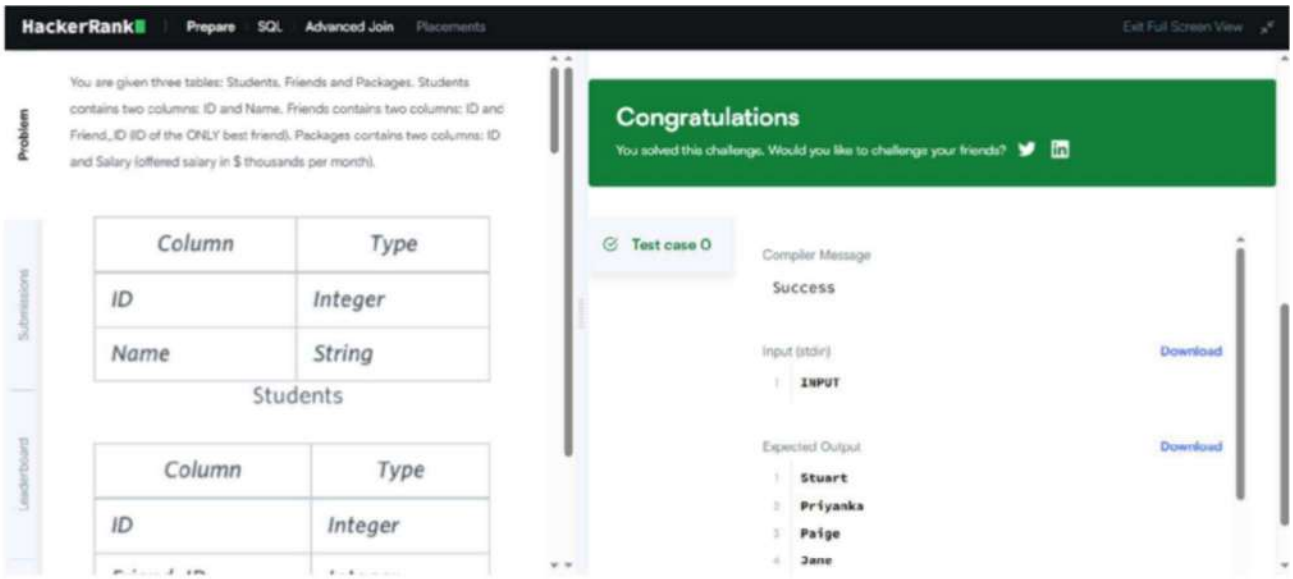
Friends

Column	Type
ID	Integer
Salary	Float

Packages

Write a query to output the names of those students whose best friends got offered a higher salary than them. Names must be ordered by the salary amount offered to the best friends. It is guaranteed that no two students got same salary offer.

```
SELECT s.Name
FROM Students s
JOIN Friends f ON s.ID = f.ID
JOIN Packages p1 ON s.ID = p1.ID
JOIN Packages p2 ON f.Friend_ID = p2.ID
WHERE p2.Salary > p1.Salary
ORDER BY p2.Salary;
```


HackerRank
Prepare
SQL
Advanced Join
Placements
Exit Full Screen View
Problem
Submissions
Leaderboard
You are given three tables: Students, Friends and Packages. Students contains two columns: ID and Name. Friends contains two columns: ID and Friend_ID (ID of the ONLY best friend). Packages contains two columns: ID and Salary (offered salary in $ thousands per month).
Column
Type
ID
Integer
Name
String
Students
Column
Type
ID
Integer
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
INPUT
Expected Output
Download
Stuart
Priyanka
Paige
Jane

SYMMETRIC PAIRS

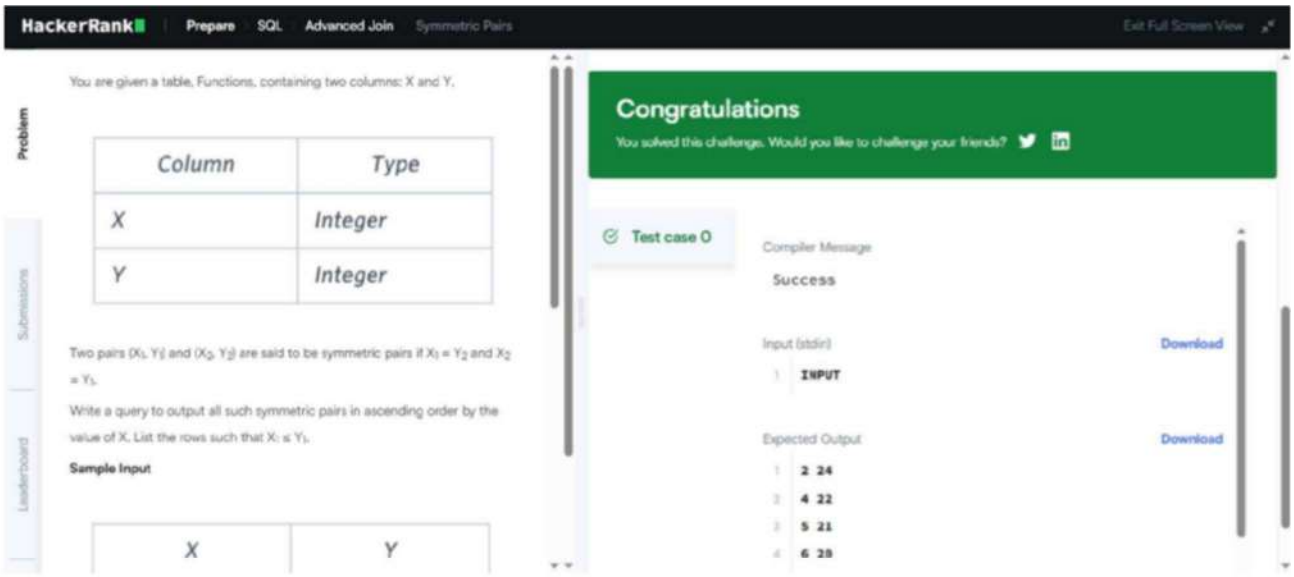
You are given a table, Functions, containing two columns: X and Y.

Column	Type
X	Integer
Y	Integer

Two pairs (X_1, Y_1) and (X_2, Y_2) are said to be symmetric pairs if $X_1 = Y_2$ and $X_2 = Y_1$.

Write a query to output all such symmetric pairs in ascending order by the value of X. List the rows such that $X_1 \leq Y_1$.

```
SELECT f1.X, f1.Y
FROM Functions f1
JOIN Functions f2
    ON f1.X = f2.Y AND f1.Y = f2.X
GROUP BY f1.X, f1.Y
HAVING f1.X < f1.Y OR COUNT(f1.X) > 1
ORDER BY f1.X;
```


HackerRank
Prepare
SQL
Advanced Join
Symmetric Pairs
Exit Full Screen View
Problem
Submissions
Leaderboard
You are given a table, Functions, containing two columns: X and Y.
Column
Type
X
Integer
Y
Integer
Two pairs (X1, Y1) and (X2, Y2) are said to be symmetric pairs if X1 = Y2 and X2 = Y1.
Write a query to output all such symmetric pairs in ascending order by the value of X. List the rows such that X1 ≤ Y1.
Sample Input
X
Y
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
1 INPUT
Expected Output
Download
1 2 24
3 4 22
3 5 21
4 6 20

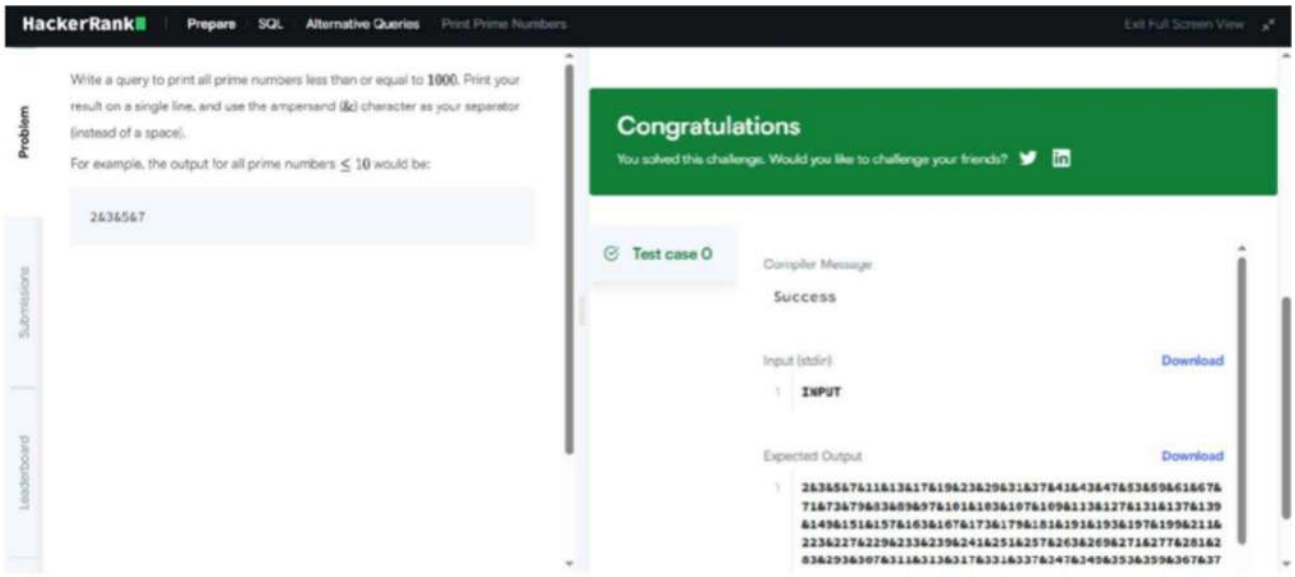
PRINT PRIME NUMBERS

Write a query to print all prime numbers less than or equal to . Print your result on a single line, and use the ampersand () character as your separator (instead of a space).

For example, the output for all prime numbers would be:

```
2&3&5&7
```

```
WITH RECURSIVE Numbers AS (
    SELECT 2 AS n
    UNION ALL
    SELECT n + 1 FROM Numbers WHERE n < 1000
)
SELECT GROUP_CONCAT(n SEPARATOR '&')
FROM Numbers n1
WHERE NOT EXISTS (
    SELECT 1
    FROM Numbers n2
    WHERE n2.n < n1.n AND n1.n % n2.n = 0
);
```


HackerRank
Prepare
SQL
Alternative Queries
Print Prime Numbers
Exit Full Screen View
Problem
Write a query to print all prime numbers less than or equal to 1000. Print your result on a single line, and use the ampersand (&) character as your separator (instead of a space).
For example, the output for all prime numbers ≤ 10 would be:
2&3&5&7
Submissions
Leaderboard
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
1 INPUT
Expected Output
Download
1 2&3&5&7&11&13&17&19&23&29&31&37&41&43&47&53&59&61&67&
71&73&79&83&89&97&101&103&107&109&113&127&131&137&139
&149&151&157&163&167&173&179&181&191&193&197&199&211&
223&227&229&233&239&241&251&257&263&269&271&277&281&2
83&293&307&311&313&317&331&337&347&349&353&359&367&37

INTERVIEWS

Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the contest_id, hacker_id, name, and the sums of total_submissions, total_accepted_submissions, total_views, and total_unique_views for each contest sorted by contest_id.

Exclude the contest from the result if all four sums are .

Note: A specific contest can be used to screen candidates at more than one college, but each college only holds screening contest.

Input Format

The following tables hold interview data:

- Contests: The contest_id is the id of the contest, hacker_id is the id of the hacker who created the contest, and name is the name of the hacker.

Column	Type
contest_id	Integer
hacker_id	Integer
name	String

- Colleges: The college_id is the id of the college, and contest_id is the id of the contest that Samantha used to screen the candidates.

Column	Type
college_id	Integer
contest_id	Integer

- Challenges: The challenge_id is the id of the challenge that belongs to one of the contests whose contest_id Samantha forgot, and college_id is the id of the college where the challenge was given to candidates.

Column	Type
challenge_id	Integer
college_id	Integer

- View_Stats: The challenge_id is the id of the challenge, total_views is the number of times the challenge was viewed by candidates, and total_unique_views is the number of times the challenge was viewed by unique candidates.

Column	Type
challenge_id	Integer
total_views	Integer
total_unique_views	Integer

- Submission_Stats: The challenge_id is the id of the challenge, total_submissions is the number of submissions for the challenge, and total_accepted_submission is the number of submissions that achieved full scores.

Column	Type
challenge_id	Integer
total_submissions	Integer
total_accepted_submissions	Integer

Contests Table:

contest_id	hacker_id	name
66406	17973	Rose
66556	79153	Angela
94828	80275	Frank

Colleges Table:

college_id	contest_id
11219	66406
32473	66556
56685	94828

Challenges Table:

challenge_id	college_id
18765	11219
47127	11219
60292	32473
72974	56685

View_Stats Table:

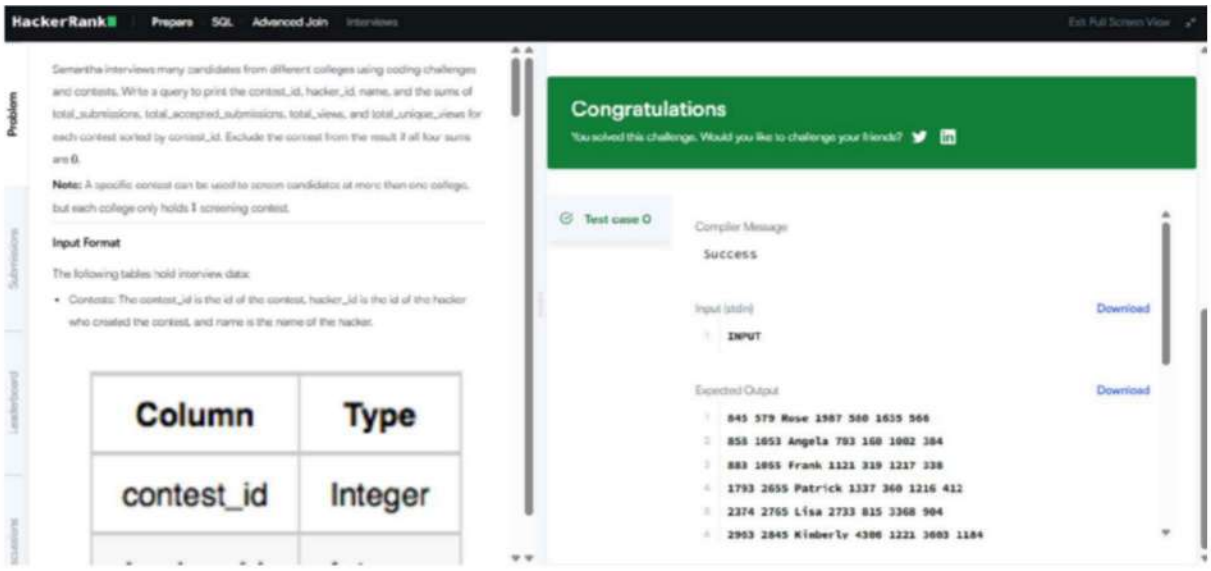
challenge_id	total_views	total_unique_views
47127	26	19
47127	15	14
18765	43	10
18765	72	13
75516	35	17
60292	11	10
72974	41	15
75516	75	11

Submission_Stats Table:

challenge_id	total_submissions	total_accepted_submissions
75516	34	12
47127	27	10
47127	56	18
75516	74	12
75516	83	8
72974	68	24
72974	82	14
47127	28	11

```
SELECT
    c.contest_id,
    c.hacker_id,
    c.name,
    SUM(COALESCE(s.total_submissions, 0)) AS total_submissions,
    SUM(COALESCE(s.total_accepted_submissions, 0)) AS total_accepted_submissions,
    SUM(COALESCE(v.total_views, 0)) AS total_views,
    SUM(COALESCE(v.total_unique_views, 0)) AS total_unique_views
FROM Contests c
JOIN Colleges col
    ON c.contest_id = col.contest_id
JOIN Challenges ch
    ON col.college_id = ch.college_id
LEFT JOIN (
    SELECT
        challenge_id,
        SUM(total_submissions) AS total_submissions,
```

```
        SUM(total_accepted_submissions) AS total_accepted_submissions
    FROM Submission_Stats
    GROUP BY challenge_id
) s
ON ch.challenge_id = s.challenge_id
LEFT JOIN
(     SELECT
        challenge_id,
        SUM(total_views) AS total_views,
        SUM(total_unique_views) AS total_unique_views
    FROM View_Stats
    GROUP BY challenge_id
) v
ON ch.challenge_id = v.challenge_id
GROUP BY
    c.contest_id,
    c.hacker_id,
    c.name
HAVING
    SUM(COALESCE(s.total_submissions, 0)) +
    SUM(COALESCE(s.total_accepted_submissions, 0)) +
    SUM(COALESCE(v.total_views, 0)) +
    SUM(COALESCE(v.total_unique_views, 0)) > 0
ORDER BY
    c.contest_id;
```


HackerRank
Prepare
SQL
Advanced Join
Interviews
Exit Full Screen View
Problem
Submissions
Leaderboard
Samantha interviews many candidates from different colleges using coding challenges and contests. Write a query to print the contest_id, hacker_id, name, and the sums of total_submissions, total_accepted_submissions, total_views, and total_unique_views for each contest sorted by contest_id. Exclude the contest from the result if all four sums are 0.
Note: A specific contest can be used to screen candidates at more than one college, but each college only holds 1 screening contest.
Input Format
The following tables hold interview data:
Contests: The contest_id is the id of the contest, hacker_id is the id of the hacker who created the contest, and name is the name of the hacker.
Column
Type
contest_id
Integer
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
INPUT
Expected Output
Download
845 579 Rose 1987 500 1655 566
855 1053 Angela 793 168 1002 384
883 1055 Frank 1121 319 1217 338
1793 2655 Patrick 1337 360 1216 412
2374 2765 Lisa 2733 815 3368 904
2953 2845 Kimberly 4306 1221 3603 1184

15 DAYS OF LEARNING SQL

Julia conducted a days of learning SQL contest. The start date of the contest was March 01, 2016 and the end date was March 15, 2016.

Write a query to print total number of unique hackers who made at least submission each day (starting on the first day of the contest), and find the hacker_id and name of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest hacker_id. The query should print this information for each day of the contest, sorted by the date.

Input Format

The following tables hold contest data:

- Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.

Column	Type
hacker_id	Integer
name	String

- Submissions: The submission_date is the date of the submission, submission_id is the id of the submission, hacker_id is the id of the hacker who made the submission, and score is the score of the submission.

Column	Type
submission_date	Date
submission_id	Integer
hacker_id	Integer
score	Integer

Sample Input

For the following sample input, assume that the end date of the contest was March 06, 2016.

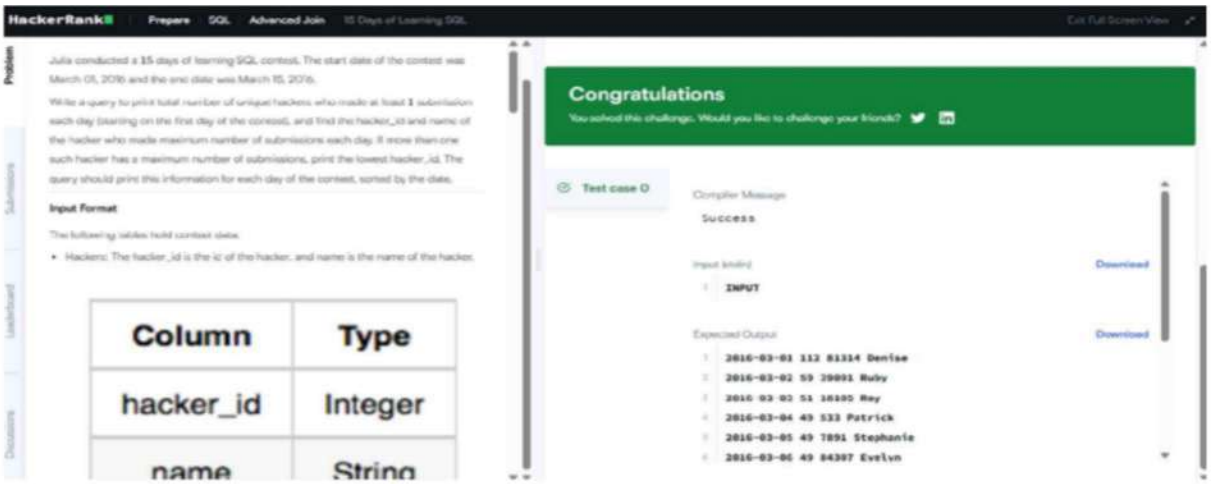
Hackers Table:

hacker_id	name
15758	Rose
20703	Angela
36396	Frank
38289	Patrick
44065	Lisa
53473	Kimberly
62529	Bonnie
79722	Michael

Submissions Table:

submission_date	submission_id	hacker_id	score
2016-03-01	8494	20703	0
2016-03-01	22403	53473	15
2016-03-01	23965	79722	60
2016-03-01	30173	36396	70
2016-03-02	34928	20703	0
2016-03-02	38740	15758	60
2016-03-02	42769	79722	25
2016-03-02	44364	79722	60
2016-03-03	45440	20703	0
2016-03-03	49050	36396	70
2016-03-03	50273	79722	5
2016-03-04	50344	20703	0
2016-03-04	51360	44065	90
2016-03-04	54404	53473	65
2016-03-04	61533	79722	45
2016-03-05	72852	20703	0
2016-03-05	74546	38289	0
2016-03-05	76487	62529	0
2016-03-05	82439	36396	10
2016-03-05	90006	36396	40
2016-03-06	90404	20703	0

```
SELECT
    s.submission_date,
    (
        SELECT COUNT(DISTINCT hacker_id)
        FROM Submissions s2
        WHERE s2.submission_date <= s.submission_date
        AND (
            SELECT COUNT(DISTINCT s3.submission_date)
            FROM Submissions s3
            WHERE s3.hacker_id = s2.hacker_id
            AND s3.submission_date <= s.submission_date
        ) = DATEDIFF(s.submission_date, '2016-03-01') + 1
    ) AS total_hackers,
    s.hacker_id,
    h.name
FROM Submissions s
JOIN Hackers h
ON s.hacker_id = h.hacker_id
WHERE s.hacker_id = (
    SELECT s1.hacker_id
    FROM Submissions s1
    WHERE s1.submission_date = s.submission_date
    GROUP BY s1.hacker_id
    ORDER BY COUNT(*) DESC, s1.hacker_id
    LIMIT 1
)
GROUP BY s.submission_date, s.hacker_id, h.name
ORDER BY s.submission_date;
```


HackerRank
Prepare
SQL
Advanced Join
Problem
Submissions
Leaderboard
Discussions
Julia conducted a 15 days of learning SQL contest. The start date of the contest was March 01, 2016 and the end date was March 15, 2016.
Write a query to print total number of unique hackers who made at least 1 submission each day (starting on the first day of the contest), and find the hacker_id and name of the hacker who made maximum number of submissions each day. If more than one such hacker has a maximum number of submissions, print the lowest hacker_id. The query should print this information for each day of the contest, sorted by the date.
Input Format
The following tables hold contest data.
Hackers: The hacker_id is the id of the hacker, and name is the name of the hacker.
Column
Type
hacker_id
Integer
name
String
Congratulations
You solved this challenge. Would you like to challenge your friends?
Test case 0
Compiler Message
Success
Input (stdin)
Download
INPUT
Expected Output
Download
2016-03-01 112 81314 Denise
2016-03-02 59 39091 Ruby
2016-03-03 51 16100 Rey
2016-03-04 49 533 Patrick
2016-03-05 49 7891 Stephanie
2016-03-06 49 84307 Evelyn

CODECHEF

BASIC COMMANDS

1. ALL PRODUCTS

Task

Write a query to output Products table.

Table name: Products

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

INPUT

select*from products;

OUTPUT

Your Output

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture

✓ Excellent work!



2. HIGH PRICE OF PRODUCTS

Task

Write a query to find all product_name and category that have a price greater than 100.00 from the **Products** table.

Table name: **Products**

product_id	product_name	price	category
1	Laptop	999.99	Electronics
2	Desk Chair	149.99	Furniture
3	Smartphone	599.99	Electronics
4	Notebook	2.99	Stationery
5	Headphones	89.99	Electronics
6	Coffee Maker	49.99	Appliances
7	Standing Desk	249.99	Furniture
8	Tablet	399.99	Electronics
9	Mouse	19.99	Electronics
10	Water Bottle	12.99	Stationery

INPUT

```
SELECT product_name, category FROM Products WHERE price > 100.00;
```

OUTPUT

Your Output

product_name	category
Laptop	Electronics
Desk Chair	Furniture
Smartphone	Electronics
Standing Desk	Furniture
Tablet	Electronics

✔ Excellent work!

3. AVERAGE SALARY

Task

- Write a query to calculate the average salary across all companies combined. Rename the column as avg_salary.

Table name: Works

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

INPUT

```
SELECT AVG(salary) AS avg_salary FROM Works;
```

OUTPUT

Your Output

avg_salary
62600.155

✔ Hooray, you did it!

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

4. LOCATE PEOPLE

Task

- Write a query to retrieve the department_name and location of departments located in a city that starts with 'S'.

Table name: **departments**

department_id	department_name	location
1	Human Resources	New York
2	Research and Development	San Francisco
3	Sales	Los Angeles
4	Marketing	New York
5	Customer Support	Boston
6	Finance	Austin
7	IT Support	Seattle
8	Product Management	San Francisco
9	Quality Assurance	Los Angeles
10	Legal	Boston

INPUT

SELECT department_name, location FROM departments WHERE location LIKE 'S%';

OUTPUT

Your Output

department_name	location
Research and Development	San Francisco
IT Support	Seattle
Product Management	San Francisco

✔ Awesome, you nailed it!

5. DISTINCT COMPANIES

Task

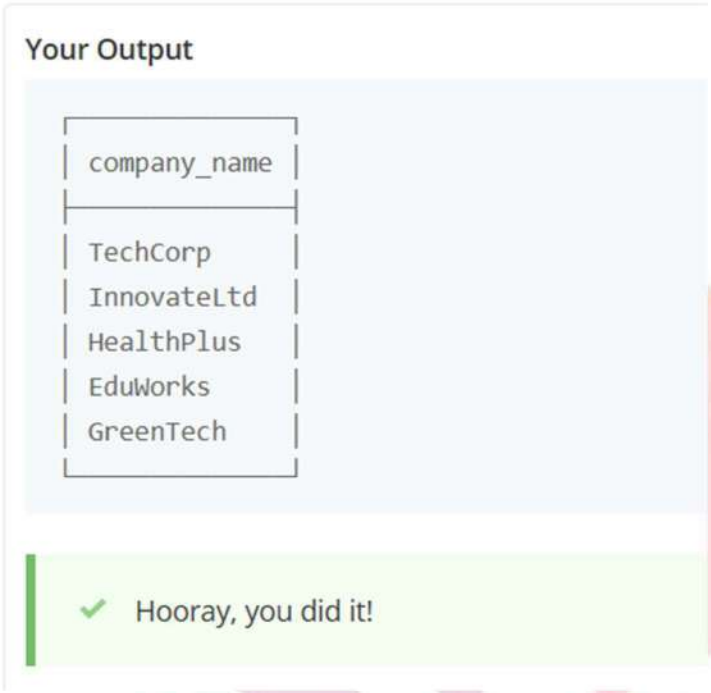
Write a query to select all the distinct companies (company_name) in the Works table.
Table name: **Works**

employee_id	company_name	salary
1	TechCorp	75000.00
2	InnovateLtd	62000.50
3	HealthPlus	54000.75
4	EduWorks	48000.00
5	GreenTech	68000.00
6	TechCorp	80000.00
7	InnovateLtd	66000.20
8	HealthPlus	50000.10
9	EduWorks	51000.00
10	GreenTech	72000.00

INPUT

```
SELECT DISTINCT company_name FROM Works;
```

OUTPUT


Your Output
company_name
TechCorp
InnovateLtd
HealthPlus
EduWorks
GreenTech
Hooray, you did it!

6. FICTION COLLECTION SIZE

Task

Write a query to find the total count of books whose genre is **Fiction**.
Note: Output column name should be **fiction_count**.

Table name: **Books**

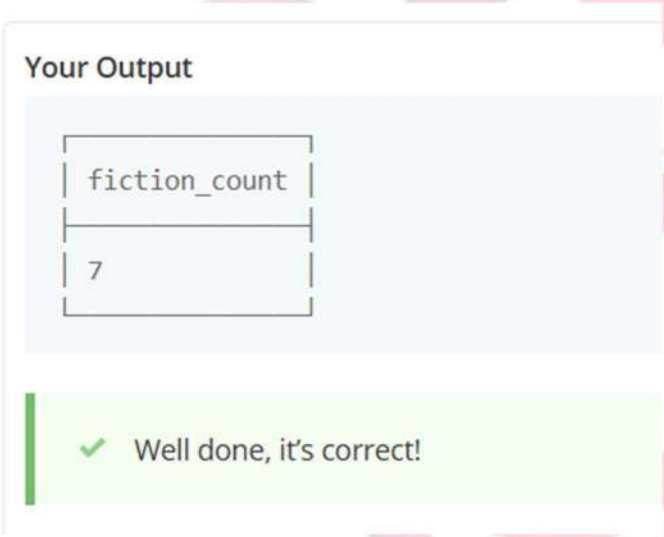
id	title	author	genre	price	published_year
1	The Great Gatsby	F. Scott Fitzgerald	Fiction	10.99	1925
2	1984	George Orwell	Dystopian	15.99	1949
3	To Kill a Mockingbird	Harper Lee	Fiction	12.99	1960
4	The Catcher in the Rye	J.D. Salinger	Fiction	14.99	1951
5	Brave New World	Aldous Huxley	Dystopian	13.99	1932
6	The Hobbit	J.R.R. Tolkien	Fantasy	9.99	1937
7	Moby Dick	Herman Melville	Fiction	18.50	1851
8	War and Peace	Leo Tolstoy	Historical	20.00	1869
9	The Picture of Dorian Gray	Oscar Wilde	Fiction	11.50	1890
10	The Alchemist	Paulo Coelho	Fiction	16.00	1988
11	Fahrenheit 451	Ray Bradbury	Dystopian	12.50	1953
12	The Chronicles of Narnia	C.S. Lewis	Fantasy	14.00	1950
13	The Handmaid's Tale	Margaret Atwood	Dystopian	15.50	1985

14	A Tale of Two Cities	Charles Dickens	Historical	13.00	1859
15	Little Women	Louisa May Alcott	Fiction	9.50	1868

INPUT

SELECT COUNT(*) AS fiction_count FROM Books WHERE genre = 'Fiction';

OUTPUT



7. LIST OF MOVIES WITH RATINGS

Task

Write a query to select only the movie names where the ratings are greater than 7 but less than 9.

Table: **Cinema**

Movie_id	Movie_name	Description	Rating
1	War	great 3D	8.9
2	Science	fiction	8.5
3	Irish	boring	6.2
4	Ice Song	Fantasy	8.6
5	House Card	Interesting	9.1
6	The Escape	Thriller	7.8
7	Solar Flare	Sci-Fi	8.3
8	The Joker	Drama	9.0
9	Lost Dreams	Mystery	7.5
10	Galaxy War	Action	8.7

INPUT

```
SELECT Movie_name FROM Cinema WHERE Rating > 7.0 AND Rating < 9.0;
```

OUTPUT

Your Output

```
┌──────────────┐
│ Movie_name   │
├──────────────┤
│ War          │
│ Science      │
│ Ice Song     │
│ The Escape   │
│ Solar Flare  │
│ Lost Dreams  │
│ Galaxy War   │
```

✔ You got it right!

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

8. HANDLING NULL VALUES

Task

Write a query to retrieve book_id, title, author and published_year of the books which have **NULL** rating for their books.

Table name: Library

book_id	title	author	published_year	rating
1	The Great Gatsby	F. Scott Fitzgerald	1925	4.2
2	To Kill a Mockingbird	Harper Lee	1960	NULL
3	1984	George Orwell	1949	4.8
4	The Catcher in the Rye	J.D. Salinger	1951	NULL
5	Brave New World	Aldous Huxley	1932	4.3

INPUT

```
select book_id, title, author, published_year from library where rating is null;
```

OUTPUT

Your Output

book_id	title	author	published_year
2	To Kill a Mockingbird	Harper Lee	1960
4	The Catcher in the Rye	J.D. Salinger	1951

✔ Excellent work!

9. SALARY OF EMPLOYEES

Task

Create a query to retrieve the employee_name, company, and salary for employees in the full-time category, ordered by salary in **descending** order

Table name: Employees

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	IT	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

INPUT

```
SELECT employee_name, company, salary FROM Employees WHERE category = 'Full-Time' ORDER BY salary DESC;
```

OUTPUT

Your Output

employee_name	company	salary
Bob Brown	FinServ	90000
David Green	TechCorp	85000
John Smith	TechCorp	80000
Grace Grey	TechCorp	70000
Frank Black	HealthPlus	60000

✔ You got it right!

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

10. DEPARTMENT OF EACH EMPLOYEE

Task

Write a query to group the employees by their department and display the total number of employees (as total_employees) in each department.

Table name: Employees

employee_id	employee_name	company	category	department	salary
1	John Smith	TechCorp	Full-Time	Engineering	80000
2	Alice Johnson	TechCorp	Part-Time	HR	30000
3	Bob Brown	FinServ	Full-Time	Finance	90000
4	Carol White	HealthPlus	Contract	IT	75000
5	David Green	TechCorp	Full-Time	Engineering	85000
6	Emma Blue	FinServ	Part-Time	Finance	32000
7	Frank Black	HealthPlus	Full-Time	HR	60000
8	Grace Grey	TechCorp	Full-Time	Marketing	70000
9	Henry Red	FinServ	Contract	Sales	95000
10	Ivy Yellow	HealthPlus	Part-Time	Marketing	28000

INPUT

```
select department, count(*) as total_employees from employees group by department;
```

OUTPUT

Your Output

department	total_employees
Engineering	2
Finance	2
HR	2
IT	1
Marketing	2
Sales	1

✔ Perfect answer!

TRUTH-TOIL-TRIUMPH
AUTONOMOUS

11. ARTICLE VIEWS

Task

Write a query to retrieve the author_id, author_name, and publication_name for authors whose articles got zero views. The result should be sorted by author_id in ascending order.

Return the result table sorted by id in **ascending order**.

Table name: Views

article_id	author_id	author_name	viewer_id	view_count	publication_name
101	1	John Doe	1	3	The Daily Times
102	2	Jane Smith	2	1	Global News
103	3	Emily Clark	4	0	Tech Monthly
104	4	Mark Lewis	4	2	Health Digest
105	5	Sara White	3	0	The Business Journal
106	1	John Doe	2	0	The Daily Times
107	2	Jane Smith	2	2	Global News

INPUT

select distinct author_id, author_name, publication_name from views where view_count = 0 order by author_id asc;

OUTPUT

Your Output

author_id	author_name	publication_name
1	John Doe	The Daily Times
3	Emily Clark	Tech Monthly
5	Sara White	The Business Journal

✔ You got it right!

12. PLAYER PERFORMANCE INSIGHTS

Task

Write a query to find the names of the **top 3 distinct players** by highest score who have **won** matches, including their **scores**.

Table 1: **Players**

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6

Table 2: **Matches**

match_id	player1	player2	winner	match_date
101	Alice	Bob	Bob	2024-01-15
102	Charlie	David	David	2024-01-16
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-18
105	Charlie	Eve	Charlie	2024-01-19

INPUT

select distinct player_name, score from players join matches where winner=player_name

order by score desc limit 3

OUTPUT

Your Output

player_name	score
David	1600
Bob	1500
Charlie	1300

✔ Awesome, you nailed it!

13. PLAYER DETAILS

Task

Write a query to retrieve the details of the **last five matches** played, including the match ID, the names of the players who participated, the name of the winning player, match date, and the final score of the winner.

There are two tables named **Players** and **Matches**.

Players

player_id	player_name	score	rank
1	Alice	1200	5
2	Bob	1500	2
3	Charlie	1300	4
4	David	1600	1
5	Eve	1100	6
6	Frank	1450	3
7	Grace	1350	7
8	Hank	1250	9
9	Ivy	1550	8
10	jack	1400	10

Matches

match_id	player_1	player_2	winner	match_date
101	Alice	Bob	Bob	2024-01-25
102	Charlie	David	David	2024-01-06
103	Eve	Bob	Bob	2024-01-17
104	Alice	David	David	2024-01-01
105	Charlie	Eve	Charlie	2024-01-15
106	Frank	Hank	Frank	2024-01-29
107	Grace	Ivy	Ivy	2024-01-10
108	Jack	Alice	Jack	2024-01-19
109	Bob	Charlie	Bob	2024-01-13
110	David	Eve	David	2024-01-24

INPUT

```
SELECT m.match_id, m.player_1, m.player_2, m.winner, m.match_date, p.score

FROM Matches m

JOIN Players p ON m.winner = p.player_name

ORDER BY m.match_date DESC

LIMIT 5;
```

OUTPUT

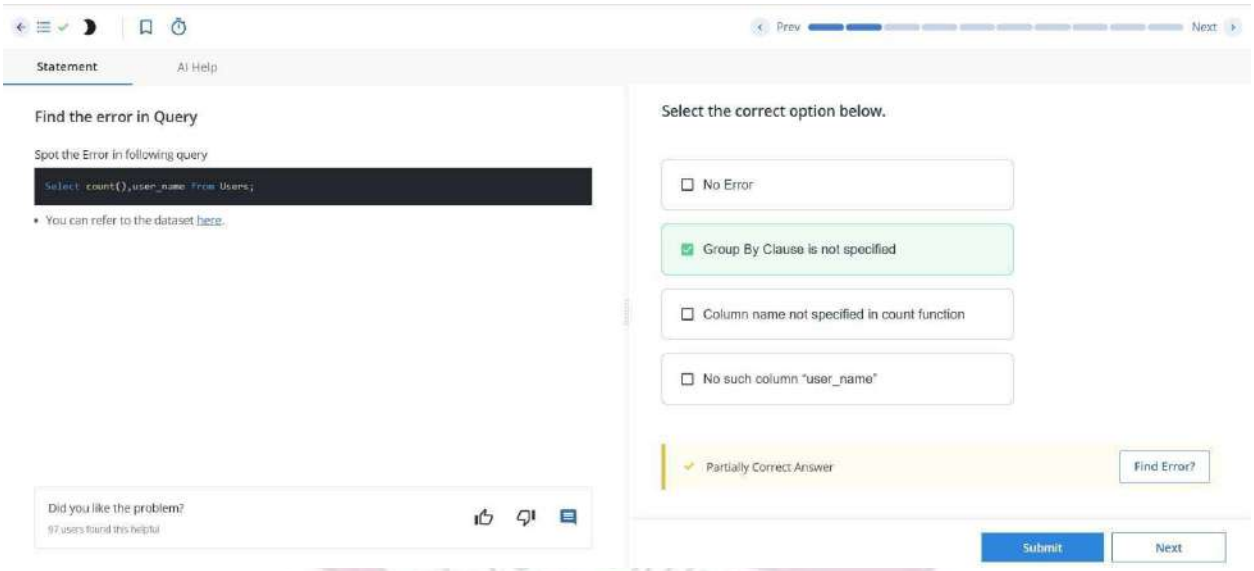
Your Output

match_id	player_1	player_2	winner	match_date	score
106	Frank	Hank	Frank	2024-01-29	1450
101	Alice	Bob	Bob	2024-01-25	1500
110	David	Eve	David	2024-01-24	1600
108	Jack	Alice	Jack	2024-01-19	1400
103	Eve	Bob	Bob	2024-01-17	1500

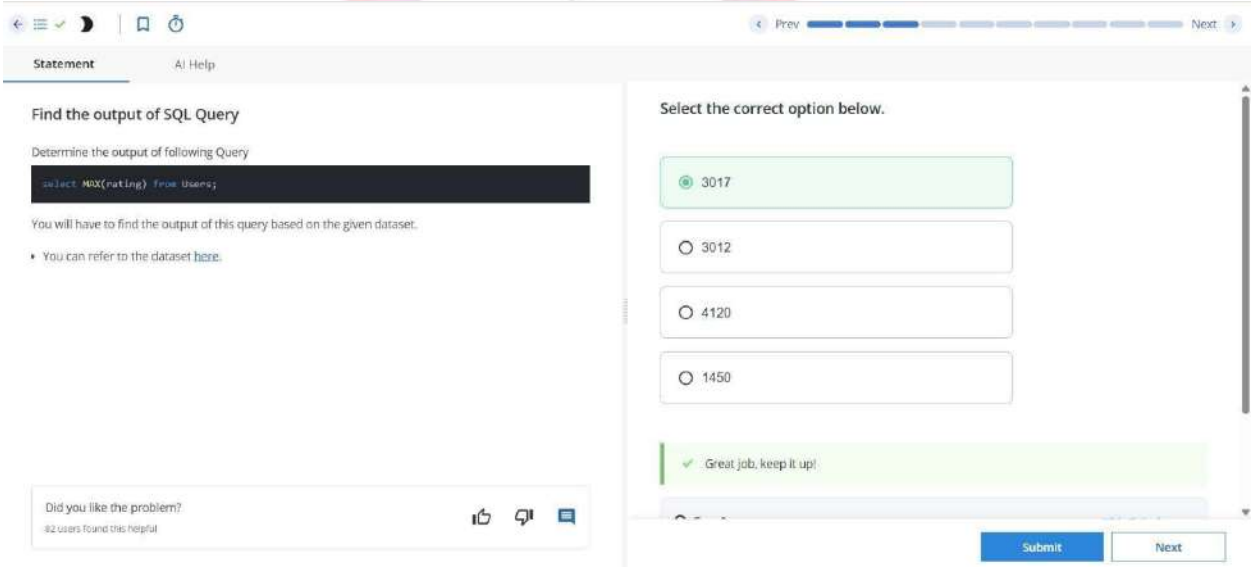
✔ Awesome, you nailed it!

CASE STUDY – CODECHEF DATABASE

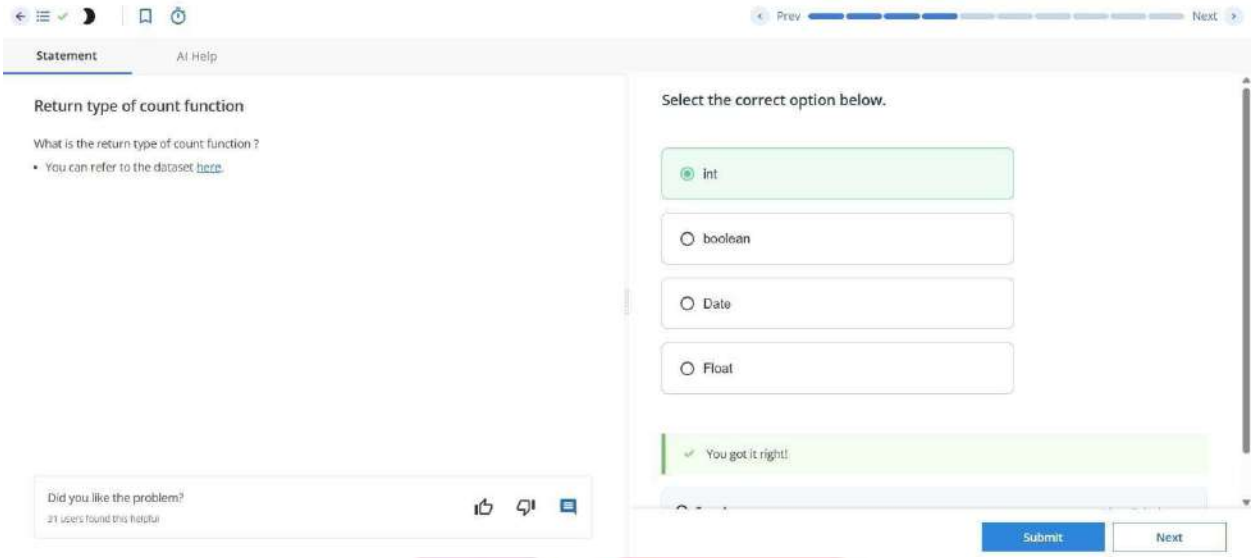
1. FIND THE ERROR IN QUERY



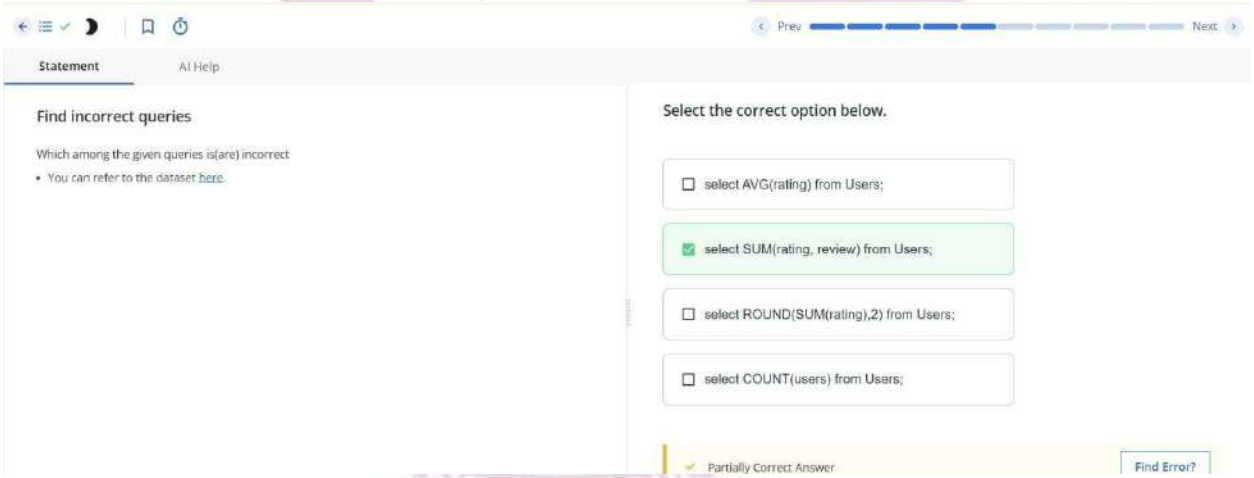
2. FIND THE OUTPUT OF SQL QUERY



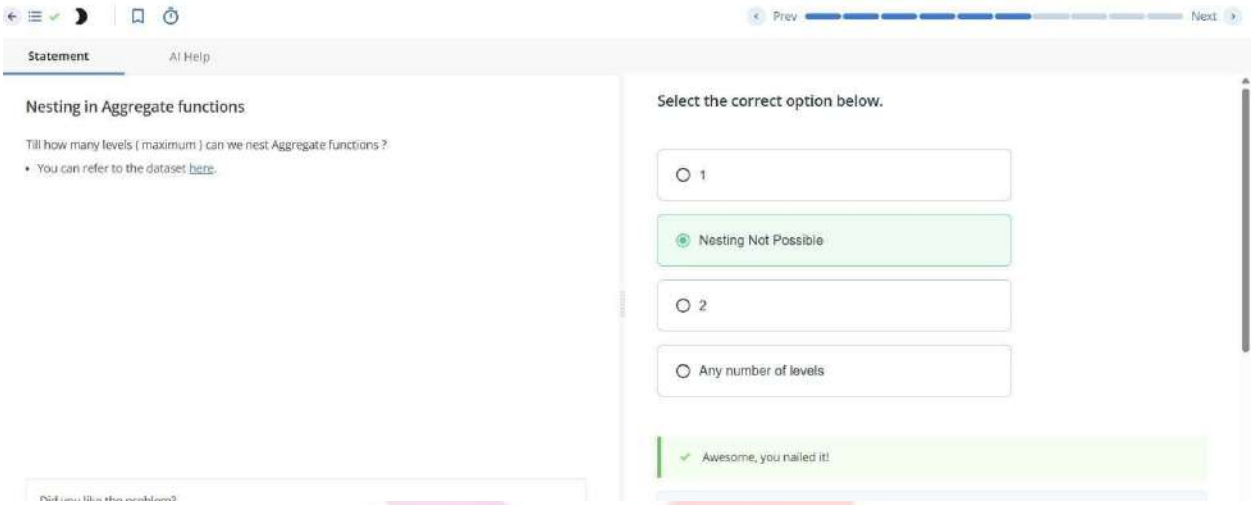
3. RETURN TYPE OF COUNT FUNCTION



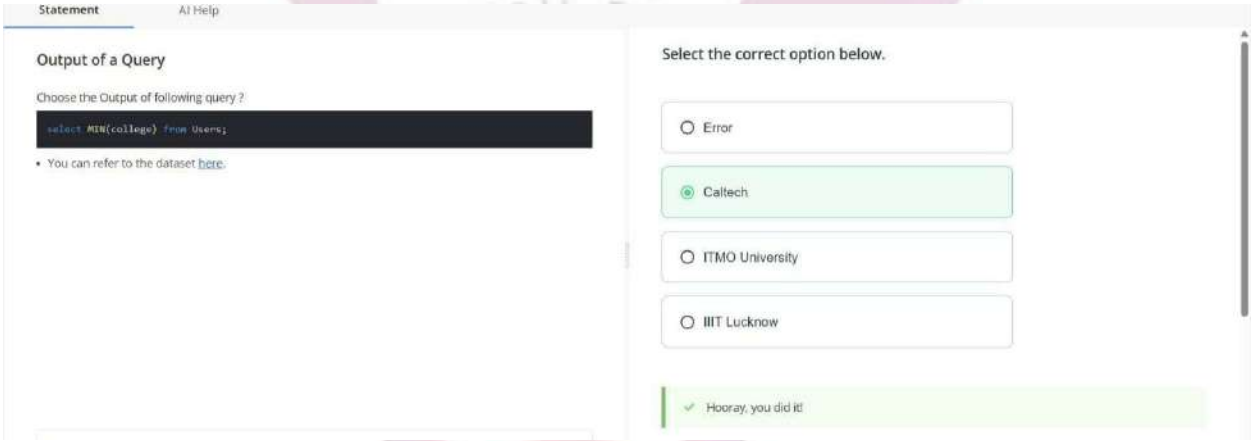
4. FIND INCORRECT QUERIES



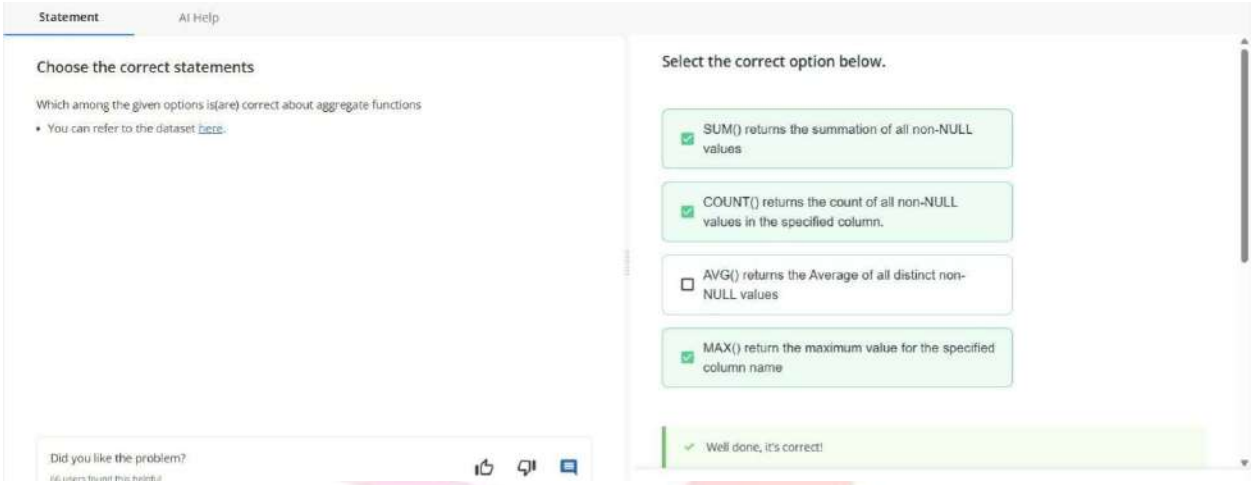
5.NESTED IN AGGREGATE FUNCTIONS



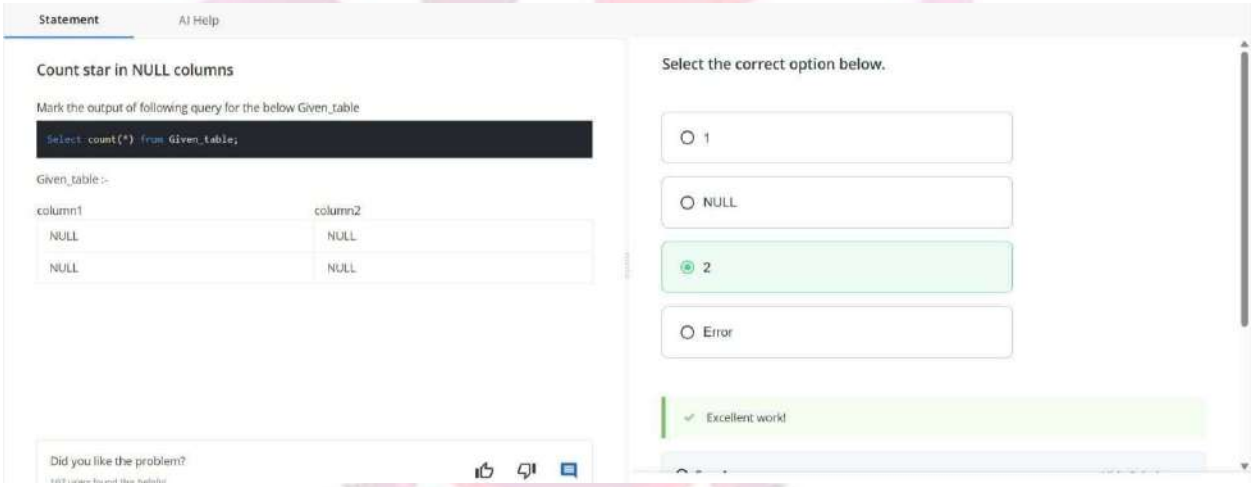
6. OUTPUT OF A QUERY



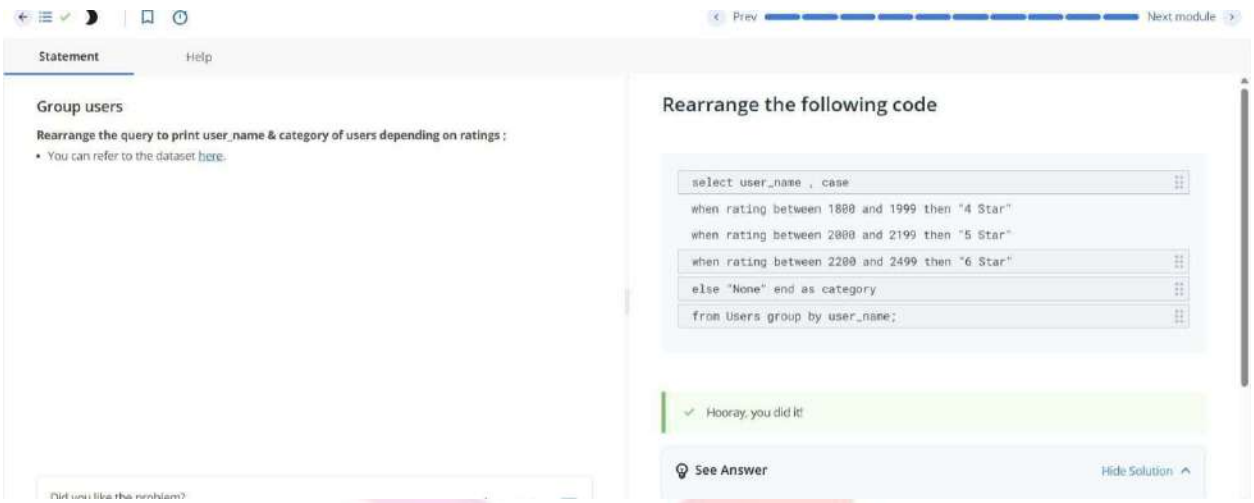
7. CHOOSE THE CORRECT STATEMENTS



8. COUNT STAR IN NULL COLUMNS



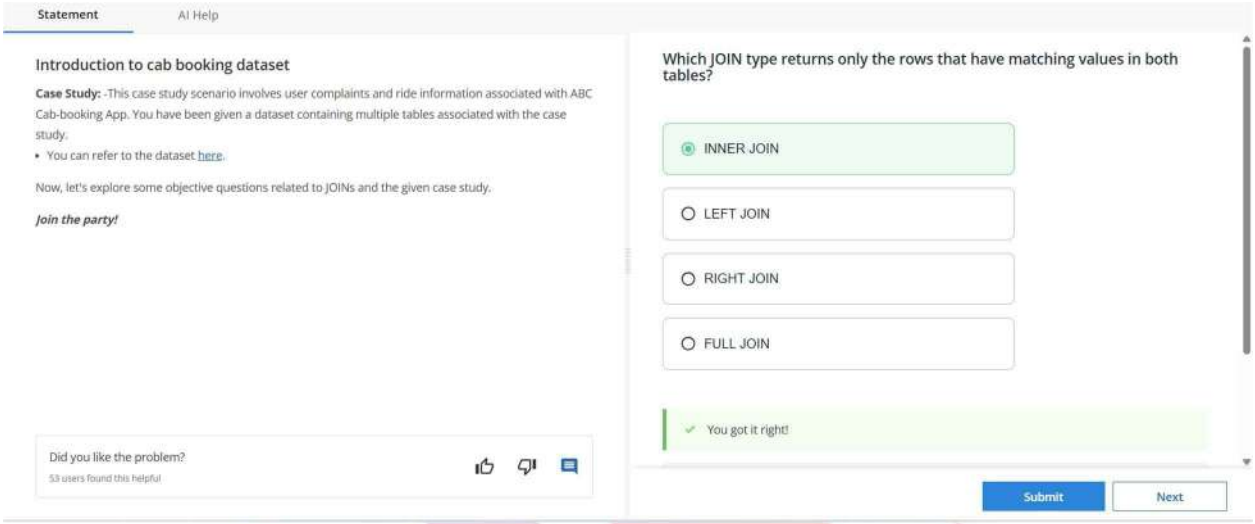
9.GROUP USERS



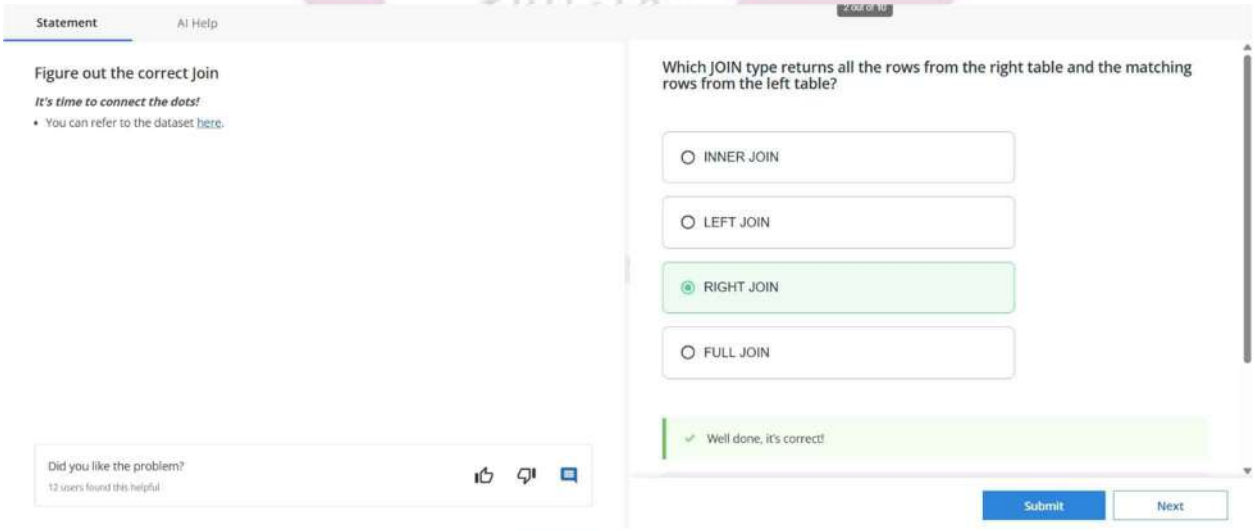
TRUTH-TOIL-TRIUMPH
AUTONOMOUS

CASE STUDY – ABC CAB BOOKIN APP

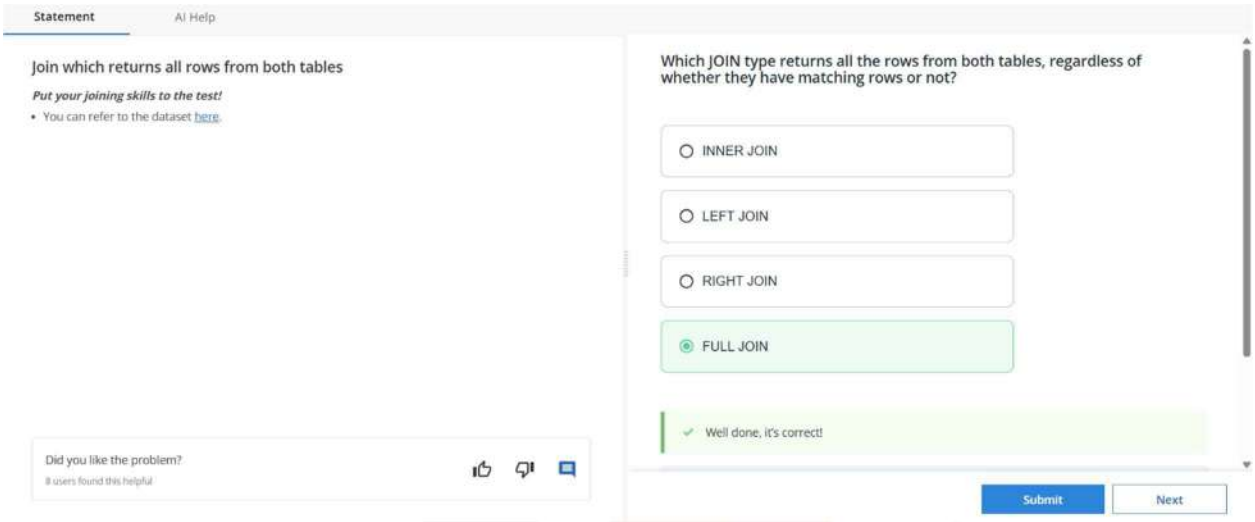
1.INTRODUCTION TO CAB BOOKING DATASET



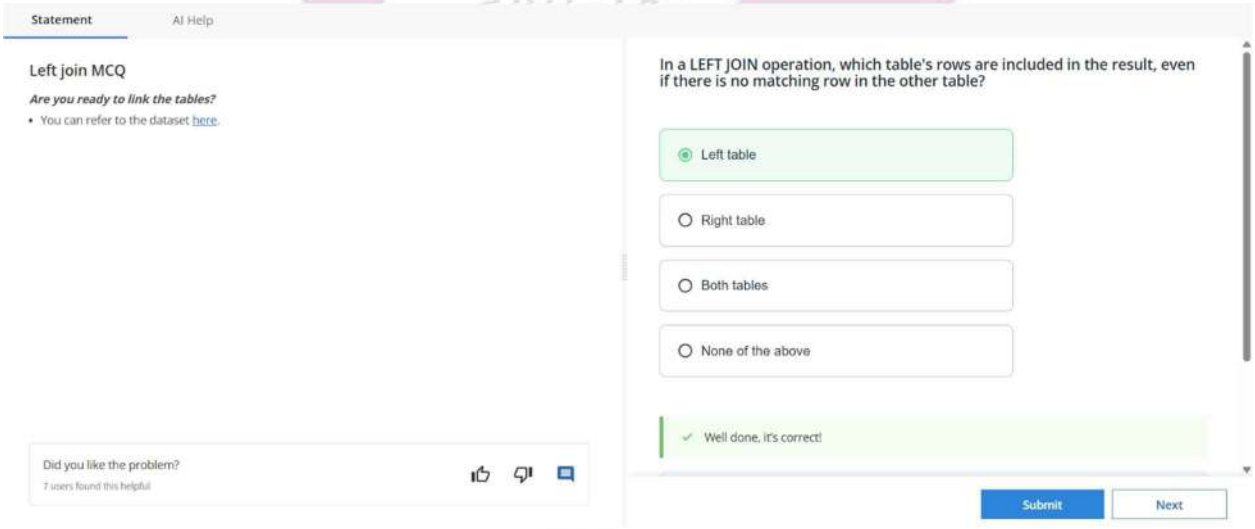
2. FIGURE OUT THE CORRECT JOIN



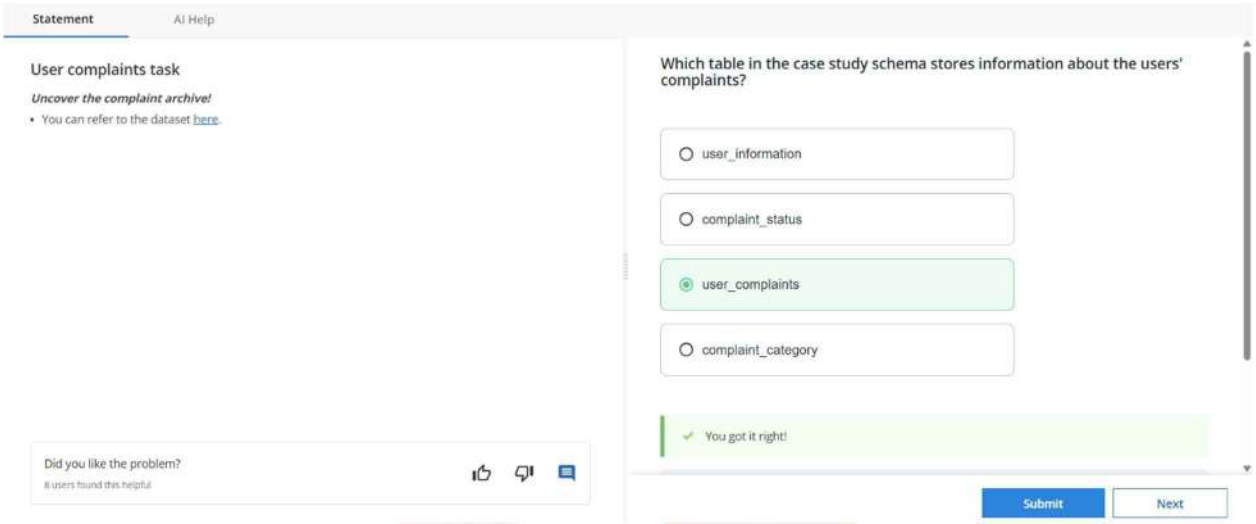
3. JOIN WHICH RETURN ALL ROWS FROM BOTH TABLES



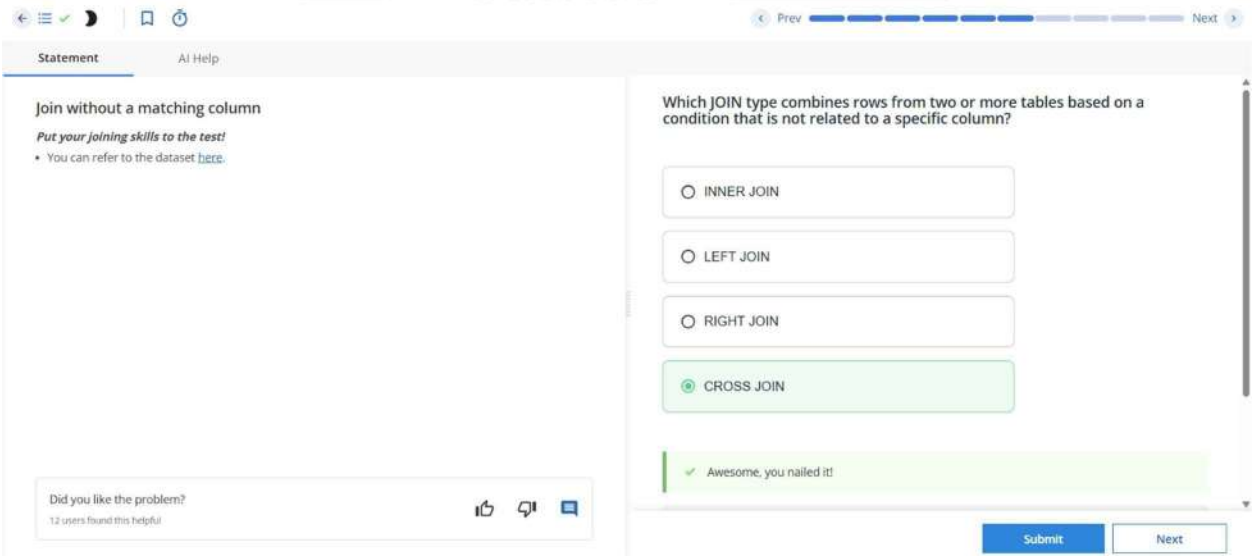
4. LEFT JOIN MCQ



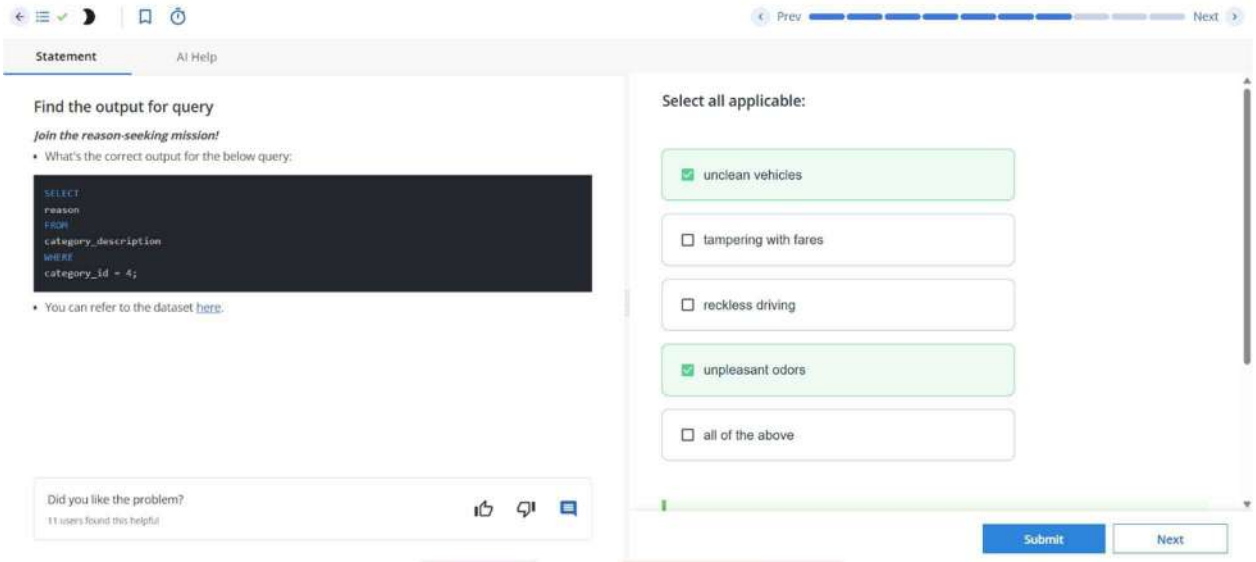
5. USER COMPLAINTS TASK



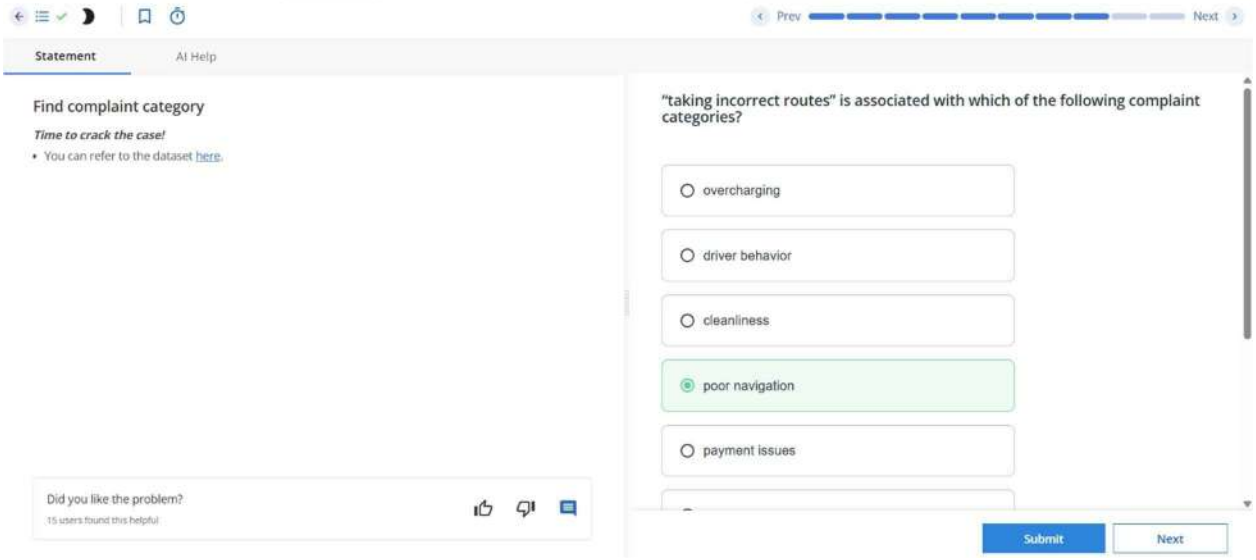
6. JOIN WITHOUT A MATCHING COLUMN



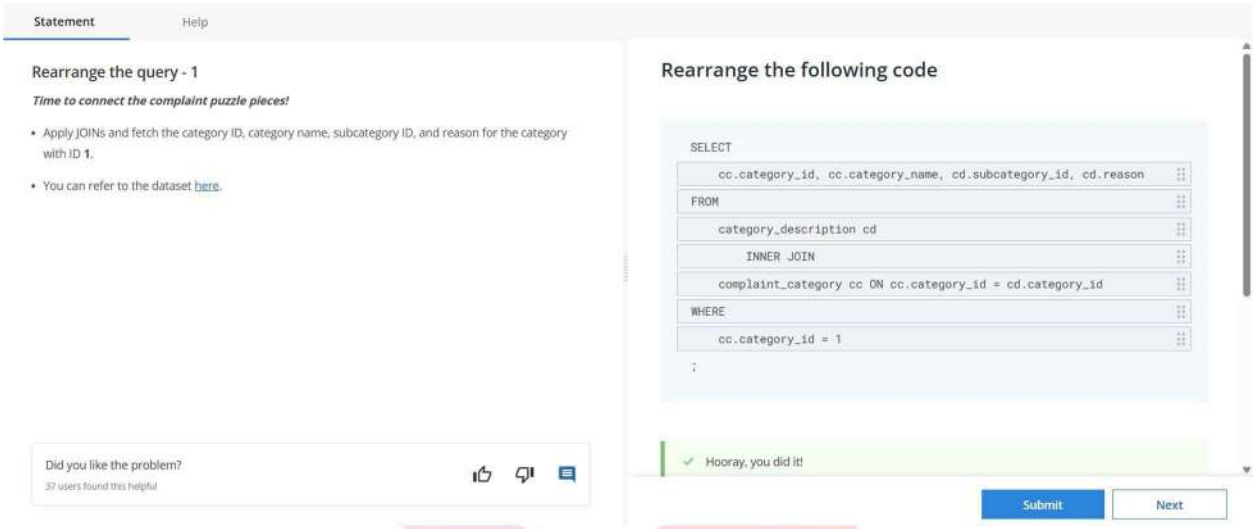
7. FIND THE OUTPUT FOR QUERY



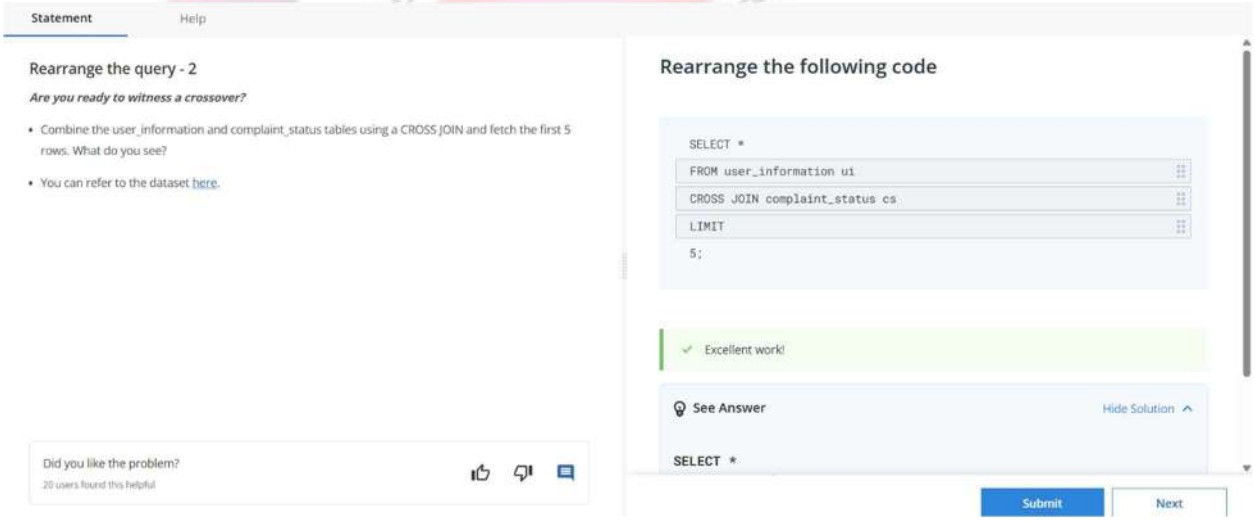
8. FIND COMPLAINT CATEGORY



9. REARRANGE THE QUERY - 1

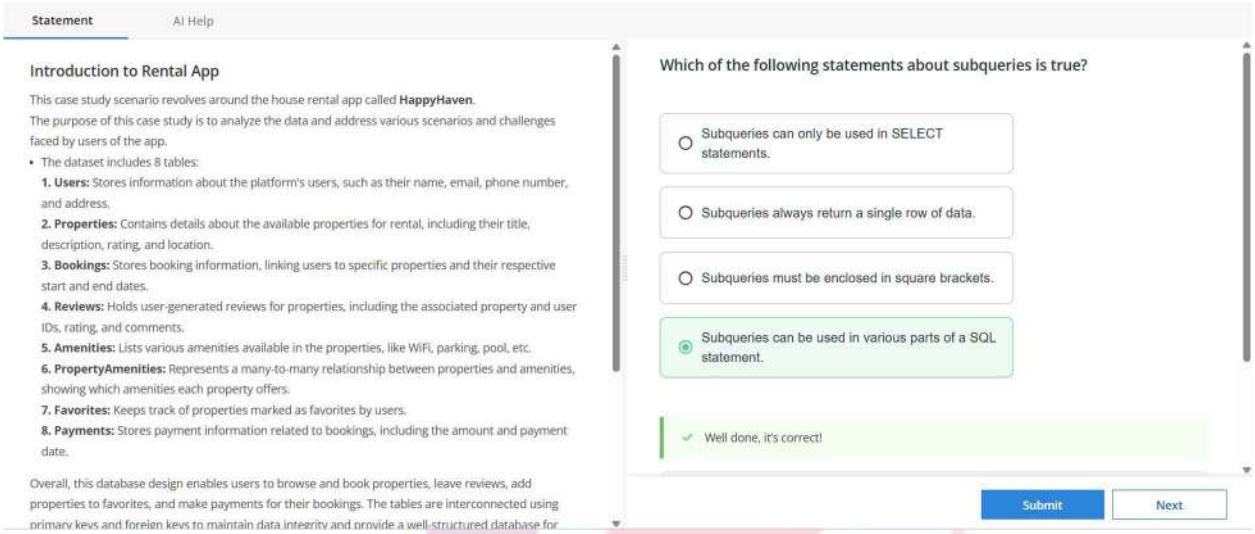


10 . REARRANGE THE QUERY - 2

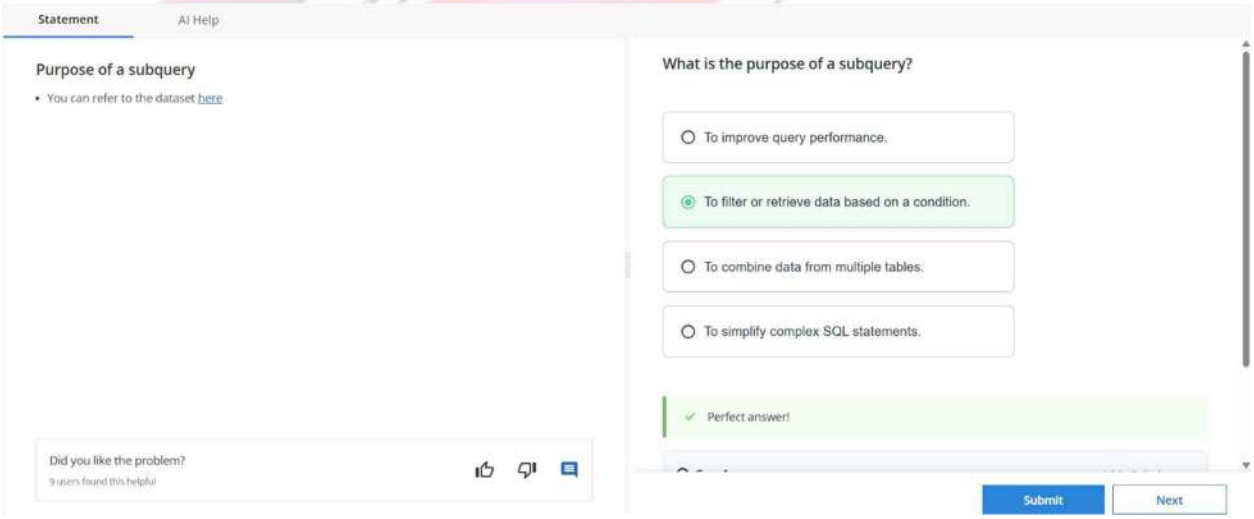


CASE STUDY – HAPPY HAVEN HOUSE RENTAL APP

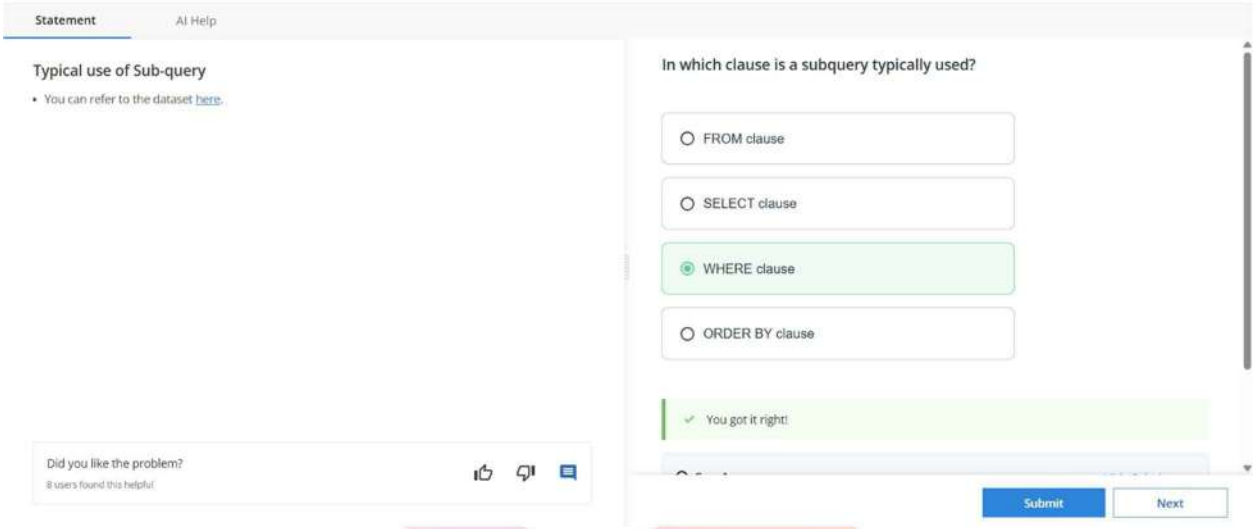
1.INTRODUCTION TO RENTAL APP



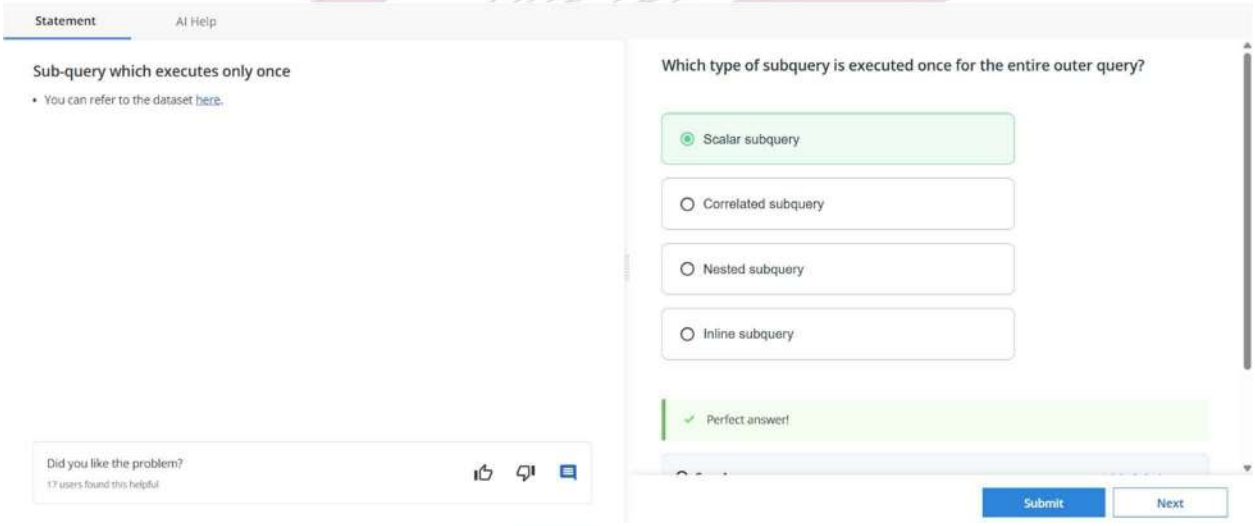
2. PURPOSE OF A SUBQUERY



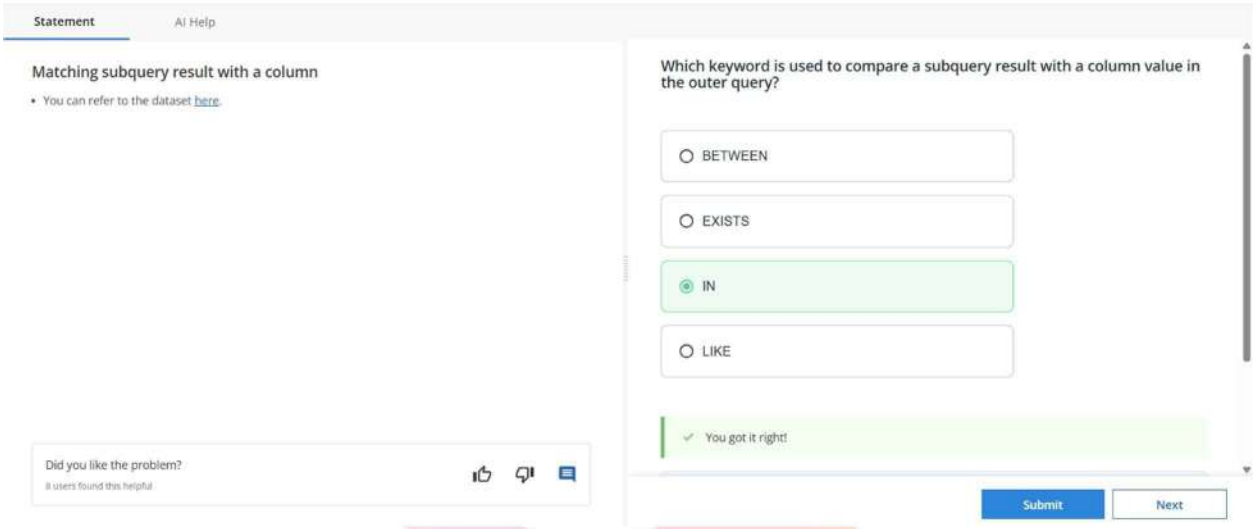
3. TYPICAL USE OF SUB-QUERY



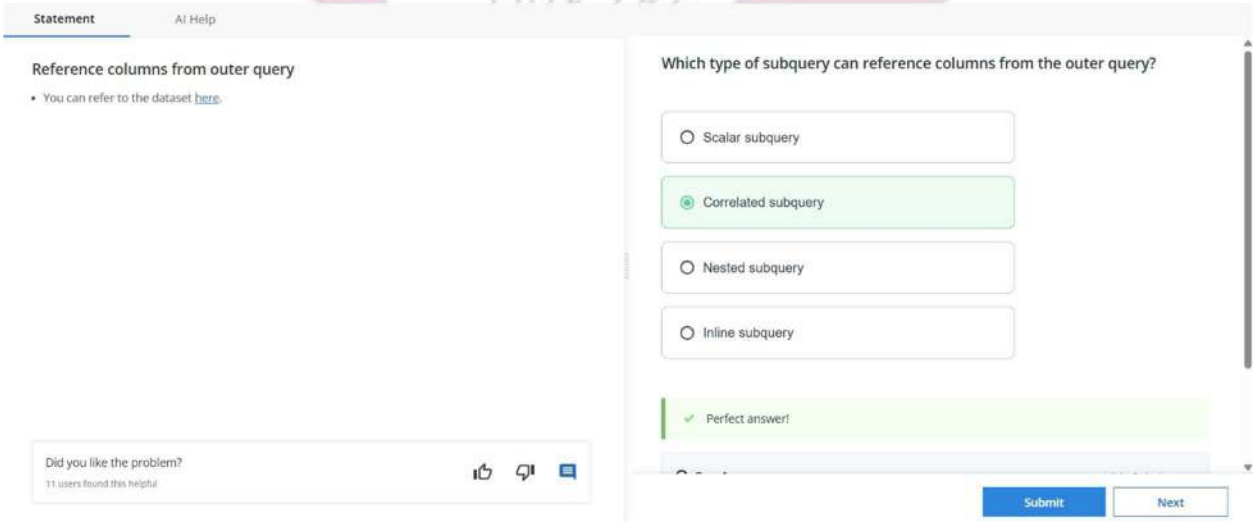
4. SUB-QUERY WHICH EXECUTES ONLY ONCE



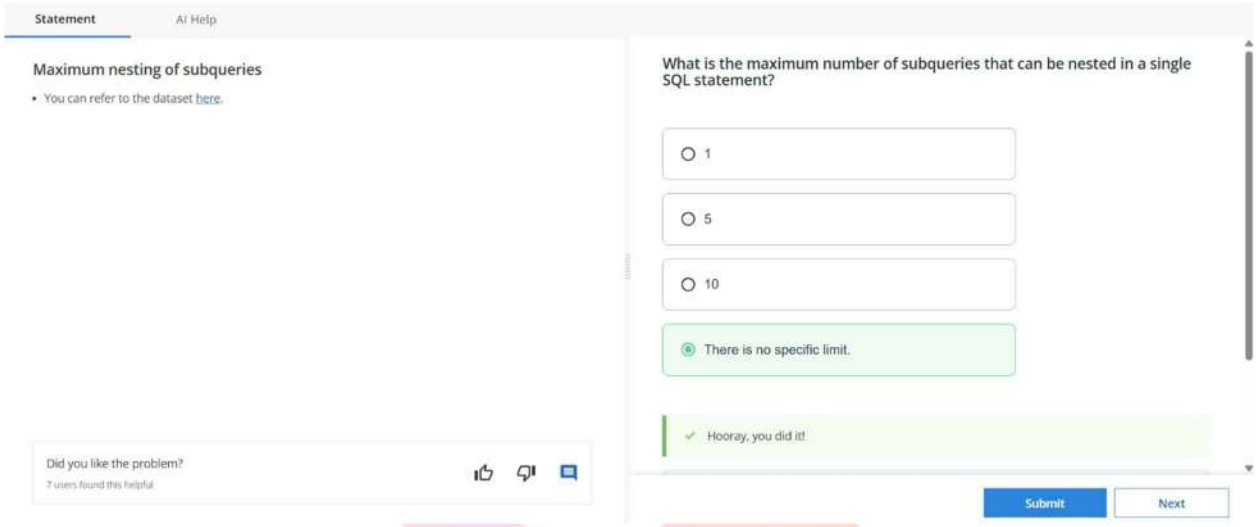
5. MATCHING SUBQUERY RESULT WITH A COLUMN



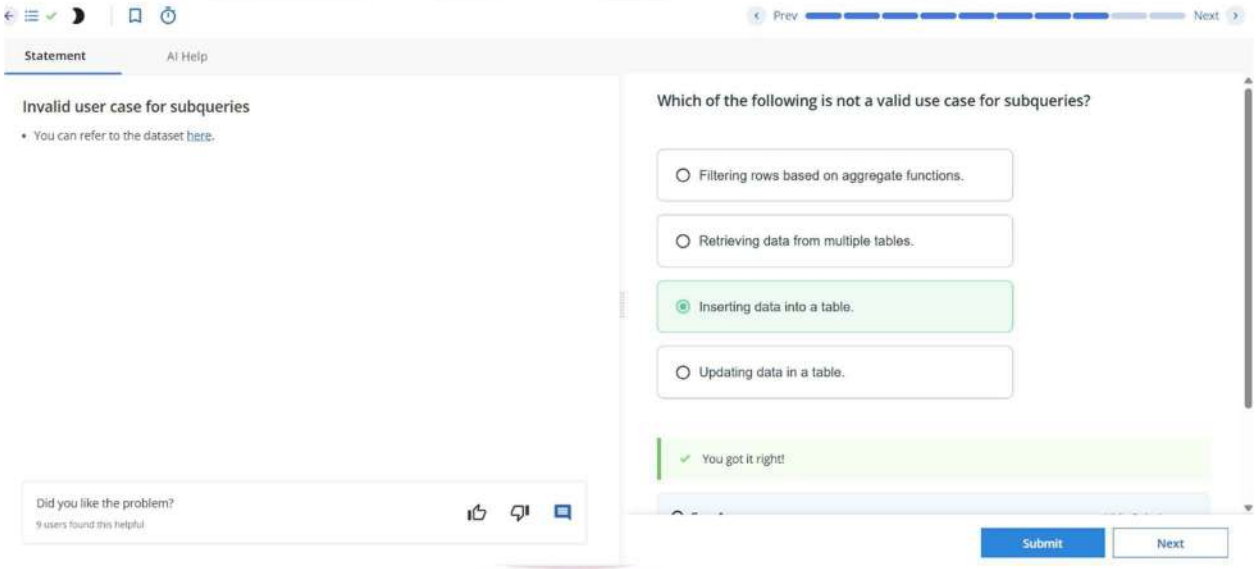
6. REFERENCE COLUMNS FROM OUTER QUERY



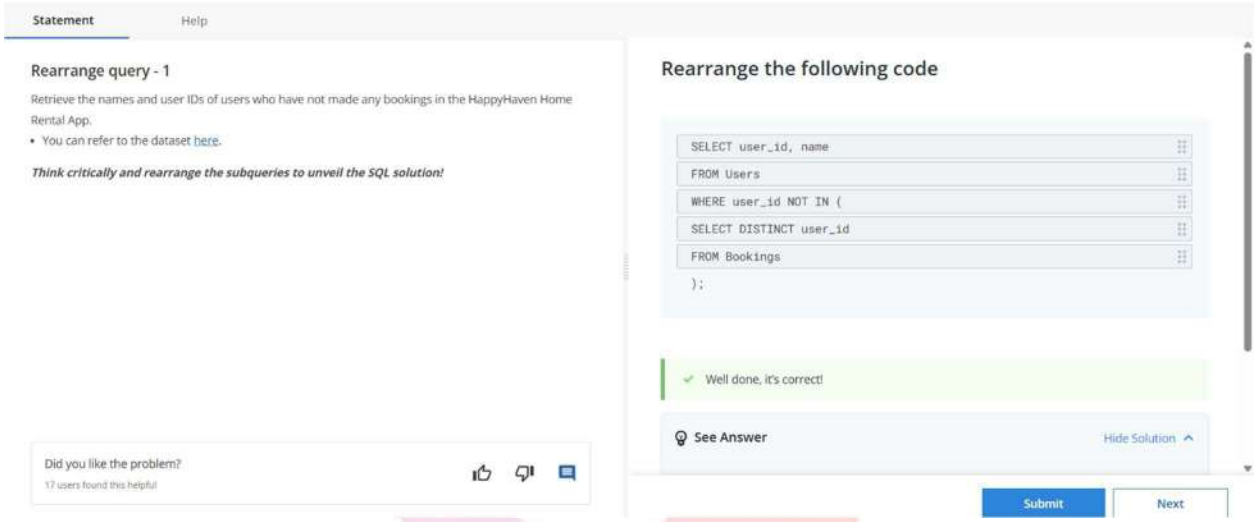
7. MAXIMUM NESTING OF SUBQUERIES



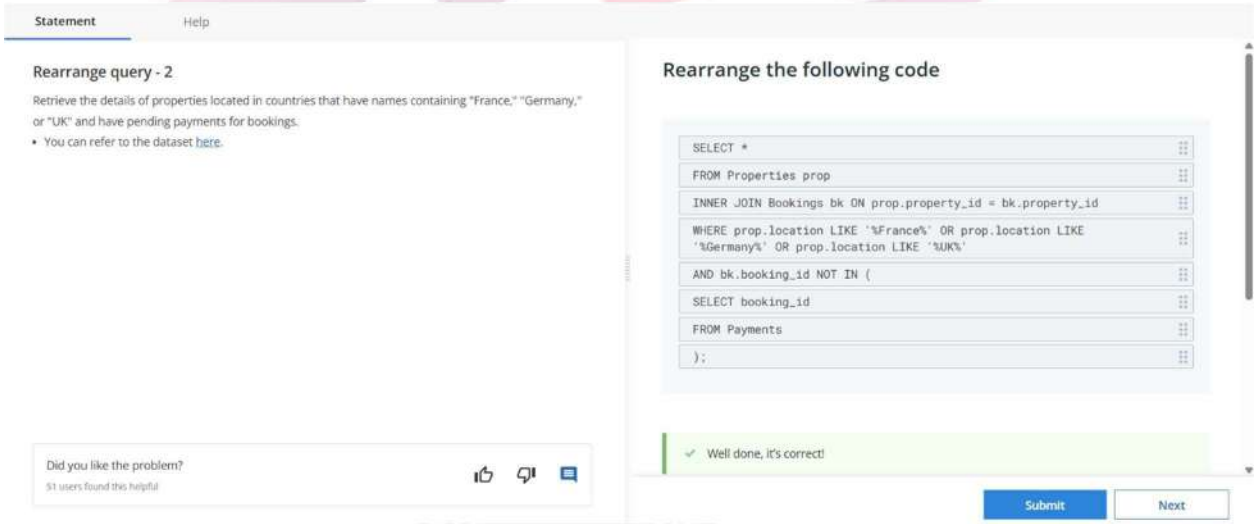
8. INVALID USER CASE FOR SUBQUERIES



9. REARRANGE QUERY-1

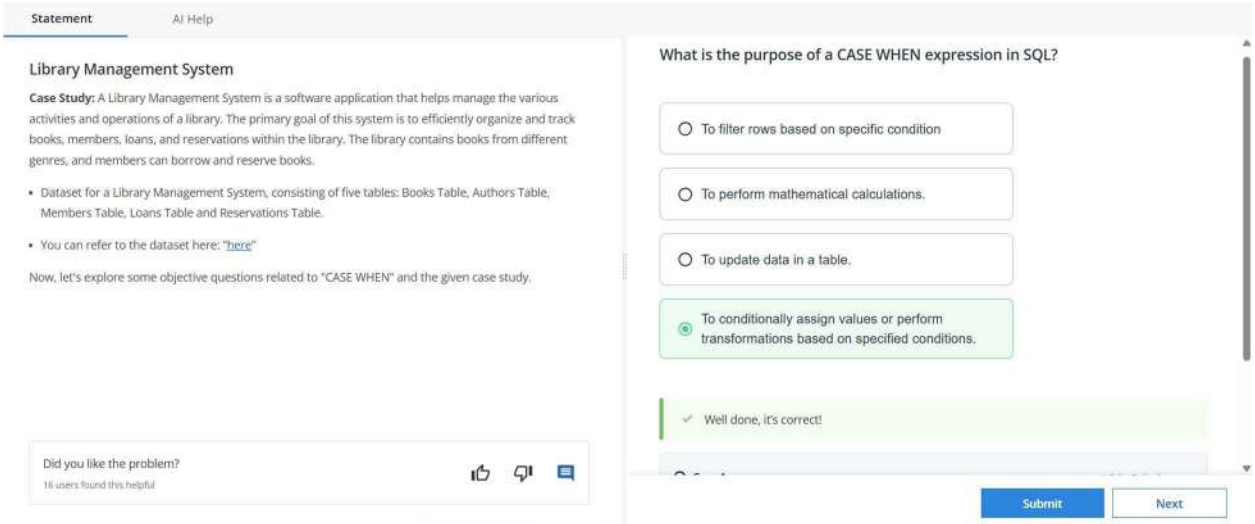


10. REARRANGE QUERY - 2

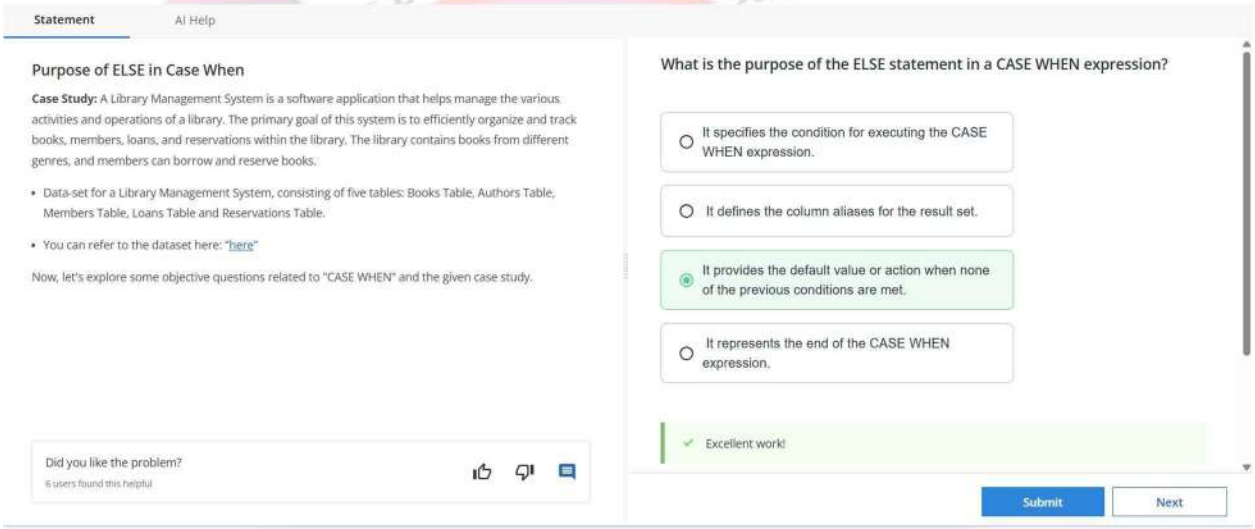


CASE STUDY – LIBRARY MANAGEMENT SYSTEM

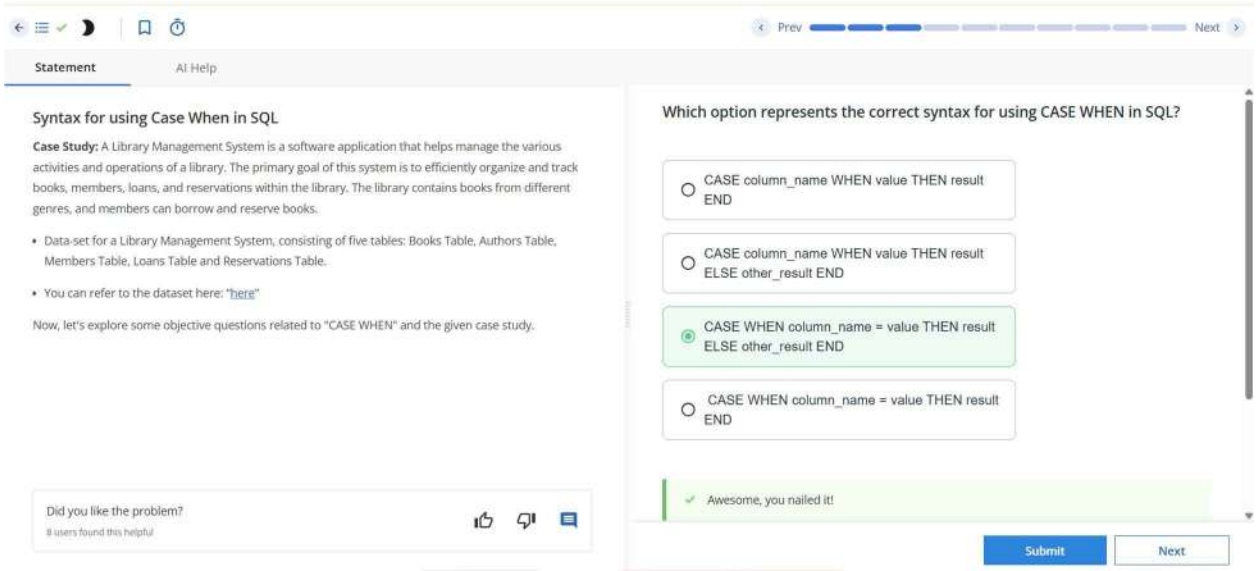
1. LIBRARY MANAGEMENT SYSTEM



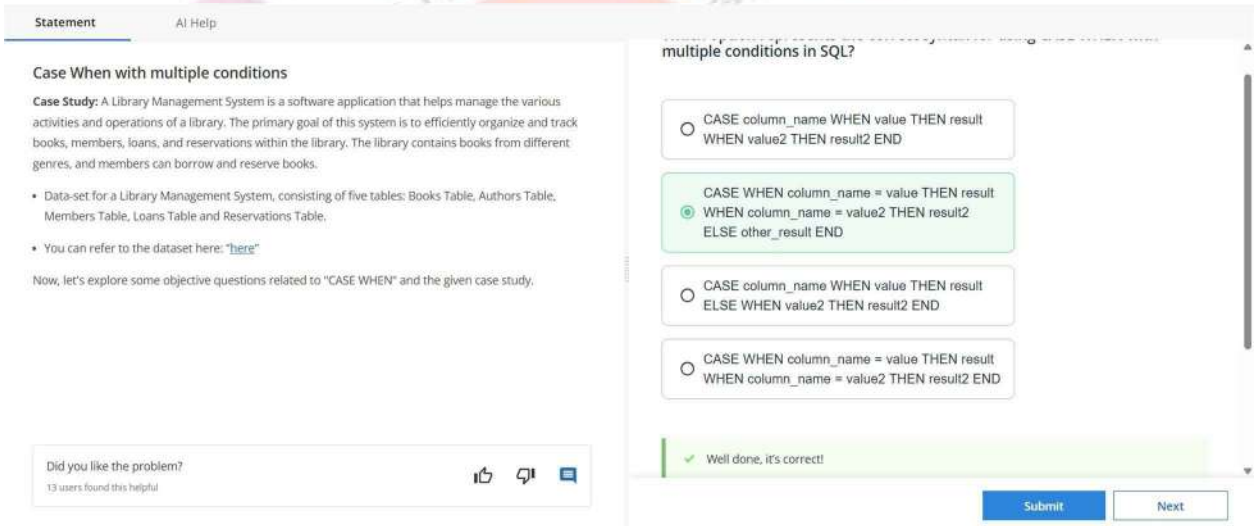
2.PURPOSE OF ELSE IN CASE WHEN



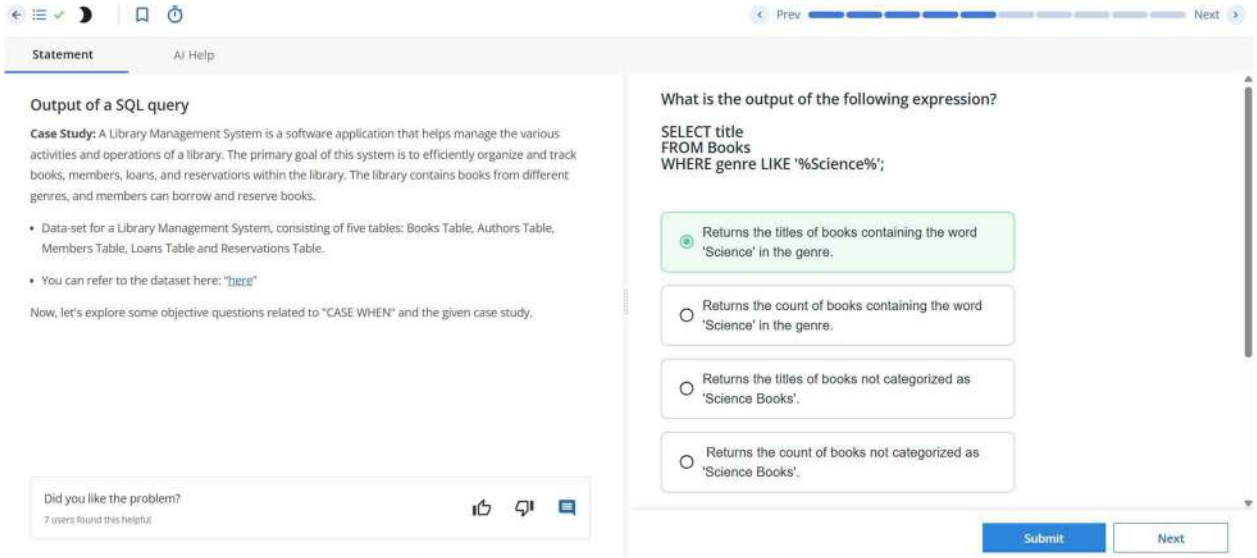
3.SYNTAX FOR USING CASE WHEN IN SQL



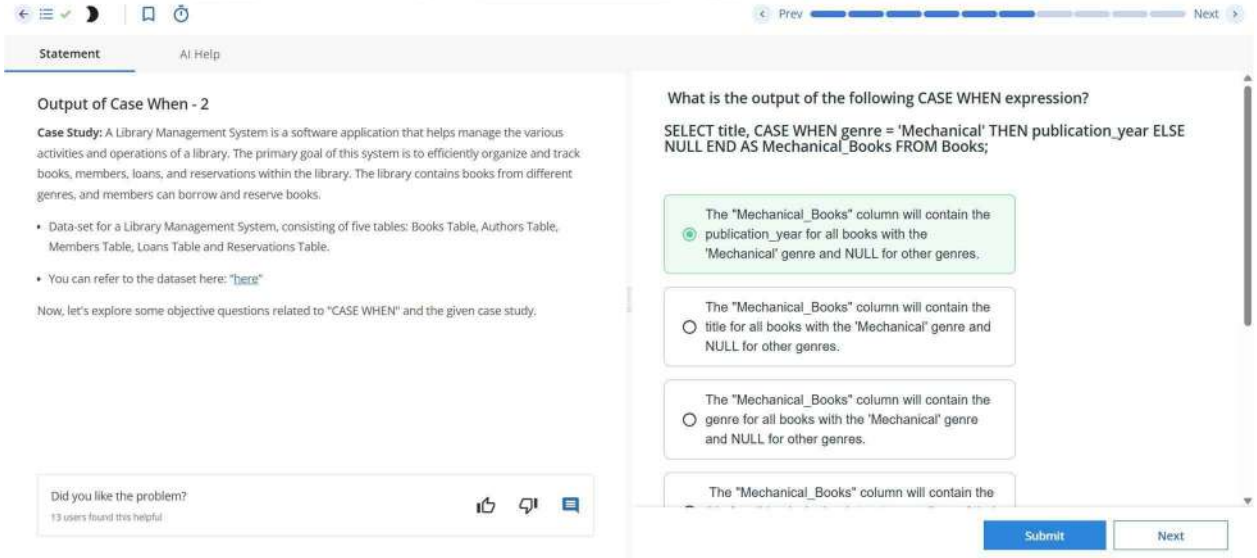
4. CASE WHEN WITH MULTIPLE CONDITIONS



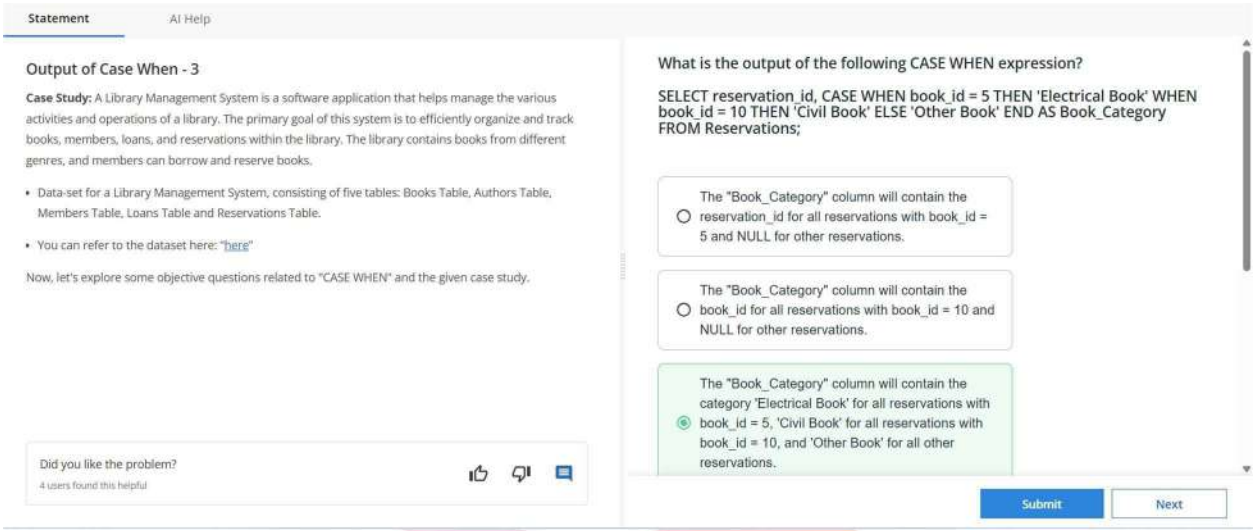
5. OUTPUT OF A SQL QUERY



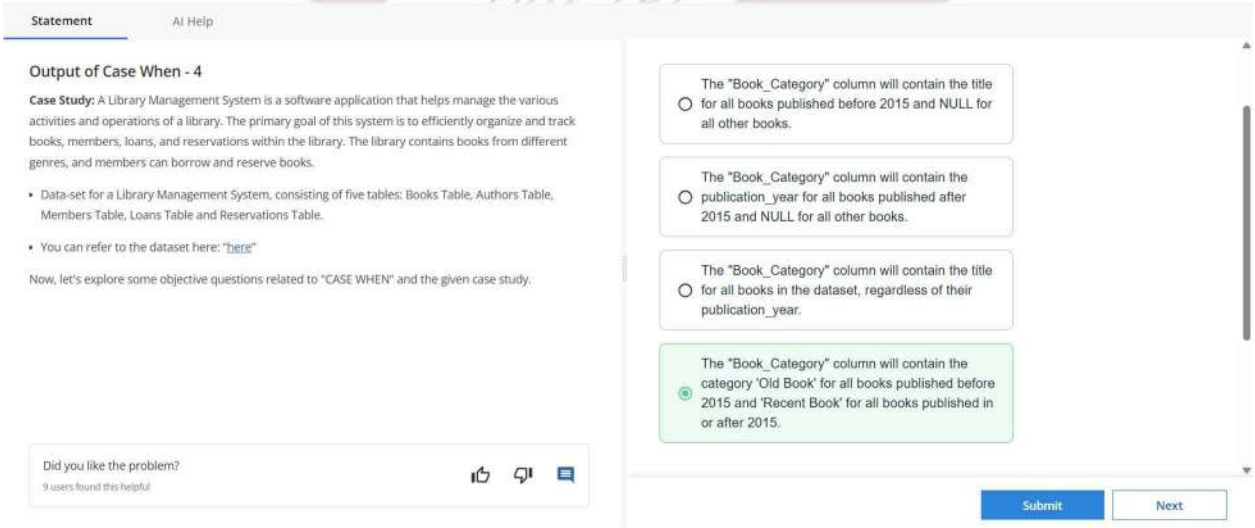
6. OUTPUT OF CASE WHEN - 2



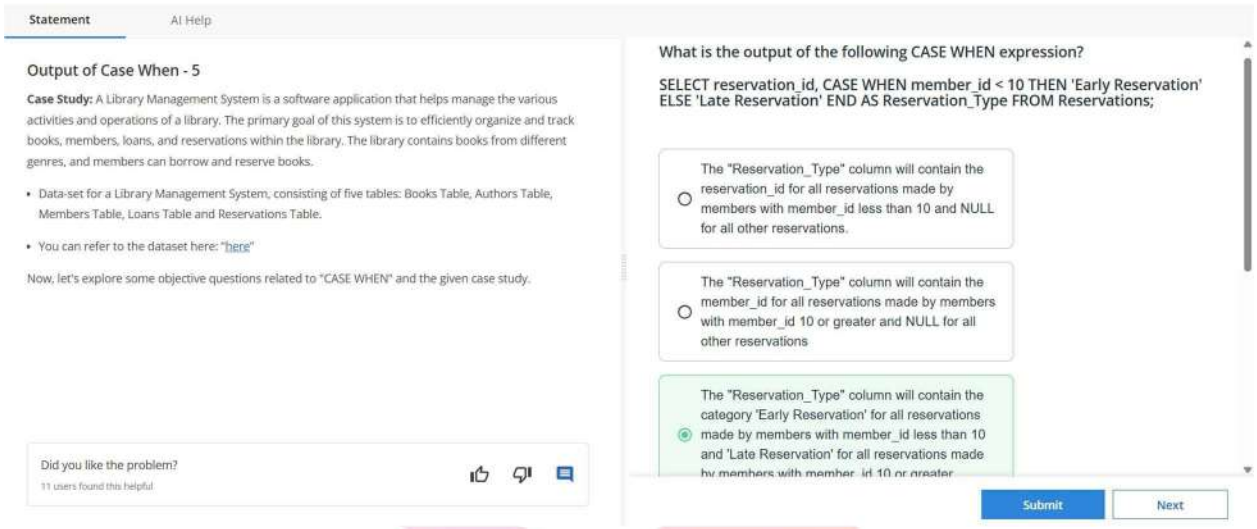
7. OUTPUT OF CASE WHEN - 3



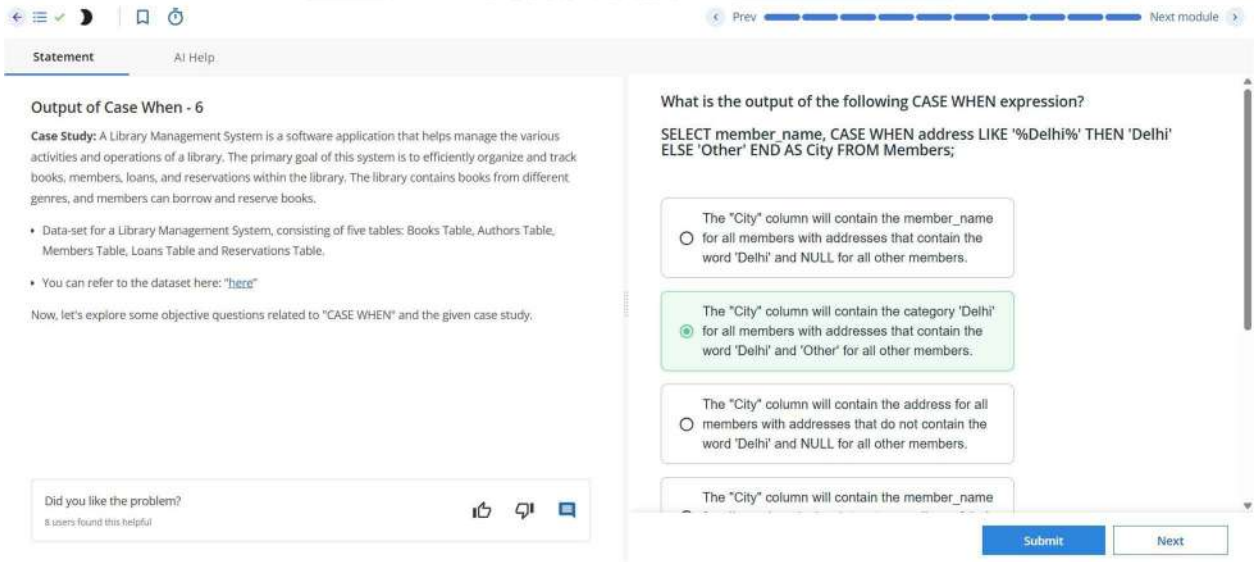
8. OUTPUT OF CASE WHEN - 4



9. OUTPUT OF CASE WHEN -5



10. OUTPUT OF CASE WHEN - 6



1. Consider the following tables CABHUB and CUSTOMER and answer (a) and (b) parts of this question:

(a) Write SQL commands for the following statements:

CREATE TABLE CABHUB (Vcode INT, VehicleName VARCHAR(20), Make VARCHAR(20), Color VARCHAR(10), Capacity INT, Charges INT);



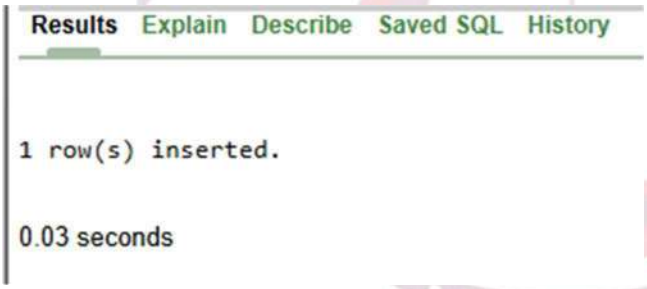
INSERT INTO CABHUB VALUES (100, 'Innova', 'Toyota', 'WHITE', 7, 15),

INSERT INTO CABHUB VALUES (102, 'SX4', 'Suzuki', 'BLUE', 4, 14),

INSERT INTO CABHUB VALUES (104, 'C-Class', 'Mercedes', 'RED', 4, 35),

INSERT INTO CABHUB VALUES (105, 'A-Star', 'Suzuki', 'WHITE', 3, 14),

INSERT INTO CABHUB VALUES (108, 'Indigo', 'Tata', 'SILVER', 3, 12);



CREATE TABLE CUSTOMER (Code INT, CName VARCHAR(30), VCode INT);



INSERT INTO CUSTOMER VALUES (1, 'Hemant Sahu', 101),

INSERT INTO CUSTOMER VALUES (2, 'Raj Lal', 108),

INSERT INTO CUSTOMER VALUES (3, 'Feroza Shah', 105),

INSERT INTO CUSTOMER VALUES (4, 'Ketan Dhal', 104);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.03 seconds
```

(i) To display the names of all the white colored vehicles.

SELECT VehicleName FROM CABHUB WHERE Color = 'WHITE';

Results Explain Describe Saved SQL History

VEHICLENAME
Innova
A-Star

2 rows returned in 0.02 seconds CSV Export

(ii) To display name of vehicle, make the capacity of vehicles in ascending order of their sitting Capacity.

SELECT VehicleName, Make, Capacity FROM CABHUB ORDER BY Capacity ASC;

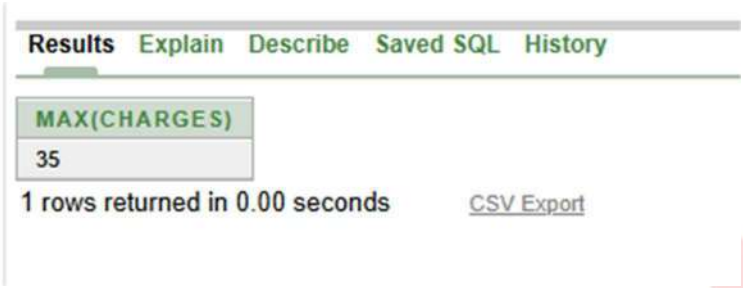
Results Explain Describe Saved SQL History

VEHICLENAME	MAKE	CAPACITY
Indigo	Tata	3
A-Star	Suzuki	3
SX4	Suzuki	4
C-Class	Mercedes	4
Innova	Toyota	7

5 rows returned in 0.00 seconds CSV Export

(iii) To display the highest charges at which a vehicle can be hired from CABHUB.

SELECT MAX(Charges) FROM CABHUB;



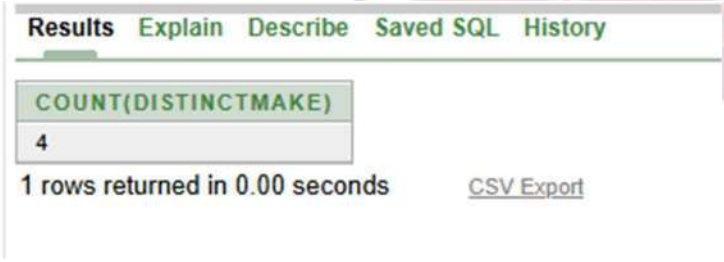
(iv) To display the customer and the corresponding name of the vehicle hired by them.

SELECT CUSTOMER.CName, CABHUB.VehicleName FROM CUSTOMER INNER JOIN CABHUB ON CUSTOMER.VCode = CABHUB.VCode;



(b) (i) SELECT COUNT (DISTINCT Make) FROM CABHUB;

SELECT COUNT(DISTINCT Make) FROM CABHUB;



(ii) SELECT MAX (CHARGES), MIN (Charges) FROM CABHUB;

SELECT MAX(Charges), MIN(Charges) FROM CABHUB;

Results Explain Describe Saved SQL History

MAX(CHARGES)	MIN(CHARGES)
35	12

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT COUNT(*), Make FROM CABHUB;

SELECT Make, COUNT(*) FROM CABHUB GROUP BY Make;

Results Explain Describe Saved SQL History

MAKE	COUNT(*)
Suzuki	2
Mercedes	1
Toyota	1
Tata	1

4 rows returned in 0.02 seconds CSV Export

(iv) SELECT VehicleName FROM CABHUB WHERE Capacity = 4;

SELECT VehicleName FROM CABHUB WHERE Capacity = 4;

Results Explain Describe Saved SQL History

VEHICLENAME
SX4
C-Class

2 rows returned in 0.00 seconds CSV Export

2. Given the following CO2_EMISSION relation, write SQL commands for questions (a) to (f).

CREATE TABLE CO2_EMISSION (VehicleID NUMBER(5), Company VARCHAR2(30), Model VARCHAR2(30), FuelType VARCHAR2(15), EngineSize NUMBER(3,1), CO2Emission NUMBER(5));



INSERT INTO CO2_EMISSION VALUES (101, 'Toyota', 'Corolla', 'Petrol', 1.8, 165);

INSERT INTO CO2_EMISSION VALUES (102, 'Hyundai', 'Creta', 'Diesel', 1.5, 145);

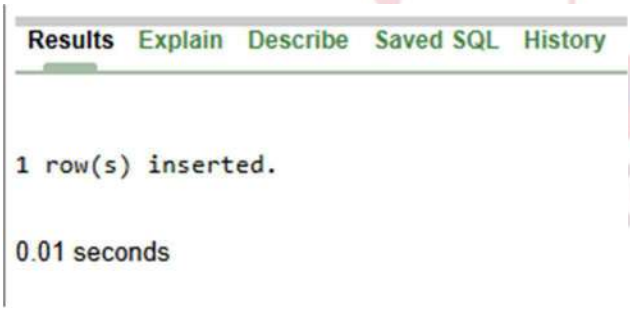
INSERT INTO CO2_EMISSION VALUES (103, 'Honda', 'City', 'Petrol', 1.5, 152);

INSERT INTO CO2_EMISSION VALUES (104, 'Tata', 'Nexon', 'Diesel', 1.5, 138);

INSERT INTO CO2_EMISSION VALUES (105, 'Maruti', 'Swift', 'Petrol', 1.2, 125);

INSERT INTO CO2_EMISSION VALUES (106, 'Kia', 'Seltos', 'Diesel', 1.5, 150);

INSERT INTO CO2_EMISSION VALUES (107, 'Mahindra', 'XUV700', 'Diesel', 2.2, 185);



(a) Display all the details of vehicles that use Petrol as the fuel type.

SELECT * FROM CO2_EMISSION WHERE FuelType = 'Petrol';

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
103	Honda	City	Petrol	1.5	152
105	Maruti	Swift	Petrol	1.2	125

3 rows returned in 0.00 seconds CSV Export

(b) Display the Company and Model of vehicles whose EngineSize is greater than 1.5 litres.

SELECT Company, Model FROM CO2_EMISSION WHERE EngineSize > 1.5;

Results Explain Describe Saved SQL History

COMPANY	MODEL
Toyota	Corolla
Mahindra	XUV700

2 rows returned in 0.00 seconds CSV Export

(c) Display the Company, Model, and CO2Emission in ascending order of CO2Emission.

SELECT Company, Model, CO2Emission FROM CO2_EMISSION ORDER BY CO2Emission ASC;

Results Explain Describe Saved SQL History

COMPANY	MODEL	CO2EMISSION
Maruti	Swift	125
Tata	Nexon	138
Hyundai	Creta	145
Kia	Seltos	150
Honda	City	152
Toyota	Corolla	165
Mahindra	XUV700	185

7 rows returned in 0.00 seconds CSV Export

(d) Display the details of vehicles whose CO2Emission is greater than 150 g/km.

SELECT * FROM CO2_EMISSION WHERE CO2Emission > 150;

Results Explain Describe Saved SQL History

VEHICLEID	COMPANY	MODEL	FUELTYPE	ENGINESIZE	CO2EMISSION
101	Toyota	Corolla	Petrol	1.8	165
103	Honda	City	Petrol	1.5	152
107	Mahindra	XUV700	Diesel	2.2	185

3 rows returned in 0.02 seconds CSV Export

(e) Count the number of vehicles that use Diesel as the fuel type.

SELECT COUNT(*) FROM CO2_EMISSION WHERE FuelType = 'Diesel';

Results Explain Describe Saved SQL History

COUNT(*)
4

1 rows returned in 0.00 seconds CSV Export

(f) Insert the following record into the CO2_EMISSION table.

INSERT INTO CO2_EMISSION VALUES (108, 'MG', 'Hector', 'Petrol', 1.5, 160);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.00 seconds

(g) Write the output of the following SQL commands.

(i) SELECT MAX(CO2Emission)FROM CO2_EMISSION;

SELECT MAX(CO2Emission) FROM CO2_EMISSION;

Results Explain Describe Saved SQL History

MAX(CO2EMISSION)
185

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT MIN(EngineSize)FROM CO2_EMISSION;

SELECT MIN(EngineSize) FROM CO2_EMISSION;



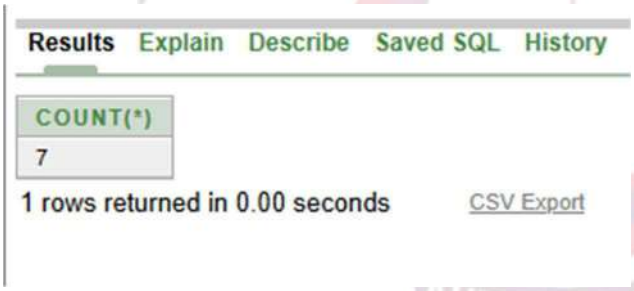
(iii) SELECT AVG(CO2Emission)FROM CO2_EMISSIONWHERE FuelType='Petrol';

SELECT AVG(CO2Emission) FROM CO2_EMISSION WHERE FuelType = 'Petrol';



(iv) SELECT COUNT(*)FROM CO2_EMISSIONWHERE EngineSize>=1.5;

SELECT COUNT(*) FROM CO2_EMISSION WHERE EngineSize >= 1.5;



3. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables APPLICANTS and COURSES.

CREATE TABLE APPLICANTS (NO NUMBER(5), NAME VARCHAR2(30), FEE NUMBER(10), GENDER VARCHAR2(10), C_ID VARCHAR2(5), JOINYEAR NUMBER(4));



INSERT INTO APPLICANTS VALUES (1012, 'Amandeep', 30000, 'M', 'A01', 2012);

INSERT INTO APPLICANTS VALUES (1102, 'Avisha', 25000, 'F', 'A02', 2009);

INSERT INTO APPLICANTS VALUES (1103, 'Ekant', 30000, 'M', 'A02', 2011);

INSERT INTO APPLICANTS VALUES (1049, 'Arun', 30000, 'M', 'A03', 2009);

INSERT INTO APPLICANTS VALUES (1025, 'Amber', 40000, 'M', 'A02', 2011);

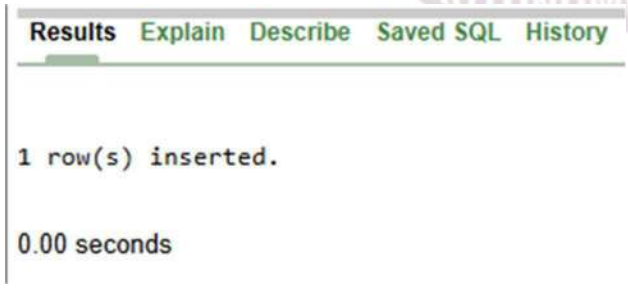
INSERT INTO APPLICANTS VALUES (1006, 'Ela', 40000, 'F', 'A05', 2010);

INSERT INTO APPLICANTS VALUES (1017, 'Nikita', 35000, 'F', 'A03', 2012);

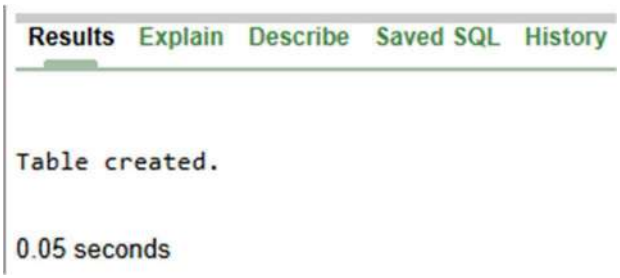
INSERT INTO APPLICANTS VALUES (1108, 'Arleena', 30000, 'F', 'A03', 2012);

INSERT INTO APPLICANTS VALUES (2109, 'Shakti', 35000, 'M', 'A04', 2011);

INSERT INTO APPLICANTS VALUES (1101, 'Kirat', 25000, 'M', 'A01', 2012);



CREATE TABLE COURSES(C_ID VARCHAR2(5), COURSE VARCHAR2(30));



INSERT INTO COURSES VALUES ('A01', 'Fashion Design');

INSERT INTO COURSES VALUES ('A02', 'Networking');

INSERT INTO COURSES VALUES ('A03', 'Hotel Management');

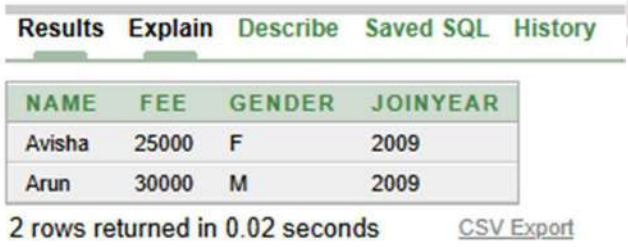
INSERT INTO COURSES VALUES ('A04', 'Event Management');

INSERT INTO COURSES VALUES ('A05', 'Office Management');



(a) To display name, fee, gender, joinyear about the applicants, who have joined before 2010.

SELECT NAME, FEE, GENDER, JOINYEAR FROM APPLICANTS WHERE JOINYEAR < 2010;



(b) To display the names of applicants, who are paying fee more than 30000.

SELECT NAME FROM APPLICANTS WHERE FEE > 30000;

Results Explain Describe Saved SQL History

NAME
Amber
Ela
Nikita
Shakti

4 rows returned in 0.00 seconds CSV Export

(c) To display name of all applicants in ascending order of their joinyear.

SELECT NAME FROM APPLICANTS ORDER BY JOINYEAR ASC;

Results Explain Describe Saved SQL History

NAME
Arun
Avisha
Ela
Ekant
Amber
Shakti
Nikita
Arleena
Amandeep
Kirat

10 rows returned in 0.00 seconds CSV Export

(d) To display the year and the total number of applicants joined in each YEAR from the table APPLICANTS.

SELECT JOINYEAR, COUNT(*) FROM APPLICANTS GROUP BY JOINYEAR;

Results Explain Describe Saved SQL History

JOINYEAR	COUNT(*)
2009	2
2010	1
2011	3
2012	4

4 rows returned in 0.00 seconds CSV Export

(e) To display the C_ID (i.e. Course ID) and the number of applicants registered in the course from the APPLICANTS table.

SELECT C_ID, COUNT(*) FROM APPLICANTS GROUP BY C_ID;

Results Explain Describe Saved SQL History

C_ID	COUNT(*)
A03	3
A04	1
A02	3
A05	1
A01	2

5 rows returned in 0.00 seconds CSV Export

(f) To display the applicant's name with their respective course's name from the tables APPLICANTS and COURSES.

SELECT APPLICANTS.NAME, COURSES.COURSE FROM APPLICANTS, COURSES WHERE APPLICANTS.C_ID = COURSES.C_ID;

Results Explain Describe Saved SQL History

NAME	COURSE
Amandeep	Fashion Design
Avisha	Networking
Ekant	Networking
Arun	Hotel Management
Amber	Networking
Ela	Office Management
Nikita	Hotel Management
Arleena	Hotel Management
Shakti	Event Management
Kirat	Fashion Design

10 rows returned in 0.02 seconds CSV Export

(g) Give the output of following SQL statements:

(g1) SELECT NAME, JOIN YEAR FROM APPLICANTS WHERE GENDER= 'F' AND C_ID= '02';

SELECT NAME, JOINYEAR FROM APPLICANTS WHERE GENDER = 'F' AND C_ID = '02';

Results Explain Describe Saved SQL History

no data found

(g2) SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE Gender= 'M';

SELECT MIN(JOINYEAR) FROM APPLICANTS WHERE GENDER = 'M';

Results Explain Describe Saved SQL History

MIN(JOINYEAR)
2009

1 rows returned in 0.00 seconds CSV Export

(g3) SELECT AVG(FEE) FROM APPLICANTS WHERE C_ID= 'A01' OR C_ID= 'A05';

SELECT AVG(FEE) FROM APPLICANTS WHERE C_ID = 'A01' OR C_ID = 'A05';

Results Explain Describe Saved SQL History

AVG(FEE)
31666.666666666666666666666666666666667

1 rows returned in 0.00 seconds CSV Export

(g4) SELECT SUM(FEE), C_ID FROM APPLICATIONS GROUP BY C_ID HAVING COUNT(*)=2;

SELECT SUM(FEE), C_ID FROM APPLICANTS GROUP BY C_ID HAVING COUNT(*) = 2;

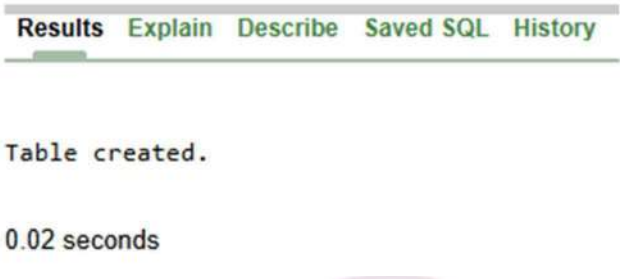
Results Explain Describe Saved SQL History

SUM(FEE)	C_ID
55000	A01

1 rows returned in 0.02 seconds CSV Export

4. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables PRODUCTS and SUPPLIERS.

CREATE TABLE PRODUCTS (PID NUMBER(5), SNAME VARCHAR2(40), QTY NUMBER(5), PRICE NUMBER(10), COMPANY VARCHAR2(30), SUPCODE VARCHAR2(5));



INSERT INTO PRODUCTS VALUES (101, 'DIGITAL CAMERA14X', 120, 12000, 'RENIX', 'S01');

INSERT INTO PRODUCTS VALUES (102, 'DIGITAL PAD II', 100, 22000, 'DIGI POP', 'S02');

INSERT INTO PRODUCTS VALUES (104, 'PEN DRIVE 16 GB', 500, 1100, 'STOREKING', 'S01');

INSERT INTO PRODUCTS VALUES (106, 'LED SCREEN', 70, 28000, 'DISEXPERTS', 'S02');

INSERT INTO PRODUCTS VALUES (105, 'CAR GPS SYSTEM', 60, 12000, 'MOVEON', 'S03');



CREATE TABLE SUPPLIERS(SUPCODE VARCHAR2(5), SNAME VARCHAR2(40), CITY VARCHAR2(30));

Results Explain Describe Saved SQL History

```
Table created.
```

0.02 seconds

INSERT INTO SUPPLIERS VALUES ('S01', 'GET ALL INC', 'KOLKATA');

INSERT INTO SUPPLIERS VALUES ('S03', 'EASY MARKET CORP', 'DELHI');

INSERT INTO SUPPLIERS VALUES ('S02', 'DIGI BUSY GROUP', 'CHENNAI');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.00 seconds

(a) To display the details of all the products in ascending order of product names (i.e. PNAME).

SELECT *FROM PRODUCTS ORDER BY SNAME ASC;

Results Explain Describe Saved SQL History

PID	SNAME	QTY	PRICE	COMPANY	SUPCODE
105	CAR GPS SYSTEM	60	12000	MOVEON	S03
101	DIGITAL CAMERA14X	120	12000	RENIX	S01
102	DIGITAL PAD II	100	22000	DIGI POP	S02
106	LED SCREEN	70	28000	DISEXPERTS	S02
104	PEN DRIVE 16 GB	500	1100	STOREKING	S01

5 rows returned in 0.01 seconds CSV Export

(b) To display product name and price of all those products, whose price is in the range of 10000 and 15000 (both values inclusive).

SELECT SNAME, PRICE FROM PRODUCTS WHERE PRICE BETWEEN 10000 AND 15000;

Results Explain Describe Saved SQL History

SNAME	PRICE
DIGITAL CAMERA14X	12000
CAR GPS SYSTEM	12000

2 rows returned in 0.02 seconds CSV Export

(c) To display the number of products which are supplied by each supplier i.e. the expected output should be

SELECT SUPCODE, COUNT(*) FROM PRODUCTS GROUP BY SUPCODE;

Results Explain Describe Saved SQL History

SUPCODE	COUNT(*)
S01	2
S02	2
S03	1

3 rows returned in 0.01 seconds CSV Export

(d) To display the price, product name (i.e. PName) and quantity (i.e. QTY) of those which have quantity more than 100.

SELECT PRICE, SNAME, QTY FROM PRODUCTS WHERE QTY > 100;

Results Explain Describe Saved SQL History

PRICE	SNAME	QTY
12000	DIGITAL CAMERA14X	120
1100	PEN DRIVE 16 GB	500

2 rows returned in 0.02 seconds CSV Export

(e) To display the names of those suppliers, who are either from DELHI or from CHENNAI.

SELECT SNAME FROM SUPPLIERS WHERE CITY = 'DELHI' OR CITY = 'CHENNAI'

Results Explain Describe Saved SQL History

SNAME
EASY MARKET CORP
DIGI BUSY GROUP

2 rows returned in 0.00 seconds CSV Export

;

(f) To display the name of the companies and the name of the products in descending order of company names.

SELECT COMPANY, SNAME FROM PRODUCTS ORDER BY COMPANY DESC;

Results Explain Describe Saved SQL History

COMPANY	SNAME
STOREKING	PEN DRIVE 16 GB
RENIX	DIGITAL CAMERA14X
MOVEON	CAR GPS SYSTEM
DISEXPERTS	LED SCREEN
DIGI POP	DIGITAL PAD II

5 rows returned in 0.00 seconds CSV Export

(g) Obtain the outputs of the following SQL queries based on the data given in tables PRODUCTS and SUPPLIERS.

(g1) SELECT DISTINCT SUPCODE FROM PRODUCTS;

SELECT DISTINCT SUPCODE FROM PRODUCTS;

Results Explain Describe Saved SQL History

SUPCODE
S01
S02
S03

3 rows returned in 0.00 seconds CSV Export

(g2) SELECT MAX(PRICE), MIN(PRICE) FROM PRODUCTS;

SELECT MAX(PRICE), MIN(PRICE) FROM PRODUCTS;

Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
28000	1100

1 rows returned in 0.00 seconds CSV Export

(g3) SELECT PRICE * QTY AMOUNT FROM PRODUCTS WHERE PID = 104;

SELECT PRICE * QTY AMOUNT FROM PRODUCTS WHERE PID = 104;

Results Explain Describe Saved SQL History

AMOUNT
550000

1 rows returned in 0.00 seconds CSV Export

(g4) SELECT PNAME, SNAME FROM PRODUCTS P, SUPPLIERS S WHERE P.SUPCODE = S.SUPCODE AND QTY>100;

SELECT P.SNAME, S.SNAME FROM PRODUCTS P, SUPPLIERS S WHERE P.SUPCODE = S.SUPCODE AND P.QTY > 100;

Results Explain Describe Saved SQL History

SNAME	SNAME
PEN DRIVE 16 GB	GET ALL INC
DIGITAL CAMERA14X	GET ALL INC

2 rows returned in 0.00 seconds CSV Export

5. Answer the questions (a) and (b) on the basis of the following tables SHOPPE and ACCESSORIES

CREATE TABLE SHOPPE (ID VARCHAR2(10), SName VARCHAR2(30), Area

VARCHAR2(30));



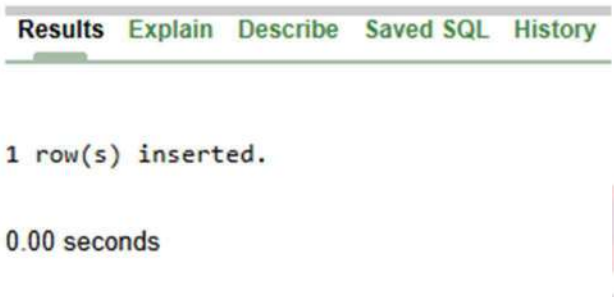
INSERT INTO SHOPPE VALUES ('S0001', 'ABC Computeronics', 'CP');

INSERT INTO SHOPPE VALUES ('S0002', 'All Infotech Media', 'GK II');

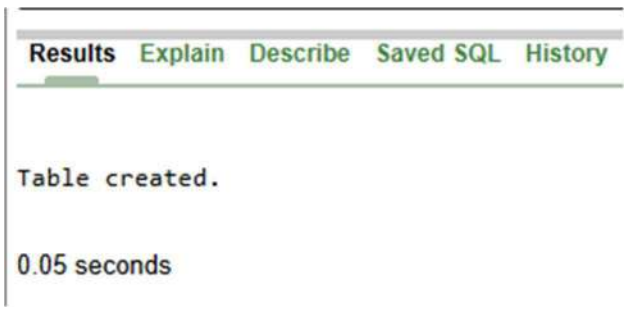
INSERT INTO SHOPPE VALUES ('S0003', 'Tech Shoppe', 'CP');

INSERT INTO SHOPPE VALUES ('S0004', 'Greeks Techno Soft', 'Nehru Place');

INSERT INTO SHOPPE VALUES ('S0005', 'Hitech Tech Store', 'Nehru Place');



CREATE TABLE ACCESSORIES (No VARCHAR2(5), Name VARCHAR2(30), Price NUMBER(10), ID VARCHAR2(5));



```
INSERT INTO ACCESSORIES VALUES ('A01', 'Mother Board', 12000, 'S01');

INSERT INTO ACCESSORIES VALUES ('A02', 'Hard Disk', 5000, 'S01');

INSERT INTO ACCESSORIES VALUES ('A03', 'Keyboard', 500, 'S02');

INSERT INTO ACCESSORIES VALUES ('A04', 'Mouse', 300, 'S01');

INSERT INTO ACCESSORIES VALUES ('A05', 'Mother Board', 13000, 'S02');

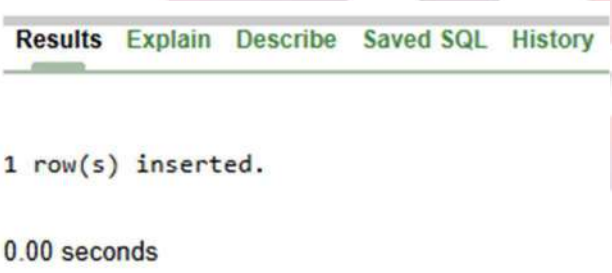
INSERT INTO ACCESSORIES VALUES ('A06', 'Keyboard', 400, 'S03');

INSERT INTO ACCESSORIES VALUES ('A07', 'LCD', 6000, 'S04');

INSERT INTO ACCESSORIES VALUES ('T08', 'LCD', 5500, 'S05');

INSERT INTO ACCESSORIES VALUES ('T09', 'Mouse', 350, 'S05');

INSERT INTO ACCESSORIES VALUES ('T10', 'Hard Disk', 4500, 'S03');
```



(a) Write the SQL queries

(i) To display Name and Price of all the accessories in ascending order of their Price.

```
SELECT Name, Price FROM ACCESSORIES ORDER BY Price ASC;
```

Results Explain Describe Saved SQL History

NAME	PRICE
Mouse	300
Mouse	350
Keyboard	400
Keyboard	500
Hard Disk	4500
Hard Disk	5000
LCD	5500
LCD	6000
Mother Board	12000
Mother Board	13000

10 rows returned in 0.00 seconds CSV Export

(ii) To display Id and SName of all Shoppe in Nehru Place.

SELECT ID, SName FROM SHOPPE WHERE Area = 'Nehru Place';

Results Explain Describe Saved SQL History

ID	SNAME
S0004	Greeks Techno Soft
S0005	Hitech Tech Store

2 rows returned in 0.00 seconds CSV Export

(iii) To display Minimum and Maximum Price of each Name of accessories.

SELECT Name, MIN(Price), MAX(Price) FROM ACCESSORIES GROUP BY Name;

Results Explain Describe Saved SQL History

NAME	MIN(PRICE)	MAX(PRICE)
Mother Board	12000	13000
LCD	5500	6000
Mouse	300	350
Keyboard	400	500
Hard Disk	4500	5000

5 rows returned in 0.01 seconds CSV Export

(iv) To display Name, Price of all accessories and their respective SName where they are available.

SELECT A.Name, A.Price, S.SName FROM ACCESSORIES A, SHOPPE S WHERE SUBSTR(A.ID, 2, 2) = SUBSTR(S.ID, 4, 2);

Results Explain Describe Saved SQL History

NAME	PRICE	SNAME
Mother Board	12000	ABC Computeronics
Hard Disk	5000	ABC Computeronics
Keyboard	500	All Infotech Media
Mouse	300	ABC Computeronics
Mother Board	13000	All Infotech Media
Keyboard	400	Tech Shoppe
LCD	6000	Greeks Techno Soft
LCD	5500	Hitech Tech Store
Mouse	350	Hitech Tech Store
Hard Disk	4500	Tech Shoppe

10 rows returned in 0.00 seconds CSV Export

;

(b) SQL Queries

(i) SELECT DISTINCT Name FROM ACCESSORIES WHERE Price>=500;

SELECT DISTINCT Name FROM ACCESSORIES WHERE Price >= 500;

Results Explain Describe Saved SQL History

NAME
Mother Board
LCD
Keyboard
Hard Disk

4 rows returned in 0.00 seconds CSV Export

(ii) SELECT Area, COUNT (*) FROM GROUP BY Area;

SELECT Area, COUNT(*) FROM SHOPPE GROUP BY Area;

Results Explain Describe Saved SQL History

AREA	COUNT(*)
GK II	1
CP	2
Nehru Place	2

3 rows returned in 0.00 seconds CSV Export

(iii) SELECT COUNT (DISTINCT Area) FROM SHOPPE;

SELECT COUNT(DISTINCT Area) FROM SHOPPE;

Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.00 seconds CSV Export

(iv) SELECT Name, Price*0.05 DISCOUNT FROM ACCESSORIES WHERE SNo IN ('S02, 'S03');

SELECT Name, Price * 0.05 DISCOUNT FROM ACCESSORIES WHERE ID IN ('S02', 'S03');

Results Explain Describe Saved SQL History

NAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	20
Hard Disk	225

4 rows returned in 0.02 seconds CSV Export

6. Write SQL commands for (a) to (f) and write the outputs for (g) on the basis of table HOSPITAL

CREATE TABLE HOSPITAL (No NUMBER, Name VARCHAR2(20), Age NUMBER, Department VARCHAR2(30), Dateofadm DATE, Charges NUMBER, Gender CHAR(1));




```
Table created.

0.08 seconds
```

INSERT INTO HOSPITAL VALUES (1, 'Arpit', 62, 'Surgery',

TO_DATE('21/01/98','DD/MM/RR'), 300, 'M');

INSERT INTO HOSPITAL VALUES (2, 'Zarina', 22, 'ENT',
TO_DATE('12/12/97','DD/MM/RR'), 250, 'F');

INSERT INTO HOSPITAL VALUES (3, 'Kareem', 32, 'Orthopedic',
TO_DATE('19/02/98','DD/MM/RR'), 200, 'M');

INSERT INTO HOSPITAL VALUES (4, 'Arun', 12, 'Surgery',

TO_DATE('11/01/98','DD/MM/RR'), 300, 'M');

INSERT INTO HOSPITAL VALUES (5, 'Zubin', 30, 'ENT',
TO_DATE('24/02/98','DD/MM/RR'), 250, 'M');

INSERT INTO HOSPITAL VALUES (6, 'Ketaki', 16, 'ENT',
TO_DATE('12/01/98','DD/MM/RR'), 250, 'M');

INSERT INTO HOSPITAL VALUES (7, 'Ankita', 29, 'Cardiology',
TO_DATE('20/08/98','DD/MM/RR'), 800, 'F');

INSERT INTO HOSPITAL VALUES (8, 'Zareen', 45, 'Gynecology',
TO_DATE('22/02/98','DD/MM/RR'), 300, 'F');

INSERT INTO HOSPITAL VALUES (9, 'Kush', 19, 'Cardiology',
TO_DATE('13/01/98','DD/MM/RR'), 800, 'M');

INSERT INTO HOSPITAL VALUES (10, 'Shilpa', 23, 'Nuclear Medicine',
TO_DATE('21/02/98','DD/MM/RR'), 400, 'F');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.01 seconds

(a) To select all the information of patients of cardiology department.

SELECT * FROM HOSPITAL WHERE Department = 'Cardiology';

Results Explain Describe Saved SQL History

NO	NAME	AGE	DEPARTMENT	DATEOFADM	CHARGES	GENDER
7	Ankita	29	Cardiology	20-AUG-98	800	F
9	Kush	19	Cardiology	13-JAN-98	800	M

2 rows returned in 0.01 seconds CSV Export

(b) To list the names of female patients who are in ENT department.

SELECT Name FROM HOSPITAL WHERE Gender = 'F' AND Department = 'ENT';

Results Explain Describe Saved SQL History

NAME
Zarina

1 rows returned in 0.00 seconds CSV Export

(c) To list name of all patients with their date of admission in ascending order.

SELECT Name, Dateofadm FROM HOSPITAL ORDER BY Dateofadm ASC;

Results Explain Describe Saved SQL History

NAME	DATEOFADM
Zarina	12-DEC-97
Arun	11-JAN-98
Ketaki	12-JAN-98
Kush	13-JAN-98
Arpit	21-JAN-98
Kareem	19-FEB-98
Shilpa	21-FEB-98
Zareen	22-FEB-98
Zubin	24-FEB-98
Ankita	20-AUG-98

10 rows returned in 0.00 seconds CSV Export

(d) To display Patient's Name, Charges, Age for only female patients.

SELECT Name, Charges, Age FROM HOSPITAL WHERE Gender = 'F';

Results Explain Describe Saved SQL History

NAME	CHARGES	AGE
Zarina	250	22
Ankita	800	29
Zareen	300	45
Shilpa	400	23

4 rows returned in 0.00 seconds CSV Export

(e) To count the number of patients with Age < 30.

SELECT COUNT(*) FROM HOSPITAL WHERE Age < 30;

Results Explain Describe Saved SQL History

COUNT(*)
6

1 rows returned in 0.00 seconds CSV Export

(f) To insert a new row in the HOSPITAL table with the given data.

INSERT INTO HOSPITAL VALUES (11, 'Aftab', 24, 'Surgery',

TO_DATE('25/02/98','DD/MM/RR'), 300, 'M');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.00 seconds

(g) Give the output of the following SQL statements

(i) Select COUNT (DISTINCT charges) from HOSPITAL;

SELECT COUNT(DISTINCT Charges) FROM HOSPITAL;

Results Explain Describe Saved SQL History

COUNT(DISTINCTCHARGES)
5

1 rows returned in 0.00 seconds CSV Export

(ii) Select MIN (Age) from HOSPITAL where Gender = "F"

SELECT MIN(Age) FROM HOSPITAL WHERE Gender = 'F';

Results Explain Describe Saved SQL History

MIN(AGE)
22

1 rows returned in 0.00 seconds CSV Export

(iii) Select SUM (Charges) from HOSPITAL where Department = "ENT"

SELECT SUM(Charges) FROM HOSPITAL WHERE Department = 'ENT';

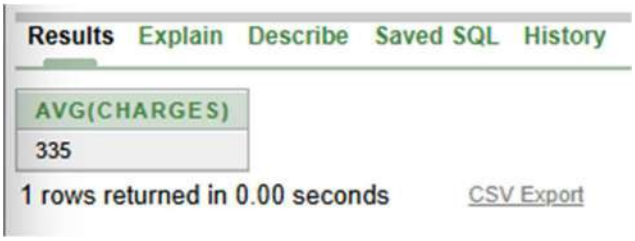
Results Explain Describe Saved SQL History

SUM(CHARGES)
750

1 rows returned in 0.00 seconds CSV Export

(iv) Select AVG (Charges) from HOSPITAL where Datofadm < {12/08/98}

SELECT AVG(Charges) FROM HOSPITAL WHERE Dateofadm < TO_DATE('12/08/98','DD/MM/RR');



7. Answer the questions (a) and (b) on the basis of the following tables STORE and ITEM

CREATE TABLE STORE (SNo VARCHAR2(5), SName VARCHAR2(30), Area VARCHAR2(20));



INSERT INTO STORE VALUES ('S01', 'ABC Computronics', 'GK II');

INSERT INTO STORE VALUES ('S02', 'All InfoTech Media', 'CP');

INSERT INTO STORE VALUES ('S03', 'Tech Shoppe', 'Nehru Place');

INSERT INTO STORE VALUES ('S04', 'Greeks Techno Soft', 'Nehru Place');

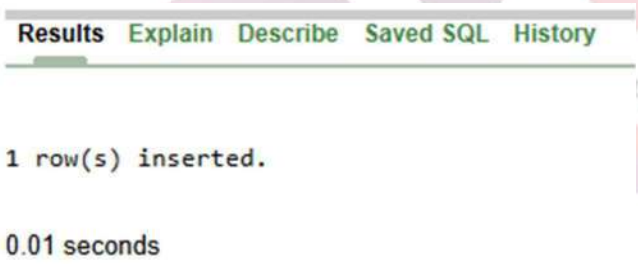
INSERT INTO STORE VALUES ('S05', 'Hitech Tech Store', 'CP');



CREATE TABLE ITEM (INo VARCHAR2(5), IName VARCHAR2(30), Price NUMBER, SNo VARCHAR2(5));



```
INSERT INTO ITEM VALUES ('T01', 'Mother Board', 12000, 'S01');
INSERT INTO ITEM VALUES ('T02', 'Hard Disk', 5000, 'S01');
INSERT INTO ITEM VALUES ('T03', 'Keyboard', 500, 'S02');
INSERT INTO ITEM VALUES ('T04', 'Mouse', 300, 'S01');
INSERT INTO ITEM VALUES ('T05', 'Mother Board', 13000, 'S02');
INSERT INTO ITEM VALUES ('T06', 'Keyboard', 400, 'S03');
INSERT INTO ITEM VALUES ('T07', 'LCD', 6000, 'S04');
INSERT INTO ITEM VALUES ('T08', 'LCD', 5500, 'S05');
INSERT INTO ITEM VALUES ('T09', 'Mouse', 350, 'S05');
INSERT INTO ITEM VALUES ('T10', 'Hard Disk', 4500, 'S03');
```



(a) Write the SQL queries

(i) To display IName and Price of all the items in ascending order of their Price.

```
SELECT IName, Price FROM ITEM ORDER BY Price ASC;
```

Results Explain Describe Saved SQL History

INAME	PRICE
Mouse	300
Mouse	350
Keyboard	400
Keyboard	500
Hard Disk	4500
Hard Disk	5000
LCD	5500
LCD	6000
Mother Board	12000
Mother Board	13000

10 rows returned in 0.00 seconds CSV Export

(ii) To display SNo and SName of all store location in CP.

SELECT SNo, SName FROM STORE WHERE Area = 'CP';

Results Explain Describe Saved SQL History

SNO	SNAME
S02	All InfoTech Media
S05	Hitech Tech Store

2 rows returned in 0.01 seconds CSV Export

(iii) To display Minimum and maximum Price of each IName from the table ITEM.

SELECT IName, MIN(Price), MAX(Price) FROM ITEM GROUP BY IName;

Results Explain Describe Saved SQL History

INAME	MIN(PRICE)	MAX(PRICE)
Mother Board	12000	13000
LCD	5500	6000
Mouse	300	350
Keyboard	400	500
Hard Disk	4500	5000

5 rows returned in 0.02 seconds CSV Export

(iv) To display IName, Price of all items and their respective SName where they are available.

SELECT I.IName, I.Price, S.SName FROM ITEM I, STORE S WHERE I.SNo = S.SNo;



Results Explain Describe Saved SQL History

INAME	PRICE	SNAME
Mother Board	12000	ABC Computronics
Hard Disk	5000	ABC Computronics
Keyboard	500	All InfoTech Media
Mouse	300	ABC Computronics
Mother Board	13000	All InfoTech Media
Keyboard	400	Tech Shoppe
LCD	6000	Greeks Techno Soft
LCD	5500	Hitech Tech Store
Mouse	350	Hitech Tech Store
Hard Disk	4500	Tech Shoppe

10 rows returned in 0.00 seconds CSV Export

(b) Write the output of the following SQL commands

(i) SELECT DISTINCT IName FROM ITEM WHERE Price >=5000;

SELECT DISTINCT IName FROM ITEM WHERE Price >= 5000;



Results Explain Describe Saved SQL History

INAME
Mother Board
LCD
Hard Disk

3 rows returned in 0.01 seconds CSV Export

(ii) SELECT Area, COUNT(*) FROM STORE GROUP BY Area;

SELECT Area, COUNT(*) FROM STORE GROUP BY Area;

Results Explain Describe Saved SQL History

AREA	COUNT(*)
GK II	1
CP	2
Nehru Place	2

3 rows returned in 0.02 seconds CSV Export

(iii) SELECT COUNT(DISTINCT Area) FROM STORE;

SELECT COUNT(DISTINCT Area) FROM STORE;

Results Explain Describe Saved SQL History

COUNT(DISTINCTAREA)
3

1 rows returned in 0.01 seconds CSV Export

(iv) SELECT IName, Price * 0.05 DISCOUNT FROM ITEM WHERE SNo IN ('S02', 'S03');

SELECT IName, Price * 0.05 DISCOUNT FROM ITEM WHERE SNo IN ('S02', 'S03');

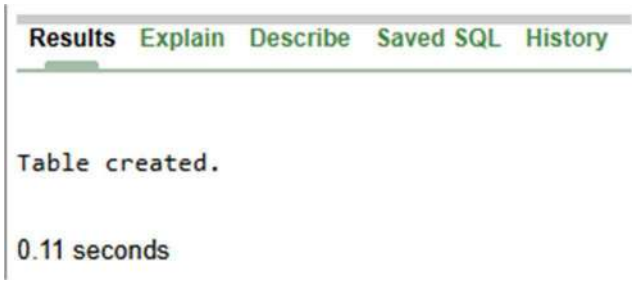
Results Explain Describe Saved SQL History

INAME	DISCOUNT
Keyboard	25
Mother Board	650
Keyboard	20
Hard Disk	225

4 rows returned in 0.00 seconds CSV Export

8. Write SQL commands for (a) to (f) and write output for (g) on the basis of Teacher relation given below:

CREATE TABLE TEACHER (No INT PRIMARY KEY, Name VARCHAR(20), Age INT, Department VARCHAR(20), Date_of_Join DATE, Salary INT, Gender CHAR(1));



INSERT INTO TEACHER VALUES
(1,'Jugal',34,'Computer',TO_DATE('10/01/97','DD/MM/RR'),12000,'M');

INSERT INTO TEACHER VALUES
(2,'Sharmila',31,'History',TO_DATE('24/03/98','DD/MM/RR'),20000,'F');

INSERT INTO TEACHER VALUES
(3,'Sandeep',32,'Maths',TO_DATE('12/12/96','DD/MM/RR'),30000,'M');

INSERT INTO TEACHER VALUES

(4,'Sangeeta',35,'History',TO_DATE('01/07/97','DD/MM/RR'),40000,'F');

INSERT INTO TEACHER VALUES

(5,'Rakesh',42,'Maths',TO_DATE('05/09/97','DD/MM/RR'),25000,'M');

INSERT INTO TEACHER VALUES

(6,'Shyam',50,'History',TO_DATE('27/06/98','DD/MM/RR'),30000,'M');

INSERT INTO TEACHER VALUES

(7,'Shiv Om',44,'Computer',TO_DATE('25/02/97','DD/MM/RR'),21000,'M');

INSERT INTO TEACHER VALUES

(8,'Shalakha',33,'Maths',TO_DATE('31/07/97','DD/MM/RR'),20000,'F');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(a) To show all information about the teacher of History department.

SELECT * FROM TEACHER WHERE Department = 'History';

Results Explain Describe Saved SQL History

NO	NAME	AGE	DEPARTMENT	DATE_OF_JOIN	SALARY	GENDER
2	Sharmila	31	History	24-MAR-98	20000	F
4	Sangeeta	35	History	01-JUL-97	40000	F
6	Shyam	50	History	27-JUN-98	30000	M

3 rows returned in 0.00 seconds CSV Export

(b) To list the names of female teacher who are in Hindi department.

SELECT Name FROM TEACHER WHERE Gender = 'F' AND Department = 'Hindi';

Results Explain Describe Saved SQL History

no data found

(c) To list names of all teachers with their date of joining in ascending order.

SELECT Name, Date_of_Join FROM TEACHER ORDER BY Date_of_Join ASC;

Results Explain Describe Saved SQL History

NAME	DATE_OF_JOIN
Sandeep	12-DEC-96
Jugal	10-JAN-97
Shiv Om	25-FEB-97
Sangeeta	01-JUL-97
Shalakha	31-JUL-97
Rakesh	05-SEP-97
Sharmila	24-MAR-98
Shyam	27-JUN-98

8 rows returned in 0.02 seconds CSV Export

(d) To display teacher's Name, Salary, Age for male teacher only.

SELECT Name, Salary, Age FROM TEACHER WHERE Gender = 'M';

Results Explain Describe Saved SQL History

NAME	SALARY	AGE
Jugal	12000	34
Sandeep	30000	32
Rakesh	25000	42
Shyam	30000	50
Shiv Om	21000	44

5 rows returned in 0.00 seconds CSV Export

(e) To count the number of teachers with Age > 23.

SELECT COUNT(*) FROM TEACHER WHERE Age > 23;

Results Explain Describe Saved SQL History

COUNT(*)
8

1 rows returned in 0.00 seconds CSV Export

(f) To insert a new row in the TEACHER table with the following data:

INSERT INTO TEACHER VALUES (9, 'Raja', 26, 'Computer',
TO_DATE('13/05/95','DD/MM/RR'), 2300, 'M');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.01 seconds
```

(g) Give the output of the following SQL statements:

(i) Select COUNT(distinct department) from TEACHER;

SELECT COUNT(DISTINCT Department) FROM TEACHER;

Results Explain Describe Saved SQL History

COUNT(DISTINCTDEPARTMENT)
3

1 rows returned in 0.00 seconds CSV Export

(ii) Select MAX(Age) from TEACHER where Gender="F";

SELECT MAX(Age) FROM TEACHER WHERE Gender = 'F';

Results Explain Describe Saved SQL History

MAX(AGE)
35

1 rows returned in 0.00 seconds CSV Export

(iii) Select AVG(Salary) from TEACHER where Date of join < {12/07/96};

SELECT AVG(Salary) FROM TEACHER WHERE Date_of_Join < TO_DATE('12/07/96','DD/MM/RR');

Results Explain Describe Saved SQL History

AVG(SALARY)
2300

1 rows returned in 0.00 seconds CSV Export

(iv) Select SUM(Salary) from TEACHER where Date of join < {12/07/96};

SELECT SUM(Salary) FROM TEACHER WHERE Date_of_Join <
TO_DATE('12/07/96','DD/MM/RR');

Results Explain Describe Saved SQL History

SUM(SALARY)
2300

1 rows returned in 0.00 seconds CSV Export



9. Given the following tables for a database LIBRARY:

Write SQL queries for (a) to (f):

CREATE TABLE BOOKS(Book_Id VARCHAR2(10) PRIMARY KEY, Book_Name VARCHAR2(30), Author_Name VARCHAR2(30), Publishers VARCHAR2(20), Price NUMBER, Type VARCHAR2(15), Quantity NUMBER);



INSERT INTO BOOKS VALUES ('F0001','The Tears','William Hopkins','First Publ.',750,'Fiction',10);

INSERT INTO BOOKS VALUES ('F0002','Thunderbolts','Anna Roberts','First Publ.',700,'Fiction',5);

INSERT INTO BOOKS VALUES ('T0001','My First C++','Briann & Brooke','EPB',250,'Text',10);

INSERT INTO BOOKS VALUES ('T0002','C++ Brainworks','A. W. Rossaine','TDH',325,'Text',5);

INSERT INTO BOOKS VALUES ('C0001','Fast Cook','LataKapoor','EPB',350,'Cookery',8);



CREATE TABLE ISSUED(Book_Id VARCHAR2(10), Quantity_Issued NUMBER);



INSERT INTO ISSUED VALUES ('F0001',3);

INSERT INTO ISSUED VALUES ('T0001',1);

INSERT INTO ISSUED VALUES ('C0001',5);



(a) To show Book name, Author name and Price of books of EPB publishers.

SELECT Book_Name, Author_Name, Price FROM BOOKS WHERE Publishers = 'EPB';



(b) To list the names of the books of Fiction Type.

SELECT Book_Name FROM BOOKS WHERE Type = 'Fiction';

Results Explain Describe Saved SQL History

BOOK_NAME
The Tears
Thunderbolts

2 rows returned in 0.00 seconds CSV Export

(c) To display the names and price of the books in descending order of their price.

SELECT Book_Name, Price FROM BOOKS ORDER BY Price DESC;

Results Explain Describe Saved SQL History

BOOK_NAME	PRICE
The Tears	750
Thunderbolts	700
Fast Cook	350
C++ Brainworks	325
My First C++	250

5 rows returned in 0.00 seconds CSV Export

(d) To increase the price of all books of first publisher by 50.

UPDATE BOOKS SET Price = Price + 50 WHERE Publishers = 'First Publ.';

Results Explain Describe Saved SQL History

```
2 row(s) updated.
```

0.00 seconds

(e) To display the Book_Id, Book_Name and Quantity issued for all books which have been issued.

The query will require contents from both the tables.

SELECT BOOKS.Book_Id, BOOKS.Book_Name, ISSUED.Quantity_Issued FROM BOOKS, ISSUED WHERE BOOKS.Book_Id = ISSUED.Book_Id;

Results Explain Describe Saved SQL History

BOOK_ID	BOOK_NAME	QUANTITY_ISSUED
F0001	The Tears	3
T0001	My First C++	1
C0001	Fast Cook	5

3 rows returned in 0.00 seconds CSV Export

(f) To insert a new row in the table ISSUED following the data: "F0002", 4

INSERT INTO ISSUED VALUES ('F0002',4);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(g) Give the output of the following queries

(i) SELECT COUNT(DISTINCT Publishers) FROM Books;

SELECT COUNT(DISTINCT Publishers) FROM BOOKS;

Results Explain Describe Saved SQL History

COUNT(DISTINCTPUBLISHERS)
3

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT SUM(Price) FROM Books WHERE Quantity > 5;

SELECT SUM(Price) FROM BOOKS WHERE Quantity > 5;

Results Explain Describe Saved SQL History

SUM(PRICE)
1400

1 rows returned in 0.02 seconds CSV Export

(iii) SELECT BOOK_NAME, AUTHOR_NAME FROM Books WHERE Price < 500;

SELECT BOOK_NAME, AUTHOR_NAME FROM BOOKS WHERE Price < 500;

Results Explain Describe Saved SQL History

BOOK_NAME	AUTHOR_NAME
My First C++	Briann & Brooke
C++ Brainworks	A. W. Rossaine
Fast Cook	LataKapoor

3 rows returned in 0.00 seconds CSV Export

(iv) SELECT COUNT(*) FROM Books;

SELECT COUNT(*) FROM BOOKS;

Results Explain Describe Saved SQL History

COUNT(*)
5

1 rows returned in 0.00 seconds CSV Export

AUTONOMOUS

10. Consider the following tables CARDEN and CUSTOMER and answer (a) and (b) parts of this question:

CREATE TABLE CARDEN (Ccode NUMBER PRIMARY KEY, CarName VARCHAR2(20), Make VARCHAR2(20), Color VARCHAR2(15), Capacity NUMBER, Charges NUMBER);



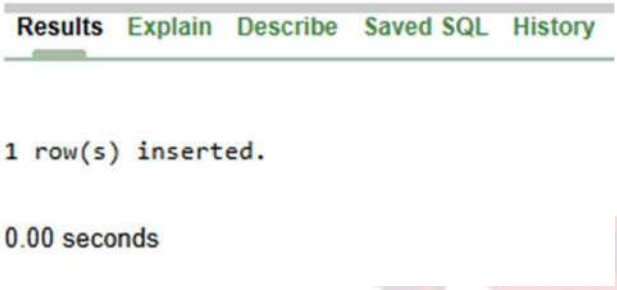
INSERT INTO CARDEN VALUES (501,'A-star','Suzuki','RED',3,14);

INSERT INTO CARDEN VALUES (503,'Indigo','Tata','SILVER',3,12);

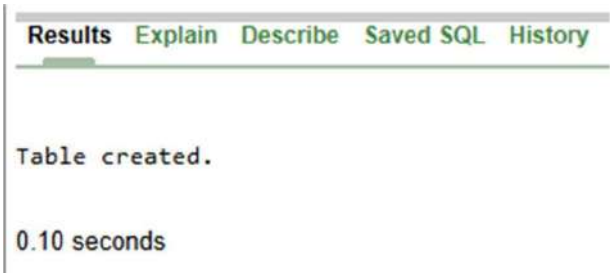
INSERT INTO CARDEN VALUES (502,'Innova','Toyota','WHITE',7,15);

INSERT INTO CARDEN VALUES (509,'SX4','Suzuki','SILVER',4,14);

INSERT INTO CARDEN VALUES (510,'C-Class','Mercedes','RED',4,35);



CREATE TABLE CUSTOMER (CCcode NUMBER PRIMARY KEY, Cname VARCHAR2(20), CarCode NUMBER);



INSERT INTO CUSTOMER VALUES (1001,'HamantSahu',501);

INSERT INTO CUSTOMER VALUES (1002,'Raj Lal',509);

INSERT INTO CUSTOMER VALUES (1003,'Feroja Shah',503);

INSERT INTO CUSTOMER VALUES (1004,'Ketan Dhal',502);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(a) Write SQL commands for the following statements

(i) To display the name of all the SILVER colored cars.

SELECT CarName FROM CARDEN WHERE Color = 'SILVER';

Results Explain Describe Saved SQL History

CARNAME
Indigo
SX4

2 rows returned in 0.02 seconds CSV Export

(ii) To display name of car, make and capacity of cars in descending order of their sitting capacity.

SELECT CarName, Make, Capacity FROM CARDEN ORDER BY Capacity DESC;

Results Explain Describe Saved SQL History

CARNAME	MAKE	CAPACITY
Innova	Toyota	7
C-Class	Mercedes	4
SX4	Suzuki	4
Indigo	Tata	3
A-star	Suzuki	3

5 rows returned in 0.00 seconds CSV Export

(iii) To display the highest Charges at which a vehicle can be hired from CARDEN.

SELECT MAX(Charges) FROM CARDEN;

Results Explain Describe Saved SQL History

MAX(CHARGES)
35

1 rows returned in 0.00 seconds CSV Export

(iv) To display the customer name and the corresponding name of the cars hired by them.

SELECT CUSTOMER.Cname, CARDEN.CarName FROM CUSTOMER, CARDEN WHERE CUSTOMER.CarCode = CARDEN.Ccode;

Results Explain Describe Saved SQL History

CNAME	CARNAME
HamantSahu	A-star
Feroja Shah	Indigo
Ketan Dhal	Innova
Raj Lal	SX4

4 rows returned in 0.00 seconds CSV Export

(b) Give the output of the following SQL queries

(i) SELECT COUNT(DISTINCT Make) FROM CARDEN;

SELECT COUNT(DISTINCT Make) FROM CARDEN;

Results Explain Describe Saved SQL History

COUNT(DISTINCTMAKE)
4

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT MAX(Charges), MIN(Charges) FROM CARDEN;

SELECT MAX(Charges), MIN(Charges) FROM CARDEN;

Results Explain Describe Saved SQL History

MAX(CHARGES)	MIN(CHARGES)
35	12

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT COUNT(*), Make FROM CARDEN;

SELECT COUNT(*), Make FROM CARDEN GROUP BY Make;

Results Explain Describe Saved SQL History

COUNT(*)	MAKE
2	Suzuki
1	Mercedes
1	Toyota
1	Tata

4 rows returned in 0.00 seconds CSV Export

(iv) SELECT CarName FROM CARDEN WHERE Capacity = 4;

SELECT CarName FROM CARDEN WHERE Capacity = 4;

Results Explain Describe Saved SQL History

CARNAME
SX4
C-Class

2 rows returned in 0.00 seconds CSV Export

11. Consider the following tables EMPLOYEE and SALGRADE and answer (a) and (b) parts of this question:

CREATE TABLE EMPLOYEE (ECODE NUMBER, NAME VARCHAR2(30), DESIG VARCHAR2(20), SGRADE VARCHAR2(5), DOJ DATE, DOB DATE);




```
Table created.

0.01 seconds
```

INSERT INTO EMPLOYEE VALUES (101, 'Abdul Ahmad', 'EXECUTIVE', 'S03', TO_DATE('23-MAR-2003','DD-MON-YYYY'), TO_DATE('13-JAN-1980','DD-MON-YYYY'));

INSERT INTO EMPLOYEE VALUES (102, 'Ravi Chander', 'HEAD-IT', 'S02', TO_DATE('12-FEB-2010','DD-MON-YYYY'), TO_DATE('22-JUL-1987','DD-MON-YYYY'));

INSERT INTO EMPLOYEE VALUES (103, 'John Ken', 'Receptionist', 'S03', TO_DATE('24-JUN-2009','DD-MON-YYYY'), TO_DATE('24-FEB-1983','DD-MON-YYYY'));

INSERT INTO EMPLOYEE VALUES (105, 'NazarAmeen', 'GM', 'S02', TO_DATE('11-AUG-2006','DD-MON-YYYY'), TO_DATE('03-MAR-1984','DD-MON-YYYY'));

INSERT INTO EMPLOYEE VALUES (108, 'PriyamSen', 'CEO', 'S01', TO_DATE('29-DEC-2004','DD-MON-YYYY'), TO_DATE('19-JAN-1982','DD-MON-YYYY'));




```
1 row(s) inserted.

0.00 seconds
```

CREATE TABLE SALGRADE (SGRADE VARCHAR2(5), SALARY NUMBER, HRA NUMBER);

Results Explain Describe Saved SQL History

```
Table created.
```

0.01 seconds

INSERT INTO SALGRADE VALUES ('S01', 56000, 18000);

INSERT INTO SALGRADE VALUES ('S02', 32000, 12000);

INSERT INTO SALGRADE VALUES ('S03', 24000, 8000);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.00 seconds

(a) Write SQL commands for the following statements:

(i) To display the detail of all the EMPLOYEE in descending order of DOJ.

SELECT * FROM EMPLOYEE ORDER BY DOJ DESC;

Results Explain Describe Saved SQL History

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
102	Ravi Chander	HEAD-IT	S02	12-FEB-10	22-JUL-87
103	John Ken	Receptionist	S03	24-JUN-09	24-FEB-83
105	NazarAmeen	GM	S02	11-AUG-06	03-MAR-84
108	PriyamSen	CEO	S01	29-DEC-04	19-JAN-82
101	Abdul Ahmad	EXECUTIVE	S03	23-MAR-03	13-JAN-80

5 rows returned in 0.02 seconds CSV Export

(ii) To display name and design of those EMPLOYEE, whose sgrade is either S02 or S03.

SELECT NAME, DESIG FROM EMPLOYEE WHERE SGRADE IN ('S02', 'S03');

Results Explain Describe Saved SQL History

NAME	DESIG
Abdul Ahmad	EXECUTIVE
Ravi Chander	HEAD-IT
John Ken	Receptionist
NazarAmeen	GM

4 rows returned in 0.00 seconds CSV Export

(iii) To display the content of all the EMPLOYEE table, whose DOJ is in between '09-FEB-2006' and '08-AUG-2009'.

SELECT * FROM EMPLOYEE WHERE DOJ BETWEEN TO_DATE('09-FEB-2006','DD-MON-YYYY') AND TO_DATE('08-AUG-2009','DD-MON-YYYY');

Results Explain Describe Saved SQL History

ECODE	NAME	DESIG	SGRADE	DOJ	DOB
103	John Ken	Receptionist	S03	24-JUN-09	24-FEB-83
105	NazarAmeen	GM	S02	11-AUG-06	03-MAR-84

2 rows returned in 0.00 seconds CSV Export

(iv) To add a new row in the EMPLOYEE table with the following data: 109, 'Harish Roy', 'HEAD-IT', 'S02', '09-SEP-2007', '21-APR-1983'.

INSERT INTO EMPLOYEE VALUES (109, 'Harish Roy', 'HEAD-IT', 'S02', TO_DATE('09-SEP-2007','DD-MON-YYYY'), TO_DATE('21-APR-1983','DD-MON-YYYY'));

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (SGRADE), SGRADE FROM EMPLOYEE GROUP BY SGRADE;

SELECT COUNT(SGRADE), SGRADE FROM EMPLOYEE GROUP BY SGRADE;

Results Explain Describe Saved SQL History

COUNT(SGRADE)	SGRADE
2	S03
3	S02
1	S01

3 rows returned in 0.00 seconds CSV Export

(ii) SELECT MIN (DOB), MAX (DOJ) FROM EMPLOYEE;

SELECT MIN(DOB), MAX(DOJ) FROM EMPLOYEE;

Results Explain Describe Saved SQL History

MIN(DOB)	MAX(DOJ)
13-JAN-80	12-FEB-10

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT NAME, SALARY FROM EMPLOYEE E, SALGRADE S WHERE E.SGRADE = S.SGRADE AND E.ECODE<103;

SELECT NAME, SALARY FROM EMPLOYEE E, SALGRADE S WHERE E.SGRADE = S.SGRADE AND E.ECODE < 103;

Results Explain Describe Saved SQL History

NAME	SALARY
Ravi Chander	32000
Abdul Ahmad	24000

2 rows returned in 0.00 seconds CSV Export

(iv) SELECT SGRADE, SALARY+HRA FROM SALGRADE WHERE SGRADE = 'S02';

SELECT SGRADE, SALARY + HRA FROM SALGRADE WHERE SGRADE = 'S02';

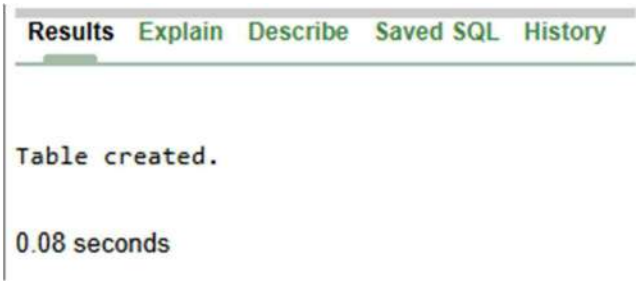
Results Explain Describe Saved SQL History

SGRADE	SALARY+HRA
S02	44000

1 rows returned in 0.00 seconds CSV Export

12. Consider the following tables WORKER and PAYLEVEL and answer (a) and (b) parts of this question.

CREATE TABLE WORKER (ECODE NUMBER, NAME VARCHAR2(30), DESIGN VARCHAR2(20), PLEVEL VARCHAR2(5), DOJ DATE, DOB DATE);



INSERT INTO WORKER VALUES (11, 'RadheShyam', 'Supervisor', 'P001', TO_DATE('13-SEP-2004','DD-MON-YYYY'), TO_DATE('23-AUG-1981','DD-MON-YYYY'));

INSERT INTO WORKER VALUES (12, 'ChanderNath', 'Operator', 'P003', TO_DATE('22-FEB-2010','DD-MON-YYYY'), TO_DATE('12-JUL-1987','DD-MON-YYYY'));

INSERT INTO WORKER VALUES (13, 'Fizza', 'Operator', 'P003', TO_DATE('14-JUN-2009','DD-MON-YYYY'), TO_DATE('14-OCT-1983','DD-MON-YYYY'));

INSERT INTO WORKER VALUES (15, 'Ahmeen Ahmad', 'Mechanic', 'P002', TO_DATE('21-AUG-2006','DD-MON-YYYY'), TO_DATE('13-MAR-1984','DD-MON-YYYY'));

INSERT INTO WORKER VALUES (18, 'Sanya', 'Clerk', 'P002', TO_DATE('19-DEC-2005','DD-MON-YYYY'), TO_DATE('09-JUN-1983','DD-MON-YYYY'));



CREATE TABLE PAYLEVEL (PLEVEL VARCHAR2(5), PAY NUMBER, ALLOWANCE NUMBER);

Results Explain Describe Saved SQL History

```
Table created.

0.08 seconds
```

INSERT INTO PAYLEVEL VALUES ('P001', 26000, 12000);

INSERT INTO PAYLEVEL VALUES ('P002', 22000, 10000);

INSERT INTO PAYLEVEL VALUES ('P003', 12000, 6000);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.05 seconds
```

(a) Write SQL commands for the following statements:

(i) To display the detail of all WORKER in descending order of DOB.

SELECT * FROM WORKER ORDER BY DOB DESC;

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
12	ChanderNath	Operator	P003	22-FEB-10	12-JUL-87
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-06	13-MAR-84
13	Fizza	Operator	P003	14-JUN-09	14-OCT-83
18	Sanya	Clerk	P002	19-DEC-05	09-JUN-83
11	RadheShyam	Supervisor	P001	13-SEP-04	23-AUG-81

5 rows returned in 0.03 seconds CSV Export

(ii) To display name and design of those WORKER, whose PLEVEL is either P001 to P002.

SELECT NAME, DESIGN FROM WORKER WHERE PLEVEL IN ('P001', 'P002');

Results Explain Describe Saved SQL History

NAME	DESIGN
RadheShyam	Supervisor
Ahmeen Ahmad	Mechanic
Sanya	Clerk

3 rows returned in 0.02 seconds CSV Export

(iii) To display the content of all the WORKER table, whose DOB is in between '19-JAN-1984' and '18-JAN-1987'.

SELECT * FROM WORKER WHERE DOB BETWEEN TO_DATE('19-JAN-1984','DD-MON-YYYY') AND TO_DATE('18-JAN-1987','DD-MON-YYYY');

Results Explain Describe Saved SQL History

ECODE	NAME	DESIGN	PLEVEL	DOJ	DOB
15	Ahmeen Ahmad	Mechanic	P002	21-AUG-06	13-MAR-84

1 rows returned in 0.00 seconds CSV Export

(iv) To add a new row with the following: 19, 'Daya Kishore', 'Operator', 'P003', '19-JUN-2008', '11-JUL-1984'.

INSERT INTO WORKER VALUES (19, 'Daya Kishore', 'Operator', 'P003', TO_DATE('19-JUN-2008','DD-MON-YYYY'), TO_DATE('11-JUL-1984','DD-MON-YYYY'));

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.01 seconds

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (PLEVEL), PLEVEL FROM WORKER GROUP BY PLEVEL;

SELECT COUNT(PLEVEL), PLEVEL FROM WORKER GROUP BY PLEVEL;

Results Explain Describe Saved SQL History

COUNT(PLEVEL)	PLEVEL
1	P001
3	P003
2	P002

3 rows returned in 0.03 seconds CSV Export

(ii) SELECT MAX (DOB), MIN (DOJ) FROM WORKER;

SELECT MAX(DOB), MIN(DOJ) FROM WORKER;

Results Explain Describe Saved SQL History

MAX(DOB)	MIN(DOJ)
12-JUL-87	13-SEP-04

1 rows returned in 0.02 seconds CSV Export

(iii) SELECT NAME, PAY FROM WORKER W, PAYLEVEL P WHERE W.PLEVEL = P.PLEVEL AND W.ECODE < 13;

SELECT NAME, PAY FROM WORKER W, PAYLEVEL P WHERE W.PLEVEL = P.PLEVEL AND W.ECODE < 13;

Results Explain Describe Saved SQL History

NAME	PAY
RadheShyam	26000
ChanderNath	12000

2 rows returned in 0.02 seconds CSV Export

(iv) SELECT PLEVEL, PAY+ALLOWANCE FROM PAYLEVEL WHERE PLEVEL = 'P003';

SELECT PLEVEL, PAY + ALLOWANCE FROM PAYLEVEL WHERE PLEVEL = 'P003';

Results Explain Describe Saved SQL History

PLEVEL	PAY+ALLOWANCE
P003	18000

1 rows returned in 0.00 seconds CSV Export

13. Consider the following tables STORE and SUPPLIERS and answer (a) and (b) parts of this question.

CREATE TABLE STORE (ItemNo NUMBER, Item VARCHAR2(30), Scode NUMBER, Qty NUMBER, Rate NUMBER, LastBuy DATE);



INSERT INTO STORE VALUES (2005, 'Sharpener Classic', 23, 60, 8, TO_DATE('30-JUN-09','DD-MON-RR'));

INSERT INTO STORE VALUES (2003, 'Ball Pen 0.25', 22, 50, 25, TO_DATE('01-FEB-09','DD-MON-RR'));

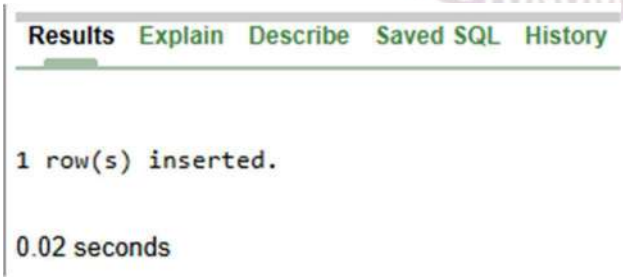
INSERT INTO STORE VALUES (2002, 'Gel Pen Premium', 21, 150, 12, TO_DATE('24-FEB-10','DD-MON-RR'));

INSERT INTO STORE VALUES (2006, 'Gel Pen Classic', 21, 250, 20, TO_DATE('11-MAY-09','DD-MON-RR'));

INSERT INTO STORE VALUES (2001, 'Eraser Small', 22, 220, 6, TO_DATE('19-JAN-09','DD-MON-RR'));

INSERT INTO STORE VALUES (2004, 'Eraser Big', 22, 110, 8, TO_DATE('02-DEC-09','DD-MON-RR'));

INSERT INTO STORE VALUES (2009, 'Ball Pen 0.5', 21, 180, 18, TO_DATE('03-NOV-09','DD-MON-RR'));



CREATE TABLE SUPPLIERS (Scode NUMBER, Sname VARCHAR2(30));

Results Explain Describe Saved SQL History

```
Table created.

0.07 seconds
```

INSERT INTO SUPPLIERS VALUES (21, 'Premium Stationers');

INSERT INTO SUPPLIERS VALUES (23, 'Soft Plastics');

INSERT INTO SUPPLIERS VALUES (22, 'Tetra Supply');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.02 seconds
```

(a) Write SQL commands for the following statements:

(i) To display details of all the items in the Store table in ascending order of LastBuy.

SELECT * FROM STORE ORDER BY LastBuy ASC;

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2001	Eraser Small	22	220	6	19-JAN-09
2003	Ball Pen 0.25	22	50	25	01-FEB-09
2006	Gel Pen Classic	21	250	20	11-MAY-09
2005	Sharpener Classic	23	60	8	30-JUN-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09
2004	Eraser Big	22	110	8	02-DEC-09
2002	Gel Pen Premium	21	150	12	24-FEB-10

7 rows returned in 0.00 seconds CSV Export

(ii) To display ItemNo and Item name of those items from STORE table whose Rate is more than 15 Rupees.

SELECT ItemNo, Item FROM STORE WHERE Rate > 15;

Results Explain Describe Saved SQL History

ITEMNO	ITEM
2003	Ball Pen 0.25
2006	Gel Pen Classic
2009	Ball Pen 0.5

3 rows returned in 0.00 seconds CSV Export

(iii) To display the details of those items whose Supplier code (Scode) is 22 or Quantity in Store (Qty) is more than 110 from the table STORE.

SELECT * FROM STORE WHERE Scode = 22 OR Qty > 110;

Results Explain Describe Saved SQL History

ITEMNO	ITEM	SCODE	QTY	RATE	LASTBUY
2003	Ball Pen 0.25	22	50	25	01-FEB-09
2002	Gel Pen Premium	21	150	12	24-FEB-10
2006	Gel Pen Classic	21	250	20	11-MAY-09
2001	Eraser Small	22	220	6	19-JAN-09
2004	Eraser Big	22	110	8	02-DEC-09
2009	Ball Pen 0.5	21	180	18	03-NOV-09

6 rows returned in 0.00 seconds CSV Export

(iv) To display minimum Rate of items for each supplier individually as per Scode from the table STORE.

SELECT Scode, MIN(Rate) FROM STORE GROUP BY Scode;

Results Explain Describe Saved SQL History

SCODE	MIN(RATE)
22	6
21	12
23	8

3 rows returned in 0.00 seconds CSV Export

(b) Give the output of the following SQL queries:

(i) SELECT COUNT (DISTINCT Scode) FROM STORE;

SELECT COUNT(DISTINCT Scode) FROM STORE;

Results Explain Describe Saved SQL History

COUNT(DISTINCTSCODE)
3

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT Rate * Qty FROM STORE WHERE ItemNo=2004;

SELECT Rate * Qty FROM STORE WHERE ItemNo = 2004;

Results Explain Describe Saved SQL History

RATE*QTY
880

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT Item,Sname FROM Store S, SUPPLIERS P WHERE S.Scode=P.Scode AND ItemNo=2006;

SELECT Item, Sname FROM STORE S, SUPPLIERS P WHERE S.Scode = P.Scode AND ItemNo = 2006;

Results Explain Describe Saved SQL History

ITEM	SNAME
Gel Pen Classic	Premium Stationers

1 rows returned in 0.01 seconds CSV Export

(iv) SELECT MAX(LastBuy) FROM STORE;

SELECT MAX(LastBuy) FROM STORE;

Results Explain Describe Saved SQL History

MAX(LASTBUY)
24-FEB-10

1 rows returned in 0.00 seconds CSV Export

14. Consider the following table GARMENT and FABRIC, write SQL commands for the statements (i) to (iv) and give outputs for the SQL queries (v) to (viii).

CREATE TABLE GARMENT (GCODE NUMBER, DESCRIPTION VARCHAR2(30), PRICE NUMBER, FCODE VARCHAR2(5), READYDATE DATE);



INSERT INTO GARMENT VALUES (10023, 'Pencil Skirt', 1150, 'F03', TO_DATE('19-DEC-08','DD-MON-RR'));

INSERT INTO GARMENT VALUES (10001, 'Formal Shirt', 1250, 'F01', TO_DATE('12-JAN-08','DD-MON-RR'));

INSERT INTO GARMENT VALUES (10012, 'Informal Shirt', 1550, 'F02', TO_DATE('06-JUN-08','DD-MON-RR'));

INSERT INTO GARMENT VALUES (10024, 'Baby Top', 750, 'F03', TO_DATE('07-APR-07','DD-MON-RR'));

INSERT INTO GARMENT VALUES (10090, 'Tulip Skirt', 850, 'F02', TO_DATE('31-MAR-07','DD-MON-RR'));

INSERT INTO GARMENT VALUES (10019, 'Evening Gown', 850, 'F03', TO_DATE('06-JUN-08','DD-MON-RR'));

INSERT INTO GARMENT VALUES (10009, 'Informal Pant', 1500, 'F02', TO_DATE('20-OCT-08','DD-MON-RR'));

INSERT INTO GARMENT VALUES (10007, 'Formal Pant', 1350, 'F01', TO_DATE('09-MAR-08','DD-MON-RR'));

INSERT INTO GARMENT VALUES (10020, 'Frock', 850, 'F04', TO_DATE('09-SEP-07','DD-MON-RR'));

INSERT INTO GARMENT VALUES (10089, 'Slacks', 750, 'F03', TO_DATE('20-OCT-08','DD-MON-RR'));




```
1 row(s) inserted.
```

0.03 seconds

CREATE TABLE FABRIC (FCODE VARCHAR2(5), TYPE VARCHAR2(20));




```
Table created.
```

0.08 seconds

INSERT INTO FABRIC VALUES ('F04', 'Polyester');

INSERT INTO FABRIC VALUES ('F02', 'Cotton');

INSERT INTO FABRIC VALUES ('F03', 'Silk');

INSERT INTO FABRIC VALUES ('F01', 'Terelene');




```
1 row(s) inserted.
```

0.03 seconds

(i) To display GCODE and DESCRIPTION of each GARMENT in descending order of GCODE.

SELECT GCODE, DESCRIPTION FROM GARMENT ORDER BY GCODE DESC;

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION
10090	Tulip Skirt
10089	Slacks
10024	Baby Top
10023	Pencil Skirt
10020	Frock
10019	Evening Gown
10012	Informal Shirt
10009	Informal Pant
10007	Formal Pant
10001	Formal Shirt

10 rows returned in 0.00 seconds CSV Export

(ii) To display the details of all the GARMENT, which have READYDATE in between 08-DEC-07 and 16-JUN-08 (inclusive if both the dates).

SELECT * FROM GARMENT WHERE READYDATE BETWEEN TO_DATE('08-DEC-07','DD-MON-RR') AND TO_DATE('16-JUN-08','DD-MON-RR');

Results Explain Describe Saved SQL History

GCODE	DESCRIPTION	PRICE	FCODE	READYDATE
10001	Formal Shirt	1250	F01	12-JAN-08
10012	Informal Shirt	1550	F02	06-JUN-08
10019	Evening Gown	850	F03	06-JUN-08
10007	Formal Pant	1350	F01	09-MAR-08

4 rows returned in 0.00 seconds CSV Export

(iii) To display the average PRICE of all the GARMENT, which are made up of fabric with FCODE as F03.

SELECT AVG(PRICE) FROM GARMENT WHERE FCODE = 'F03';

Results Explain Describe Saved SQL History

AVG(PRICE)
875

1 rows returned in 0.00 seconds CSV Export

(iv) To display fabric wise highest and lowest price of GARMENT from GARMENT table. (Display FCODE of each GARMENT along with highest and lowest Price).

SELECT FCODE, MAX(PRICE), MIN(PRICE) FROM GARMENT GROUP BY FCODE;

Results Explain Describe Saved SQL History

FCODE	MAX(PRICE)	MIN(PRICE)
F02	1550	850
F04	850	850
F03	1150	750
F01	1350	1250

4 rows returned in 0.00 seconds CSV Export

(v) SELECT SUM (PRICE) FROM GARMENT WHERE FCODE = 'F01';

SELECT SUM(PRICE) FROM GARMENT WHERE FCODE = 'F01';

Results Explain Describe Saved SQL History

SUM(PRICE)
2600

1 rows returned in 0.00 seconds CSV Export

(vi) SELECT DESCRIPTION, TYPE FROM GARMENT, FABRIC WHERE GARMENT.FCODE = FABRIC.FCODE AND GARMENT.PRICE >=1260;

SELECT DESCRIPTION, TYPE FROM GARMENT, FABRIC WHERE GARMENT.FCODE = FABRIC.FCODE AND GARMENT.PRICE >= 1260;

Results Explain Describe Saved SQL History

DESCRIPTION	TYPE
Informal Pant	Cotton
Informal Shirt	Cotton
Formal Pant	Terelene

3 rows returned in 0.00 seconds CSV Export

(vii) SELECT MAX (FCODE) FROM FABRIC;

SELECT MAX(FCODE) FROM FABRIC;

Results Explain Describe Saved SQL History

MAX(FCODE)
F04

1 rows returned in 0.00 seconds CSV Export

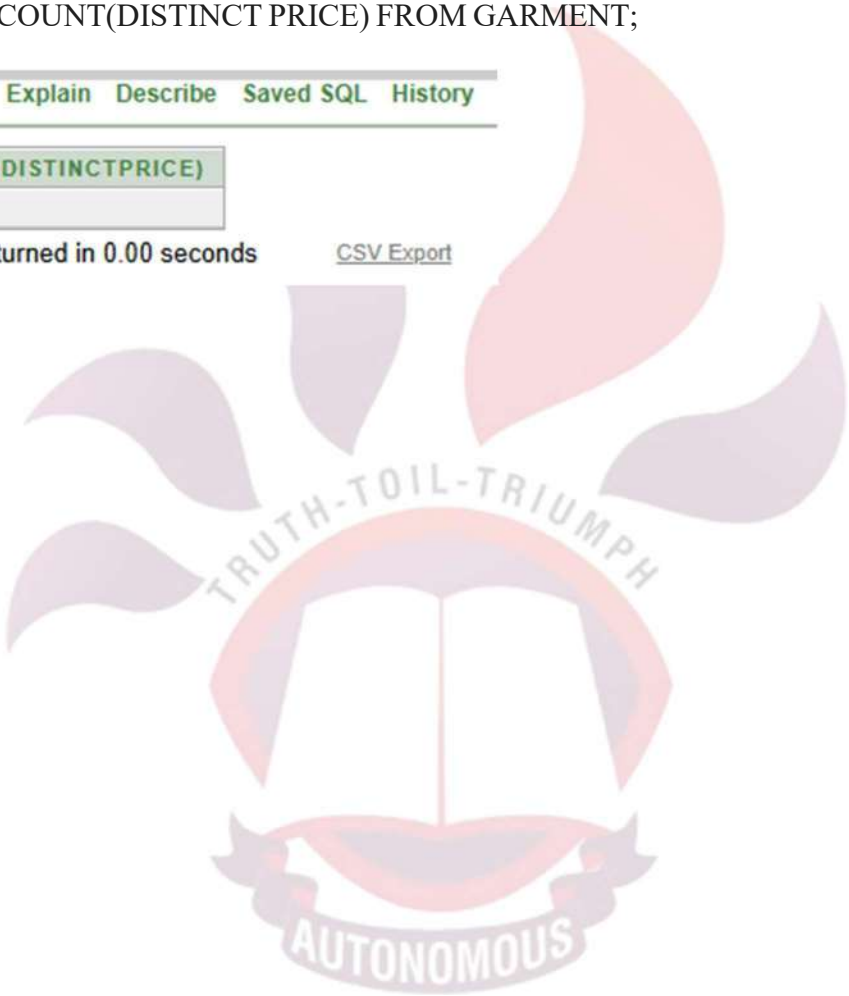
(viii) SELECT COUNT (DISTINCT PRICE) FROM GARMENT;

SELECT COUNT(DISTINCT PRICE) FROM GARMENT;

Results Explain Describe Saved SQL History

COUNT(DISTINCTPRICE)
7

1 rows returned in 0.00 seconds CSV Export



15. Given the following tables for a database FURNITURE: Write SQL command for (a) to (f) and write the outputs for (g) on the basis of tables FURNITURE and ARRIVALS.

CREATE TABLE FURNITURE (No NUMBER, ItemName VARCHAR2(30), Type VARCHAR2(20), DateofStock DATE, Price NUMBER, Discount NUMBER);



INSERT INTO FURNITURE VALUES (1, 'White lotus', 'Double Bed', TO_DATE('23/02/02','DD/MM/RR'), 30000, 25);

INSERT INTO FURNITURE VALUES (2, 'Pink feather', 'Baby cot', TO_DATE('20/01/02','DD/MM/RR'), 7000, 20);

INSERT INTO FURNITURE VALUES (3, 'Dolphin', 'Baby cot', TO_DATE('19/02/02','DD/MM/RR'), 9500, 20);

INSERT INTO FURNITURE VALUES (4, 'Decent', 'Office Table', TO_DATE('01/01/02','DD/MM/RR'), 25000, 30);

INSERT INTO FURNITURE VALUES (5, 'Comfort zone', 'Double Bed', TO_DATE('12/01/02','DD/MM/RR'), 25000, 25);

INSERT INTO FURNITURE VALUES (6, 'Donald', 'Baby cot', TO_DATE('24/02/02','DD/MM/RR'), 6500, 15);

INSERT INTO FURNITURE VALUES (7, 'Royal Finish', 'Office Table', TO_DATE('20/02/02','DD/MM/RR'), 18000, 30);

INSERT INTO FURNITURE VALUES (8, 'Royal tiger', 'Sofa', TO_DATE('22/02/02','DD/MM/RR'), 31000, 30);

INSERT INTO FURNITURE VALUES (9, 'Econo sitting', 'Sofa', TO_DATE('13/12/01','DD/MM/RR'), 9500, 25);

INSERT INTO FURNITURE VALUES (10, 'Eating paradise', 'Dining Table', TO_DATE('19/02/02','DD/MM/RR'), 11500, 25);



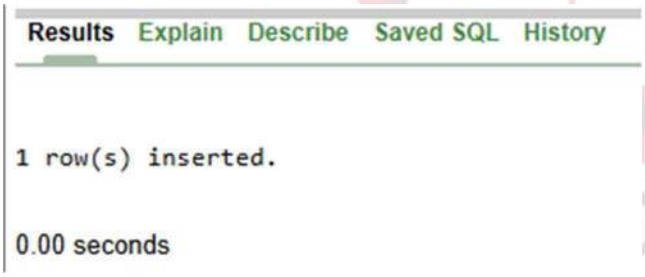
CREATE TABLE ARRIVALS (No NUMBER, ItemName VARCHAR2(30), Type VARCHAR2(20), DateofStock DATE, Price NUMBER, Discount NUMBER);



INSERT INTO ARRIVALS VALUES (11, 'Wood Comfort', 'Double Bed', TO_DATE('23/03/03','DD/MM/RR'), 25000, 25);

INSERT INTO ARRIVALS VALUES (12, 'Old Fox', 'Sofa', TO_DATE('20/02/03','DD/MM/RR'), 17000, 20);

INSERT INTO ARRIVALS VALUES (13, 'Micky', 'Baby cot', TO_DATE('21/02/02','DD/MM/RR'), 7500, 15);



(a) To show all information about the baby cots from the FURNITURE table.

SELECT * FROM FURNITURE WHERE Type = 'Baby cot';

Results Explain Describe Saved SQL History

NO	ITEMNAME	TYPE	DATEOFSTOCK	PRICE	DISCOUNT
2	Pink feather	Baby cot	20-JAN-02	7000	20
3	Dolphin	Baby cot	19-FEB-02	9500	20
6	Donald	Baby cot	24-FEB-02	6500	15

3 rows returned in 0.00 seconds CSV Export

(b) To list the ITEMNAME which are priced at more than 15000 from the FURNITURE table.

SELECT ItemName FROM FURNITURE WHERE Price > 15000;

Results Explain Describe Saved SQL History

ITEMNAME
White lotus
Decent
Comfort zone
Royal Finish
Royal tiger

5 rows returned in 0.00 seconds CSV Export

(c) To list ITEMNAME AND TYPE of those items, in which DATEOFSTOCK is before 22/01/02 from the FURNITURE table in descending order of ITEMNAME.

SELECT ItemName, Type FROM FURNITURE WHERE DateofStock < TO_DATE('22/01/02','DD/MM/RR') ORDER BY ItemName DESC;

Results Explain Describe Saved SQL History

ITEMNAME	TYPE
Pink feather	Baby cot
Econo sitting	Sofa
Decent	Office Table
Comfort zone	Double Bed

4 rows returned in 0.00 seconds CSV Export

(d) To display ITEMNAME and DATEOFSTOCK of those items, in which the DISCOUNT percentage is more than 25 from FURNITURE table.

SELECT ItemName, DateofStock FROM FURNITURE WHERE Discount > 25;

Results Explain Describe Saved SQL History

ITEMNAME	DATEOFSTOCK
Decent	01-JAN-02
Royal Finish	20-FEB-02
Royal tiger	22-FEB-02

3 rows returned in 0.00 seconds CSV Export

(e) To count the number of items, whose TYPE is "Sofa" from FURNITURE table.

SELECT COUNT(*) FROM FURNITURE WHERE Type = 'Sofa';

Results Explain Describe Saved SQL History

COUNT(*)
2

1 rows returned in 0.00 seconds CSV Export

(f) To insert a new row in the ARRIVALS table with the following data: 14, "Velvet touch", "Double bed", {25/03/03}, 25000, 30.

INSERT INTO ARRIVALS VALUES (14, 'Velvet touch', 'Double Bed',
TO_DATE('25/03/03','DD/MM/RR'), 25000, 30);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.
```

0.00 seconds

(g) Give the output of following SQL statement:

(i) SELECT COUNT(DISTINCT TYPE) FROM FURNITURE;

SELECT COUNT(DISTINCT Type) FROM FURNITURE;

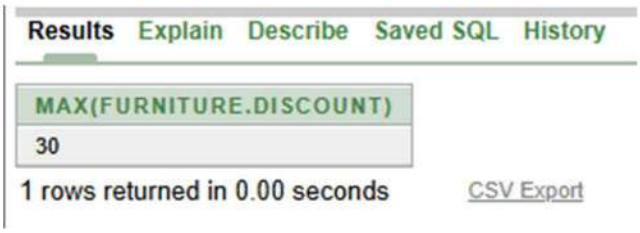
Results Explain Describe Saved SQL History

COUNT(DISTINCTTYPE)
5

1 rows returned in 0.01 seconds CSV Export

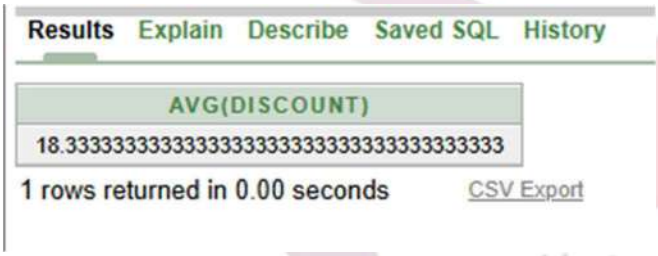
(ii) SELECT MAX(DISCOUNT) FROM FURNITURE, ARRIVALS;

SELECT MAX(FURNITURE.Discount) FROM FURNITURE, ARRIVALS;



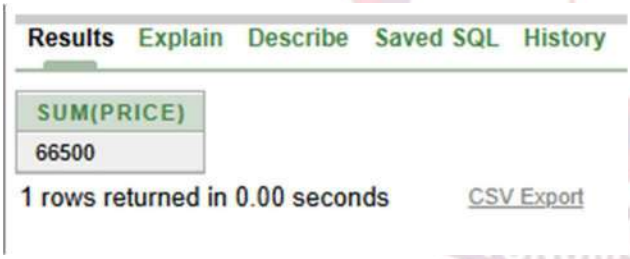
(iii) SELECT AVG(DISCOUNT) FROM FURNITURE WHERE TYPE = "Baby cot";

SELECT AVG(Discount) FROM FURNITURE WHERE Type = 'Baby cot';



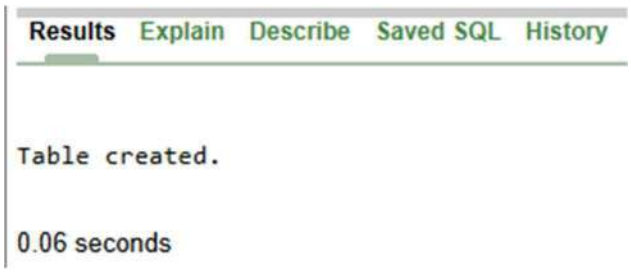
(iv) SELECT SUM(PRICE) FROM FURNITURE WHERE DATEOFSTOCK < {12/02/02};

SELECT SUM(Price) FROM FURNITURE WHERE DateofStock <
TO_DATE('12/02/02','DD/MM/RR');



16. Given the following COVID19 relation, write SQL commands for questions (a) to (f).

CREATE TABLE COVID19 (PatientID NUMBER, PatientName VARCHAR2(30), State VARCHAR2(30), Age NUMBER, Status VARCHAR2(20), Vaccinated VARCHAR2(5));



INSERT INTO COVID19 VALUES (101, 'Arun', 'Tamil Nadu', 35, 'Recovered', 'Yes');

INSERT INTO COVID19 VALUES (102, 'Priya', 'Kerala', 42, 'Active', 'Yes');

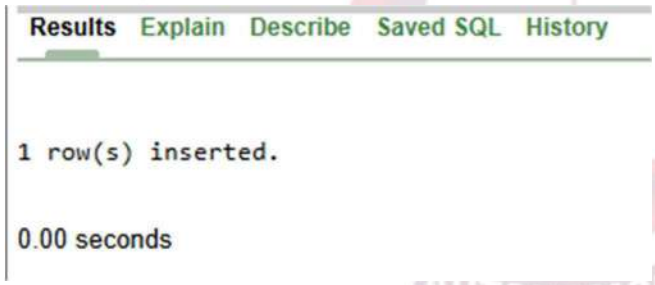
INSERT INTO COVID19 VALUES (103, 'Karthik', 'Karnataka', 28, 'Recovered', 'No');

INSERT INTO COVID19 VALUES (104, 'Divya', 'Tamil Nadu', 31, 'Active', 'Yes');

INSERT INTO COVID19 VALUES (105, 'Rahul', 'Andhra Pradesh', 55, 'Deceased', 'No');

INSERT INTO COVID19 VALUES (106, 'Meena', 'Kerala', 24, 'Recovered', 'Yes');

INSERT INTO COVID19 VALUES (107, 'Suresh', 'Tamil Nadu', 48, 'Active', 'No');



(a) Display all details of patients from Tamil Nadu.

SELECT * FROM COVID19 WHERE State = 'Tamil Nadu';

Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	STATE	AGE	STATUS	VACCINATED
101	Arun	Tamil Nadu	35	Recovered	Yes
104	Divya	Tamil Nadu	31	Active	Yes
107	Suresh	Tamil Nadu	48	Active	No

3 rows returned in 0.02 seconds CSV Export

(b) Display the names of patients whose Status is Recovered.

SELECT PatientName FROM COVID19 WHERE Status = 'Recovered';

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Karthik
Meena

3 rows returned in 0.00 seconds CSV Export

(c) Display PatientName and Age in ascending order of Age.

SELECT PatientName, Age FROM COVID19 ORDER BY Age ASC;

Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Meena	24
Karthik	28
Divya	31
Arun	35
Priya	42
Suresh	48
Rahul	55

7 rows returned in 0.02 seconds CSV Export

(d) Display the names of patients who are Vaccinated = 'Yes'.

SELECT PatientName FROM COVID19 WHERE Vaccinated = 'Yes';

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Priya
Divya
Meena

4 rows returned in 0.00 seconds CSV Export

(e) Count the number of patients whose status is Active.

SELECT COUNT(*) FROM COVID19 WHERE Status = 'Active';

Results Explain Describe Saved SQL History

COUNT(*)
3

1 rows returned in 0.00 seconds CSV Export

(f) Insert the following record into the COVID19 table:

108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes'

INSERT INTO COVID19 VALUES (108, 'Anitha', 'Puducherry', 39, 'Recovered', 'Yes');

Results Explain Describe Saved SQL History

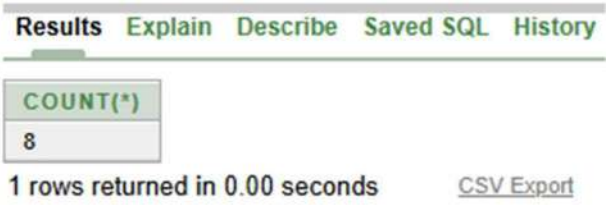
```
1 row(s) inserted.

0.00 seconds
```

(g) Write the output of the following SQL commands.

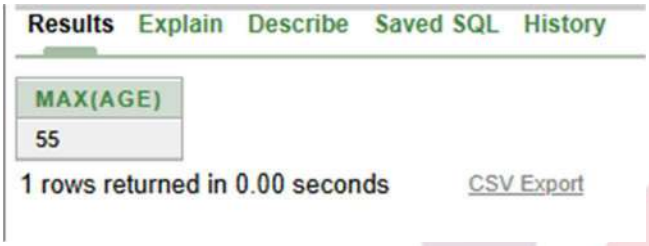
(a) SELECT COUNT(*) FROM COVID19;

SELECT COUNT(*) FROM COVID19;



(b) SELECT MAX(Age) FROM COVID19;

SELECT MAX(Age) FROM COVID19;



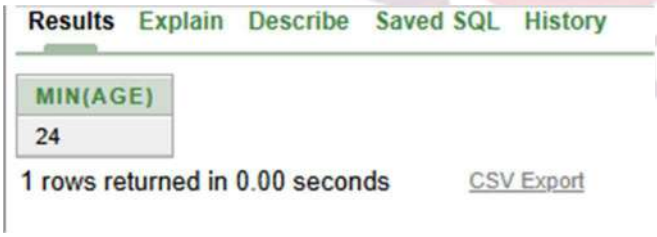
(c) SELECT COUNT(*) FROM COVID19 WHERE Vaccinated='Yes';

SELECT COUNT(*) FROM COVID19 WHERE Vaccinated = 'Yes';



(d) SELECT MIN(Age) FROM COVID19 WHERE Status='Recovered';

SELECT MIN(Age) FROM COVID19 WHERE Status = 'Recovered';



17. Given the following HOUSEPRICE relation, write SQL commands for questions (a) to (f).

CREATE TABLE HOUSEPRICE (HouseID NUMBER, Location VARCHAR2(30), Bedrooms NUMBER, AreaSqft NUMBER, Price NUMBER, HouseType VARCHAR2(20));



INSERT INTO HOUSEPRICE VALUES (201, 'Chennai', 2, 1100, 5200000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES (202, 'Coimbatore', 3, 1450, 6800000, 'Villa');

INSERT INTO HOUSEPRICE VALUES (203, 'Madurai', 2, 980, 4100000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES (204, 'Chennai', 4, 2200, 9800000, 'Villa');

INSERT INTO HOUSEPRICE VALUES (205, 'Trichy', 3, 1650, 7200000, 'Independent');

INSERT INTO HOUSEPRICE VALUES (206, 'Salem', 2, 1200, 5000000, 'Apartment');

INSERT INTO HOUSEPRICE VALUES (207, 'Erode', 3, 1800, 7600000, 'Independent');



(a) Display all houses located in Chennai.

SELECT * FROM HOUSEPRICE WHERE Location = 'Chennai';

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
201	Chennai	2	1100	5200000	Apartment
204	Chennai	4	2200	9800000	Villa

2 rows returned in 0.01 seconds CSV Export

(b) Display the Location and Price of houses having 3 Bedrooms.

SELECT Location, Price FROM HOUSEPRICE WHERE Bedrooms = 3;

Results Explain Describe Saved SQL History

LOCATION	PRICE
Coimbatore	6800000
Erode	7600000

2 rows returned in 0.00 seconds CSV Export

(c) Display Location, AreaSqft, and Price in descending order of Price.

SELECT Location, AreaSqft, Price FROM HOUSEPRICE ORDER BY Price DESC;

Results Explain Describe Saved SQL History

LOCATION	AREASQFT	PRICE
Chennai	2200	9800000
Erode	1800	7600000
Coimbatore	1450	6800000
Chennai	1100	5200000
Salem	1200	5000000
Madurai	980	4100000

6 rows returned in 0.00 seconds CSV Export

(d) Display the details of houses costing more than ₹70,00,000.

SELECT * FROM HOUSEPRICE WHERE Price > 7000000;

Results Explain Describe Saved SQL History

HOUSEID	LOCATION	BEDROOMS	AREASQFT	PRICE	HOUSETYPE
204	Chennai	4	2200	9800000	Villa
207	Erode	3	1800	7600000	Independent

2 rows returned in 0.00 seconds CSV Export

(e) Count the number of Apartments.

SELECT COUNT(*) FROM HOUSEPRICE WHERE HouseType = 'Apartment';

Results Explain Describe Saved SQL History

COUNT(*)
3

1 rows returned in 0.00 seconds CSV Export

(f) Insert the following record:

208, 'Vellore', 3, 1700, 7300000, 'Villa'

INSERT INTO HOUSEPRICE VALUES (208, 'Vellore', 3, 1700, 7300000, 'Villa');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(g) Write the output of the following SQL commands.

(i) SELECT MAX(Price) FROM HOUSEPRICE;

SELECT MAX(Price) FROM HOUSEPRICE;

Results Explain Describe Saved SQL History

MAX(PRICE)
9800000

1 rows returned in 0.01 seconds CSV Export

(ii) SELECT MIN(AreaSqft) FROM HOUSEPRICE;

SELECT MIN(AreaSqft) FROM HOUSEPRICE;

Results Explain Describe Saved SQL History

MIN(AREASQFT)
980

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT AVG(Price) FROM HOUSEPRICE WHERE HouseType='Apartment';

SELECT AVG(Price) FROM HOUSEPRICE WHERE HouseType = 'Apartment';

Results Explain Describe Saved SQL History

AVG(PRICE)
4766666.666666666666666666666666666667

1 rows returned in 0.00 seconds CSV Export

(iv) SELECT COUNT(*) FROM HOUSEPRICE WHERE Bedrooms=3;

SELECT COUNT(*) FROM HOUSEPRICE WHERE Bedrooms = 3;

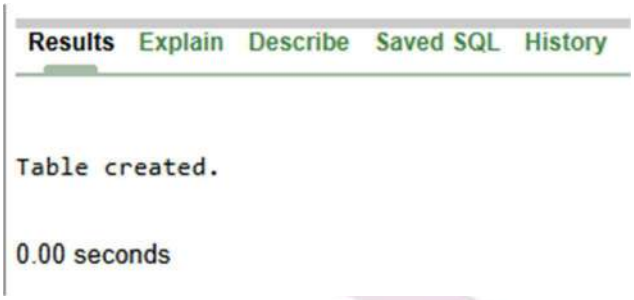
Results Explain Describe Saved SQL History

COUNT(*)
3

1 rows returned in 0.00 seconds CSV Export

18. Given the following CANCERDATA relation, write SQL commands for questions (a) to (f).

CREATE TABLE CANCERDATA (PatientID NUMBER, PatientName VARCHAR2(30), CancerType VARCHAR2(20), Age NUMBER, Stage VARCHAR2(5), SurvivalYears NUMBER);



INSERT INTO CANCERDATA VALUES (301, 'Kavya', 'Breast', 45, 'II', 8);

INSERT INTO CANCERDATA VALUES (302, 'Arun', 'Lung', 60, 'III', 4);

INSERT INTO CANCERDATA VALUES (303, 'Divya', 'Breast', 38, 'I', 10);

INSERT INTO CANCERDATA VALUES (304, 'Hari', 'Colon', 52, 'II', 6);

INSERT INTO CANCERDATA VALUES (305, 'Meena', 'Lung', 57, 'IV', 2);

INSERT INTO CANCERDATA VALUES (306, 'Ravi', 'Prostate', 64, 'III', 5);

INSERT INTO CANCERDATA VALUES (307, 'Priya', 'Breast', 41, 'II', 9);



(a) Display all patients diagnosed with Breast Cancer.

SELECT * FROM CANCERDATA WHERE CancerType = 'Breast';

Results Explain Describe Saved SQL History

PATIENTID	PATIENTNAME	CANCERTYPE	AGE	STAGE	SURVIVALYEARS
301	Kavya	Breast	45	II	8
303	Divya	Breast	38	I	10
307	Priya	Breast	41	II	9

3 rows returned in 0.00 seconds CSV Export

(b) Display the names of patients whose Stage is III.

SELECT PatientName FROM CANCERDATA WHERE Stage = 'III';

Results Explain Describe Saved SQL History

PATIENTNAME
Arun
Ravi

2 rows returned in 0.02 seconds CSV Export

(c) Display PatientName and Age in ascending order of Age.

SELECT PatientName, Age FROM CANCERDATA ORDER BY Age ASC;

Results Explain Describe Saved SQL History

PATIENTNAME	AGE
Divya	38
Priya	41
Kavya	45
Hari	52
Meena	57
Arun	60
Ravi	64

7 rows returned in 0.00 seconds CSV Export

(d) Display the names of patients having SurvivalYears greater than 5.

SELECT PatientName FROM CANCERDATA WHERE SurvivalYears > 5;

Results Explain Describe Saved SQL History

PATIENTNAME
Kavya
Divya
Hari
Priya

4 rows returned in 0.00 seconds CSV Export

(e) Count the number of patients diagnosed with Lung Cancer.

SELECT COUNT(*) FROM CANCERDATA WHERE CancerType = 'Lung';

Results Explain Describe Saved SQL History

COUNT(*)
2

1 rows returned in 0.00 seconds CSV Export

(f) Insert the following record:

308, 'Anand', 'Leukemia', 50, 'II', 7

INSERT INTO CANCERDATA VALUES (308, 'Anand', 'Leukemia', 50, 'II', 7);

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(g) Write the output of the following SQL commands.

(i) SELECT COUNT(DISTINCT CancerType) FROM CANCERDATA;

SELECT COUNT(DISTINCT CancerType) FROM CANCERDATA;

Results Explain Describe Saved SQL History

COUNT(DISTINCTCANCERTYPE)
5

1 rows returned in 0.00 seconds CSV Export

(ii) SELECT MAX(SurvivalYears) FROM CANCERDATA;

SELECT MAX(SurvivalYears) FROM CANCERDATA;

Results Explain Describe Saved SQL History

MAX(SURVIVALYEARS)
10

1 rows returned in 0.00 seconds CSV Export

(iii) SELECT AVG(Age) FROM CANCERDATA WHERE CancerType='Breast';

SELECT AVG(Age) FROM CANCERDATA WHERE CancerType = 'Breast';

Results Explain Describe Saved SQL History

AVG(AGE)
41.333333333333333333333333333333333333

1 rows returned in 0.00 seconds CSV Export

(iv) SELECT MIN(Age) FROM CANCERDATA WHERE Stage='II';

SELECT MIN(Age) FROM CANCERDATA WHERE Stage = 'II';

Results Explain Describe Saved SQL History

MIN(AGE)
41

1 rows returned in 0.01 seconds CSV Export

19. Write SQL queries for (a) to (f) and write the outputs for the SQL queries mentioned shown in (g1) to (g4) parts on the basis of tables ITEMS and TRADERS.

CREATE TABLE ITEMS (CODE NUMBER, INAME VARCHAR2(30), QTY NUMBER, PRICE NUMBER, COMPANY VARCHAR2(30), TCODE VARCHAR2(5));



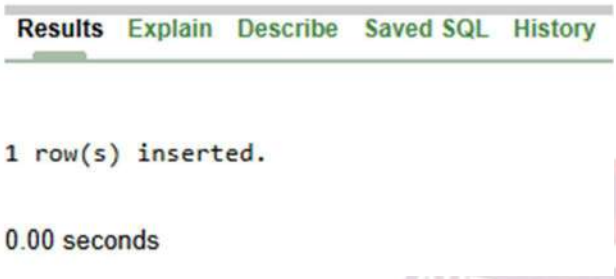
INSERT INTO ITEMS VALUES (1001, 'DIGITAL PAD12i', 120, 11000, 'XENITA', 'T01');

INSERT INTO ITEMS VALUES (1006, 'LED SCREEN 40', 70, 38000, 'SANTORA', 'T02');

INSERT INTO ITEMS VALUES (1004, 'CAR GPS SYSTEM', 50, 21500, 'GEOKNOW', 'T01');

INSERT INTO ITEMS VALUES (1003, 'DIGITAL CAMERA 12X', 160, 8000, 'DIGICLICK', 'T02');

INSERT INTO ITEMS VALUES (1005, 'PEN DRIVE 32 GB', 600, 1200, 'STOREHOME', 'T03');



CREATE TABLE TRADERS (TCODE VARCHAR2(5), TNAME VARCHAR2(30), CITY VARCHAR2(20));



INSERT INTO TRADERS VALUES ('T01', 'ELECTRONIC SALES', 'MUMBAI');

INSERT INTO TRADERS VALUES ('T03', 'BUSY STORE CORP', 'DELHI');

INSERT INTO TRADERS VALUES ('T02', 'DISP HOUSE INC', 'CHENNAI');

Results Explain Describe Saved SQL History

```
1 row(s) inserted.

0.00 seconds
```

(a) To display the details of all the items in ascending order of item names (i.e. INAME).

SELECT * FROM ITEMS ORDER BY INAME ASC;

Results Explain Describe Saved SQL History

CODE	INAME	QTY	PRICE	COMPANY	TCODE
1004	CAR GPS SYSTEM	50	21500	GEOKNOW	T01
1003	DIGITAL CAMERA 12X	160	8000	DIGICLICK	T02
1001	DIGITAL PAD12i	120	11000	XENITA	T01
1006	LED SCREEN 40	70	38000	SANTORA	T02
1005	PEN DRIVE 32 GB	600	1200	STOREHOME	T03

5 rows returned in 0.00 seconds CSV Export

(b) To display item name and price of all those items, whose price is in the range of 10000 and 22000 (both values inclusive).

SELECT INAME, PRICE FROM ITEMS WHERE PRICE BETWEEN 10000 AND 22000;

Results Explain Describe Saved SQL History

INAME	PRICE
DIGITAL PAD12i	11000
CAR GPS SYSTEM	21500

2 rows returned in 0.00 seconds CSV Export

(c) To display the number of items, which are traded by each trader.

SELECT TCODE, COUNT(*) FROM ITEMS GROUP BY TCODE;

Results Explain Describe Saved SQL History

TCODE	COUNT(*)
T01	2
T03	1
T02	2

3 rows returned in 0.00 seconds CSV Export

(d) To display the price, item name (i.e. INAME) and quantity (i.e. QTY) of those items which have quantity more than 150.

SELECT PRICE, INAME, QTY FROM ITEMS WHERE QTY > 150;

Results Explain Describe Saved SQL History

PRICE	INAME	QTY
8000	DIGITAL CAMERA 12X	160
1200	PEN DRIVE 32 GB	600

2 rows returned in 0.00 seconds CSV Export

(e) To display the names of those traders, who are either from DELHI or from MUMBAI.

SELECT TNAME FROM TRADERS WHERE CITY = 'DELHI' OR CITY = 'MUMBAI';

Results Explain Describe Saved SQL History

TNAME
ELECTRONIC SALES
BUSY STORE CORP

2 rows returned in 0.00 seconds CSV Export

(f) To display the name of the companies and the name of the items in descending order of company names.

SELECT COMPANY, INAME FROM ITEMS ORDER BY COMPANY DESC;

Results Explain Describe Saved SQL History

COMPANY	INAME
XENITA	DIGITAL PAD12i
STOREHOME	PEN DRIVE 32 GB
SANTORA	LED SCREEN 40
GEOKNOW	CAR GPS SYSTEM
DIGICLICK	DIGITAL CAMERA 12X

5 rows returned in 0.00 seconds CSV Export

(g) Obtain the outputs of the following SQL queries based on the data given in tables ITEMS and TRADERS.

(g1) SELECT MAX(PRICE), MIN(PRICE) FROM ITEMS;

SELECT MAX(PRICE), MIN(PRICE) FROM ITEMS;

Results Explain Describe Saved SQL History

MAX(PRICE)	MIN(PRICE)
38000	1200

1 rows returned in 0.00 seconds CSV Export

(g2) SELECT PRICE * QTY AMOUNT FROM ITEMS WHERE CODE = 1004;

SELECT PRICE * QTY AS AMOUNT FROM ITEMS WHERE CODE = 1004;

Results Explain Describe Saved SQL History

AMOUNT
1075000

1 rows returned in 0.00 seconds CSV Export

(g3) SELECT DISTINCT TCODE FROM ITEMS;

SELECT DISTINCT TCODE FROM ITEMS;

Results Explain Describe Saved SQL History

TCODE
T01
T03
T02

3 rows returned in 0.00 seconds CSV Export

(g4) SELECT INAME, TNAME FROM ITEMS I, TRADERS T WHERE I.TCODE = T.TCODE AND QTY < 100;

SELECT I.INAME, T.TNAME FROM ITEMS I, TRADERS T WHERE I.TCODE = T.TCODE AND I.QTY < 100;

Results Explain Describe Saved SQL History

INAME	TNAME
CAR GPS SYSTEM	ELECTRONIC SALES
LED SCREEN 40	DISP HOUSE INC
LED SCREEN 40	DISP HOUSE INC

3 rows returned in 0.00 seconds CSV Export

TRUTH RIUMPH AUTONOMOUS